Algorithm Specification and Supporting Documentation

SHUTTLE: A Lattice Based Digital Signature Scheme

Long Chen[1], Yifan Ming[1], Jipeng Zhang[2], Cong Zhang[3], Zihan Liu[1], Guofeng Tang[4], Si Gao[1], Pengfei Chen[3], Yutao Sun[5], Taiyu Wang[3], Minxuan Mei[3], and Jianhua Hou[6]

[1]Institute of Software, Chinese Academy of Sciences, China
[2]National University of Singapore, Singapore
[3]Zhejiang University, China
[4]Singapore Management University, Singapore
[5]Beijing Electronic Science and Technology Institute, China
[6]Information Engineering University, China

October 9, 2026

Abstract

This document provides the complete specification, design rationale, security analysis, and performance evaluation for SHUTTLE, submitted to the New Generation Commercial Cryptography (NGCC) algorithm call by the Commercial Cryptography Standard Research Institute.

SHUTTLE is an exceptionally compact, post-quantum digital signature scheme based on the conservative Module Learning with Errors (MLWE) and Module Short Integer Solution (MSIS) assumptions. Designed within the Fiat-Shamir with aborts paradigm, it leverages an advanced rejection-free iterative sampling technique to achieve one of the smallest signature sizes among integer-based lattice implementations. To address the inherent challenges of practical deployment, SHUTTLE introduces three core innovations: (1) an asymmetric structural compression mechanism tailored for MLWE that dramatically reduces transmission overhead; (2) a rejection-free iterative sampling approach that ensures zero-knowledge simulatability while completely eliminating timing side-channel vulnerabilities; and (3) logarithmic optimizations for transcendental functions, which drastically reduce computational overhead.

Our comprehensive benchmarks demonstrate that SHUTTLE offers an outstanding balance of compactness and high performance. At the 128-bit security level, SHUTTLE-128 achieves a combined public key and signature size of just 2447 bytes, representing a 34.4% size reduction compared to the NIST standard ML-DSA-44 (3732 bytes), with its standalone signature ($\sigma = 1183$ bytes) fitting comfortably within the standard 1500-byte Ethernet MTU. On the performance front, our AVX2-optimized signing implementation runs in only 614k cycles, which is 1.9× faster than the state-of-the-art bimodal lattice signature HAETAE-2 (1163k cycles). Crucially, by eliminating the data-dependent rejection loop, SHUTTLE completely eradicates the long-tail latency bottleneck: its worst-case (p99) signing latency is over 11× faster than that of HAETAE-2 (638k vs. 7074k cycles). These results confirm that SHUTTLE delivers a highly secure, hardware-friendly, and worst-case optimal signature solution perfectly suited for both high-throughput servers and resource-constrained, real-time embedded platforms.

Contents

1 Introduction **4**
1.1 Main Features and Advantages 4

2 Algorithm Description **6**
2.1 Preliminaries and Notation 6
2.1.1 Basic Notation 6
2.1.2 Rings, Vectors, and Splittings 7
2.1.3 Modular Arithmetic 7
2.1.4 Number Theoretic Transform (NTT) 8
2.1.5 Rényi Divergence 8
2.2 Algorithm Overview 9
2.3 Data Encoding and Hash Functions 9
2.3.1 Base Bit Packing and Unpacking 9
2.3.2 Extendable-Output Function Interface 11
2.3.3 Message Hashing and Challenge Generation 13
2.4 Scheme Specification 14
2.4.1 Key Generation 14
2.4.2 Signing 16
2.4.3 Verifying 18
2.5 Auxiliary Functions 18
2.5.1 Number Theoretic Transform 18
2.5.2 Conversion and Seed Expansion Primitives 20
2.5.3 Encodings and Decodings 22
2.5.4 Pseudorandom Sampling 26
2.5.5 Compression and Hint Mechanism 29
2.5.6 RejectSampling 31
2.5.7 Sample in Ball 33
2.5.8 Uniform Sampling and Logarithm Approximation 34
2.5.9 Gaussian Sampler Specification 38
2.5.10 Entropy Coding via Range Asymmetric Numeral Systems (rANS) 44
2.6 Correctness 50
2.7 Parameter Sets 51

3 Design Rationale **53**
3.1 Underlying Hard Problems 53
3.2 Architectural Evolution and Core Innovations 53
3.3 Parameter Selection Strategy 56
3.4 Rejection Sampling on the Key 57
3.5 Signing Failure and Restart Probability Analysis 57

4 Security Statement and Analysis **58**
4.1 Distributional Deviation of Approximate Iterative Rejection Sampling 58
4.1.1 Intrinsic Error of the Ideal Transition 58
4.1.2 Implementation Errors 61
4.1.3 Multi-Step Composition and Rényi Divergence Bound 69
4.1.4 Concrete Rényi-Divergence Estimate 71
4.2 Theoretical Security 75
4.2.1 Security Models 75
4.2.2 UF-CMA and sUF-CMA Security 76
4.2.3 Security and Correctness of the Canonical Identification Scheme 85

4.2.4 UF-NMA Security . . . 92
4.2.5 Reduction from BimodalSelfTargetMSIS to MSIS . . . 94
4.3 Concrete Security Estimates . . . 94
4.3.1 Cost of MLWE . . . 95
4.3.2 Cost of MSIS . . . 95
4.3.3 Cost of Self-Target-MSIS . . . 95
4.4 Side-Channel Resistance and Implementation Security . . . 96
4.4.1 Timing attacks. . . . 96
4.4.2 Power/EM and other implementation analysis. . . . 101
4.5 Other Security Properties . . . 101

5 Performance Evaluation and Implementation Analysis 101
5.1 Performance on x86 architecture. . . . 102
5.1.1 Benchmarking Results . . . 102
5.1.2 Implementation Analysis . . . 102
5.2 Performance on ARM Cortex-M4 . . . 102
5.2.1 Benchmark Results . . . 102
5.3 Transmission and Storage Overhead . . . 105

6 Features Statement 105
6.1 Innovation . . . 105
6.2 Compatibility and Real-World Applicability . . . 106
6.3 Flexibility and Scalability . . . 106

1 Introduction

This document provides the complete specification, security analysis, and performance evaluation for SHUTTLE, a lattice-based digital signature scheme submitted to the New Generation Commercial Cryptography (NGCC) algorithm call. SHUTTLE is designed to achieve an optimal balance between high computational efficiency, extremely compact transmission sizes, and robust post-quantum security.

At its core, SHUTTLE builds upon the Fiat-Shamir with Aborts paradigm [32, 33] and inherits the compact, iterative framework of Gärtner [22] to achieve remarkably small signature sizes. However, SHUTTLE introduces a fundamental paradigm shift by proposing a novel *rejection-free iterative sampling* mechanism. This "two-birds-with-one-stone" construction simultaneously resolves two of the most critical vulnerabilities in lattice-based signatures: (1) it removes the secret-dependent abort branch, thereby eliminating the primary timing- and cache-observable leakage channel associated with rejection decisions, and (2) it eradicates the notorious "long-tail" latency bottleneck, delivering a perfectly deterministic and isochronous execution profile essential for real-time systems.

By combining this rejection-free design with a tailored asymmetric structural compression mechanism under standard MLWE and MSIS assumptions, SHUTTLE achieves unprecedented compactness without relying on non-standard algebraic assumptions. At security Level 1, SHUTTLE requires only 1183 bytes for a standalone signature σ—a *51.1% reduction* compared to ML-DSA-44 (2420 bytes)—approaching the compactness of the floating-point-based Falcon within 30% while retaining pure, robust integer-only arithmetic. Crucially, this extreme compactness does not come at the cost of speed; thanks to our logarithmic-domain optimizations, SHUTTLE maintains highly competitive signing and verification performance, outperforming state-of-the-art bimodal schemes like HAETAE [14] by up to 1.9× in AVX2. This makes SHUTTLE exceptionally well-suited for high-throughput, bandwidth-constrained, and real-time network protocols such as IPsec [28] and TLS 1.3 [42].

1.1 Main Features and Advantages

The distinct advantages of SHUTTLE stem from three primary theoretical and engineering innovations, directly addressing the core limitations of existing NIST standards.

Standardized Security with Structural Compression (The Best of Both Worlds). Lattice-based signature standards currently present a polarizing trade-off: ML-DSA (Dilithium) [19, 38] is simple and relies on standard Module-LWE (MLWE) [30] and Module-SIS (MSIS) [30] assumptions, but suffers from relatively large public keys and signatures that complicate network integration. Conversely, Falcon [41] is extremely compact but relies on complex, high-precision floating-point arithmetic and Fast Fourier Transforms (FFT) for its Gaussian sampling, making secure, constant-time implementation on embedded hardware notoriously difficult.

SHUTTLE breaks this dichotomy. By strictly basing its security on the well-studied MLWE and MSIS problems, it inherits the conservative security foundation of ML-DSA. To eliminate the size penalty of the MLWE secret key structure—specifically, the extra components that must be masked during rejection sampling—we introduce a tailored asymmetric stretch-and-compress mechanism. This technique aggressively compresses the ℓ_2-norm of the response vector z.

Consequently, SHUTTLE achieves a dramatic reduction in communication overhead. At security Level 1, the combined public key and signature size of SHUTTLE is only 2447 bytes, representing an approximate 34.4% savings compared to ML-DSA-44 (3732 bytes). More importantly, in practical deployment scenarios where the public key is pre-distributed, cached, or transmitted out-of-band, the bandwidth advantage of SHUTTLE becomes even more pronounced. When comparing the standalone signature size $|\sigma|$ alone, SHUTTLE requires only 1183

bytes, which is a *51.1% reduction* compared to ML-DSA-44 (2420 bytes). This brings the signature size of our pure integer-based scheme remarkably close to that of the floating-point-based Falcon-512 (666 bytes), making SHUTTLE exceptionally well-suited for bandwidth-constrained network protocols and resource-limited IoT applications.

Elimination of the "Long-Tail" Rejection Bottleneck for Real-Time Systems. A major, often overlooked drawback of conventional Fiat-Shamir with aborts schemes (such as ML-DSA [38] and HAETAE [14]) is their inherent rejection probability during signing. Because the signer must abort and restart the entire process if the response leaks secret information, the signing time exhibits a high-variance, "long-tail" distribution [15]. In many critical, real-time industrial applications—such as automotive CAN bus authentication, industrial control loops, high-frequency trading, and embedded IoT devices—cryptographic operations must fit within strict, deterministic *fixed time-windows* [25, 44]. A sudden spike in signing latency due to multiple rejection aborts can disrupt the scheduling of time-triggered networks and cause critical system failures [46].

SHUTTLE completely resolves this bottleneck through its rejection-free iterative sampling mechanism, delivering an exceptionally stable and flat execution profile. As demonstrated by our extensive benchmarks over 10,000 runs (Table 15), traditional schemes suffer from extreme tail latency: ML-DSA-44 exhibits a 99th-percentile (p99) to median (p50) signing time ratio of approximately 3.3× to 4.4× (jumping from 186k to 758k cycles in AVX2), while HAETAE2 suffers from an even more severe 6.1× tail spike (jumping from 1163k to 7074k cycles). In stark contrast, the p99 signing latency of SHUTTLE-128 remains virtually identical to its median latency, maintaining a ratio of remarkably 1.01× *to* 1.05× across all implementation backends (e.g., 614k vs. 638k cycles for AVX2). This near-perfect isochronism eliminates the long-tail latency risk entirely, guaranteeing worst-case optimal performance and making SHUTTLE uniquely qualified for mission-critical, real-time deployments.

SHUTTLE resolves this by employing a novel rejection-free iterative sampling mechanism. Instead of strictly enforcing that each intermediate step yields an identical target distribution (which necessitates aborts), we relax this constraint and instead bound the accumulated distributional drift via Rényi divergence [40, 22]. This theoretical relaxation guarantees a *zero rejection rate* during the internal iterations of the signing loop. By eliminating data-dependent aborts, SHUTTLE completely eradicates the long-tail latency bottleneck, delivering a regular, predictable, and constant-time execution profile that is highly isochronous [25] and perfectly compatible with real-time system constraints.

Logarithmic Optimization for Streamlined Integer Arithmetic. While iterative rejection sampling [22] offers extreme compactness, evaluating the transition probabilities in prior frameworks required computing infinite Gaussian sums and heavy transcendental functions (such as exponentials), representing a massive computational bottleneck. SHUTTLE overcomes this by mapping the randomized decision logic into the logarithmic domain. By leveraging the monotonicity of logarithmic functions and splitting complex probability checks into simple, nested integer interval comparisons, we reduce the computational overhead from approximately 100 transcendental operations per iteration to just a single logarithmic evaluation on public randomness. Crucially, the secret key material is completely decoupled from any non-linear arithmetic.

This mathematical streamlining translates directly into outstanding software performance, as evidenced by the benchmarking results in Table 15. In portable reference implementations ('ref'), our SHUTTLE-128 signing requires only 2083k cycles, outperforming HAETAE2 (3065k cycles) by 1.47×. When vectorized, the efficiency gap widens significantly: under AVX2, SHUTTLE-128 signing takes just 614k cycles, achieving a 1.90× *speedup* over HAETAE2 (1163k cycles). At the 256-bit security level, SHUTTLE-256 ('avx2') signs in 1058k cycles, which is 1.71× *faster*

than HAETAE5 (1804k cycles). Furthermore, by leveraging AVX-512 vector extensions, SHUTTLE achieves a massive performance leap: SHUTTLE-128 and SHUTTLE-256 signing times drop to a mere *444k and 765k cycles* respectively, representing a 2.62× *and* 2.36× *speedup* compared to HAETAE's state-of-the-art AVX2 implementations. This seamlessly combines embedded-friendly integer arithmetic with worst-case optimal, high-speed performance.

2 Algorithm Description

2.1 Preliminaries and Notation

This section defines the mathematical spaces, rings, basic notations, and fundamental algebraic transformations used throughout the specification of SHUTTLE.

2.1.1 Basic Notation

$\mathbb{N}$	the set of nonnegative integers
$\mathbb{Z}$	the ring of integers
λ	the target security level $\lambda \in \{128, 256, 512\}$ (Table 10); the same parameter governs seed/hash/digest byte-lengths (e.g. $\frac{\lambda}{8} = \lambda/8$), Rényi budgets in Section 4, and security-game inputs 1^λ
$\|\mathbf{x}\|_2$	the Euclidean norm of vector $\mathbf{x}$
$\langle \mathbf{x_1}, \mathbf{x_2} \rangle$	the inner product of vectors $\mathbf{x_1}$ and $\mathbf{x_2}$
$\Pr[\mathcal{E}]$	the probability that event $\mathcal{E}$ occurs
$\mathbf{s}_1$	the second component of the secret key, a vector of l ring elements
$\mathbf{s}_2$	the third component of the secret key, a vector of m ring elements
$\mathbf{A_{gen}}$	the public matrix of $m \times l$ ring elements generated from $seed_A$
$\hat{\mathbf{A}}_{gen}$	the matrix $\mathbf{A_{gen}}$ stored in NTT representation
$\mathbf{a_{gen}}$	the public offset vector of m ring elements generated from $seed_A$
$\mathbf{b}$	the rounded public vector $\mathbf{b} = \mathsf{RoundB}(\mathbf{b}_0, \alpha_b)$, where $\mathbf{b}_0 = \mathbf{a_{gen}} + \mathbf{A_{gen}} \cdot \mathbf{s}_1 + \mathbf{s}_2 \pmod q$
$\mathbf{j}$	the first unit vector $[1, 0, \ldots, 0]^T$, with dimension implicit from the context
$\mathbf{A}$	the derived public matrix $\mathbf{A} = [\, 2(\mathbf{a_{gen}} - \mathbf{b}) + q\mathbf{j} \mid 2\mathbf{A_{gen}} \mid 2\mathbf{I}_m \,]$
$seed_A, seed_{sk}, seed_c$	seeds to generate $\mathbf{A}$, s, and c respectively
$K, seed_y$	master seed and ephemeral seed to generate the masking vector $\mathbf{y}$
$\mathbf{y}$	the ephemeral masking vector in R^{l+m+1}
$\mathbf{z}$	the response vector, typically split into $(\mathbf{z}_1, \mathbf{z}_2)$
$\mathbf{h}$	the boolean verification hint vector
$\circ$	point-wise (Hadamard) multiplication of polynomials or vectors in the NTT domain; also matrix–vector/matrix products evaluated in the NTT domain (Section 2.1.4)
μ	message digest used for challenge generation
$\mathbf{w}$	commitment vector computed from the ephemeral vector $\mathbf{y}$
c	challenge polynomial with fixed Hamming weight τ
σ	the generated signature

2.1.2 Rings, Vectors, and Splittings

We work over the polynomial ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$, where n is a power of 2 and q is a prime. Vectors over R_q are denoted by bold lowercase letters (e.g., $\mathbf{v}$), and matrices by bold uppercase letters (e.g., $\mathbf{A}$).

In our algorithms, we frequently concatenate or split vectors of polynomials. Let $\mathbf{v} \in R_q^m$. When we write $(\mathbf{v}_1, \mathbf{v}_2) \leftarrow \mathbf{v}$, where $\mathbf{v}_1$ has dimension m_1 and $\mathbf{v}_2$ has dimension m_2 (with $m_1 + m_2 = m$), it means $\mathbf{v}_1$ consists of the first m_1 polynomials of $\mathbf{v}$, and $\mathbf{v}_2$ consists of the remaining m_2 polynomials.

2.1.3 Modular Arithmetic

To support our compression techniques, we define operations to extract the high-order and low-order bits of elements in $\mathbb{Z}_q$. Let α_h be a chosen compression parameter that divides $2(q-1)$, and write

$$H_h := \frac{2(q-1)}{\alpha_h}$$

for the corresponding number of high-order buckets. All high-bit reductions performed by MakeHint and UseHint are taken modulo H_h.

For any element $x \in \mathbb{Z}_q$, its central remainder $x \bmod^{\pm} \alpha$ is the unique integer $r_0 \equiv x \pmod{\alpha}$ lying in $[-\alpha/2, \alpha/2)$ when α is even, or in $[-(\alpha-1)/2, (\alpha-1)/2]$ when α is odd. For any integer x, the least significant bit is

$$\mathsf{LSB}(x) := (x \bmod 2q) \bmod 2 \in \{0, 1\},$$

applied coefficient-wise to polynomials in R.

Unless it is written as $\bmod^{\pm}$, a reduction $\bmod q$ (respectively $\bmod 2q$) returns the *non-negative* representative in $\{0, \ldots, q-1\}$ (respectively $\{0, \ldots, 2q-1\}$); the centered representative is used only where $\bmod^{\pm}$ is written explicitly. In particular, the forward/inverse NTT outputs, the raw public vector $\mathbf{b}_0$, and the commitments computed in Sign and UseHint all use the non-negative representative, which is precisely the input convention required by LiftToModTwoQ below.

Since q is odd, the map $x \mapsto (x \bmod q,\ x \bmod 2)$ is a bijection from $\mathbb{Z}_{2q}$ onto $\mathbb{Z}_q \times \mathbb{Z}_2$. We write $\mathsf{LiftToModTwoQ}(\bar{x}, b)$ for its inverse on a single coordinate: given $\bar{x} \in \mathbb{Z}_q$ and a parity bit $b \in \{0, 1\}$, it returns the unique integer $x \in \{0, \ldots, 2q-1\}$ satisfying

$$x \equiv \bar{x} \pmod{q}, \qquad x \equiv b \pmod{2}.$$

Concretely, let $\bar{x}$ be represented in $\{0, \ldots, q-1\}$. If $\bar{x} \bmod 2 = b$, then $\mathsf{LiftToModTwoQ}(\bar{x}, b) = \bar{x}$; otherwise $\mathsf{LiftToModTwoQ}(\bar{x}, b) = \bar{x} + q$.

Implementation note (parity source). At the call sites $\mathbf{w} \leftarrow \mathsf{LiftToModTwoQ}(\overline{\mathbf{w}}, \mathsf{LSB}(\mathbf{y}_0')\,\mathbf{j})$ in Sign and $\widetilde{\mathbf{w}} \leftarrow \mathsf{LiftToModTwoQ}(\overline{\mathbf{w}}, \mathbf{w}_0')$ in UseHint, the parity bit b must be the low bit of the *original signed* coefficient, $b = x \bmod 2 = (x \bmod 2q) \bmod 2$ (equivalently $x \,\&\, 1$), which is exactly $\mathsf{LSB}(\cdot)$ as defined above. It must *not* be taken from the mod-q representative as $(x \bmod q) \,\&\, 1$: since q is odd, reducing mod q adds an odd q to negative coefficients and flips their parity, and the inputs $\mathbf{y}_0', c$ are signed, so a freeze-then-&1 shortcut would corrupt the lift.

For a real number x, the rounding bracket $\lfloor x \rceil := \lfloor x + 1/2 \rfloor$ denotes rounding to the nearest integer with ties resolved *upward* (round-half-up). This convention is shift-invariant under integer translation, i.e. $\lfloor x + k \rceil = \lfloor x \rceil + k$ for $k \in \mathbb{Z}$, which is required for the commitment-reconstruction identity used by CompressY, MakeHint, and UseHint to hold for both odd and even divisors.

For polynomials and vectors, all of these functions are applied coefficient-wise.

2.1.4 Number Theoretic Transform (NTT)

To achieve high-performance implementations, polynomial multiplication in R_q is accelerated using the Number Theoretic Transform (NTT). We require $q \equiv 1 \pmod{2n}$, ensuring the existence of a primitive $2n$-th root of unity $\zeta \in \mathbb{Z}_q$.

Since $q \equiv 1 \pmod{2n}$, the polynomial $X^n + 1$ splits into n distinct linear factors over $\mathbb{Z}_q$, so all three parameter sets admit a *complete* NTT decomposition down to point evaluations. For a polynomial $a \in R_q$, the forward NTT evaluates a at the n primitive $2n$-th roots of unity. As is standard for the in-place Cooley–Tukey schedule of Algorithm 4, the output array $\hat{a} \in \mathbb{Z}_q^n$ is produced in bit-reversed order:

$$\hat{a} = \mathsf{NTT}(a) = \big(a(\zeta^{2\,\mathrm{brv}(0)+1}),\, a(\zeta^{2\,\mathrm{brv}(1)+1}),\, \ldots,\, a(\zeta^{2\,\mathrm{brv}(n-1)+1})\big) \bmod q,$$

where $\mathrm{brv}(\cdot)$ denotes the bit reversal of a $\log_2 n$-bit index. The inverse transform NTT^{-1} recovers a from $\hat{a}$; it is realized by the matching Gentleman–Sande schedule of Algorithm 5 followed by a final multiplication by $n^{-1} \bmod q$.

Both the root of unity ζ and the scaling factor n^{-1} are determined by the parameter pair (n, q) of Table 10. We fix ζ to be the *smallest* primitive $2n$-th root of unity modulo q (so that $\zeta^n \equiv -1 \pmod{q}$); concretely, for the three parameter sets SUF-128/256/512 this gives $\zeta = 98,\ 21,\ 3$ and the inverse-transform scaling factor $n^{-1} \bmod q = 15301,\ 61321,\ 59335$, respectively. The forward and inverse butterflies (line 10 of Algorithm 4 and line 10 of Algorithm 5) read their twiddle factors from a single precomputed table $\mathsf{zetas}[0\,..\,n-1]$ defined by

$$\mathsf{zetas}[k] \;=\; \zeta^{\mathrm{brv}(k)} \bmod q, \qquad k = 0, 1, \ldots, n-1,$$

where the entries are visited in increasing k by the forward transform and in decreasing k by the inverse transform. Entry $\mathsf{zetas}[0] = \zeta^0 = 1$ is never consumed by a butterfly, so the schedules only index $\mathsf{zetas}[1\,..\,n-1]$. A Montgomery-arithmetic implementation stores the table in Montgomery form, i.e. as $\zeta^{\mathrm{brv}(k)} \cdot 2^{16} \bmod q$.

The symbol $\circ$ denotes point-wise multiplication in the NTT domain: $\hat{c} = \hat{a} \circ \hat{b} \implies \hat{c}_i = \hat{a}_i \cdot \hat{b}_i \pmod{q}$. The standard polynomial multiplication is efficiently computed as $c = \mathsf{NTT}^{-1}(\mathsf{NTT}(a) \circ \mathsf{NTT}(b))$. In our scheme, the public matrix $\mathbf{A_{gen}}$ is generated and stored directly in the NTT domain (denoted as $\hat{\mathbf{A}}_{gen}$) to minimize computational overhead.

We further overload $\circ$ to matrix–vector and matrix–matrix products evaluated entirely in the NTT domain. When $\hat{\mathbf{M}}$ and $\hat{\mathbf{v}}$ are an NTT-domain matrix and vector of compatible dimensions, $\hat{\mathbf{M}} \circ \hat{\mathbf{v}}$ is the ordinary matrix–vector product in which every scalar-entry multiplication is the point-wise product above and the partial products are summed coefficient-wise modulo q; that is, $(\hat{\mathbf{M}} \circ \hat{\mathbf{v}})_i = \sum_j \hat{\mathbf{M}}_{i,j} \circ \hat{\mathbf{v}}_j$. Because the NTT is a ring isomorphism, this agrees with the schoolbook product after transforming back, $\mathsf{NTT}^{-1}(\hat{\mathbf{A}}_{gen} \circ \mathsf{NTT}(\mathbf{s}_1)) = \mathbf{A_{gen}} \cdot \mathbf{s}_1$; this is exactly how the matrix–vector products $\mathbf{A_{gen}} \cdot \mathbf{s}_1$ in KeyGen (Algorithm 1) and the analogous products in signing and verification are evaluated.

The in-place algorithms realizing the forward and inverse transforms (Algorithms 4 and 5) are deferred to the auxiliary functions in Section 2.5.1.

2.1.5 Rényi Divergence

We measure the closeness of two distributions by the *Rényi divergence*, which is the central tool of the implementation-error analysis and the security reduction of Section 4.

Definition 1 (Rényi divergence)**.** *Let P, Q be probability distributions over a common discrete (or measurable) space with* $\mathrm{Supp}(P) \subseteq \mathrm{Supp}(Q)$*. For a real order $a \in (1, \infty)$, the (multiplicative)*

Rényi divergence of order a is

$$R_a(P \,\|\, Q) := \left(\sum_{x \in \mathrm{Supp}(P)} \frac{P(x)^a}{Q(x)^{a-1}} \right)^{\frac{1}{a-1}} = \left(\mathbb{E}_{x \sim Q}\left[(P(x)/Q(x))^a \right] \right)^{\frac{1}{a-1}},$$

with the convention $0/0 = 0$. We write $\bar{D}_a(P \,\|\, Q) := \mathbb{E}_Q\left[(dP/dQ)^a\right] = R_a(P \,\|\, Q)^{\,a-1}$ for the associated unnormalized order-a divergence. The two boundary orders are recovered as limits: $R_1(P\|Q) := \exp\big(\mathrm{KL}(P\|Q)\big)$ as $a \to 1^+$ and $R_\infty(P\|Q) := \max_{x \in \mathrm{Supp}(P)} P(x)/Q(x)$ as $a \to \infty$.

We use the following standard properties throughout (see [40]); fix $a \in (1, \infty)$ and $P \ll Q$.

- *Non-negativity and monotonicity.* $R_a(P\|Q) \geq 1$, with equality iff $P = Q$, and $a \mapsto R_a(P\|Q)$ is non-decreasing.
- *Probability preservation.* For every event $E \subseteq \mathrm{Supp}(Q)$,

  $$P(E) \;\leq\; \big(R_a(P \,\|\, Q) \cdot Q(E)\big)^{\,\frac{a-1}{a}}.$$

  This is the inequality used to transport a success probability from Q to P, keeping the divergence *inside* the $(a-1)/a$ exponent.
- *Multiplicativity.* For independent components, $R_a\big(\prod_i P_i \,\|\, \prod_i Q_i\big) = \prod_i R_a(P_i \,\|\, Q_i)$; equivalently $\bar{D}_a$ tensorizes.
- *Data-processing inequality.* For any (possibly randomized) map f, $R_a(f(P) \,\|\, f(Q)) \leq R_a(P \,\|\, Q)$ (Lemma 6).

2.2 Algorithm Overview

Before presenting the full normative specification, we provide a highly simplified algebraic skeleton of SHUTTLEin Figure 1 to highlight its core design. This simplified view completely omits public-key rounding, signature compression, rANS entropy coding, and the hint-based compression mechanism (i.e., we assume the entire response vector $\mathbf{z}$ is transmitted without omitting $\mathbf{z}_2$).

The fundamental innovation lies in the transition from the traditional one-step rejection sampling to our rejection-free iterative sign-selection. To make this contrast explicit, we juxtapose the standard bimodal rejection step with our approximate transition step $\mathsf{T}_v(\mathbf{y})$ below.

2.3 Data Encoding and Hash Functions

To ensure interoperability and exact reproducibility of test vectors, we strictly define the base encoding primitives and the cryptographic hashing mechanisms used to process arbitrary-length messages. The specific serialization formats for the public key, secret key, and signature will be detailed alongside their respective algorithms in Section 2.

2.3.1 Base Bit Packing and Unpacking

The fundamental building blocks for serialization are the generic polynomial-to-byte packing and unpacking functions, assuming a little-endian byte- and bit-order convention. Both take the per-coefficient bit length d as an explicit second argument.

- $\mathsf{PolyToBytes}(w, d)$**:** Let $w \in R$ be a polynomial (equivalently an array of its n coefficients) with every coefficient $w_i \in [0, 2^d - 1]$, and let d be the number of bits used to represent each coefficient. The function writes each w_i as its d-bit binary expansion (least significant

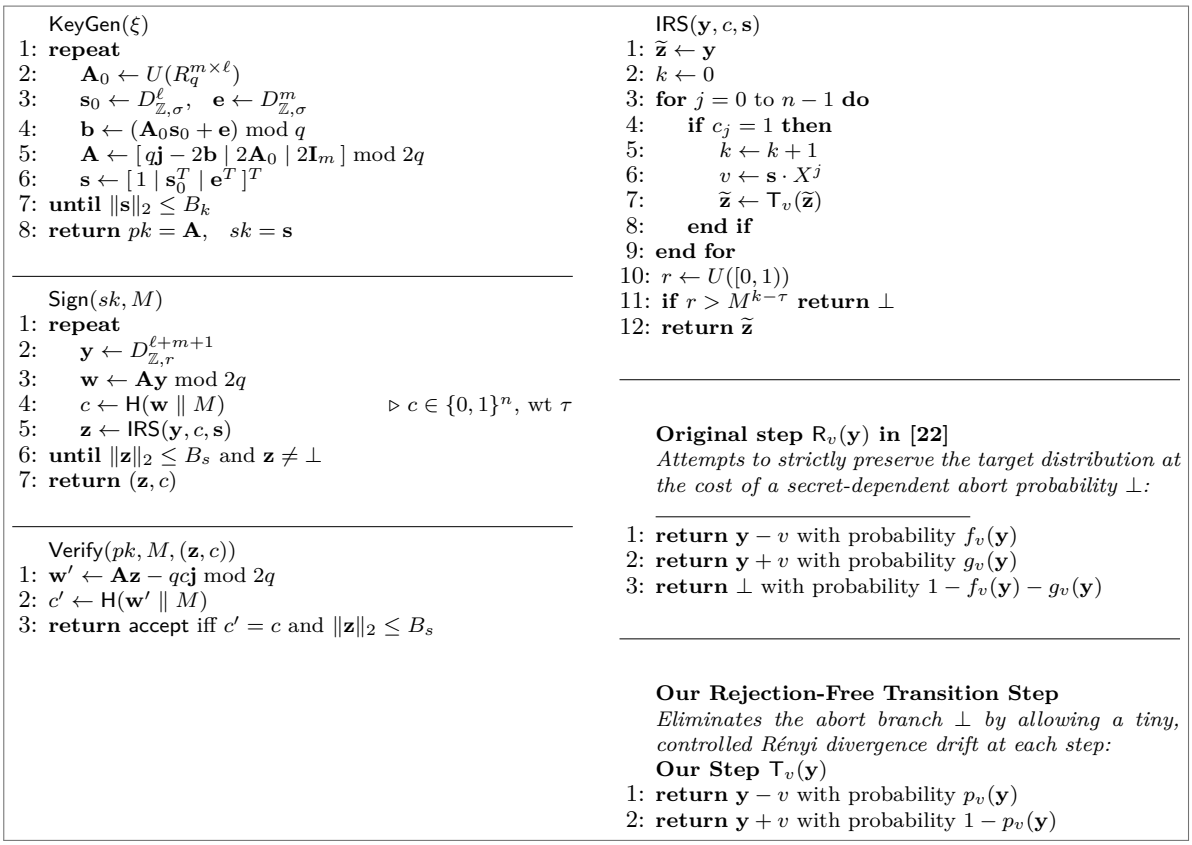


Figure 1: Simplified algebraic skeleton of SHUTTLE(left) and the comparison of the core sampling step (right). By replacing the traditional secret-dependent aborting step with our rejection-free transition $\mathsf{T}_v(\mathbf{y})$, the signing algorithm completely avoids intermediate restarts during the iterative coefficient-wise traversal of the challenge c.

bit first), concatenates the resulting $n \cdot d$ bits, and groups them into blocks of 8 bits to form a byte array. If $n \cdot d$ is not a multiple of 8, the final byte is padded with zero bits. The output is a byte string of length $\lceil nd/8 \rceil$.

- $\mathsf{BytesToPoly}(B, d)$**:** The inverse of PolyToBytes. Given a byte array B and the per-coefficient bit length d, it reads B as a sequence of bits (least significant bit first) and regroups every d consecutive bits into a coefficient in $[0, 2^d - 1]$, returning the polynomial $w \in R$ of n coefficients.

For a vector of polynomials, PolyToBytes is applied to each component in turn with the same bit length d and the resulting byte strings are concatenated; BytesToPoly inverts this component by component. The formal constant-time procedures realizing these two functions are given as Algorithms 8 and 9 in the Auxiliary Functions of Section 2.5.2.

2.3.2 Extendable-Output Function Interface

Several subroutines of SHUTTLE—the seed expansions in key generation and signing, the secret-vector and masking-vector samplers, and the challenge sampler—require a stream of pseudorandom bytes derived deterministically from a seed. We abstract this stream through an *extendable-output function* (XOF) exposing the incremental, sponge-style interface below. It mirrors the absorb/squeeze API of the SHAKE functions built on the Keccak sponge construction of FIPS 202 [37]: a seed is absorbed once, after which an arbitrary number of output bytes can be squeezed off a single deterministic stream.

- $ctx \leftarrow \mathsf{XOF.Init}(str)$: initialize a fresh XOF context ctx and absorb the byte string str into it.
- $(ctx, B) \leftarrow \mathsf{XOF.Squeeze}(ctx, \ell)$: extract ℓ output bytes B from the squeezing phase and update the context ctx, so that a subsequent XOF.Squeeze continues the same byte stream.

As with the unified hash H of the following subsection, this XOF is treated as a black-box primitive instantiated by the unified cryptographic API provided by the institute (via `API_PKC.zip`); the sponge interface above is the only structural property our algorithms rely on.

Regarding the security level required of the XOF, the subroutines split into two classes. ExpandA expands only *public* material—the matrix $\mathbf{A}_{\mathbf{gen}}$ and the vector $\mathbf{a}_{\mathbf{gen}}$ of the public key—so its XOF need only behave as a public random function strong enough to preclude structured public keys; for the analogous expansion ML-DSA instantiates the XOF with SHAKE128 [38]. All remaining XOF-driven subroutines—ExpandSeeds and ExpandSigningSeeds (Section 2.5.2), ExpandS, SampleY (hence the underlying SampleDGauss and BaseSampler), SampleC, and the iterative rejection sampler (RejectSample, R, SamplerU via domain tag 0x09)—process secret-key or signature-dependent material, so their XOF must provide pseudorandomness commensurate with the full signature security level; ML-DSA correspondingly uses SHAKE256 for the analogous routines.

Domain separation. Every XOF-driven subroutine prepends a distinct one-byte domain tag to its input before passing it to XOF.Init, so that no two subroutines ever evaluate the underlying sponge on the same string; domain separation is therefore enforced explicitly by the tags rather than implicitly by the assumption that inputs happen to differ. We also write $\mathsf{XOF}(S, \ell)$ for the one-shot call that initializes a fresh context on the (already tagged) byte string S and squeezes ℓ output bytes, i.e. $ctx \leftarrow \mathsf{XOF.Init}(S)$; $(ctx, B) \leftarrow \mathsf{XOF.Squeeze}(ctx, \ell)$; return B. The tag assignment is fixed as follows.

Tag	Subroutine
0x00	ExpandSeeds (key-generation seed expansion)
0x01	ExpandSigningSeeds (signing masking-seed expansion)
0x02	ExpandA (public matrix/vector expansion)
0x03	ExpandS (secret-vector expansion)
0x04	H_{ch} (challenge-seed hash)
0x05	H_{pk} (public-key hash)
0x06	$\mathsf{H}_{\mathsf{msg}}$ (message digest)
0x07	SampleC (in-ball challenge sampler)
0x08	SampleY (masking-vector sampler)
0x09	RejectSample / R / SamplerU (iterative rejection-sampling transition randomness)

Per-MODE instantiation. The unified XOF admits two concrete instantiations defining two distinct KAT sets; within each, the `ref`, AVX2, and AVX512 backends are byte-exact. Under the default **NGCC** mode the API collapses onto a single SM3 Hash-DRBG, so the public-strength and full-strength instantiations (the two security classes above) are byte-identical—the split is a placeholder for a future NGCC hash of 256-bit strength. Each XOF.Squeeze is then one DRBG *generate* that advances the state once regardless of the requested length; consequently the per-routine fixed squeeze-block sizes—392 bytes per ExpandS mini-batch, the initial sign-prefix block and $\gamma_Y = 676$-byte refill blocks of SampleY, and the analogous fixed blocks of the other samplers—are part of the byte schedule and are pinned identically across the three backends. Under **SHA-3** mode the two classes are genuinely distinct, SHAKE128 for public material (tag 0x02) and SHAKE256 for all secret/signature routines (tags 0x00/0x01/0x03–0x09); its rate-buffered squeeze makes the byte stream independent of the squeeze granularity.

Per-stream squeeze blocks (binding). The paragraph above has a sharp consequence that must be spelled out, because it determines the KAT: under NGCC mode the bytes produced by $\mathsf{XOF.Squeeze}(ctx, \ell)$ depend on how the requested length ℓ is split. An implementation that follows the per-coefficient and per-candidate squeezes of Algorithms 15 and 24 literally therefore does *not* reproduce the reference $\mathsf{pk}/\mathsf{sk}/\sigma$. The prescribed schedule opens each stream once and advances it by fixed-size *blocks* off the persisted context, slicing a block across its boundary where a logical unit straddles one. Writing L for the squeeze quantum ($\mathsf{L} = 128$ bytes under NGCC, $\mathsf{L} = 168$ under SHA-3) and $\lceil x \rceil_{\mathsf{L}} = \mathsf{L}\lceil x/\mathsf{L} \rceil$ for rounding up to a whole number of quanta, and using $p = 8$ for the byte over-read pad of the vectorised 64-bit window reads of SampleY, the blocks are listed below. For SampleY, β_Y is the initial block length; each subsequent refill is one $\gamma_Y = 21G + G/8 = 676$-byte candidate mini-batch.

Routine	Block	Definition	NGCC	SHA-3
ExpandA	β_A	$\lceil 2b_q \frac{mn(1+l)}{16} \rceil_{\mathsf{L}}$	768 / 1024 / 2048	840 / 1176 / 2184
ExpandS	β	$12G + G/4$	392	392
SampleY (initial)	β_Y	$\lceil w/8 \rceil + p + \gamma_Y$	698 / 708 / 732	698 / 708 / 732
SampleY (refill)	γ_Y	$21G + G/8$	676	676
SampleC	β_C	$\lceil 2\tau b_n \rceil_{\mathsf{L}}$	128 / 256 / 512	168 / 336 / 504
RejectSample	—	one squeeze of 18τ bytes	756 / 1044 / 2070	756 / 1044 / 2070

The first row is right-sized: each lane needs about $mn(1 + l)/16$ candidates, so $2b_q$ bytes per candidate with a factor two of slack covers the lane's mean demand; if a block is exhausted, the

stream simply continues in further blocks of the same size. The last row is not a per-transition block at all: RejectSample draws the complete 18τ-byte IRS stream in a *single* squeeze and slices it in ascending-j order. For SampleY, the initial block contains the sign bytes, then p discarded padding bytes, then the first γ_Y-byte mini-batch. Refills contain only candidate mini-batches; neither the sign prefix nor the padding is repeated. In NGCC mode the initial block must be obtained by one generate call of length β_Y, followed by one generate call of length γ_Y per refill. Splitting the initial call, or combining multiple refill calls, changes the random stream and is not permitted. Two remarks.

- Under SHA-3 a lane's own stream is rate-buffered and prefix-stable, so how it is split makes no difference; the N-way vectorised batch must nevertheless request each lane's $K_{\mathsf{b}}\beta$ bytes *contiguously*, since its blocks are whole-rate and drop sub-rate tails. The block *formulas* and the rounding rule are the same in both modes, with the mode's quantum L substituted.
- The AVX2/AVX-512 implementations may pre-fetch K_{b} blocks per lane N-way at a time ($K_{\mathsf{b}} = 6/9/16$ for ExpandS and $5/7/13$ for SampleY at levels 128/256/512). K_{b} is a pure performance knob: it changes neither the bytes a lane consumes nor any KAT, which is why it is not part of the byte schedule above. In particular, pre-fetching for the revised SampleY must preserve the generate-call boundaries just specified in NGCC mode; it may not replace several calls by a single longer call.

2.3.3 Message Hashing and Challenge Generation

In the Fiat–Shamir with aborts paradigm, the signature must cryptographically bind the message M, the public key pk, and the prover's commitment $\mathbf{w}$. All hashing is performed through the single extendable-output function XOF of Section 2.3.2, invoked with the disjoint one-byte domain tags fixed there. **In strict accordance with the NGCC submission guidelines**, XOF is treated as a black-box primitive instantiated by the unified cryptographic API provided by the institute (via `API_PKC.zip`); it can be replaced by the ultimately standardized NGCC primitive without any change to the signature logic.

We define three domain-separated hashes. All three are sized for collision resistance and produce $\frac{\lambda}{4} = \lambda/4$ bytes (i.e. 2λ bits) of output, giving generic (birthday) collision resistance up to 2^{λ} over a 2λ-bit digest:

$$\begin{aligned} \mathsf{H}_{\mathsf{pk}}(S) &:= \mathsf{XOF}(\texttt{0x05} \parallel S,\ \lambda/4), \\ \mathsf{H}_{\mathsf{msg}}(S) &:= \mathsf{XOF}(\texttt{0x06} \parallel S,\ \lambda/4), \\ \mathsf{H}_{\mathsf{ch}}(S) &:= \mathsf{XOF}(\texttt{0x04} \parallel S,\ \lambda/4). \end{aligned}$$

The public-key hash tr, the message digest μ, and the challenge seed $seed_c$ are then computed as

$$tr \leftarrow \mathsf{H}_{\mathsf{pk}}(pk), \qquad \mu \leftarrow \mathsf{H}_{\mathsf{msg}}(tr \parallel M), \qquad seed_c \leftarrow \mathsf{H}_{\mathsf{ch}}\big(\mathsf{EncodeCom}(\mathbf{w}_h, \mathbf{w}_0) \parallel \mu\big),$$

with $tr, \mu, seed_c \in \mathbb{B}^{2\lambda}$ (i.e. $\frac{\lambda}{4} = \lambda/4$ bytes each), where pk is the *already-encoded* public key (so no further pkEncode is applied), and EncodeCom is the canonical commitment encoding of Eq. (1). Here $\mathbf{w}_h$ is the high-order part of the commitment and $\mathbf{w}_0$ its least significant bit, reduced as specified by MakeHint/UseHint. The arbitrary-length message $M \in \{0,1\}^*$ enters only through $\mathsf{H}_{\mathsf{msg}}$; since tr has fixed length, μ may be precomputed for large or streamed messages.

Why H_{ch} is sized as a collision-resistant transcript hash. Although $seed_c$ is consumed only as the seed of SampleC, it is simultaneously the Fiat–Shamir hash that binds the transcript $(\mathbf{w}_h, \mathbf{w}_0, \mu)$ to the challenge; we therefore size it as a collision-resistant transcript hash—2λ bits,

matching the choice of the challenge seed $\widetilde{c}$ in ML-DSA/FIPS 204 [38]—rather than as a mere λ-bit seed. For plain EUF-CMA the directly exploitable event is a *second preimage* (finding $\mu_2 \neq \mu_1$ with $\mathsf{H}_{\mathsf{ch}}(\cdot \parallel \mu_2) = \mathsf{H}_{\mathsf{ch}}(\cdot \parallel \mu_1)$ for an already-signed commitment), which already costs 2^{λ} even at λ-bit output. The stronger guarantees that Shuttle targets—strong unforgeability (SUF-CMA), the BUFF-style message-binding and exclusive-ownership properties, and a clean Self-Target-MSIS reduction—instead incur an explicit transcript-*collision* term; only a 2λ-bit digest keeps this birthday term at $Q^2/2^{2\lambda} \leq 2^{-\lambda}$ for up to $Q \leq 2^{\lambda/2}$ signing queries. Since Shuttle transmits the challenge seed $seed_c$ verbatim (Section 2.5.10), this 2λ-bit digest is exactly the $\frac{\lambda}{4}$-byte seed field carried in each signature, so the larger transcript hash contributes its full length to the signature—the deliberate cost of binding the transcript at the 2λ-bit collision level.

The fixed-length seed $seed_c$ is subsequently used to deterministically sample the challenge polynomial c via the $\mathsf{SampleC}$ function. This hierarchical, domain-separated hashing is one ingredient of the strong-unforgeability argument of Section 2.5.10: any modification to the message, the public key, or the commitment yields an independent challenge seed, while the canonical signature encoding supplies the complementary injectivity property required for SUF-CMA.

2.4 Scheme Specification

2.4.1 Key Generation

The key generation algorithm KeyGen takes a seed ξ as input and outputs a public key and a secret key, both encoded as byte strings. The seed ξ is expanded as needed to produce a public seed $seed_A$, a secret seed $seed_{sk}$, and an auxiliary secret seed K.

Using $seed_A$, the public parameters $\mathbf{a_{gen}}$ and $\mathbf{A_{gen}}$ are pseudorandomly generated. Here, $\mathbf{a_{gen}}$ is a public vector and $\mathbf{A_{gen}}$ is a public matrix; in implementation, $\mathbf{A_{gen}}$ is typically stored or recomputed in NTT representation for efficiency in subsequent arithmetic. Using $seed_{sk}$, the secret vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ are pseudorandomly sampled from the bimodal distribution specified by our scheme.

The raw public vector is first computed as

$$\mathbf{b}_0 = \mathbf{a_{gen}} + \mathbf{A_{gen}} \cdot \mathbf{s}_1 + \mathbf{s}_2 \pmod{q}.$$

The public-key component is the rounded value

$$\mathbf{b} = \mathsf{RoundB}(\mathbf{b}_0, \alpha_b).$$

Let $\delta_{\mathbf{b}} = \mathbf{b} - \mathbf{b}_0$ be the rounding error, interpreted as its centered representative. In KeyGen, this rounding error is absorbed into the secret error vector by updating $\mathbf{s}_2' \leftarrow \mathbf{s}_2 + \delta_{\mathbf{b}}$. After this update, the symbol $\mathbf{s}_2'$ denotes the adjusted error vector. Therefore, the rounded public-key component satisfies

$$\mathbf{b} = \mathbf{a_{gen}} + \mathbf{A_{gen}} \cdot \mathbf{s}_1 + \mathbf{s}_2' \pmod{q}.$$

Indeed, before the update we have

$$\mathbf{b}_0 = \mathbf{a_{gen}} + \mathbf{A_{gen}} \cdot \mathbf{s}_1 + \mathbf{s}_2 \pmod{q},$$

and after replacing $\mathbf{s}_2$ by $\mathbf{s}_2 + \delta_{\mathbf{b}}$, we obtain

$$\mathbf{a_{gen}} + \mathbf{A_{gen}} \cdot \mathbf{s}_1 + \mathbf{s}_2' = \mathbf{b}_0 + \delta_{\mathbf{b}} \quad = \mathbf{b} \pmod{q}.$$

Accordingly, the public key consists of an encoding of $(seed_A, \mathbf{b})$, while the secret key consists of an encoding of the secret material needed for signing, including the adjusted secret vectors and the auxiliary seed K.

For verification purposes, the public quantities $(\mathbf{a_{gen}}, \mathbf{A_{gen}}, \mathbf{b})$ define the derived matrix

$$\mathbf{A} = [\, 2(\mathbf{a_{gen}} - \mathbf{b}) + q\mathbf{j} \mid 2\mathbf{A_{gen}} \mid 2\mathbf{I}_m \,].$$

We now derive the shifted public relation. Since, after absorbing the rounding error, we have

$$\mathbf{b} = \mathbf{a_{gen}} + \mathbf{A_{gen}} \cdot \mathbf{s}_1 + \mathbf{s}_2' \pmod q,$$

there exists an integer vector $\mathbf{z}$ such that

$$\mathbf{a_{gen}} + \mathbf{A_{gen}} \cdot \mathbf{s}_1 + \mathbf{s}_2' - \mathbf{b} = q\mathbf{z}.$$

Multiplying by 2 gives

$$2(\mathbf{a_{gen}} + \mathbf{A_{gen}} \cdot \mathbf{s}_1 + \mathbf{s}_2' - \mathbf{b}) = 2q\mathbf{z} \equiv \mathbf{0} \pmod{2q}.$$

By the definition of $\mathbf{A}$, we have

$$\begin{aligned}
\mathbf{A} \cdot [1, \mathbf{s}_1, \mathbf{s}_2']^T &= \big(2(\mathbf{a_{gen}} - \mathbf{b}) + q\mathbf{j}\big) \cdot 1 + 2\mathbf{A_{gen}} \cdot \mathbf{s}_1 + 2\mathbf{I}_m \cdot \mathbf{s}_2' \\
&= q\mathbf{j} + 2(\mathbf{a_{gen}} - \mathbf{b}) + 2\mathbf{A_{gen}} \cdot \mathbf{s}_1 + 2\mathbf{s}_2' \\
&= q\mathbf{j} + 2(\mathbf{a_{gen}} + \mathbf{A_{gen}} \cdot \mathbf{s}_1 + \mathbf{s}_2' - \mathbf{b}) \\
&\equiv q\mathbf{j} \pmod{2q}.
\end{aligned}$$

The same structure of $\mathbf{A}$ also fixes the parity of any product $\mathbf{Au}$, a fact used by both Sign and $\mathsf{UseHint}$ to lift a commitment computed modulo q to its representative modulo $2q$.

Implementation note (cached matrix). An implementation that evaluates the matrix–vector products of Sign and $\mathsf{UseHint}$ in the NTT domain caches $\mathbf{A}$ column-wise in NTT form. For the constant column, the factor 2 and the shift $q\mathbf{j}$ are applied during the modulo-$2q$ lift, so the cached NTT slot must hold $\mathsf{NTT}\big((\mathbf{b} - \mathbf{a_{gen}}) \bmod q\big)$ rather than $\mathsf{NTT}(\mathbf{b})$: the subtraction $\mathbf{b} - \mathbf{a_{gen}}$ is performed in the coefficient domain *before* the transform. With this convention the constant term contributes $(\mathbf{a_{gen}} - \mathbf{b}) \circ x_0$, which is consistent with the derived column $2(\mathbf{a_{gen}} - \mathbf{b}) + q\mathbf{j}$ above.

Lemma 1 (Commitment parity)**.** *Let q be odd. For any vector $\mathbf{u} = (u_0, \mathbf{u}_s, \mathbf{u}_e) \in R^{1+l+m}$,*

$$\mathbf{A}\,\mathbf{u} \equiv u_0\,\mathbf{j} \pmod 2,$$

so in particular the commitment $\mathbf{w} = \mathbf{Ay}'$ satisfies $\mathsf{LSB}(\mathbf{w}) = \mathsf{LSB}(\mathbf{y}_0')\,\mathbf{j}$, where $\mathbf{y}_0'$ is the first component of $\mathbf{y}'$.

Proof. By definition $\mathbf{A} = [\, 2(\mathbf{a_{gen}} - \mathbf{b}) + q\mathbf{j} \mid 2\mathbf{A_{gen}} \mid 2\mathbf{I}_m \,]$, so $\mathbf{Au} = (2(\mathbf{a_{gen}} - \mathbf{b}) + q\mathbf{j})u_0 + 2\mathbf{A_{gen}}\mathbf{u}_s + 2\mathbf{u}_e$. Reducing modulo 2, every block carrying an explicit factor 2 vanishes, leaving $\mathbf{Au} \equiv q\mathbf{j}\,u_0 \equiv u_0\mathbf{j} \pmod 2$ because q is odd. Applying this coefficient-wise to $\mathbf{u} = \mathbf{y}'$ gives $\mathsf{LSB}(\mathbf{w}) = \mathsf{LSB}(\mathbf{y}_0')\mathbf{j}$. □

After absorbing the rounding error, KeyGen performs a norm-window check on the stretched secret vector:

$$B_k' \le \|\mathsf{StretchS}(1, \mathbf{s}_1, \mathbf{s}_2')\|_2 \le B_k.$$

If this condition is not satisfied, the seed expansion is repeated with an incremented counter. This check serves two purposes.

First, the upper bound B_k ensures that every secret-dependent shift vector used later in the signing sampler has norm at most the value assumed in the distributional analysis. In particular, the concrete Rényi-divergence estimate uses the conservative lower bound

$$\beta_{\min} = \frac{r}{B_k},$$

and the Gaussian-input relative-error bound is valid uniformly for all keys passing the check. Without this upper-bound check, a rare key with a larger secret norm could lead to a smaller effective β and hence a larger sampling-distribution deviation than claimed in the concrete analysis.

Second, the lower bound B'_k excludes atypically small stretched secrets. Such keys could make the secret-dependent shift too small compared with the parameter window used by the iterative sampler, leading to a distributional profile that is outside the range covered by the fixed-parameter analysis. The lower-bound check therefore keeps all accepted keys in the same norm regime and makes the signing distribution depend only on the prescribed parameter window rather than on exceptional low-norm keys.

Since the seeds are expanded deterministically together with the counter κ, this rejection step does not introduce any secret-dependent timing during signing. It only affects key generation and ensures that all generated keys satisfy the norm conditions required by the signing algorithm and by the concrete distributional-error proof.

Algorithm 1 KeyGen(ξ)

Input: Seed $\xi \in \mathbb{B}^\lambda$
Output: Encoded public key pk and encoded secret key sk
$\kappa \leftarrow 0$
$\kappa \leftarrow \kappa + 1$
$(seed_A, seed_{sk}, K) \leftarrow \mathsf{ExpandSeeds}(\xi \| \mathsf{IntegerToBytes}(m, 2) \| \mathsf{IntegerToBytes}(\kappa, 4), \frac{5\lambda}{8})$ ▷ expand the input seed: $seed_A \in \{0,1\}^\lambda, seed_{sk} \in \{0,1\}^{2\lambda}, K \in \{0,1\}^{2\lambda}$
$(\mathbf{a_{gen}}, \hat{\mathbf{A}}_{gen}) \leftarrow \mathsf{ExpandA}(seed_A)$ ▷ generate $\hat{\mathbf{A}}_{gen}$ in NTT representation and $\mathbf{a_{gen}}$
$(\mathbf{s}_1, \mathbf{s}_2) \leftarrow \mathsf{ExpandS}(seed_{sk})$ ▷ generate $\mathbf{s}_1$ and $\mathbf{s}_2$ in coefficient representation
$\mathbf{b}_0 \leftarrow \mathbf{a_{gen}} + \mathsf{NTT}^{-1}(\hat{\mathbf{A}}_{gen} \circ \mathsf{NTT}(\mathbf{s}_1)) + \mathbf{s}_2 \pmod q$ ▷ compute the uncompressed value $\mathbf{b}_0$
$\mathbf{b} \leftarrow \mathsf{RoundB}(\mathbf{b}_0, \alpha_b)$ ▷ round $\mathbf{b}_0$ to obtain the compressed public-key component
$\delta_{\mathbf{b}} \leftarrow (\mathbf{b} - \mathbf{b}_0) \bmod^{\pm} q$ ▷ rounding error as its unique centered representative modulo q
$\mathbf{s}'_2 \leftarrow \mathbf{s}_2 + \delta_{\mathbf{b}}$ ▷ absorb the rounding error into the secret error
if $\|\mathsf{StretchS}(1, \mathbf{s}_1, \mathbf{s}'_2)\|_2 > B_k$ or $\|\mathsf{StretchS}(1, \mathbf{s}_1, \mathbf{s}'_2)\|_2 < B'_k$ **then**
    goto 4
end if
$pk \leftarrow \mathsf{pkEncode}(seed_A, \mathbf{b})$
$tr \leftarrow \mathsf{H}_{\mathsf{pk}}(pk)$ ▷ public-key hash with domain tag 0x05
$sk \leftarrow \mathsf{skEncode}(seed_A, \mathbf{b}, K, tr, \mathbf{s}_1, \mathbf{s}'_2)$
return (pk, sk)

2.4.2 Signing

The signing algorithm Sign takes as input an encoded secret key sk, a message M, and optional randomness rnd, and outputs a signature σ. The secret key is first decoded to recover the public seed $seed_A$, the public vector $\mathbf{b}$, the auxiliary secret seed K, and the secret vectors $\mathbf{s}_1$ and $\mathbf{s}_2$. Using $seed_A$, the public parameters are regenerated, and the derived public matrix $\hat{\mathbf{A}}$ is reconstructed as needed.

To ensure strong unforgeability and mitigate multi-target attacks, the message is hashed together with the hash of the public key to obtain the message digest μ. Then, together with the auxiliary secret seed, the optional randomness input, and a counter, μ is used to derive a seed from which the ephemeral masking vector $\mathbf{y}$ is sampled. From $\mathbf{y}$, the signer computes a public commitment $\mathbf{w}$, extracts the information required for challenge generation, and derives a challenge seed $seed_c$. The challenge c is then sampled from $seed_c$.

Next, the response vector $\mathbf{z}$ is computed from $\mathbf{y}$, the challenge c, and the secret key material through the internal rejection-free sampling procedure. The response is written as $\mathbf{z} = (\mathbf{z}_1, \mathbf{z}_2)$,

and auxiliary information $\mathbf{h}$ is generated from $\mathbf{z}_2$ and $\mathbf{w}$ to support verification. The signer then explicitly reconstructs the verifier-side value $\mathbf{z}_2'$ from $\mathbf{h}$, $\mathbf{z}_1$, and the low-order commitment, and repeats the process unless the reconstructed response $(\mathbf{z}_1, \mathbf{z}_2')$ satisfies the verification bound B_v. The final signature contains the canonical encoding of $(seed_c, \mathbf{z}_1, \mathbf{h})$. The challenge seed $seed_c$ is transmitted verbatim; seed aliases are precluded not by the encoding but by the verifier's check $seed_c' = seed_c$ against the seed recomputed from the reconstructed commitment, which pins $seed_c$ to a unique value as required for strong unforgeability. If encoding fails, the signer starts a fresh iteration.

Algorithm 2 $\mathrm{Sign}(sk, M, rnd)$

Input: Encoded secret key sk, message M, optional randomness rnd
Output: Signature σ
$(seed_A, \mathbf{b}, K, tr, \mathbf{s}_1, \mathbf{s}_2') \leftarrow \mathsf{skDecode}(sk)$ ▷ decode the secret key
$s \leftarrow [1, \mathbf{s}_1, \mathbf{s}_2']^T$ ▷ form the complete secret vector used in rejection sampling
$(\mathbf{a_{gen}}, \hat{\mathbf{A}}_{gen}) \leftarrow \mathsf{ExpandA}(seed_A)$ ▷ regenerate $\mathbf{a_{gen}}$ and $\mathbf{A_{gen}}$ from $seed_A$, with $\mathbf{A_{gen}}$ in NTT representation as $\hat{\mathbf{A}}_{gen}$
$\hat{\mathbf{A}} \leftarrow [\,\mathsf{NTT}(2(\mathbf{a_{gen}} - \mathbf{b}) + q\mathbf{j}) \mid 2\hat{\mathbf{A}}_{gen} \mid 2\hat{\mathbf{I}}_m\,]$ ▷ reconstruct the derived public matrix in NTT representation
$\mu \leftarrow \mathsf{H_{msg}}(tr \parallel M)$ ▷ message digest bound to the public key (domain tag 0x06)
$\kappa \leftarrow 0$
while true do
  $seed_y \leftarrow \mathsf{ExpandSigningSeeds}(K \| rnd \| \mu \| \mathsf{IntegerToBytes}(\kappa, 4), \frac{\lambda}{8})$
  $\kappa \leftarrow \kappa + 1$
  $\mathbf{y} \leftarrow \mathsf{SampleY}(seed_y)$ ▷ sample the ephemeral masking vector
  $\mathbf{y}' \leftarrow \mathsf{CompressY}(y)$
  $\overline{\mathbf{w}} \leftarrow \mathsf{NTT}^{-1}(\hat{\mathbf{A}} \cdot \mathsf{NTT}(\mathbf{y}')) \bmod q$
  $\mathbf{w} \leftarrow \mathsf{LiftToModTwoQ}(\overline{\mathbf{w}}, \mathsf{LSB}(\mathbf{y}_0')\,\mathbf{j})$ ▷ lift to $[0, 2q)$ using the known low bit (Lemma 1)
  $\mathbf{w}_0 \leftarrow \mathsf{LSB}(\mathbf{w})$
  $\mathbf{w}_h \leftarrow \lfloor \mathbf{w}/\alpha_h \rceil \bmod H_h$
  $seed_c \leftarrow \mathsf{H_{ch}}(\mathsf{EncodeCom}(\mathbf{w}_h, \mathbf{w}_0) \| \mu)$ ▷ derive the challenge seed (domain tag 0x04)
  $c \leftarrow \mathsf{SampleC}(seed_c)$ ▷ sample the challenge with τ entries set to $+1$ and the rest to 0
  $\widetilde{s} \leftarrow \mathsf{StretchS}(s)$ ▷ stretch the full secret vector
  $ctx_{\mathsf{R}} \leftarrow \mathsf{XOF.Init}(\texttt{0x09} \parallel seed_y)$
  $(ctx_{\mathsf{R}}, \widetilde{\mathbf{z}}) \leftarrow \mathsf{RejectSample}(ctx_{\mathsf{R}}, \mathbf{y}, c, \widetilde{s})$ ▷ iterative rejection sampling (Algorithm 23)
  $\mathbf{z} \leftarrow \mathsf{CompressY}(\widetilde{\mathbf{z}})$
  $(\mathbf{z}_1, \mathbf{z}_2) \leftarrow \mathbf{z}$ ▷ Decompose $\mathbf{z}$ into the first $l+1$ entries and the subsequent m entries.
  $\mathbf{h} \leftarrow \mathsf{MakeHint}(\mathbf{z}_2, \mathbf{w})$ ▷ generate auxiliary information for verification
  $\mathbf{z}_0 \leftarrow$ first polynomial of $\mathbf{z}_1$
  $\mathbf{w}_0' \leftarrow \mathsf{LSB}(\mathbf{z}_0 - c)\,\mathbf{j}$ ▷ reconstruct the low bit exactly as the verifier does
  $\widetilde{\mathbf{w}} \leftarrow \mathbf{w} - 2\mathbf{z}_2 \bmod 2q$
  $\mathbf{w}^{\mathrm{app}} \leftarrow \alpha_h\big(\mathbf{h} + \lfloor \widetilde{\mathbf{w}}/\alpha_h \rceil \bmod H_h\big) + \mathbf{w}_0'$ ▷ reassemble as UseHint does
  $\mathbf{z}_2' \leftarrow (\mathbf{w}^{\mathrm{app}} - \widetilde{\mathbf{w}})/2 \bmod^{\pm} q$
  if $\|(\mathbf{z}_1, \mathbf{z}_2')\|_2 > B_v$ **then**
    continue
  end if
  $\sigma \leftarrow \mathsf{sigEncode}(seed_c, \mathbf{z}_1, \mathbf{h})$
  if $\sigma \neq \perp$ **then**
    return σ
  end if
end while

2.4.3 Verifying

The verification algorithm Verify takes as input an encoded public key pk, a message M, and a signature σ, and outputs either accept or reject. The public key is first decoded to recover the public seed $seed_A$ and the public vector $\mathbf{b}$, while the signature is decoded to recover the challenge seed $seed_c$—from which the challenge $c = \mathsf{SampleC}(seed_c)$ is recomputed—together with the first component $\mathbf{z}_1$ of the compressed response and the auxiliary information $\mathbf{h}$.

Using $seed_A$, the public parameters are regenerated as needed. The verifier recomputes the message digest μ and reconstructs the low-bit part $\mathbf{w}'_0$ of the commitment from $\mathbf{z}_1$ and c. It then reconstructs the relevant derived public matrix and uses the auxiliary information $\mathbf{h}$ to recover the high-bit commitment information $\mathbf{w}_h$ together with the reconstructed second response component $\mathbf{z}'_2$.

Finally, the verifier recomputes the challenge seed $seed'_c$ from the recovered commitment information and accepts if and only if $seed'_c = seed_c$ and the reconstructed response vector $(\mathbf{z}_1, \mathbf{z}'_2)$ satisfies the verification bound B_v. The signer applies this same reconstructed-response bound before outputting a signature, so the value checked during verification is already guaranteed to be short for honestly generated signatures.

Algorithm 3 Verify(pk, M, σ)

Input: Encoded public key pk, message M, signature σ
Output: accept or reject
$(seed_A, \mathbf{b}) \leftarrow \mathsf{pkDecode}(pk)$ ▷ decode the public key
$tr \leftarrow \mathsf{H}_{\mathsf{pk}}(pk)$ ▷ public-key hash (domain tag 0x05)
$\mu \leftarrow \mathsf{H}_{\mathsf{msg}}(tr \parallel M)$ ▷ message digest (domain tag 0x06)
$(seed_c, \mathbf{z}_1, \mathbf{h}) \leftarrow \mathsf{sigDecode}(\sigma)$ ▷ decode the signature
$c \leftarrow \mathsf{SampleC}(seed_c)$ ▷ recompute the challenge from the transmitted seed
$(\mathbf{a_{gen}}, \hat{\mathbf{A}}_{gen}) \leftarrow \mathsf{ExpandA}(seed_A)$
$\mathbf{z}_0 \leftarrow$ the first polynomial component of $\mathbf{z}_1$
$\mathbf{w}'_0 \leftarrow \mathsf{LSB}(\mathbf{z}_0 - c)\,\mathbf{j}$ ▷ reconstruct the low-bit part of the commitment (coefficient-wise LSB)
$\hat{\mathbf{A}}_1 \leftarrow [\,\mathsf{NTT}(2(\mathbf{a_{gen}} - \mathbf{b}) + q\mathbf{j}) \mid 2\hat{\mathbf{A}}_{gen}]$
$(\mathbf{w}_h, \mathbf{z}'_2) \leftarrow \mathsf{UseHint}(\mathbf{h}, \mathbf{z}_1, \mathbf{w}'_0, \hat{\mathbf{A}}_1, c)$ ▷ recover high-bit commitment and second response component
$seed'_c \leftarrow \mathsf{H}_{\mathsf{ch}}(\mathsf{EncodeCom}(\mathbf{w}_h, \mathbf{w}'_0) \| \mu)$ ▷ recompute the challenge seed (domain tag 0x04)
if $seed_c = seed'_c$ and $\|(\mathbf{z}_1, \mathbf{z}'_2)\|_2 \le B_v$ **then**
    return accept
else
    return reject
end if

2.5 Auxiliary Functions

This section explicitly defines the auxiliary data conversion and serialization functions used in the main protocol algorithms. These definitions ensure strict interoperability and exact reproducibility of the Known Answer Tests (KATs).

2.5.1 Number Theoretic Transform

The forward and inverse transforms introduced in Section 2.1.4 are specified in Algorithms 4 and 5. Both operate in place on an integer array of size n and share the single twiddle table $\mathsf{zetas}[1\,..\,n-1]$. The forward NTT uses a Cooley–Tukey butterfly and walks the table upward ($k = 1, \ldots, n-1$); the inverse NTT uses the dual Gentleman–Sande butterfly, walks the table downward with negated twiddles, and finishes by scaling every coefficient by $n^{-1} \bmod q$. With

(n, q) from Table 10 and the resulting ζ of Section 2.1.4 fixed per parameter set, the same two algorithms instantiate the transform for $n \in \{256, 512, 1024\}$. All additions, subtractions, and products below are reduced modulo q.

Algorithm 4 $\mathsf{NTT}(w)$ — forward NTT (complete, Cooley–Tukey, bit-reversed output)

Input: Polynomial $w = \sum_{j=0}^{n-1} w_j X^j \in R_q$
Output: $\hat{w} = (\hat{w}[0], \ldots, \hat{w}[n-1]) \in \mathbb{Z}_q^n$ in NTT representation
for $j = 0$ to $n - 1$ **do**
    $\hat{w}[j] \leftarrow w_j$
end for
$k \leftarrow 0$
for $len = n/2, n/4, \ldots, 1$ **do**
    for $start = 0$ to $n - 1$ **step** $2 \cdot len$ **do**
        $k \leftarrow k + 1$
        $z \leftarrow \mathsf{zetas}[k]$ ▷ $z = \zeta^{\mathrm{brv}(k)} \bmod q$
        for $j = start$ to $start + len - 1$ **do**
            $t \leftarrow z \cdot \hat{w}[j + len] \bmod q$
            $\hat{w}[j + len] \leftarrow (\hat{w}[j] - t) \bmod q$
            $\hat{w}[j] \leftarrow (\hat{w}[j] + t) \bmod q$
        end for
    end for
end for
return $\hat{w}$

Algorithm 5 $\mathsf{NTT}^{-1}(\hat{w})$ — inverse NTT (Gentleman–Sande, with n^{-1} scaling)

Input: $\hat{w} = (\hat{w}[0], \ldots, \hat{w}[n-1]) \in \mathbb{Z}_q^n$
Output: Polynomial $w = \sum_{j=0}^{n-1} w_j X^j \in R_q$ with $\mathsf{NTT}(w) = \hat{w}$
for $j = 0$ to $n - 1$ **do**
    $w_j \leftarrow \hat{w}[j]$
end for
$k \leftarrow n$
for $len = 1, 2, \ldots, n/2$ **do**
    for $start = 0$ to $n - 1$ **step** $2 \cdot len$ **do**
        $k \leftarrow k - 1$
        $z \leftarrow -\mathsf{zetas}[k] \bmod q$ ▷ $z = -\zeta^{\mathrm{brv}(k)} \bmod q$
        for $j = start$ to $start + len - 1$ **do**
            $t \leftarrow w_j$
            $w_j \leftarrow (t + w_{j+len}) \bmod q$
            $w_{j+len} \leftarrow z \cdot (t - w_{j+len}) \bmod q$
        end for
    end for
end for
$f \leftarrow n^{-1} \bmod q$ ▷ mode-dependent; see Section 2.1.4
for $j = 0$ to $n - 1$ **do**
    $w_j \leftarrow f \cdot w_j \bmod q$
end for
return w

2.5.2 Conversion and Seed Expansion Primitives

To ensure exact interoperability, we define the fundamental integer/byte conversion primitives and the seed expansion functions used throughout SHUTTLE. Let $\mathbb{B} = \{0,1\}$ denote the set of bits. A byte string of length ℓ is an element of $(\mathbb{B}^8)^\ell$, equivalently $\mathbb{B}^{8\ell}$; we identify byte strings with bit strings whose length is a multiple of eight, and write $\mathbb{B}^*$ for the set of all such strings. Throughout, $\lambda \in \{128, 256, 512\}$ denotes the *target security level* (Table 10); the same λ fixes all seed, hash, and digest byte-lengths and enters the concrete Rényi bounds of Section 4. The expansion seeds $seed_A$, $seed_{sk}$, K, and the masking seed $seed_y$ each use the base unit $\frac{\lambda}{8} = \lambda/8$ bytes (with $seed_{sk}$ and K occupying two such units), whereas the collision-resistant transcript hashes tr, μ and the challenge seed $seed_c$ are double-length, $\frac{\lambda}{4} = \lambda/4$ bytes.

Integer Serialization. The conversion between non-negative integers and fixed-length byte strings is defined by the two mutually inverse primitives IntegerToBytes (Algorithm 6) and BytesToInteger (Algorithm 7), both using a little-endian byte order convention. IntegerToBytes(x, ℓ) serializes a non-negative integer x into a fixed ℓ-byte array (returning $\perp$ when $x \geq 2^{8\ell}$ does not fit); it is primarily used to serialize loop counters and dimension parameters so as to prevent parsing ambiguities in hash inputs. BytesToInteger(B, ℓ) reads an ℓ-byte string back into the integer $\sum_{i=0}^{\ell-1} B[i] \cdot 2^{8i} \in [0, 2^{8\ell})$; it is the exact inverse of IntegerToBytes on $[0, 2^{8\ell})$ and is used to assemble integers from freshly squeezed XOF output before the rejection or masking logic of the samplers is applied.

Algorithm 6 IntegerToBytes(x, ℓ)

Input: Non-negative integer $x \in \mathbb{N}$ and target byte length $\ell \in \mathbb{N}^+$.
Output: The little-endian ℓ-byte encoding $B \in \mathbb{B}^{8\ell}$ of x, or $\perp$.
if $x \geq 2^{8\ell}$ **then**
    return $\perp$ ▷ x does not fit in ℓ bytes
end if
$y \leftarrow x$
for $i = 0$ to $\ell - 1$ **do**
    $B[i] \leftarrow y \bmod 256$
    $y \leftarrow \lfloor y/256 \rfloor$
end for
return $B = (B[0], \ldots, B[\ell - 1])$

Algorithm 7 BytesToInteger(B, ℓ)

Input: An ℓ-byte string $B = (B[0], \ldots, B[\ell - 1]) \in \mathbb{B}^{8\ell}$ and byte length $\ell \in \mathbb{N}^+$.
Output: The non-negative integer $x \in [0, 2^{8\ell})$ encoded by B in little-endian order.
$x \leftarrow 0$
for $i = \ell - 1$ **downto** 0 **do**
    $x \leftarrow 256 \cdot x + B[i]$
end for
return x

Polynomial Bit Packing. The polynomial-level serialization primitives PolyToBytes and BytesToPoly introduced in Section 2.3 are realized by Algorithms 8 and 9. Both run in constant time for a fixed (n, d) and use the little-endian bit order of IntegerToBytes within each coefficient.

Algorithm 8 $\mathsf{PolyToBytes}(w, d)$

Input: Polynomial $w \in R$ with coefficients $w_i \in [0, 2^d - 1]$; per-coefficient bit length d.
Output: Byte string $B \in \mathbb{B}^{nd}$.
$z \leftarrow (\,)$ ▷ empty bit string
for $i = 0$ to $n - 1$ **do**
  for $j = 0$ to $d - 1$ **do**
    $z \leftarrow z \parallel \big(\lfloor w_i / 2^j \rfloor \bmod 2\big)$ ▷ bits of w_i, least significant first
  end for
end for
pad z on the right with zero bits until $|z|$ is a multiple of 8
return the byte string whose k-th byte equals $\sum_{t=0}^{7} z[8k + t]\, 2^t$

Algorithm 9 $\mathsf{BytesToPoly}(B, d)$

Input: Byte string $B \in \mathbb{B}^{nd}$; per-coefficient bit length d.
Output: Polynomial $w \in R$ with coefficients in $[0, 2^d - 1]$.
$z \leftarrow$ the bit string with $z[8k + t] = \lfloor B_k / 2^t \rfloor \bmod 2$ for $0 \le t < 8$ ▷ expand each byte, least significant bit first
for $i = 0$ to $n - 1$ **do**
  $w_i \leftarrow \sum_{j=0}^{d-1} z[i\, d + j]\, 2^j$
end for
return w

Commitment Encoding. The challenge seed binds the high-order commitment $\mathbf{w}_h$ together with its least significant bit $\mathbf{w}_0$. With

$$H_h = \frac{2(q-1)}{\alpha_h}, \qquad d_h = \lceil \log_2 H_h \rceil,$$

(the parameters of Table 10 are chosen so that H_h is a positive integer), the canonical commitment encoding is

$$\mathsf{EncodeCom}(\mathbf{w}_h, \mathbf{w}_0) := \mathsf{PolyToBytes}(\mathbf{w}_h, d_h) \parallel \mathsf{PolyToBytes}(\mathbf{w}_0, 1). \tag{1}$$

Every coefficient of $\mathbf{w}_h$ must lie in $[0, H_h)$ and every coefficient of $\mathbf{w}_0$ in $\{0, 1\}$; the coefficients are serialized in the polynomial-major, coefficient-major order of $\mathsf{PolyToBytes}$. Inputs violating these ranges are rejected as non-canonical. Decoding reverses this exactly: each d_h-bit field is read verbatim and *range-checked* into $[0, H_h)$, with the gap $[H_h, 2^{d_h})$ *rejected*—never folded modulo H_h (the moduli $H_h \in \{30, 120, 58\}$ are not powers of two, so a mod-H_h reduction would alias distinct codewords). The hint carried in a signature is decoded by the same range-check-then-reject rule. Rejecting rather than reducing makes both encodings injective, a property the strong-unforgeability (SUF-CMA) argument relies on.

KeyGen Seed Expansion. The deterministic seed expansion in KeyGen is governed by the function $\mathsf{ExpandSeeds}(S, \ell)$, which maps a seed string $S \in \mathbb{B}^*$ to an ℓ-byte output. This function is dedicated exclusively to expanding the initial entropy during key generation; it prepends the domain tag `0x00` to S, initializes a fresh XOF context on the result, and squeezes the requested ℓ output bytes:

$$\mathsf{ExpandSeeds}(S, \ell) := T,$$

where $ctx \leftarrow \mathsf{XOF.Init}(\texttt{0x00} \parallel S)$; and $(ctx, T) \leftarrow \mathsf{XOF.Squeeze}(ctx, \ell)$. During KeyGen (Algorithm 1), the joint seed material is derived by invoking:

$$T \leftarrow \mathsf{ExpandSeeds}\left(\xi \parallel \mathsf{IntegerToBytes}(m, 2) \parallel \mathsf{IntegerToBytes}(\kappa, 4), \frac{5\lambda}{8}\right),$$

where $T \in \mathbb{B}^{5\lambda}$ is partitioned into the public seed $seed_A$, the secret seed $seed_{sk}$, and the master seed K as:

$$\begin{aligned} seed_A &:= T[\,0 \;\ldots\; \lambda/8 - 1\,], & seed_A &\in \mathbb{B}^{\lambda}, \\ seed_{sk} &:= T[\,\lambda/8 \;\ldots\; 3\lambda/8 - 1\,], & seed_{sk} &\in \mathbb{B}^{2\lambda}, \\ K &:= T[\,3\lambda/8 \;\ldots\; 5\lambda/8 - 1\,]. & K &\in \mathbb{B}^{2\lambda}. \end{aligned}$$

The counter κ is local to KeyGen and is incremented upon rejection. Its inclusion in the XOF input guarantees that successive re-sampling attempts derived from the same initial seed ξ yield independent, statistically uncorrelated seed triples.

Signing Masking-Seed Expansion. The deterministic derivation of ephemeral randomness during signing is governed by the function $\mathsf{ExpandSigningSeeds}(S, \ell)$, which maps a seed string $S \in \mathbb{B}^*$ to an ℓ-byte output. This function is dedicated exclusively to deriving the masking seed $seed_y$; like ExpandSeeds, it prepends its domain tag `0x01` to S, initializes a fresh XOF context on the result, and squeezes the requested ℓ bytes:

$$\mathsf{ExpandSigningSeeds}(S, \ell) := seed_y,$$

where $ctx \leftarrow \mathsf{XOF.Init}(\texttt{0x01} \parallel S)$; and $(ctx, seed_y) \leftarrow \mathsf{XOF.Squeeze}(ctx, \ell)$. Although ExpandSeeds and ExpandSigningSeeds invoke the same underlying XOF, their distinct domain tags `0x00` and `0x01` place the two streams in separate domains, so their outputs are independent even on related inputs; this separation is enforced by the tags rather than by any assumption that the inputs differ. During Sign (Algorithm 2), the ephemeral masking seed $seed_y$ is generated at each iteration by computing:

$$seed_y \leftarrow \mathsf{ExpandSigningSeeds}\left(K \parallel rnd \parallel \mu \parallel \mathsf{IntegerToBytes}(\kappa, 4), \frac{\lambda}{8}\right).$$

The resulting $\frac{\lambda}{8}$-byte seed $seed_y \in \mathbb{B}^{\lambda}$ is subsequently consumed by SampleY to sample the masking vector $\mathbf{y}$. The four-byte little-endian encoding of the loop counter κ ensures that each rejection-loop iteration in the signing algorithm utilizes a fresh, independent masking vector, even when the master seed K, optional randomness rnd, and message digest μ remain static.

2.5.3 Encodings and Decodings

Building upon the base PolyToBytes and BytesToPoly functions defined in Section 2.3, we specify the exact byte layouts for the public key, secret key, and signature. Let $\parallel$ denote byte array concatenation.

Public Key Encoding (pkEncode / pkDecode): The public key consists of the $\frac{\lambda}{8}$-byte seed $seed_A$ and the rounded polynomial vector $\mathbf{b} = \mathsf{RoundB}(\mathbf{b}_0, \alpha_b)$. Since RoundB zeroes the low-order part modulo α_b, every coefficient of $\mathbf{b}$ is an *exact multiple of* α_b lying in $[0, q)$; packing it with the full $d_q = \lceil \log_2 q \rceil$ bits would store $\log_2 \alpha_b$ guaranteed-zero trailing bits per coefficient. We therefore transmit only the high part $\mathbf{b}/\alpha_b \in [0, \lceil q/\alpha_b \rceil)$, which needs

$$d_b \;:=\; \left\lceil \log_2 \lceil q/\alpha_b \rceil \right\rceil$$

bits per coefficient. For the three parameter sets this gives $d_b = 13, 15, 14$ (versus $d_q = 14, 16, 16$), i.e. a saving of $\log_2 \alpha_b \in \{1, 1, 2\}$ bits per coefficient, or $m\,n\,(d_q - d_b)/8 = 96, 128, 512$ bytes of public key for the three sets respectively. Hence

$$\mathsf{pkEncode}(seed_A, \mathbf{b}) := seed_A \parallel \mathsf{PolyToBytes}(\mathbf{b}_1[0], d_b) \parallel \cdots \parallel \mathsf{PolyToBytes}(\mathbf{b}_1[m-1], d_b),$$

where $\mathbf{b}_1 := \mathbf{b}/\alpha_b$ is the high part packed one polynomial at a time, and pkDecode reads the first $\frac{\lambda}{8}$ bytes as $seed_A$, splits the remaining bytes into m equal blocks, unpacks each block with $\mathsf{BytesToPoly}(\cdot, d_b)$, and rescales by α_b to recover $\mathbf{b}$ (the rescaling is lossless because $\mathbf{b}$ is a multiple of α_b).

The encoded public key consists of the $\frac{\lambda}{8} = \lambda/8$ seed bytes followed by the packed high part of $\mathbf{b}$, so its total size in bytes is

$$|pk| \;=\; \frac{\lambda}{8} + \left\lceil \frac{m\,n\,d_b}{8} \right\rceil \;=\; \frac{\lambda}{8} + \left\lceil \frac{m\,n\,d_b}{8} \right\rceil \quad \text{bytes.}$$

For the three parameter sets (λ, m, n, d_b) this evaluates to $16 + 1248 = 1264$, $32 + 1920 = 1952$, and $64 + 3584 = 3648$ bytes, respectively, matching the public-key sizes of Table 10.

Algorithm 10 $\mathsf{pkEncode}(seed_A, \mathbf{b})$

Input: Public seed $seed_A \in \mathbb{B}^\lambda$ and rounded public vector $\mathbf{b} \in R^m$ (every coefficient a multiple of α_b).
Output: Encoded public key pk.
$\mathbf{b}_1 \leftarrow \mathbf{b}/\alpha_b$ ▷ coefficient-wise; exact, since $\mathbf{b} \equiv 0 \pmod{\alpha_b}$ after RoundB
$pk \leftarrow seed_A$
for $i = 0$ to $m - 1$ **do**
    $pk \leftarrow pk \,\|\, \mathsf{PolyToBytes}(\mathbf{b}_1[i], d_b)$
end for
return pk

Algorithm 11 $\mathsf{pkDecode}(pk)$

Input: Encoded public key pk.
Output: $(seed_A, \mathbf{b})$.
$(seed_A, z_0, \ldots, z_{m-1}) \leftarrow pk$, where $seed_A \in \mathbb{B}^\lambda$ and each $z_i \in \mathbb{B}^{nd_b}$
for $i = 0$ to $m - 1$ **do**
    $\mathbf{b}_1[i] \leftarrow \mathsf{BytesToPoly}(z_i, d_b)$
end for
$\mathbf{b} \leftarrow \alpha_b \cdot \mathbf{b}_1$ ▷ lossless rescale
return $(seed_A, \mathbf{b})$

Secret Key Encoding (skEncode / skDecode): The secret key contains the public key components, the master seed K, the public-key hash tr, and the secret vectors $\mathbf{s}_1$ and the *adjusted* error vector $\mathbf{s}_2'$ (which, after the rounding-error absorption of KeyGen, is the vector actually used in signing). Caching tr in the secret key lets the signer bind the message to the public key without re-encoding and re-hashing pk at every invocation. The two secret vectors are packed with *separate* bit-widths, because they have different dimensions ($\mathbf{s}_1 \in R^l$, $\mathbf{s}_2' \in R^m$) and different coefficient ranges—in particular $\mathbf{s}_2' = \mathbf{s}_2 + \delta_{\mathbf{b}}$ absorbs the rounding error and is therefore wider than the raw $\mathbf{s}_2$. Let B_s^{enc} and B_e^{enc} be the per-coefficient encoding bounds, i.e. the smallest integers such that every coefficient of $\mathbf{s}_1$ lies in $[-B_s^{\mathsf{enc}}, B_s^{\mathsf{enc}}]$ and every coefficient of $\mathbf{s}_2'$ lies in $[-B_e^{\mathsf{enc}}, B_e^{\mathsf{enc}}]$ for all keys accepted by KeyGen (concretely, the σ_1 and σ_2 samplers hard-truncate their one-sided support at the table lengths L_σ of Tables 5–7, namely $L_{0.85} = 9$, $L_{0.9} = 10$, $L_1 = 11$, so after the sign flip every coefficient of $\mathbf{s}_1$ satisfies $|\mathbf{s}_1| \le L_{\sigma_1}$, i.e. $B_s^{\mathsf{enc}} = L_{\sigma_1}$; the absorbed rounding error $\delta_{\mathbf{b}} = \mathbf{b} - \mathbf{b}_0$ adds at most $\lceil \alpha_b/2 \rceil$ per coefficient, so $\mathbf{s}_2' = \mathbf{s}_2 + \delta_{\mathbf{b}}$ satisfies $B_e^{\mathsf{enc}} = L_{\sigma_2} + \lceil \alpha_b/2 \rceil$). Substituting the per-mode values of $(\sigma_1, \sigma_2, \alpha_b)$ from Table 10 gives $B_s^{\mathsf{enc}} = 9, 10, 10$ and $B_e^{\mathsf{enc}} = 10, 12, 12$ for SUF-128/256/512, which are also listed in Table 10. Set

$$d_s = \lceil \log_2(2B_s^{\mathsf{enc}} + 1) \rceil, \qquad d_e = \lceil \log_2(2B_e^{\mathsf{enc}} + 1) \rceil,$$

which evaluate to $d_s = d_e = 5$ for all three parameter sets (also listed in Table 10). Shift coefficients by the respective bound to make them non-negative:

$$\mathsf{skEncode}(seed_A, \mathbf{b}, K, tr, \mathbf{s}_1, \mathbf{s}'_2) := seed_A \parallel \mathsf{PolyToBytes}(\mathbf{b}/\alpha_b, d_b) \parallel K \parallel tr \\ \parallel \mathsf{PolyToBytes}(\mathbf{s}_1 + B_s^{\mathsf{enc}}, d_s) \parallel \mathsf{PolyToBytes}(\mathbf{s}'_2 + B_e^{\mathsf{enc}}, d_e).$$

The skDecode function reverses this process by parsing fixed-length segments from left to right:

1. The first $\frac{\lambda}{8}$ bytes are read as the public seed $seed_A$.
2. The next $\lceil m\,n\,d_b/8 \rceil$ bytes are decoded with $\mathsf{BytesToPoly}(\cdot, d_b)$ and rescaled by α_b to reconstruct the public polynomial vector $\mathbf{b}$.
3. The subsequent $\lambda/4$ bytes are read as the master seed K.
4. The next $\lambda/4$ bytes are read as the public-key hash tr.
5. The next $l\,n\,d_s$ bits are decoded with $\mathsf{BytesToPoly}(\cdot, d_s)$ and shifted by $-B_s^{\mathsf{enc}}$ to recover $\mathbf{s}_1$, and the final $m\,n\,d_e$ bits are decoded with $\mathsf{BytesToPoly}(\cdot, d_e)$ and shifted by $-B_e^{\mathsf{enc}}$ to recover $\mathbf{s}'_2$.

The two secret segments are therefore *not* of equal length in general, since $l \neq m$ and $d_s \neq d_e$ are both possible.

Algorithm 12 $\mathsf{skEncode}(seed_A, \mathbf{b}, K, tr, \mathbf{s}_1, \mathbf{s}'_2)$

Input: Public seed $seed_A \in \mathbb{B}^\lambda$, rounded public vector $\mathbf{b} \in R^m$, master seed $K \in \mathbb{B}^{2\lambda}$, public-key hash $tr \in \mathbb{B}^{2\lambda}$, secret vector $\mathbf{s}_1 \in R^l$ (coefficients in $[-B_s^{\mathsf{enc}}, B_s^{\mathsf{enc}}]$), adjusted error vector $\mathbf{s}'_2 \in R^m$ (coefficients in $[-B_e^{\mathsf{enc}}, B_e^{\mathsf{enc}}]$).
Output: Encoded secret key sk.
$\mathbf{b}_1 \leftarrow \mathbf{b}/\alpha_b$ ▷ coefficient-wise; exact, since $\mathbf{b} \equiv 0 \pmod{\alpha_b}$ after RoundB
$sk \leftarrow seed_A$
for $i = 0$ to $m - 1$ **do**
    $sk \leftarrow sk \parallel \mathsf{PolyToBytes}(\mathbf{b}_1[i], d_b)$
end for
$sk \leftarrow sk \parallel K \parallel tr$
for $i = 0$ to $l - 1$ **do**
    $sk \leftarrow sk \parallel \mathsf{PolyToBytes}(\mathbf{s}_1[i] + B_s^{\mathsf{enc}}, d_s)$ ▷ shift into $[0, 2^{d_s} - 1]$
end for
for $i = 0$ to $m - 1$ **do**
    $sk \leftarrow sk \parallel \mathsf{PolyToBytes}(\mathbf{s}'_2[i] + B_e^{\mathsf{enc}}, d_e)$ ▷ shift into $[0, 2^{d_e} - 1]$
end for
return sk

Algorithm 13 skDecode(sk)

Input: Encoded secret key sk.
Output: $(seed_A, \mathbf{b}, K, tr, \mathbf{s}_1, \mathbf{s}'_2)$.
$(seed_A,\, p_0, \ldots, p_{m-1},\, K,\, tr,\, y_0, \ldots, y_{l-1},\, w_0, \ldots, w_{m-1}) \leftarrow sk$,
    where $seed_A \in \mathbb{B}^{\lambda}$, $K, tr \in \mathbb{B}^{2\lambda}$, $p_i \in \mathbb{B}^{nd_b}$, $y_i \in \mathbb{B}^{nd_s}$, $w_i \in \mathbb{B}^{nd_e}$
for $i = 0$ to $m-1$ **do**
    $\mathbf{b}_1[i] \leftarrow \mathsf{BytesToPoly}(p_i, d_b)$
end for
$\mathbf{b} \leftarrow \alpha_b \cdot \mathbf{b}_1$ ▷ lossless rescale
for $i = 0$ to $l-1$ **do**
    $\mathbf{s}_1[i] \leftarrow \mathsf{BytesToPoly}(y_i, d_s) - B_s^{\mathsf{enc}}$
end for
for $i = 0$ to $m-1$ **do**
    $\mathbf{s}'_2[i] \leftarrow \mathsf{BytesToPoly}(w_i, d_e) - B_e^{\mathsf{enc}}$
end for
return $(seed_A, \mathbf{b}, K, tr, \mathbf{s}_1, \mathbf{s}'_2)$

Public-Key Rounding (RoundB): The function RoundB takes the raw public vector $\mathbf{b}_0 \in R^m$ and compresses it by eliminating the low-order bits of each coefficient, returning the rounded vector $\mathbf{b} \in R_q^m$. It depends only on the compression parameter α_b (see Table 10) and is applied independently to each of the n integer coefficients of every one of the m ring-element components of $\mathbf{b}_0$, as made explicit by the double loop of Algorithm 14.

Algorithm 14 RoundB($\mathbf{b}_0, \alpha_b$)

Input: Raw public vector $\mathbf{b}_0 \in R^m$, compression parameter α_b.
Output: Rounded public vector $\mathbf{b} \in R_q^m$ (every coefficient a multiple of α_b).
for $i = 0$ to $m-1$ **do** ▷ over the m polynomial components
    for $j = 0$ to $n-1$ **do** ▷ over the n integer coefficients of $\mathbf{b}_0[i]$
        $v^+ \leftarrow \mathbf{b}_0[i]_j \bmod q$ ▷ representative in $\{0, \ldots, q-1\}$
        $v_0 \leftarrow v^+ \bmod^{\pm} \alpha_b$ ▷ centered low-order part (Section 2.1); $v_0 \in [-\alpha_b/2, \alpha_b/2)$
        $\mathbf{b}[i]_j \leftarrow v^+ - v_0$ ▷ zero out the low-order part modulo α_b
    end for
end for
return b

The returned value coincides with $\mathbf{b}_0$ modulo α_b, so the rounding error $\delta_{\mathbf{b}} = \mathbf{b} - \mathbf{b}_0$ is bounded in each coefficient by $\alpha_b/2$. This error is absorbed into the secret error vector $\mathbf{s}_2$ during key generation (Algorithm 1, line 11), which preserves the shifted public relation $\mathbf{A} \cdot [1, \mathbf{s}_1, \mathbf{s}'_2]^T = q\mathbf{j} \pmod{2q}$.

Signature Encoding (sigEncode / sigDecode): A signature canonically encodes the triple $(seed_c, \mathbf{z}_1, \mathbf{h})$. The response quotients and hint coefficients are entropy-coded with static rANS models, a block-dependent number of low bits of each response coefficient is stored verbatim, and the challenge seed $seed_c$—an incompressible fixed-length hash output—is copied in verbatim without compression. The normative byte layout, static models, rejection conditions, canonical-decoding checks, and the encoding/decoding procedures sigEncode and sigDecode themselves (Algorithms 32 and 33) are specified in Section 2.5.10. If any component lies outside its allowed support, sigEncode returns $\perp$, triggering a fresh signing iteration.

2.5.4 Pseudorandom Sampling

Expansion of the public matrix. The subroutine ExpandA generates the public vector $\mathbf{a_{gen}} \in R_q^m$ and the public matrix $\hat{\mathbf{A}}_{gen} \in R_q^{m \times l}$, the latter directly in NTT representation, from the seed $seed_A$. Every coefficient of both objects is sampled independently and uniformly from $\mathbb{Z}_q$ by rejection sampling: writing $d_q := \lceil \log_2 q \rceil$ for the coefficient bit-length and $b_q := \lceil d_q/8 \rceil$ for the number of bytes that hold it, the sampler squeezes b_q bytes off its stream, converts them to an integer with BytesToInteger, masks off the low d_q bits, and rejects the candidate if it is not below q.

Because the NTT is a bijection on R_q, sampling each coefficient of $\hat{\mathbf{A}}_{gen}$ uniformly and treating the result *directly* as the NTT representation yields exactly the same distribution as sampling a uniform $\mathbf{A_{gen}}$ and applying NTT; the implicitly defined $\mathbf{A_{gen}} := \mathsf{NTT}^{-1}(\hat{\mathbf{A}}_{gen})$ is then uniform over $R_q^{m \times l}$, matching its specification. Consequently ExpandA performs *no* NTT internally and emits $\hat{\mathbf{A}}_{gen}$ as it is produced. The narrow public offset vector $\mathbf{a_{gen}}$ is only ever used in the coefficient domain (Algorithm 1), so it is sampled in the coefficient domain.

To expose sufficient parallelism for a vectorized XOF implementation, the output is produced by $P = 16$ independent XOF contexts: context t (for $0 \le t < 16$) is initialized by absorbing $\texttt{0x02} \parallel seed_A \parallel \mathsf{IntegerToBytes}(t, 2)$, where the one-byte tag 0x02 separates ExpandA from the other subroutines and the two-byte index separates the 16 contexts from one another. The vector $\mathbf{a_{gen}}$ is flattened into a one-dimensional array of $m\,n$ coefficients and split into 16 equal contiguous chunks, chunk t being filled from context t; the matrix $\hat{\mathbf{A}}_{gen}$ is partitioned in the same way over its $m\,l\,n$ coefficients. Both flattenings are zero-cost in the implementation, since each object is already laid out as a single contiguous memory block.

Algorithm 15 $\mathsf{ExpandA}(seed_A)$

Input: Seed $seed_A$
Output: $\mathbf{a_{gen}} \in R_q^m$ and $\hat{\mathbf{A}}_{gen} \in R_q^{m \times l}$ in NTT representation
$d_q \leftarrow \lceil \log_2 q \rceil, \quad b_q \leftarrow \lceil d_q/8 \rceil, \quad P \leftarrow 16$ ▷ coeff. bit/byte length; P parallel lanes for the vectorized XOF
for $t = 0$ to $P-1$ **do**
    $ctx_t \leftarrow \mathsf{XOF.Init}(\texttt{0x02} \parallel seed_A \parallel \mathsf{IntegerToBytes}(t, 2))$
end for
view $\mathbf{a_{gen}}$ as the flat array $\bar{a}[\,0\,..\,m\,n-1\,]$; $w_a \leftarrow m\,n/P$ ▷ flattening is zero-cost
for $t = 0$ to $P-1$ **do**
    for $k = t\,w_a$ to $(t{+}1)\,w_a - 1$ **do**
        repeat
            $(ctx_t, B) \leftarrow \mathsf{XOF.Squeeze}(ctx_t, b_q)$ ▷ one β_A-byte block, sliced
            $a \leftarrow \mathsf{BytesToInteger}(B, b_q)\ \&\ (2^{d_q} - 1)$ ▷ keep the low d_q bits
        until $a < q$
        $\bar{a}[k] \leftarrow a$ ▷ uniform in $\mathbb{Z}_q$
    end for
end for
view $\hat{\mathbf{A}}_{gen}$ as the flat array $\bar{h}[\,0\,..\,m\,l\,n-1\,]$; $w_h \leftarrow m\,l\,n/P$ ▷ flattening is zero-cost
for $t = 0$ to $P-1$ **do**
    for $k = t\,w_h$ to $(t{+}1)\,w_h - 1$ **do**
        repeat
            $(ctx_t, B) \leftarrow \mathsf{XOF.Squeeze}(ctx_t, b_q)$ ▷ one β_A-byte block, sliced
            $a \leftarrow \mathsf{BytesToInteger}(B, b_q)\ \&\ (2^{d_q} - 1)$ ▷ keep the low d_q bits
        until $a < q$
        $\bar{h}[k] \leftarrow a$ ▷ uniform in $\mathbb{Z}_q$: directly a valid NTT-domain coefficient
    end for
end for
return $(\mathbf{a_{gen}}, \hat{\mathbf{A}}_{gen})$ ▷ $\hat{\mathbf{A}}_{gen}$ already in NTT representation; no NTT performed

Expansion of the secret key. The subroutine ExpandS deterministically derives the secret vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ from the seed $seed_{sk}$. Since the standard deviations σ_1 and σ_2 are small (Table 10), the convolution and exponential rejection steps of SampleDGauss are unnecessary: each coefficient is the signed value $(-1)^b x$ of one BaseSampler draw $x \sim D_{\mathbb{Z}^+,\sigma}$ from the mode-dependent table RCDT_σ ($\sigma = \sigma_1$ for $\mathbf{s}_1$, $\sigma = \sigma_2$ for $\mathbf{s}_2$). At $x = 0$ the $+0$ and -0 branches coincide, so a fold bit f rejects half of the zero mass to restore the two-sided density of $D_{\mathbb{Z},\sigma}$.

As in ExpandA, $P = 16$ independent XOF contexts expose lane parallelism: context t is initialized *once* by $\mathsf{XOF.Init}(\texttt{0x03} \parallel seed_{sk} \parallel \mathsf{IntegerToBytes}(t, 2))$ (the one-byte tag 0x03 separates ExpandS, the two-byte index the 16 contexts) and is thereafter only squeezed. The flat coefficient arrays of $\mathbf{s}_1$ (length $l\,n$) and then $\mathbf{s}_2$ (length $m\,n$) are split into 16 equal contiguous chunks, chunk t filled from context t over the shared stream (flattening is zero-cost). To match the vectorized implementation byte-for-byte, the stream is consumed in fixed mini-batches of $G = 32$ candidates—a block of $\beta = 12G + G/4 = 392$ bytes per mini-batch, *not* one squeeze per coefficient: the first $12G = 384$ bytes carry G base-sampler inputs in the batched limb-major layout used by BaseSampler (defined with Algorithm 27), and the trailing $G/4 = 8$ bytes carry G two-bit fields packed 4 per byte, LSB first, so candidate j uses bits $2(j \bmod 4)$ and $2(j \bmod 4)+1$ of byte $384 + \lfloor j/4 \rfloor$—bit 0 the sign b, bit 1 the fold bit f. The whole mini-batch is consumed unconditionally; candidate j is *skipped* (its output slot left to a later candidate) iff $x = 0$ and $f = 1$.

Algorithm 16 ExpandS($seed_{sk}$)

Input: Seed $seed_{sk}$ **Output:** $\mathbf{s}_1 \in R^l$, $\mathbf{s}_2 \in R^m$
$P \leftarrow 16$; $G \leftarrow 32$; $\beta \leftarrow 12G + G/4$ ▷ 16 lanes; mini-batch of G; block $\beta = 392$ bytes
for $t = 0$ to $P - 1$ **do** $ctx_t \leftarrow \mathsf{XOF.Init}(\texttt{0x03} \parallel seed_{sk} \parallel \mathsf{IntegerToBytes}(t, 2))$
end for
for $(\bar{v}, \sigma, w) \in \{(\bar{s}, \sigma_1, ln/P),\ (\bar{e}, \sigma_2, mn/P)\}$ **do** ▷ $\mathbf{s}_1$ then $\mathbf{s}_2$, shared stream
  for $t = 0$ to $P - 1$ **do**
    $c \leftarrow 0$ ▷ accepted count in lane chunk t
    while $c < w$ **do**
      $(ctx_t, \rho) \leftarrow \mathsf{XOF.Squeeze}(ctx_t, \beta)$ ▷ one fixed mini-batch block
      for $j = 0$ to $G - 1$ **do**
        $x \leftarrow \mathsf{BaseSampler}(\mathrm{RCDT}_\sigma, \rho_u^{(j)})$; $m \leftarrow j \bmod 4$ ▷ $\rho_u^{(j)}$: limb-major slot j
        $g \leftarrow \rho[384 + \lfloor j/4 \rfloor]$; $b \leftarrow (g \gg 2m) \,\&\, 1$; $f \leftarrow (g \gg (2m{+}1)) \,\&\, 1$
        if $c < w$ **and** $\neg(x = 0 \wedge f = 1)$ **then** $\bar{v}[tw + c] \leftarrow (-1)^b x$; $c \leftarrow c + 1$
        end if
      end for
    end while
  end for
end for
return $(\mathbf{s}_1, \mathbf{s}_2)$

SampleY SampleY takes a seed $seed_y$ and outputs the masking vector $\mathbf{y} \in R^{l+m+1}$, reproducible from the seed alone. As in ExpandA and ExpandS, $P = 16$ independent XOF contexts expose lane parallelism: context t is initialized *once* by $\mathsf{XOF.Init}(\texttt{0x08} \parallel seed_y \parallel \mathsf{IntegerToBytes}(t, 2))$ (tag 0x08 separating SampleY) and thereafter only squeezed. The flat array of $(l + m + 1)\,n$ coefficients is split into 16 equal contiguous chunks of width $w = (l + m + 1)n/P$, chunk t filled from context t. The revised byte schedule is binding for all backends. Each lane first squeezes one β_Y-byte block containing $\lceil w/8 \rceil$ *output-indexed* sign bytes, $p = 8$ discarded padding bytes, and the first candidate mini-batch: the sign b of the c-th accepted coefficient is bit c of this stream (LSB first), *not* a per-candidate draw. It fills the chunk in fixed mini-batches of $G = 32$ candidates, using the initial mini-batch first and squeezing one further $\gamma_Y = 21G + G/8 = 676$-byte block whenever more candidates are needed. Each mini-batch is laid out as 384 bytes of G base-sampler inputs (batched limb-major layout), then G low-part bytes y, G eight-byte Bernoulli words u, and $G/8$ bytes carrying one independent fold bit per candidate. The whole mini-batch is consumed unconditionally; a candidate accepted by SampleDGauss is placed at the next free slot, advancing the sign index only on a placement. The fold bit is separate from the output-indexed sign bit, so zero folding never conditions acceptance on the sign.

Algorithm 17 SampleY($seed_y$)

Input: Seed $seed_y$ **Output:** $\mathbf{y} \in R^{l+m+1}$
$P \leftarrow 16$; $G \leftarrow 32$; $w \leftarrow (l+m+1)\,n/P$
$s_Y \leftarrow \lceil w/8 \rceil$; $p \leftarrow 8$; $\gamma_Y \leftarrow 21G + G/8$
$\beta_Y \leftarrow s_Y + p + \gamma_Y$
for $t = 0$ to $P-1$ **do** $ctx_t \leftarrow \mathsf{XOF.Init}(\texttt{0x08} \parallel seed_y \parallel \mathsf{IntegerToBytes}(t, 2))$
end for
for $t = 0$ to $P-1$ **do**
  $(ctx_t, B) \leftarrow \mathsf{XOF.Squeeze}(ctx_t, \beta_Y)$
  $S \leftarrow B[0 : s_Y]$; $\rho \leftarrow B[s_Y + p : \beta_Y]$ ▷ sign prefix, discard pad, first mini-batch
  $c \leftarrow 0$
  while $c < w$ **do**
    for $j = 0$ to $G-1$ **do**
      $x \leftarrow \mathsf{BaseSampler}(\mathrm{RCDT}_{\sigma_s}, \rho_u^{(j)})$ ▷ $\rho_u^{(j)}$: limb-major slot j
      $y \leftarrow \rho[384 + j]$; $u \leftarrow \mathsf{BytesToInteger}(\rho[416 + 8j : 416 + 8j + 8], 8)$
      $d \leftarrow \mathsf{ct_sel}([\![c = w]\!], w-1, c)$ ▷ safe dummy sign index after the lane fills
      $b \leftarrow (S[\lfloor d/8 \rfloor] \gg (d \bmod 8)) \,\&\, 1$
      $f \leftarrow (\rho[672 + \lfloor j/8 \rfloor] \gg (j \bmod 8)) \,\&\, 1$
      $(a, z) \leftarrow \mathsf{SampleDGauss}(x, y, u, b, f)$
      if $c < w$ **and** $a = 1$ **then** $\bar{y}[tw + c] \leftarrow z$; $c \leftarrow c + 1$
      end if
    end for
    if $c < w$ **then**
      $(ctx_t, \rho) \leftarrow \mathsf{XOF.Squeeze}(ctx_t, \gamma_Y)$
    end if
  end while
end for
return y

2.5.5 Compression and Hint Mechanism

CompressY and StretchS. We divide a vector

$$\mathbf{x} = (x_0, \mathbf{x}_s, \mathbf{x}_e) \in R^{1+l+m}$$

into the constant block, the l secret blocks, and the m error blocks. The three blocks are respectively associated with the scaling parameters α_1, α_s, and α_e. The stretching map multiplies each block by its corresponding scaling parameter, whereas the compression map divides each block by the same parameter and rounds coefficient-wise to the nearest integer.

Algorithm 18 CompressY($\mathbf{x}$)

Input: $\mathbf{x} = (x_0, \mathbf{x}_s, \mathbf{x}_e) \in R^{1+l+m}$
Output: $\mathbf{x}' \in R^{1+l+m}$
$x'_0 \leftarrow \lfloor x_0/\alpha_1 + 1/2 \rfloor$
$\mathbf{x}'_s \leftarrow \lfloor \mathbf{x}_s/\alpha_s + 1/2 \rfloor$ ▷ applied coefficient-wise
$\mathbf{x}'_e \leftarrow \lfloor \mathbf{x}_e/\alpha_e + 1/2 \rfloor$
return $\mathbf{x}' = (x'_0, \mathbf{x}'_s, \mathbf{x}'_e)$

Algorithm 19 $\mathsf{StretchS}(\mathbf{x})$

Input: $\mathbf{x} = (x_0, \mathbf{x}_s, \mathbf{x}_e) \in R^{1+l+m}$
Output: $\widetilde{\mathbf{x}} \in R^{1+l+m}$
$\widetilde{x}_0 \leftarrow \alpha_1 x_0$
$\widetilde{\mathbf{x}}_s \leftarrow \alpha_s \mathbf{x}_s$
$\widetilde{\mathbf{x}}_e \leftarrow \alpha_e \mathbf{x}_e$
return $\widetilde{\mathbf{x}} = (\widetilde{x}_0, \widetilde{\mathbf{x}}_s, \widetilde{\mathbf{x}}_e)$

Here $\lfloor \cdot + 1/2 \rfloor$ is applied coefficient-wise. Both $\mathsf{StretchS}$ and $\mathsf{CompressY}$ perform *no* modular reduction: $\mathsf{StretchS}$ is the exact integer scaling $\mathbf{x}_i \mapsto \alpha_i \mathbf{x}_i$ and $\mathsf{CompressY}$ the matching coefficient-wise shifted floor. Consequently the translation identity

$$\mathsf{CompressY}\big(\mathbf{y} + \mathsf{StretchS}(\mathbf{x})\big) = \mathsf{CompressY}(\mathbf{y}) + \mathbf{x}$$

holds *unconditionally* for all $\mathbf{x}, \mathbf{y} \in R^{1+l+m}$ over the integers, because the integer shift $\mathbf{x}$ passes through the floor coefficient-wise: $\lfloor (y + \alpha_i x_i)/\alpha_i + 1/2 \rfloor = \lfloor y/\alpha_i + 1/2 \rfloor + x_i$ for all y and integer x_i.

Consequently, if the iterative sampler outputs

$$\widetilde{\mathbf{z}} = \mathbf{y} + \mathsf{StretchS}(s)c',$$

then the compressed response satisfies

$$\mathbf{z} = \mathsf{CompressY}(\widetilde{\mathbf{z}}) = \mathsf{CompressY}(\mathbf{y}) + sc',$$

which is the response relation required for verification.

Generation of the hint. To avoid transmitting the full commitment $\mathbf{w}$, we only transmit its least significant bits and a highly compressible boolean hint $\mathbf{h}$. The subroutine $\mathsf{MakeHint}$ computes this auxiliary hint to compensate for the information loss caused by omitting the second part $\mathbf{z}_2$ of the response from the signature.

The signer first forms $\widetilde{\mathbf{w}} = \mathbf{w} - 2\mathbf{z}_2 \bmod 2q$. It then separately computes the high-order parts of $\mathbf{w}$ and $\widetilde{\mathbf{w}}$ with respect to the compression parameter α_h. The hint is defined as their difference modulo $2(q-1)/\alpha_h$. This allows the verifier to later recover the exact high-order commitment information.

Algorithm 20 $\mathsf{MakeHint}(\mathbf{z}_2, \mathbf{w})$

Input: $\mathbf{z}_2$, $\mathbf{w}$
Output: $\mathbf{h}$
$\widetilde{\mathbf{w}} \leftarrow \mathbf{w} - 2\mathbf{z}_2 \bmod 2q$
$\mathbf{h} \leftarrow \lfloor \mathbf{w}/\alpha_h \rceil - \lfloor \widetilde{\mathbf{w}}/\alpha_h \rceil \bmod H_h$
return $\mathbf{h}$

Recovery using the hint. The subroutine $\mathsf{UseHint}$ uses the transmitted hint to recover the high-order commitment information needed during verification. The verifier first recomputes the modified commitment corresponding to the transmitted partial response $\mathbf{z}_1$, extracts its high-order part, and combines it with the hint to recover the high-order part of the original commitment $\mathbf{w}_h$.

Together with the reconstructed least significant bit $\mathbf{w}'_0$, this yields an approximate version of the original commitment, from which the omitted response part $\mathbf{z}_2$ can be recovered up to the tolerated short error.

Algorithm 21 UseHint$(\mathbf{h}, \mathbf{z}_1, \mathbf{w}'_0, \hat{\mathbf{A}}_1, c)$

Input: $\mathbf{h}, \mathbf{z}_1, \mathbf{w}'_0, \hat{\mathbf{A}}_1, c$
Output: $\mathbf{w}_h, \mathbf{z}'_2$
$\overline{\mathbf{w}} \leftarrow \big(\mathsf{NTT}^{-1}(\hat{\mathbf{A}}_1 \cdot \mathsf{NTT}(\mathbf{z}_1)) - q\, c\, \mathbf{j}\big) \bmod q$ ▷ recompute the modified commitment modulo q (matrix–vector product); non-negative representative in $\{0, \ldots, q-1\}$
$\widetilde{\mathbf{w}} \leftarrow \mathsf{LiftToModTwoQ}(\overline{\mathbf{w}}, \mathbf{w}'_0)$ ▷ lift to the representative in $[0, 2q)$ using the reconstructed low bit
$\mathbf{w}_h \leftarrow \mathbf{h} + \lfloor \widetilde{\mathbf{w}}/\alpha_h \rceil \bmod H_h$ ▷ recover the high-order part
$\mathbf{w}^{\text{app}} \leftarrow \alpha_h \mathbf{w}_h + \mathbf{w}'_0$ ▷ approximate commitment value; lies in $[0, 2q)$ provided $\mathbf{w}_h \in [0, H_h)$ and $\mathbf{w}'_0 \in \{0, 1\}$
$\mathbf{z}'_2 \leftarrow (\mathbf{w}^{\text{app}} - \widetilde{\mathbf{w}})/2 \bmod^{\pm} q$ ▷ recover the omitted response part, centered representative
return $(\mathbf{w}_h, \mathbf{z}'_2)$

2.5.6 RejectSampling

Our rejection-sampling procedure follows an iterative, rejection-free design. Let the challenge be a sparse binary polynomial with τ non-zero monomials. Instead of forming the full secret-dependent shift $\widetilde{s}c$ at once, we process the non-zero coefficients in a fixed order: for each index $j \in \{0, \ldots, n-1\}$ in ascending order, if the j-th coefficient $c_j = 1$, the corresponding shift is $v = \widetilde{s} \cdot X^j$, where $\widetilde{s} = \mathsf{StretchS}(s)$ is the stretched secret vector. At each iteration, the transition procedure $\mathsf{R}(ctx, y, v, V)$ draws its randomness from the XOF context ctx (initialized in Sign on $\texttt{0x09} \parallel seed_y$) and decides whether the current state should be shifted in the positive or negative direction of v. Hence, after all non-zero coefficients have been processed in this deterministic order, the output has the form

$$z = y + \widetilde{s}c',$$

where c' differs from c only in the signs of its non-zero coefficients.

The main point of this design is that the transition R never outputs an abort symbol. It always returns one of the two neighbouring candidates $y - v$ and $y + v$. Therefore, the sampler runs for a fixed number of iterations, equal to the weight of the challenge, and avoids the variable-time accept/reject loop used in traditional Fiat–Shamir-with-aborts signatures. The purpose of the probabilistic choice in R is not to preserve the input distribution exactly after each individual step. Instead, each transition is chosen so that the accumulated output distribution remains close to a public target discrete Gaussian distribution, independent of the secret key.

IRS randomness and traversal order. Each signing attempt derives a fresh masking seed $seed_y$ from ExpandSigningSeeds (domain tag 0x01). The masking vector $\mathbf{y}$ and the IRS transitions consume independent XOF streams from the *same* $seed_y$, separated only by the domain tags 0x08 (SampleY) and 0x09 (RejectSample). Before calling RejectSample, Sign initializes $ctx_{\mathsf{R}} \leftarrow \mathsf{XOF.Init}(\texttt{0x09} \parallel seed_y)$; this context is threaded through RejectSample $\rightarrow$ R $\rightarrow$ SamplerU, with every XOF.Squeeze updating ctx_{R}; the whole 18τ-byte IRS stream is drawn in a single squeeze and sliced in ascending-j order (block table in Section 2.3.2) (as in SampleY). Because RejectSample applies transitions in strictly increasing index order $j = 0, \ldots, n-1$ (Algorithm 23), the output $\widetilde{\mathbf{z}}$ is fully determined by $(seed_y, c, \widetilde{s})$ even though SampleC places its τ nonzero coefficients via an unordered partial Fisher–Yates shuffle.

The probability implemented by R is derived from the alternating series of Gaussian ratios along the one-dimensional lattice generated by the shift v. Directly evaluating this probability would require several Gaussian ratio computations. We instead use an equivalent logarithmic interval test. A uniform sample $U \sim \mathcal{U}(0, 1]$ is transformed into

$$u = 2r^2 \ln U,$$

and the algorithm checks whether this value lies between consecutive squared norm differences

$$\|y\|^2 - \|y + (2i+1)v\|^2 \quad \text{and} \quad \|y\|^2 - \|y + 2iv\|^2.$$

This realizes the truncated transition probability using only fixed arithmetic comparisons in the secret-dependent loop. The parameter N controls the truncation length of the alternating sum (the implemented test of Algorithm 22 retains the N terms $i = 0, \ldots, N-1$, covered by the $\lceil N/2 \rceil$ boundary pairs of its loop), and is chosen so that the resulting statistical error is within the overall distributional error budget. The concrete fixed-point realization of this test—the number of uniform random bits drawn to form $U \sim \mathcal{U}(0,1]$, the integer-only approximation of the logarithm with additive error $\eta_{\log}$, and the rounding rule used for the boundary comparison—is fixed by the implementation so that the induced statistical error stays within the budget analyzed in Section 4.1.2. The logarithm is approximated in *base two* by SamplerU (Section 2.5.8), which returns $\ell \approx \log_2 U$; the natural-logarithm test variable $u = 2r^2 \ln U$ is then obtained by the single multiplication $u = (2r^2 \ln 2)\,\ell$ in line 2 of Algorithm 22, using the reproducible constant

$$2r^2 \ln 2 = 2 \cdot 825^2 \cdot \ln 2 = 943546.5995372256\ldots$$

(with $2r^2 = 1{,}361{,}250$). Carrying ℓ in base two makes the integer exponent contribution exact and confines the whole logarithm error budget $\eta_{\log}$ to the approximation of $\log_2 b$ on $b \in [1,2)$.

Fixed-point form (implementation note). For bit-level reproducibility this constant is held as the Q44 integer $K = \lfloor 2r^2 \ln 2 \cdot 2^{44} \rceil = 16599047320634951608$, and u is formed in signed Q44 from the two parts of ℓ returned by SamplerU—its Q62 fraction frac and integer exponent a, with $\ell = \mathrm{frac}/2^{62} - a$—as $u = \big((K \cdot \mathrm{frac} + 2^{61}) \gg 62\big) - Ka$ (the fractional term rounds to nearest, the exponent term is exact); the interval bounds of Algorithm 22 are compared after the same left shift by 44.

A naive evaluation of the interval bounds would recompute the squared norms $\|y + mv\|_2^2$ for every shift m inside the loop. We avoid this by expanding

$$\|y\|_2^2 - \|y + mv\|_2^2 \;=\; -2m\,\langle y, v\rangle - m^2\|v\|_2^2,$$

so that each bound is a closed-form expression in the single inner product $t = \langle y, v\rangle$ and the squared shift norm $V = \|v\|_2^2$. Thus one call to $\mathsf{R}(ctx, y, v, V)$ requires only one evaluation of $\langle y, v\rangle$, and the whole loop reduces to additions and multiplications by the small integers $m = 0, 1, \ldots$. Moreover, every shift has the form $v = \widetilde{s} \cdot X^j$ for an index j with $c_j = 1$, and multiplication by X^j is an isometry of $R = \mathbb{Z}[X]/(X^n+1)$; hence $\|v\|_2^2 = \|\widetilde{s} \cdot X^j\|_2^2 = \|\widetilde{s}\|_2^2 = \langle \widetilde{s}, \widetilde{s}\rangle$ is the same for all such indices. Consequently, a single invocation of RejectSample computes $\langle \widetilde{s}, \widetilde{s}\rangle$ exactly once and one inner product $\langle y, v\rangle$ per nonzero coefficient, i.e. τ inner products in total, eliminating all per-iteration norm evaluations. The challenge c itself has only $\{0,1\}$ entries; the sign of each applied shift is determined by the transition procedure R, not by the challenge polynomial.

Before performing the interval test, the sign of v is normalized so that $\langle y, v\rangle > 0$. This ensures that the Gaussian ratios along the sequence $y, y+v, y+2v, \ldots$ have decreasing magnitudes, making the alternating interval representation valid. The flag computed by R then determines which of the two signed shifts is applied. Repeating this transition for every nonzero coefficient of c in ascending index order yields the final response z used in the signature.

Algorithm 22 $\mathsf{R}(ctx, y, v, V)$

Require: XOF context ctx, input vector y, shift vector v, squared shift norm $V = \|v\|_2^2$
Ensure: Updated context ctx and output $y' \in \{y - v, y + v\}$

1: $(ctx, \ell) \leftarrow \mathsf{SamplerU}(ctx, \kappa_a, \kappa_b)$
2: $u \leftarrow 2r^2 \ln 2 \cdot \ell$ ▷ $\ell \approx \log_2 U$; the factor $\ln 2$ gives $u \approx 2r^2 \ln U$ for $U \sim \mathcal{U}(0, 1]$
3: $flg \leftarrow -1$
4: $t \leftarrow \langle y, v \rangle$ ▷ single inner product per transition
5: **if** $t \leq 0$ **then**
6: $\quad v \leftarrow -v$
7: $\quad t \leftarrow -t$ ▷ normalize the sign so that $\langle y, v \rangle > 0$
8: **end if**
9: **for** $i = 0$ to $\lceil N/2 \rceil - 1$ **do** ▷ each iteration tests the boundary pair $k = 2i$ and $k = 2i + 1$, covering all N truncated terms
10: $\quad$ **if** $-2(2i + 1)\, t - (2i + 1)^2 V \; < \; u \; \leq \; -4i\, t - 4i^2 V$ **then**
11: $\quad\quad flg \leftarrow 1$
12: $\quad$ **end if**
13: **end for**
14: **return** $(ctx,\, y - flg \cdot v)$[1]

Algorithm 23 $\mathsf{RejectSample}(ctx, y, c, \widetilde{s})$

Require: XOF context ctx, input vector y, challenge c, stretched secret vector $\widetilde{s}$
Ensure: Updated context ctx and output z

1: $z \leftarrow y$
2: $V \leftarrow \langle \widetilde{s}, \widetilde{s} \rangle$ ▷ computed once; $\|\widetilde{s} \cdot X^j\|_2^2 = \|\widetilde{s}\|_2^2$ for every index j
3: **for** $j = 0$ **to** $n - 1$ **do** ▷ fixed ascending traversal order
4: $\quad$ **if** $c_j = 1$ **then**
5: $\quad\quad (ctx, z) \leftarrow \mathsf{R}(ctx, z, \widetilde{s} \cdot X^j, V)$
6: $\quad$ **end if**
7: **end for**
8: **return** (ctx, z)

2.5.7 Sample in Ball

The challenge sampler $\mathsf{SampleC}$ deterministically expands the challenge seed $seed_c$ into a binary polynomial $c \in \{0, 1\}^n$ of fixed Hamming weight τ. It follows the in-ball construction of Dilithium, specialized to the unsigned setting of Shuttle: since the transmitted challenge uses only $+1$ entries (the signs are recovered later by the iterative sampler), no sign bits are drawn. The partial Fisher–Yates shuffle in lines 7–11 places exactly τ nonzero coefficients, so the output is uniform over the weight-τ subset of positions; the rejection condition in the repeat-until loop makes each drawn index j uniform over $\{0, \ldots, i\}$. The sampler itself is deterministic given $seed_c$, and different seeds may map to the same challenge. Although the partial Fisher–Yates construction does not output the nonzero indices in sorted order, $\mathsf{RejectSample}$ always consumes them in strictly increasing index order $j = 0, \ldots, n - 1$ (Algorithm 23), so the IRS output is independent of the internal shuffle order. The signature transmits the challenge seed $seed_c$ itself, stored verbatim (Section 2.5.10); signature aliases are precluded because the verifier recomputes $seed'_c$ from the reconstructed commitment and accepts only when $seed'_c = seed_c$, pinning the seed to a unique value.

[1] An earlier version of this specification incorrectly returned $y + flg \cdot v$ on this line, which swapped the two branches relative to the definition of $p_v(y)$ (probability of returning $y - v$). The present revision corrects that sign so that an interval hit ($flg = +1$) applies $y - v$, matching the analysis and the intended transition.

Algorithm 24 $\mathsf{SampleC}(seed_c)$

1: **Input:** Challenge seed $seed_c \in \mathbb{B}^{2\lambda}$ ($\frac{\lambda}{4}$ bytes)
2: **Output:** Challenge polynomial $c \in \{0,1\}^n$ of Hamming weight τ
3: $d_n \leftarrow \lceil \log_2 n \rceil, \quad b_n \leftarrow \lceil d_n/8 \rceil$ ▷ index bit-length and byte-length
4: $ctx \leftarrow \mathsf{XOF.Init}(\texttt{0x07} \parallel seed_c)$
5: $c \leftarrow (0,0,\ldots,0) \in \{0,1\}^n$ ▷ n coefficients, all zero
6: **for** $i = n - \tau$ **to** $n-1$ **do**
7: **repeat**
8: $(ctx, B) \leftarrow \mathsf{XOF.Squeeze}(ctx, b_n)$ ▷ one β_C-byte block, sliced
9: $j \leftarrow \mathsf{BytesToInteger}(B, b_n)\ \&\ (2^{d_n} - 1)$ ▷ keep the low d_n bits
10: **until** $j \le i$ ▷ rejection sampling of a uniform index $j \in \{0,\ldots,i\}$
11: $c_i \leftarrow c_j$
12: $c_j \leftarrow 1$
13: **end for**
14: **return** c

2.5.8 Uniform Sampling and Logarithm Approximation

To avoid explicitly representing a uniform random value that may be exponentially close to zero, we use an exponent–mantissa representation. For every $u \in (0,1)$, there exist an integer $a \ge 1$ and a value $b \in [1,2)$ such that

$$u = 2^{-a} b.$$

Consequently, working in base two,

$$\log_2 u = \log_2 b - a.$$

This representation separates the dynamic range of u, which is captured by the integer a, from its significant bits, which are captured by b. Hence, the implementation never needs to construct the potentially very small value u explicitly. **Working in base two is deliberate**: the integer exponent a then contributes *exactly*, and only $\log_2 b$ for $b \in [1,2)$ requires a polynomial approximation—no approximation of $\log 2$ enters the exponent term. (In a natural-logarithm formulation one would instead write $\log u = \log b - a \log 2$ and carry a stored approximation $\widehat{L}_2 \approx \log 2$; the base-two choice removes that term and its error entirely.) The transition test of Algorithm 22 consumes $\log_2 u$ after a single multiplication by the constant $2r^2 \ln 2$, which restores the natural-logarithm test variable $u = 2r^2 \ln U$ (line 2 of Algorithm 22).

In the ideal continuous model, let a follow the geometric distribution

$$\Pr[a = j] = 2^{-j}, \qquad j \ge 1,$$

and let b be uniform over $[1,2)$, independently of a. Then

$$u = 2^{-a} b$$

is uniform over $(0,1)$. Indeed, conditioned on $a = j$, the value u is uniform over

$$[2^{-j}, 2^{-j+1}),$$

with conditional density 2^j. Multiplying this density by $\Pr[a = j] = 2^{-j}$ gives unit density over each dyadic interval.

In the implementation, the exponent is sampled from a fixed-length bit string. Let κ_a denote the number of exponent bits and let κ_b denote the precision of the mantissa. The exponent

a is determined by the number of leading zero bits before the first one, while the mantissa is sampled from the dyadic set

$$\mathcal{B}_{\kappa_b} := \left\{ 1 + \frac{m}{2^{\kappa_b}} : m \in \{0, \ldots, 2^{\kappa_b} - 1\} \right\}.$$

If all κ_a exponent bits are zero, we set $a = \kappa_a + 1$. This case represents the entire remaining tail $u < 2^{-\kappa_a}$. Provided that all comparison thresholds used by the transition algorithm are at least $2^{-\kappa_a}$, collapsing this tail does not change any comparison result.

Segmented base-two logarithm. A single polynomial on the whole interval $[1, 2)$ would require a high degree, because the minimax approximation rate of $\log_2$ on $[\beta, 2\beta)$ is governed by the endpoint *ratio* (here 2), independent of any recentering. We therefore split $[1, 2)$ into 2^g equal segments indexed by the top g mantissa bits. Writing

$$j = \left\lfloor m / 2^{\kappa_b - g} \right\rfloor \in \{0, \ldots, 2^g - 1\}, \qquad \beta_j = 1 + \frac{j}{2^g},$$

every $b \in [\beta_j, \beta_j + 2^{-g})$ is normalized to

$$x = (b - \beta_j)\, 2^g = \frac{m \bmod 2^{\kappa_b - g}}{2^{\kappa_b - g}} \in [0, 1).$$

On segment j a degree-d polynomial in the normalized argument x,

$$P_j(x) = 2^{-Q} \sum_{i=0}^{d} c_{j,i}\, x^i \;\approx\; \log_2\!\big(\beta_j + x\, 2^{-g}\big) = \log_2 b,$$

with signed integer coefficients $c_{j,i}$ at fixed-point scale 2^{-Q}, reproduces $\log_2 b$; the segment offset $\log_2 \beta_j$ is absorbed into the constant term $c_{j,0}$. We fix

$$g = 2, \qquad d = 13, \qquad Q = 62,$$

that is, four segments, each a degree-13 polynomial with signed Q62 coefficients. The 4×14 integer coefficients $c_{j,i}$ are listed verbatim in Table 1. By construction $c_{0,0} = 0$ exactly, so $\mathsf{ApproxLog}$ maps the boundary value $b = 1$ (segment $j = 0$, $x = 0$) to 0, i.e. $\mathsf{ApproxLog}(0, 1) = 0$.

Table 1: Segmented base-two logarithm coefficients $c_{j,i}$ (signed Q62 integers): on segment $j \in \{0, 1, 2, 3\}$ the polynomial $P_j(x) = 2^{-62} \sum_{i=0}^{13} c_{j,i} x^i$ approximates $\log_2(\beta_j + x\, 2^{-2}) = \log_2 b$ for the normalized argument $x \in [0, 1)$, $\beta_j = 1 + j/4$.

i	$c_{0,i}$	$c_{1,i}$	$c_{2,i}$	$c_{3,i}$
0	0	1484631294131014398	2697663385880076776	3723267405961586381
1	1663314137230540005	1330651309784432187	1108876091487026868	950465221274594463
2	−207914267153782834	−133065130978439107	−92406340957251847	−67890372948185266
3	34652377857794422	17742017463685572	10267371217462318	6465749804587722
4	−6497320829889967	−2661302618120493	−1283421402046772	−692758907616243
5	1299464001399016	425808407352253	171122852511038	79172446438655
6	−270720747985221	−70968007646487	−23767057140815	−9425290482070
7	58008183734025	12165731010280	3395273701197	1154114515161
8	−12680634058615	−2128477788302	−495094418506	−144257705268
9	2802591328231	377485364884	73261264270	18306992888
10	−611697204761	−66839897685	−10884612082	−2339854304
11	123141961189	11228353229	1561954161	292286931
12	−19530076391	−1554229086	−191070343	−31958889
13	1717255301	124985134	14130118	2187745

The approximation is evaluated by Horner's rule on the selected segment (Algorithm 25). In the constant-time implementation the coefficient row $c_{j,\cdot}$ is selected by a full-table scan—so the

secret-dependent segment index j touches every row and reveals no address—and the products use rounded fixed-point high-half multiplies; these realization details do not affect the value computed below.

Algorithm 25 $\mathsf{ApproxLog}(a, m)$

Input: An integer $a \geq 1$ and a mantissa integer $m \in \{0, \ldots, 2^{\kappa_b} - 1\}$ (so $b = 1 + m/2^{\kappa_b}$)
Output: An approximation ℓ of $\log_2(2^{-a}b) = \log_2 b - a$
$j \leftarrow \lfloor m/2^{\kappa_b - g} \rfloor$ ▷ top g bits: segment index
$x \leftarrow (m \bmod 2^{\kappa_b - g})/2^{\kappa_b - g}$ ▷ normalized reduced argument in $[0, 1)$
$z \leftarrow c_{j,d}$
for $i = d - 1, d - 2, \ldots, 0$ **do**
    $z \leftarrow z \cdot x + c_{j,i}$ ▷ Horner evaluation of $2^Q P_j(x)$
end for
$\ell \leftarrow 2^{-Q} z - a$ ▷ $= P_j(x) - a \approx \log_2 b - a$
return ℓ

The exponent–mantissa representation and the corresponding logarithm approximation are generated by $\mathsf{SamplerU}$, which draws all randomness from the XOF context ctx threaded in from $\mathsf{RejectSample}$.

Bit consumption in $\mathsf{SamplerU}$. Let $\mathsf{BytesToBits}(B, \ell)$ expand an ℓ-byte string into a bit string $(R_0, \ldots, R_{8\ell-1})$ by reading the bytes in squeeze order and, within each byte, taking bits from most significant to least significant. Indices are 0-based, and a slice $[p{:}q]$ denotes the inclusive run $R_p, \ldots, R_q$. Each call to $\mathsf{SamplerU}$ consumes randomness in the fixed order below; every $\mathsf{XOF.Squeeze}$ updates ctx before the next draw.

1. **Exponent block (κ_a bits).** Squeeze $\lceil \kappa_a/8 \rceil$ bytes ρ_a and set $R \leftarrow \mathsf{BytesToBits}(\rho_a, \lceil \kappa_a/8 \rceil)$ (the first squeezed bit is R_0, the most significant bit). Because $\kappa_a = 80$ is a whole number of bytes, $\lceil \kappa_a/8 \rceil \cdot 8 = \kappa_a$, so R already has exactly κ_a bits and no truncation $[0{:}\kappa_a - 1]$ is needed. Set the exponent
$$a \leftarrow \mathsf{CLZ}(R) + 1,$$
the 1-indexed position of the first one-bit, equivalently one plus the number of leading zeros. (When R is the all-zero string, $\mathsf{CLZ}(R) = \kappa_a$ and $a = \kappa_a + 1$, the collapsed tail.)

2. **Mantissa block (κ_b bits).** Squeeze $\lceil \kappa_b/8 \rceil$ bytes ρ_b, let $M \leftarrow \mathsf{BytesToBits}(\rho_b, \lceil \kappa_b/8 \rceil)[0{:}\kappa_b - 1]$, and set $m \leftarrow \sum_{i=0}^{\kappa_b - 1} M_i \, 2^{\kappa_b - 1 - i}$ (MSB-first integer in $\{0, \ldots, 2^{\kappa_b} - 1\}$).

The exponent a (above) and the mantissa $b = 1 + m/2^{\kappa_b}$ are then consumed by Algorithm 26. We set $\kappa_a = 80$, so that the exponent block occupies exactly $\lceil \kappa_a/8 \rceil = 10$ whole bytes with no discarded bit (the same 10-byte squeeze previously used for $\kappa_a = 79$ now uses its last bit as well); the mantissa block occupies $\lceil \kappa_b/8 \rceil = 8$ bytes, of which the low $8\lceil \kappa_b/8 \rceil - \kappa_b = 7$ bits are discarded.

Algorithm 26 SamplerU$(ctx, \kappa_a, \kappa_b)$

Input: XOF context ctx, exponent precision κ_a, mantissa precision κ_b
Output: Updated context ctx and an approximation ℓ of $\log_2 u$
$(ctx, \rho_a) \leftarrow \mathsf{XOF.Squeeze}(ctx, \lceil \kappa_a/8 \rceil)$ ▷ first 10 B of the one-shot 18τ-byte IRS block
$R \leftarrow \mathsf{BytesToBits}(\rho_a, \lceil \kappa_a/8 \rceil)$ ▷ $\kappa_a = 80$ whole bytes: all κ_a bits used; R_0 is the MSB
$a \leftarrow \mathsf{CLZ}(R) + 1$ ▷ exponent $= 1 +$ (#leading zeros); $a = \kappa_a + 1$ if R all-zero
$(ctx, \rho_b) \leftarrow \mathsf{XOF.Squeeze}(ctx, \lceil \kappa_b/8 \rceil)$
$M \leftarrow \mathsf{BytesToBits}(\rho_b, \lceil \kappa_b/8 \rceil)[0{:}\kappa_b - 1]$
$m \leftarrow \sum_{i=0}^{\kappa_b - 1} M_i \cdot 2^{\kappa_b - 1 - i}$ ▷ MSB-first top κ_b bits; mantissa $b = 1 + m/2^{\kappa_b} \in [1, 2)$
$\ell \leftarrow \mathsf{ApproxLog}(a, m)$ ▷ ApproxLog derives j, x from m; $\ell \approx \log_2(2^{-a} b)$
return (ctx, ℓ)

When R is not the all-zero string, the probability that SamplerU outputs a particular exponent $a = j$ is

$$\Pr[a = j] = \Pr[R_0 = \cdots = R_{j-2} = 0 \ \wedge \ R_{j-1} = 1] = 2^{-j}.$$

The all-zero event has probability $2^{-\kappa_a}$, which is exactly the probability mass of the ideal geometric tail $a > \kappa_a$. Thus, the exponent sampler preserves the probability mass of the entire tail, although all tail values are represented by the single exponent $a = \kappa_a + 1$.

The pair (a, b) is an implicit representation of $u = 2^{-a} b$. The transition algorithm only uses $\log_2 u$ in comparisons (it forms $u = 2r^2 \ln 2 \cdot \ell$ on line 2 of Algorithm 22), and therefore it can directly use the value $\ell = \mathsf{ApproxLog}(a, m)$ returned by SamplerU, without explicitly computing u.

Up to the rounding errors introduced by the underlying fixed-point arithmetic, the implemented logarithm is

$$\mathsf{ApproxLog}(a, m) = P_j(x) - a,$$

where j and x are the segment index and normalized argument derived from $b = 1 + m/2^{\kappa_b}$. Because the exponent contributes the *exact* integer $-a$, the pointwise absolute error is

$$\begin{aligned} \Delta_{a,b} &:= \log_2(2^{-a} b) - \mathsf{ApproxLog}(a, m) \\ &= \big(\log_2 b - P_j(x)\big) + \Delta_{\mathrm{arith}}(a, m), \end{aligned}$$

where $\Delta_{\mathrm{arith}}(a, m)$ denotes the accumulated fixed-point rounding error, with its sign absorbed into the definition. The base-two formulation removes the term $-a(\log 2 - \widehat{L}_2)$ that a natural-logarithm approximation would carry, and with it the amplifying factor $a \le \kappa_a + 1$; no $\log 2$-approximation error enters.

If

$$\max_{0 \le j < 2^g} \sup_{x \in [0,1)} \left| \log_2(\beta_j + x\, 2^{-g}) - P_j(x) \right| \le \delta_{\mathrm{poly}}$$

and

$$|\Delta_{\mathrm{arith}}(a, m)| \le \delta_{\mathrm{arith}},$$

then, uniformly in a,

$$|\Delta_{a,b}| \le \delta_{\mathrm{poly}} + \delta_{\mathrm{arith}}.$$

We now check that the chosen κ_a leaves every comparison performed by Algorithm 22 unaffected by the tail collapse. After dividing the boundary $\|y\|_2^2 - \|y + mv\|_2^2 = -2mt - m^2 V$ by $2r^2$ and exponentiating, the test on $u = 2r^2 \ln U$ against this boundary is exactly the test on the raw uniform $U = 2^{-a} b$ against the Gaussian ratio

$$a_m = \exp\left(\frac{-2mt - m^2 V}{2r^2} \right), \qquad t = \langle y, v \rangle, \quad m = 0, 1, \ldots, N.$$

Thus the comparison thresholds used in a transition step are precisely $a_0, \ldots, a_N$, and since these ratios are decreasing in m, the smallest one is a_N. By the tail-collapse condition above, the collapse $U < 2^{-\kappa_a} \mapsto a = \kappa_a + 1$ leaves every comparison result unchanged provided

$$a_N \geq 2^{-\kappa_a} \quad \Longleftrightarrow \quad 2N\,t + N^2 V \leq \kappa_a\,(\ln 2)\,2r^2,$$

With the $N+1 = 30$ retained shifts $m = 0, \ldots, N$ (the implemented test's range), the dominant term is $N^2 V = 841V$. With $r = 825$ ($2r^2 = 1{,}361{,}250$) and the largest admissible shift norm $\|v\|_2 \leq 296.19$ (the largest of the three B_k, attained at SUF-256; $V = 87728.5$), this term alone contributes $841V/(2r^2) \approx 54.20$, i.e. $a_{29} \approx 2^{-78.2}$ at $t = 0$, already about 1.8 bits above 2^{-80}. More generally, the condition above is an upper bound on the (sign-normalized, hence nonnegative) inner product $t = \langle y, v \rangle$:

$$\kappa_a = 79 : \; t \lesssim 1.03 \times 10^5, \qquad \kappa_a = 80 : \; t \lesssim 1.20 \times 10^5,$$

the latter corresponding to an in-direction component $\|y_{\|v}\|_2 \lesssim 404$. This bound need not hold on every reachable state; when it fails the collapse merges only intervals whose Gaussian ratio is already below $2^{-\kappa_a}$, perturbing the transition probability by at most $2 \cdot 2^{-80}$ (absorbed by the budget), while the smallest retained threshold a_{29} still exceeds 2^{-80} at $t = 0$. We therefore set $\kappa_a = 80$, which covers all 29 comparisons of Algorithm 22 with one bit of margin beyond the minimal requirement, at no additional randomness cost since $\lceil 80/8 \rceil = \lceil 79/8 \rceil = 10$ bytes. The segmentation g, the per-segment degree d, the mantissa precision κ_b, and the fixed-point scale Q are selected so that $|\Delta_{a,b}| \leq 2^{-57}$ for every non-tail input used in a transition comparison. With the values fixed above ($g = 2$, $d = 13$, $Q = 62$, coefficients in Table 1) the realized bound is $|\Delta_{a,b}| \leq 2^{-59.37}$, i.e. more than two bits below the budget $\eta_{\log} = 2^{-57}$.

We set $\kappa_b = 57$. This choice satisfies two requirements simultaneously. First, the resulting function ApproxLog must be *strictly increasing* with respect to the represented value $u = 2^{-a}b$ and satisfy $\mathsf{ApproxLog}(0, 1) = 0$, as required by Theorem 2. The latter holds exactly because $c_{0,0} = 0$ (segment $j = 0$, $x = 0$ maps to 0). For monotonicity, the increase in $\log_2 b$ across one mantissa step, $\approx 2^{-\kappa_b}/(b \ln 2)$, must exceed the only non-monotone effect—the cross-segment fixed-point rounding wobble, at most two units in the last place of the Q62 output, i.e. $\leq 2^{-61}$; this holds for every $\kappa_b \leq 61$, and at $\kappa_b = 57$ with more than three bits of margin, so ApproxLog is strictly increasing on the dyadic inputs $b \in \mathcal{B}_{\kappa_b}$. Second, the mantissa granularity $2^{-\kappa_b}$ is accounted for as an explicit absolute-probability term $\varepsilon_{\mathrm{q}} \leq 2^{-58}$ in M_5 (Section 4.1.4): U' is uniform on a grid of spacing $2^{-a-\kappa_b}$ with $a \geq 1$, whose Kolmogorov distance to $\mathcal{U}(0, 1]$ is $2^{-\kappa_b - 1} = 2^{-58}$. The mantissa block then occupies $\lceil \kappa_b/8 \rceil = 8$ bytes.

2.5.9 Gaussian Sampler Specification

The wide Gaussian sampling driven by SampleY produces independent samples from the centered discrete Gaussian distribution $D_{\mathbb{Z},r}$ over the integers, with the per-candidate convolution kernel SampleDGauss (Algorithm 28). All three SUF parameter sets share a single wide standard deviation $r = 825$ (Table 10), so one wide-Gaussian sampler instance serves every mode. We adopt the BLISS-style convolution sampler of [17], in the variant formalized as Algorithm 12 of [43]: a wide Gaussian over $\mathbb{Z}$ is realized as $z = k \cdot x + y$, where x is drawn from a small one-sided Gaussian via a cumulative distribution table and y is drawn uniformly from $\{0, \ldots, k-1\}$, followed by an exponential rejection step. Throughout this section we fix the convolution multiplier

$$k = 256, \qquad \sigma_s \;=\; r/k \;=\; 825/256 \;=\; 3.22265625.$$

Since r is identical across all modes, the wide sampler – including its cumulative distribution table – is shared by SHUTTLE-SUF-128, SHUTTLE-SUF-256, and SHUTTLE-SUF-512.

Choice of the small standard deviation. For the fixed wide standard deviation $r = 825$, the convolution parameters (σ_s, k) trade off two competing costs. A larger σ_s enlarges the cumulative distribution table that backs the small-Gaussian sampler, while a smaller σ_s (obtained from a larger k) raises the worst-case exponent in the rejection test and thereby lowers the average acceptance rate. Sweeping $k \in \{256, 512, 1024\}$ for $r = 825$ yields the candidate pairs $\sigma_s \in \{3.22265625, 1.61132813, 0.80566406\}$ with average acceptance rates 89.0%, 80.2%, and 66.9%, respectively. We therefore select $k = 256$ (a power of two enabling a cheap bit-shift decomposition of y), giving $\sigma_s = 3.22265625$ and the highest acceptance rate.

Cumulative distribution tables. We precompute a single reverse cumulative distribution table RCDT of length L for the wide sampler, where L is the truncation index. Truncating at 11 standard deviations of σ_s gives $L = 36$ (since $\lceil 11\,\sigma_s \rceil = \lceil 35.45 \rceil = 36$). Each entry is stored with $\theta = 96$ bits of precision and represents

$$\mathrm{RCDT}[i] \;=\; \left\lfloor 2^{\theta} \cdot \Pr_{X \sim D_{\mathbb{Z}^+,\sigma_s}} \big[X \geq i+1\big] \right\rfloor, \qquad i = 0, \ldots, L-1.$$

The values are listed in Table 4 as triples of 32-bit limbs, so that the θ-bit number is recovered as $\mathrm{RCDT}[i] = \sum_{j=0}^{2} \mathrm{limb}_j \cdot 2^{32j}$.

Base sampler. Algorithm 27 samples a non-negative integer x approximately distributed as $D_{\mathbb{Z}^+,\sigma_s}$ by a constant-time scan over the precomputed RCDT. The construction is the natural transposition of the BaseSampler of [43, 25] (and Algorithm 12 of [25, Section 3.9]-style Falcon documentation) to the table precision θ bits fixed for each table: $\theta = 96$ for both the wide signing table (Table 4) and the secret-key tables (Tables 5–7). Rather than drawing its own randomness, the base sampler receives θ uniform bits as a $\lceil \theta/8 \rceil$-byte string ρ_u supplied by the caller; it converts ρ_u to the integer $u = \mathsf{BytesToInteger}(\rho_u, \lceil \theta/8 \rceil) \bmod 2^{\theta}$, compares u against every table entry, and counts the indices for which $u < \mathrm{RCDT}[i]$; the count is the returned sample. To guarantee constant-time execution, the comparison is implemented via branchless limb comparisons that propagate a carry across the limb boundary (three 32-bit limbs in all tables).

Algorithm 27 $\mathsf{BaseSampler}(\mathrm{RCDT}, \rho_u)$

Input: The table RCDT of length L at precision θ bits, and a uniform string $\rho_u \in \mathbb{B}^{\theta}$ supplied by the caller ($\theta = 96$ for both the wide table and the secret-key tables).
Output: An integer $x \in \{0, \ldots, L\}$ approximately distributed as $D_{\mathbb{Z}^+,\sigma_s}$.
$u \leftarrow \mathsf{BytesToInteger}(\rho_u, \lceil \theta/8 \rceil) \bmod 2^{\theta}$ ▷ θ-bit uniform integer
$x \leftarrow 0$
for $i = 0$ to $L - 1$ **do**
    $x \leftarrow x + [\![u < \mathrm{RCDT}[i]]\!]$ ▷ constant-time θ-bit comparison
end for
return x

Batched limb-major input layout. When a block of G base-sampler inputs is squeezed at once—as in $\mathsf{ExpandS}$ and $\mathsf{SampleDGauss}$—the $12G$ magnitude bytes are read in the grouped, limb-major order of the vectorized implementation, *not* as G contiguous 12-byte strings. The batch is partitioned into groups of 8 candidates; within group γ (byte base 96γ), candidate $s \in \{0, \ldots, 7\}$ (batch index $j = 8\gamma + s$) takes the three little-endian 32-bit limbs of its 96-bit input $\rho_u^{(j)}$ from byte offsets $96\gamma{+}4s$, $96\gamma{+}4s{+}32$, and $96\gamma{+}4s{+}64$, so that $\rho_u^{(j)} = \sum_{i=0}^{2} \mathrm{limb}_i\, 2^{32i}$ is then consumed exactly as in Algorithm 27.

Discrete Gaussian sampler. The per-candidate kernel SampleDGauss (Algorithm 28) realizes the BLISS-style convolution [43, Algorithm 12]. Given a small one-sided Gaussian $x \sim D_{\mathbb{Z}^+,\sigma_s}$ from BaseSampler, a uniform low part $y \in \{0,\dots,k-1\}$, a 64-bit Bernoulli word u, an output sign bit b, and an independent candidate fold bit f, it forms the candidate $z = kx+y$ and accepts it with probability $\exp(-y(y+2kx)/(2r^2))$ through the test $u < \widehat{p}$, where $\widehat{p} = \mathsf{ApproxExp}(x,y) \approx 2^{64}\exp(\cdot)$ compensates exactly for the convolution. Since $k = 256$, y is one byte. The fold correction uses f: an accepted $z = 0$ is dropped iff $f = 1$, halving the zero mass without conditioning on the output sign, and only candidates with $a = 1$ are emitted by SampleY. Under ideal base sampling and exact exponential acceptance, these emitted signed values follow $D_{\mathbb{Z},r}$; truncation and finite-precision errors are treated separately below. The inputs x, y, u, f come from the candidate mini-batch, while b comes from the separate output-indexed sign prefix. The kernel itself draws no randomness.

Algorithm 28 $\mathsf{SampleDGauss}(x, y, u, b, f)$

1: **Input:** magnitude x, low part $y \in \{0,\dots,k-1\}$, Bernoulli word $u \in \{0,\dots,2^{64}-1\}$, sign bit b, fold bit f; $k = 256$
2: **Output:** accept flag $a \in \{0,1\}$ and signed candidate z_{out}
3: $z \leftarrow kx+y$; $\quad \widehat{p} \leftarrow \mathsf{ApproxExp}(x,y)$ $\quad \triangleright\ \widehat{p} \approx 2^{64}\exp\big(-y(y+2kx)/(2r^2)\big)$
4: $a \leftarrow [\![u < \widehat{p}]\!] \wedge \neg(z = 0 \wedge f = 1)$ $\triangleright$ Bernoulli accept, then fold zero with an independent bit
5: **return** $(a,\ (-1)^b z)$

Correctness of independent zero folding. Work in the independent-uniform-coins/XOF model and first assume an untruncated, exact base Gaussian and exact exponential acceptance. For $v = kx+y \geq 0$, the unnormalised mass after exponential acceptance is

$$e^{-x^2/(2\sigma_s^2)} e^{-y(y+2kx)/(2r^2)} = e^{-v^2/(2r^2)}, \qquad \sigma_s = r/k.$$

Let $h(v) = e^{-v^2/(2r^2)}$ and $C = h(0)/2 + \sum_{v\geq 1} h(v)$. The fresh independent fold bit leaves accepted magnitude masses $h(0)/(2C)$ at zero and $h(v)/C$ for $v > 0$. Acceptance is independent of the fixed output sign b, so applying that fair sign gives mass $h(|z|)/(2C)$ at every $z \in \mathbb{Z}$, including zero. Since $2C = \sum_{z\in\mathbb{Z}} h(|z|)$, this is exactly $D_{\mathbb{Z},r}$. Thus retaining b across retries is valid only because f is refreshed for every candidate and the acceptance test does not use b. For the table-driven, finite-precision sampler the same construction preserves sign symmetry; its deviation from the ideal Gaussian still requires the truncation and numerical-error analysis.

Test-vector update. Separating the fold and sign bits changes the random-byte schedule and hence the seed-to-mask mapping. Implementations must use the new fold bytes and the specified initial/refill squeeze boundaries. Signing KATs and backend consistency checks must be regenerated for this schedule; earlier KATs and implementation audit results do not validate the revised sampler.

ApproxExp. The exponential rejection in Algorithm 28 is evaluated by the integer-only, branchless routine $\mathsf{ApproxExp}(x,y)$ of Algorithm 29, which returns a Q64 fixed-point approximation $\widehat{p} \in \{0,\dots,2^{64}-1\}$ of $2^{64}\exp(p)$, where $p = -y(y+2kx)/(2r^2) \leq 0$ is the log-domain exponent of the candidate. With $k = 256$ the exponent numerator is the integer $N = y(y+512x)$, and enumerating the $37 \cdot 256 = 9472$ admissible inputs $(x,y) \in \{0,\dots,36\} \times \{0,\dots,255\}$ gives $p \in [-3.5006, 0]$ and $N \leq 255 \cdot (255 + 512 \cdot 36) = 4{,}765{,}185 < 2^{23}$.

Principle. Rather than approximating $\exp(p)$ directly on the full range (which would need a degree-31 polynomial with coefficients exceeding 64 bits), we split the exponent into 128 equal parts. Writing

$$s = \frac{N}{2^{23}} \in [0,1), \qquad a = \frac{2^{23}}{128 \cdot 2r^2} = 0.0481439853\ldots,$$

one has the exact identity $\exp(p) = \big(\exp(-as)\big)^{128}$. Because $0 \le s < 1$ and a is small, the small factor $\exp(-as)$ is approximated by its degree-8 truncated Taylor series

$$q(s) = \sum_{i=0}^{8} \frac{(-a)^i}{i!}\, s^i,$$

evaluated in Q64 by Horner's rule, after which seven successive squarings reconstruct $q(s)^{128} \approx \exp(p)$. The routine has no secret-dependent branch, no runtime division (the scaling by 2^{23} is a bit shift), and no table access.

Fixed-point semantics. Coefficients $C_0, \ldots, C_8$ are the Q64 integers $C_i = \mathrm{round}\big(2^{64}(-a)^i/i!\big)$ for $i \ge 1$, with the constant term pinned to $C_0 = 2^{64} - 1$; their normative values are listed in Table 2. The residual s is held as the Q63 integer $s = N \ll 40$ (exact, since $s/2^{63} = N/2^{23}$). All wide products consume only the high half of the $64 \times 64 \to 128$ result: we write

$$\mathrm{mul}_{64,64}(u,w) = \left\lfloor \tfrac{uw}{2^{64}} \right\rfloor \quad (\text{Q64}\times\text{Q64} \to \text{Q64}), \qquad \mathrm{mul}_{64,63}(u,w) = 2\left\lfloor \tfrac{uw}{2^{64}} \right\rfloor \quad (\text{Q64}\times\text{Q63} \to \text{Q64}),$$

so the Horner chain (8 multiply–adds) and the seven squarings together use 15 high-half multiplications.

Algorithm 29 ApproxExp(x, y) – branchless fixed-point acceptance probability

Input: $x \in \{0, \ldots, 36\}$ (the BaseSampler output) and the uniform low part $y \in \{0, \ldots, 255\}$ $(k = 256)$.
Output: Q64 integer $\widehat{p} \in \{0, \ldots, 2^{64} - 1\}$ with $\widehat{p}\, 2^{-64} \approx \exp\big(-y(y + 512x)/(2r^2)\big)$.
$N \leftarrow y\,(y + 512\,x)$ ▷ $512 = 2k$; exponent numerator $N \in \{0, \ldots, 255 \cdot (255 + 512 \cdot 36)\}$
$s \leftarrow N \ll 40$ ▷ Q63 encoding of $s = N/2^{23} \in [0, 1)$; exact, 2^{23} a power of two
$v \leftarrow C_8$ ▷ Q64 Horner accumulator; coefficients $C_0, \ldots, C_8$ from Table 2
for $i = 7$ **down to** 0 **do**
    $v \leftarrow C_i + \mathrm{mul}_{64,63}(v, s)$ ▷ Horner step; after the loop $v \approx 2^{64}\, q(s) = 2^{64} \exp(-as)$
end for
for $i = 1$ **to** 7 **do**
    $v \leftarrow \mathrm{mul}_{64,64}(v, v)$ ▷ seven squarings: $v \approx 2^{64}\, q(s)^{128} = 2^{64} \exp(p)$
end for
return $\widehat{p} \leftarrow v$

Table 2: Normative Q64 Horner coefficients of ApproxExp (Algorithm 29). Each coefficient is the rounded fixed-point Taylor coefficient $C_i = \mathrm{round}(2^{64}(-a)^i/i!)$ for $i \ge 1$, with the constant term pinned to $C_0 = 2^{64} - 1$, where $a = 2^{23}/(128 \cdot 2 \cdot 825^2)$. This closed form fully determines the table; no other data is required to reproduce it.

i	C_i
0	$2^{64} - 1 = 18446744073709551615$
1	−888099775658129789
2	21378331275493551
3	−343079355609610
4	4129301863954
5	−39760209654
6	319035825
7	−2194237
8	13205

Accuracy. Because the input domain is finite, the accuracy is established by exhaustive enumeration rather than a worst-case bound: evaluating Algorithm 29 at every one of the 9472 admissible pairs (x, y) and comparing the fixed-point output $\widehat{p}\,2^{-64}$ against a high-precision reference for $\exp(p)$ gives the worst-case accept-branch relative error

$$\eta = \max_{x,y} \left| \frac{\widehat{p}\,2^{-64} - \exp(p)}{\exp(p)} \right| = 3.9644 \cdot 10^{-17} \approx 2^{-54.49},$$

attained at $(x, y) = (36, 248)$, comfortably above the 53-bit design target. As shown below, this is precisely the quantity that condition (C1) constrains, and it leaves about 1.5 bits of margin against the binding threshold of Table 3. Since the coefficients of Table 2 follow the stated closed form and the enumeration ranges over the entire input domain, both the coefficients and this precision figure are fully reproducible from this specification alone.

Distribution quality. The output of Algorithm 28 differs from the ideal distribution $D_{\mathbb{Z},r}$ only through (i) the truncation of RCDT, (ii) the θ-bit rounding of the table entries, and (iii) the finite-precision evaluation of the rejection probability. We bound this gap by specializing the isochronous-sampling analysis of [25, Section 5] to the *emitted-sample* distribution, which is all that signing ever reveals. Throughout, $\mathsf{ApproxExp}(x) \in [0, 1]$ denotes the realized acceptance probability $\widehat{p}\,2^{-64}$ of Algorithm 29 viewed as a function of the log-domain exponent $x = p \le 0$, and $\epsilon_{\exp}(x) = \mathsf{ApproxExp}(x) - \exp(x)$ is its absolute error.

Theorem 1 (Concrete wide sampler)**.** *Let λ denote the target security level of Table 10 (128/256/512), let $Q_{\exp}$ be the number of* ApproxExp *evaluations and Q_{bs} the number of* BaseSampler *invocations made over the lifetime of the key, and let*

$$\eta = \max_{x<0} \left| \frac{\epsilon_{\exp}(x)}{\exp(x)} \right|$$

be the accept-branch *relative error of* ApproxExp*. Suppose the following two conditions hold:*

$$\frac{2\,\eta}{1-\eta} \;\le\; \frac{1}{\sqrt{2\,(2\lambda-1)\,Q_{\exp}}}, \tag{C1}$$

$$R_{2\lambda+1}\big(\mathsf{BaseSampler} \,\big\|\, D_{\mathbb{Z}^+,\sigma_s}\big) \;\le\; 1 + \frac{1}{4\,Q_{\mathrm{bs}}}. \tag{C2}$$

Then replacing the ideal sampler $D_{\mathbb{Z},r}$ by Algorithm 28 costs at most two bits of security for the underlying search problem – one bit per condition.

Proof. The two error sources are independent and we charge one bit to each.

The BaseSampler step is exactly the Rényi-divergence case of [25, Theorem 7]: under condition (C2) the order-$(2\lambda+1)$ divergence between the table-driven half-Gaussian and the ideal $D_{\mathbb{Z}^+,\sigma_s}$ is at most $1 + 1/(4Q_{\mathrm{bs}})$, so probability preservation over the Q_{bs} samples loses one bit.

For ApproxExp, observe that Algorithm 28 is a rejection sampler: it emits a candidate z with probability proportional to $q(z)\exp(a(z))$, where $q(z)$ is the ApproxExp-independent proposal mass (the convolution of BaseSampler with the uniform low part, together with the fold and sign corrections) and $\exp(a(z))$ with $a(z) \le 0$ is the ideal acceptance probability [25, Lemma 8]. The implementation replaces $\exp(a(z))$ by $\mathsf{ApproxExp}(a(z)) = \exp(a(z))\,(1+\theta_z)$ with $|\theta_z| \le \eta$ by the definition of η. Hence the emitted distribution is the ideal one reweighted by $1+\theta_z$ and renormalized by the proposal-average $1+\bar{\theta}$ of the same factors, so each emitted point carries relative error

$$\left| \frac{1+\theta_z}{1+\bar{\theta}} - 1 \right| \;\le\; \frac{2\,\eta}{1-\eta},$$

which is exactly the quantity bounded by condition (C1). Crucially, only the *emitted* sample z leaves the constant-time sampler: rejected candidates are resampled internally, and the accept/reject bit is neither part of a signature nor a function of the long-term secret. Consequently the reject-branch ratio $\left|\epsilon_{\exp}(x)/(1-\exp(x))\right|$ – which the stronger Condition (1) of [25, Theorem 7] would *also* have to bound, and which blows up as $x \to 0^-$ since $1-\exp(x) \approx |x| \to 0$ – never needs to be controlled. With the per-output distortion $2\eta/(1-\eta)$ in place of η, probability preservation [25, Lemma 11] over the $Q_{\exp}$ emitted samples loses one further bit, for a total of two. □

It is precisely the improved Condition (C1) that distinguishes our analysis from [25, Theorem 7]: the original two-sided bound (the inner max over both $|\epsilon_{\exp}/\exp|$ and $|\epsilon_{\exp}/(1-\exp)|$) forces an unattainably tight reject-branch precision near $x = 0$, whereas the emitted-sample argument above retains only the accept-branch error η. Condition (C1) therefore controls the precision of the exponential used in the rejection test, while condition (C2) – the focus of the remainder of this paragraph – is precisely the Rényi-divergence requirement on the table-driven BaseSampler that backs the wide convolution sampler.

Query budget. The NGCC call caps the number of signatures per key at 2^{80}. Each signature draws the masking vector $\mathbf{y} \in R^{l+m+1}$, i.e. $n\,(l+m+1)$ wide-Gaussian samples, and each wide-Gaussian sample issues on average $1/p_{\text{acc}}$ BaseSampler calls because of the exponential rejection. Our ApproxExp-based rejection accepts with average probability $p_{\text{acc}} \approx 0.89$; to stay conservative we budget with $p_{\text{acc}} = 0.8$ (the value hard-coded in the parameter scripts). Hence

$$Q_{\text{bs}} \;=\; Q_{\exp} \;=\; \frac{2^{80} \cdot n\,(l+m+1)}{0.8},$$

which we tabulate together with the resulting threshold $1 + 1/(4Q_{\text{bs}})$ of condition (C2) in Table 3.

Table 3: Query budget and the Rényi-divergence threshold of condition (C2). Here λ is the target security level of Table 10 (128/256/512); the wide sampler is shared, so the binding requirement is the smallest threshold (SUF-512, $\lambda = 512$).

Mode	λ	$n\,(l+m+1)$	Q_{bs}	$\frac{1}{4Q_{\text{bs}}}$ (C2)	$\frac{1}{\sqrt{2(2\lambda-1)Q_{\exp}}}$ (C1)
SHUTTLE-SUF-128	128	$256 \cdot 7$	$2^{91.13}$	$2^{-93.13}$	$2^{-50.06}$
SHUTTLE-SUF-256	256	$512 \cdot 6$	$2^{91.91}$	$2^{-93.91}$	$2^{-50.95}$
SHUTTLE-SUF-512	512	$1024 \cdot 6$	$2^{92.91}$	$2^{-94.91}$	$2^{-51.95}$

Verification of condition (C2). Because the wide standard deviation $r = 825$ is shared by all three modes, the wide sampler must meet the strictest (smallest) threshold in Table 3, namely $1 + 2^{-94.91}$ at the SUF-512 security level ($\lambda = 512$). The Rényi order in condition (C2) is $a = 2\lambda + 1$ (for SUF-512, $a = 1025$). For the precomputed table of Table 4 at $\theta = 96$ bits, a direct numerical evaluation at this order gives

$$R_{2\lambda+1}\big(\mathsf{BaseSampler}_{\sigma_s=3.22265625} \,\big\|\, D_{\mathbb{Z}^+,\sigma_s}\big) \;=\; 1 + 2^{-95.40} \;<\; 1 + 2^{-94.91},$$

so condition (C2) holds with roughly half a bit of margin (the order-∞ divergence is $1+2^{-89.55}$).

Verification of condition (C1). The pinned ApproxExp of Algorithm 29 has accept-branch relative error $\eta \le 2^{-54.49}$ (enumerated over all 9472 inputs; see the ApproxExp paragraph above), so the emitted-sample distortion is $2\eta/(1-\eta) \le 2^{-53.49}$. This is below the binding (smallest) threshold $1/\sqrt{2(2\lambda-1)Q_{\exp}} = 2^{-51.95}$ at the SUF-512 level in the last column of Table 3, so condition (C1) holds at all three levels with about 1.5 bits of margin. Consequently the concrete wide sampler can replace the ideal $D_{\mathbb{Z},r}$ in the security reduction at a cost of at most two bits for the search problem (Theorem 1).

Table 4: Reverse cumulative distribution table for the shared wide sampler $\sigma_s = 825/256 = 3.22265625$ ($\theta = 96$, $L = 36$, identical for SHUTTLE-SUF-128/256/512). $\mathrm{RCDT}[i] = \mathrm{lo} + \mathrm{mid} \cdot 2^{32} + \mathrm{hi} \cdot 2^{64}$. The 36 entries are listed in two side-by-side blocks ($i = 0$–17 on the left, $i = 18$–35 on the right).

i	lo	mid	hi	i	lo	mid	hi
0	0x402F7FA2	0x12C69654	0xC7999458	18	0x13DE9A9D	0x89A793B6	0x0000001F
1	0x8E7FB711	0xE6782481	0x91D9D07A	19	0xBA01FE36	0xBF71FA38	0x00000004
2	0x08E4AE6C	0x9C4E92B4	0x63548C39	20	0x7C388C2B	0xA673B059	0x00000000
3	0xF198871F	0x153DAC5B	0x3EC320F8	21	0xA0C47B37	0x14BB85E9	0x00000000
4	0x5B379FE3	0xC0D5F1C9	0x24A7FB57	22	0x82A7DEC0	0x02592B23	0x00000000
5	0xA2890D59	0x0A373A8C	0x13BAD6C5	23	0xDC786C9A	0x003DE9B7	0x00000000
6	0x85ECF03B	0x77F6EF00	0x09C34493	24	0xF203DEDB	0x0005CC00	0x00000000
7	0x06BD7625	0x657531BD	0x046EB351	25	0xC474C30E	0x00007E4E	0x00000000
8	0x0AE5CC87	0xF6A4CE61	0x01D7ED5B	26	0x85F8ADDF	0x000009C5	0x00000000
9	0x8E8C4290	0x80C6FB16	0x00B39140	27	0xE4013000	0x000000AF	0x00000000
10	0x945A8C25	0xFC91CEB4	0x003E7104	28	0x3D57DDC7	0x0000000B	0x00000000
11	0xFFE7F159	0xE3BC50BA	0x0013D35D	29	0xA7163F8F	0x00000000	0x00000000
12	0xDBA3E1F9	0x799C55C0	0x0005BE50	30	0x08D11741	0x00000000	0x00000000
13	0x16FF3692	0x1CB2C576	0x00018465	31	0x006C385F	0x00000000	0x00000000
14	0x654B748F	0x2018307D	0x00005D81	32	0x0004B6EA	0x00000000	0x00000000
15	0xCAAE107A	0xB2CD4BA0	0x00001481	33	0x00002FC3	0x00000000	0x00000000
16	0x5842B889	0x6D1B1765	0x00000418	34	0x000001B6	0x00000000	0x00000000
17	0x030C33B1	0x9BDF6112	0x000000BE	35	0x0000000D	0x00000000	0x00000000

Secret-key RCDT tables. As described in Section 2.1, the secret vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ are sampled directly via BaseSampler with the small standard deviations σ_1 and σ_2 listed in Table 10; across the three modes these take the three distinct values $\sigma \in \{0.85, 0.9, 1\}$, for each of which a separate RCDT table is precomputed at precision $\theta = 96$ with 3×32-bit limbs:

$$\mathrm{RCDT}[i] = \mathrm{lo} + \mathrm{mid} \cdot 2^{32} + \mathrm{hi} \cdot 2^{64}.$$

This packing is *identical* to that of the wide signing table (Table 4), which also uses $\theta = 96$ and 32-bit limbs, so the same reconstruction formula applies to all tables. The hex values in Tables 5–7 are normative. Unlike the wide signing sampler, the secret-key sampler is invoked only at key generation, and its output never feeds the per-signature Rényi-divergence argument of condition (C2). The relevant quality metric is therefore not the Rényi divergence but the closeness of the *induced* standard deviation to its target, since the MLWE hardness depends on the secret distribution essentially through its width. Accordingly we measure quality by the absolute deviation of the induced standard deviation from the ideal value (after the sign flip and zero-fold correction). As reported in the captions below, these deviations are only $2^{-15.98}$, $2^{-18.22}$, and $2^{-23.17}$ for $\sigma = 0.85$, 0.9, and 1 respectively—far too small to have any measurable effect on the MLWE security estimates of Table 10.

2.5.10 Entropy Coding via Range Asymmetric Numeral Systems (rANS)

Source model. The signature transmits the triple $(seed_c, \mathbf{z}_1, \mathbf{h})$, and each component is coded according to its own statistics, following one principle: split every quantity into a near-uniform part that is stored verbatim and a structured part that carries the entropy and is rANS-coded, while genuinely uniform objects are stored in their information-theoretic minimum.

After CompressY, the response $\mathbf{z}_1 = (z_0, \mathbf{z}_s)$ is statistically close to a centered rounded Gaussian. With the normalizer $Z_r = \sum_{y \in \mathbb{Z}} \exp\!\left(-y^2/2r^2\right)$, the per-coefficient laws of the two

Table 5: RCDT for $\sigma = 0.85$ ($\theta = 96$, $L = 9$; $\mathrm{RCDT}[i] = \mathrm{lo} + \mathrm{mid} \cdot 2^{32} + \mathrm{hi} \cdot 2^{64}$). Induced $\sigma_{\mathrm{actual}} = 0.849984$, $|\sigma_{\mathrm{actual}} - \sigma| = 2^{-15.98}$.

i	lo (hex)	mid (hex)	hi (hex)
0	0xC3CB73B8	0xF67BCC8D	0x5C747AA6
1	0x110C8F76	0xC2A05C28	0x0A978F6E
2	0x9AB7E17D	0x2DDCC90A	0x00533DE7
3	0xBC9B329A	0xE0213A50	0x0000A6CA
4	0xB024ADB1	0x1B1BFD11	0x00000054
5	0xC041BCDF	0x0AA35023	0x00000000
6	0xDD3862DD	0x00005653	0x00000000
7	0xAF89D7B3	0x00000000	0x00000000
8	0x0000596F	0x00000000	0x00000000

Table 6: RCDT for $\sigma = 0.9$ ($\theta = 96$, $L = 10$; $\mathrm{RCDT}[i] = \mathrm{lo} + \mathrm{mid} \cdot 2^{32} + \mathrm{hi} \cdot 2^{64}$). Induced $\sigma_{\mathrm{actual}} = 0.899997$, $|\sigma_{\mathrm{actual}} - \sigma| = 2^{-18.22}$.

i	lo (hex)	mid (hex)	hi (hex)
0	0x63C3A91B	0x73D08C99	0x62C00D07
1	0x3E38EB38	0x9625D715	0x0DEDB092
2	0x53FCB3E2	0xE73FB18A	0x009DB3B9
3	0x1CCE8231	0xBAF2F8FA	0x00021365
4	0x81920B96	0x79103336	0x0000020C
5	0x73DC07FB	0x96E7AD8B	0x00000000
6	0x9BDF1816	0x000CA3F6	0x00000000
7	0xE1610238	0x0000004E	0x00000000
8	0x008F3A79	0x00000000	0x00000000
9	0x0000004B	0x00000000	0x00000000

Table 7: RCDT for $\sigma = 1$ ($\theta = 96$, $L = 11$; $\mathrm{RCDT}[i] = \mathrm{lo} + \mathrm{mid} \cdot 2^{32} + \mathrm{hi} \cdot 2^{64}$). Induced $\sigma_{\mathrm{actual}} = 0.9999999$, $|\sigma_{\mathrm{actual}} - \sigma| = 2^{-23.17}$.

i	lo (hex)	mid (hex)	hi (hex)
0	0x2BFDBDFC	0xB7D318D4	0x6DFDA4E6
1	0x2AA2B001	0xB85F106C	0x156E867A
2	0x15454C5F	0x1625B451	0x01ABEA39
3	0x26C842F8	0xE66F73EE	0x000CADCC
4	0xB773BC08	0x4710A6BD	0x000023CE
5	0x0F92A8AD	0x5D28DCBB	0x00000025
6	0xD8D1C6D8	0x0E5DF25B	0x00000000
7	0xB536290E	0x00020893	0x00000000
8	0x1CBE1E4D	0x0000001B	0x00000000
9	0x0084FE20	0x00000000	0x00000000
10	0x000000EF	0x00000000	0x00000000

blocks are

$$\Pr[z_0 = k] = \frac{1}{Z_r} \sum_{y:\, \lfloor y/\alpha_1 + 1/2 \rfloor = k} \exp\Big(-\tfrac{y^2}{2r^2}\Big), \tag{2}$$

$$\Pr[z_{s,i} = k] = \frac{1}{Z_r} \sum_{y:\, \lfloor y/\alpha_s + 1/2 \rfloor = k} \exp\Big(-\tfrac{y^2}{2r^2}\Big), \tag{3}$$

so their effective standard deviations are r/α_1 and r/α_s, respectively; the omitted block $\mathbf{z}_2$ obeys the same law with divisor α_e and effective deviation r/α_e. These deviations differ substantially (see Table 10): z_0 is narrow, whereas $\mathbf{z}_s$ and $\mathbf{z}_2$ are wide—the spread the coder exploits.

For a single coefficient, let w be the corresponding coefficient of the commitment $\mathbf{w}$. By MakeHint, the canonical hint coefficient is

$$h = \left(\left\lfloor \frac{w}{\alpha_h} \right\rceil - \left\lfloor \frac{w - 2z_2}{\alpha_h} \right\rceil \right) \bmod H \in [0, H), \qquad H = \frac{2(q-1)}{\alpha_h},$$

which depends on z_2 only through $2z_2$. Modelling the phase $w \bmod \alpha_h$ as uniform, h takes only the two values $\lfloor 2z_2/\alpha_h \rfloor$ and $\lfloor 2z_2/\alpha_h \rfloor + 1$, the larger occurring with probability $\{2z_2/\alpha_h\}$; that is,

$$\Pr[h = k \mid z_2] = \begin{cases} 1 - \{2z_2/\alpha_h\}, & k \equiv \lfloor 2z_2/\alpha_h \rfloor, \\ \{2z_2/\alpha_h\}, & k \equiv \lfloor 2z_2/\alpha_h \rfloor + 1, \end{cases} \pmod H.$$

Averaging over the rounded Gaussian of z_2 (divisor α_e) gives the marginal law

$$\Pr[h = k] = \sum_{z_2} \Pr[z_2]\ \Pr[h = k \mid z_2],$$

a small integer distribution on $[0, H)$ concentrated near 0. A model mismatch affects only the compressed length, not decoding correctness or security.

Block-adaptive low-bit splitting. A coefficient of effective standard deviation σ is compressed by peeling off a number of low bits and entropy-coding only the remainder. Writing $z = 2^b Q + R$ with $R = z \bmod {}^+ 2^b \in [0, 2^b)$ and $Q = \lfloor z/2^b \rfloor$, the low part R is essentially uniform whenever $2^b \lesssim \sigma$ and is therefore stored verbatim at exactly b bits, while the quotient Q carries the Gaussian shape and is rANS-coded. The total cost is then $b + H(Q) \approx H(z)$, but only when b is chosen well:

- if b is too large ($2^b \gg \sigma$) the retained low bits are no longer uniform, so storing them raw wastes space;
- if b is too small ($2^b \ll \sigma$) the quotient support—hence the static model and the renormalization overhead—grows needlessly.

A moderate choice $b \approx \lfloor \log_2 \sigma \rfloor - 1$ balances the two.

Because the blocks of $\mathbf{z}_1$ have very different effective deviations, they use *different* split widths rather than one global value. The narrow block z_0 ($\sigma \approx r/\alpha_1$, only a few units) uses a small b_0—in the limit $b_0 = 0$ it is rANS-coded in full—whereas the wide secret block $\mathbf{z}_s$ ($\sigma \approx r/\alpha_s$) uses a larger b_s. A single large b shared by both would trigger the first failure mode for z_0: its low bits are far from uniform and must not be stored raw. The omitted block $\mathbf{z}_2$ is never transmitted; its information is carried by the hint h, which has a small bounded support and is likewise rANS-coded in full (no split).

Coefficients are flattened in polynomial-major then coefficient-major order, giving the logical symbol sequence $\big(Q_0[\cdot],\ \mathbf{Q}_s[\cdot],\ h[\cdot]\big)$, where $Q_0, \mathbf{Q}_s$ are the retained quotients of $z_0, \mathbf{z}_s$ under their

respective split widths. The challenge is transmitted as its seed $seed_c$, a uniformly random fixed-length hash output that is incompressible; it is therefore stored verbatim and never entered into the rANS stream. The signature packet is

$$\sigma = seed_c \parallel rlen \parallel com \parallel R,$$

where R concatenates the raw low bits of all split coefficients, *rlen* is the two-byte little-endian length of the actual rANS stream, and *com* is a fixed-size reserve containing that rANS stream followed by zero padding. The rANS stream itself holds the explicitly serialized final state followed by the emitted renormalization bytes.

The fixed signature length is therefore not a fitted constant but the sum of the fixed fields

$$|\sigma| = \tfrac{\lambda}{4} + 2 + R_{\text{rans}} + \left\lceil \frac{nb_0}{8} \right\rceil + \left\lceil \frac{lnb_s}{8} \right\rceil,$$

where the two-byte field records the actual rANS stream length inside the zero-padded reserve R_{rans}, and b_0, b_s are the block-specific low-bit widths. The reserve is selected from the measured source model with a per-stream overflow target of 2^{-35}; i.e., the fixed packet is large enough that the probability of an rANS stream exceeding the reserve is bounded by the design target 2^{-35} before the signer restarts.

Table 8: Concrete rANS packet-size constants used by sigEncode.

Mode	$\frac{\lambda}{4}$	b_0	b_s	R_{rans}	$\lvert R_0\rvert$	$\lvert R_s\rvert$	$\lvert\sigma\rvert$
SHUTTLE-SUF-128	32	2	6	509	64	576	1183
SHUTTLE-SUF-256	64	2	7	879	128	1344	2417
SHUTTLE-SUF-512	128	2	7	1927	256	2688	5001

Static rANS codec. Each block is coded by a static byte-renormalized rANS coder sharing a common denominator $M = 2^{10} = 1024$, with byte-renormalization interval $[L, 256L) = [2^{23}, 2^{31})$ and two interleaved rANS streams. Its model is built once from the (truncated) target law of that block: the ideal probabilities p_s are quantized to integer frequencies $f_s = \max(1, \lfloor Mp_s \rfloor)$, adjusted on the largest-residual symbols until $\sum_s f_s = M$, with cumulatives $F_s = \sum_{u<s} f_u$; every in-support symbol keeps a nonzero frequency so it stays decodable. Starting from the fixed state $x = L$, the encoder visits symbols in reverse order, renormalizes by emitting low bytes of x while x exceeds the symbol-dependent threshold, and updates

$$C(x, s) = \left\lfloor \tfrac{x}{f_s} \right\rfloor M + (x \bmod f_s) + F_s.$$

The decoder inverts this with $u = x \bmod M$, the unique s such that $F_s \le u < F_s + f_s$, and

$$D(x, s) = f_s \left\lfloor \tfrac{x}{M} \right\rfloor + u - F_s,$$

re-injecting bytes while $x < L$. The canonical *com* is the explicitly serialized final state followed by the emitted renormalization bytes in reverse, so the decoder reproduces the symbol sequence exactly. The encoder rejects and restarts the signing iteration if any quotient falls outside its modelled support; the discarded Gaussian tail mass is negligible. Concretely, the Gaussian quotient blocks Q_0 and $\mathbf{Q}_s$ are truncated at 11σ, so the per-coordinate truncation-failure probability is $\operatorname{erfc}(11/\sqrt{2}) \approx 2^{-87.5}$, far below the per-stream overflow target 2^{-35}; the hint block h carries no Gaussian tail, as it is a bucket-crossing value range-checked into $[0, H)$ and therefore stays in range for every in-support coefficient. Moreover, the signer's B_v norm gate means an

accepted signature never reaches this tail, so the truncation can fire only on an already-rejected attempt.

The per-mode dimensions and supports for the quotient blocks $Q_0, \mathbf{Q}_s$ and the hint h are fixed in Table 9. The frequency and cumulative tables are then the unique tables obtained by applying the quantization rule above with $M = 1024$ and with every in-support symbol kept nonzero.

Table 9: Concrete rANS model constants. Q_0 and Q_s use contiguous supports $[\mathrm{lo}, \mathrm{lo} + N - 1]$; h uses the full support $[0, H)$, where $H = 2(q-1)/\alpha_h$.

Mode	Q_0 lo	Q_0 N	Q_s lo	Q_s N	h lo	h N	M	L
SHUTTLE-SUF-128	-26	52	-15	30	0	30	1024	2^{23}
SHUTTLE-SUF-256	-17	35	-15	30	0	120	1024	2^{23}
SHUTTLE-SUF-512	-16	33	-24	48	0	58	1024	2^{23}

Codec and serialization algorithms. Algorithms 30 and 31 specify the static codec above as a single $N_\mathrm{r} = 2$-stream coder over the merged symbol sequence $T = (Q_0 \parallel \mathbf{Q}_s \parallel \mathbf{h})$, where position t selects model $\mathrm{md}(t) \in \{Q_0, Q_s, h\}$ from its block and stream $s = t \bmod N_\mathrm{r}$; the encoder pushes symbols in reverse t and the decoder pulls them in forward t. The per-model constants are lo_md, N_md, frequencies $f^\mathrm{md}[\cdot]$ (Table 9, $M = 1024$), and their prefix sums $F^\mathrm{md}[\cdot]$. Algorithms 32 and 33 wrap this codec into the full signature serialization: sigEncode performs the block-adaptive low-bit split, rANS-codes the quotient and hint symbols, and emits $seed_c \parallel rlen \parallel com \parallel R$; sigDecode reverses it and additionally enforces the canonical checks below.

Algorithm 30 rANSEncode$(Q_0, \mathbf{Q}_s, \mathbf{h}; \mathrm{cap})$ — merged static codec, $N_\mathrm{r} = 2$ streams

Input: symbol blocks $Q_0, \mathbf{Q}_s, \mathbf{h}$; reserve cap $= R_\mathrm{rans}$ **Output:** stream com, or $\perp$
$T \leftarrow (Q_0 \parallel \mathbf{Q}_s \parallel \mathbf{h})$ of length N; $x_0 \leftarrow L$; $x_1 \leftarrow L$; $pos \leftarrow$ cap ▷ write backward
for $t = N - 1$ **downto** 0 **do**
  $s \leftarrow t \bmod N_\mathrm{r}$; $slot \leftarrow T[t] - \mathrm{lo}_{\mathrm{md}(t)}$
  if $slot \notin [0, N_{\mathrm{md}(t)})$ **or** $pos < 4N_\mathrm{r} + 4$ **then return** $\perp$
  end if ▷ out-of-support ⇒ resign
  $f \leftarrow f^{\mathrm{md}(t)}[slot]$; $F \leftarrow F^{\mathrm{md}(t)}[slot]$
  while $x_s \geq (256L/M)\, f$ **do** $out[\,{-}{-}pos\,] \leftarrow x_s \bmod 256$; $x_s \leftarrow \lfloor x_s/256 \rfloor$
  end while
  $x_s \leftarrow \lfloor x_s/f \rfloor\, M + (x_s \bmod f) + F$ ▷ $C(x, s)$
end for
for $s = 0$ **to** $N_\mathrm{r} - 1$ **do** prepend the 4 big-endian bytes of x_s to $out[\,{-}{-}pos\,]$
end for
return $com \leftarrow out[pos \,..\, \mathrm{cap})$

Algorithm 31 rANSDecode($com, rlen; n_{q0}, n_{qs}, n_h$) — canonical static decode

Input: stream com of length $rlen$; block sizes n_{q0}, n_{qs}, n_h **Output:** $(Q_0, \mathbf{Q}_s, \mathbf{h})$, or $\perp$
if $rlen < 4N_{\mathrm{r}}$ **then return** $\perp$
end if
$bp \leftarrow 0$
for $s = N_{\mathrm{r}} - 1$ **downto** 0 **do**
  $x_s \leftarrow$ next 4 bytes of com as a big-endian integer; $bp \leftarrow bp + 4$
  if $x_s \notin [L, 256L)$ **then return** $\perp$
  end if ▷ initial-state range check
end for
for $t = 0$ **to** $N - 1$ **do**
  $s \leftarrow t \bmod N_{\mathrm{r}}$; $u \leftarrow x_s \bmod M$; $slot \leftarrow$ unique j with $F^{\mathrm{md}(t)}[j] \le u < F^{\mathrm{md}(t)}[j{+}1]$
  $T[t] \leftarrow \mathrm{lo}_{\mathrm{md}(t)} + slot$; $x_s \leftarrow f^{\mathrm{md}(t)}[slot]\,\lfloor x_s/M \rfloor + u - F^{\mathrm{md}(t)}[slot]$ ▷ $D(x, s)$
  while $x_s < L$ **do if** $bp \ge rlen$ **then return** $\perp$; $x_s \leftarrow 256\,x_s + com[bp{+}{+}]$
  end while
end for
if $bp \ne rlen$ **or** $x_s \ne L$ for some s **then return** $\perp$
end if ▷ full-consume + terminal state
return the three blocks $(Q_0, \mathbf{Q}_s, \mathbf{h})$ of T

Algorithm 32 sigEncode($seed_c, \mathbf{z}_1, \mathbf{h}$)

Input: seed $seed_c$, response $\mathbf{z}_1 = (z_0, \mathbf{z}_s) \in R^{1+l}$, hint $\mathbf{h} \in R^m$ **Output:** σ, or $\perp$
$Q_0[k] \leftarrow \lfloor z_0[k]/2^{b_0} \rfloor$; $\mathbf{Q}_s[i,k] \leftarrow \lfloor \mathbf{z}_{s,i}[k]/2^{b_s} \rfloor$ (all k, $i < l$) ▷ heads
$com \leftarrow$ rANSEncode$(Q_0, \mathbf{Q}_s, \mathbf{h}; R_{\mathrm{rans}})$, **or return** $\perp$ ▷ out-of-support ⇒ resign
$rlen \leftarrow |com|$; pad com to R_{rans} bytes with zeros
$R \leftarrow \langle\, z_0[k] \bmod {}^{+}2^{b_0} \,\rangle_k \parallel \langle\, \mathbf{z}_{s,i}[k] \bmod {}^{+}2^{b_s} \,\rangle_{i,k}$ ▷ raw low bits, LSB-first
return $\sigma \leftarrow seed_c \parallel$ IntegerToBytes$(rlen, 2) \parallel com \parallel R$

Algorithm 33 sigDecode(σ)

Input: σ **Output:** $(seed_c, \mathbf{z}_1, \mathbf{h})$, or $\perp$
parse $\sigma = seed_c \parallel lenfield \parallel com \parallel R$ with $|seed_c| = \frac{\lambda}{4}$, $|com| = R_{\mathrm{rans}}$
$rlen \leftarrow$ BytesToInteger$(lenfield, 2)$
if $rlen > R_{\mathrm{rans}}$ **or** $com[rlen\,..\,R_{\mathrm{rans}})$ is not all-zero **then return** $\perp$
end if ▷ length + zero-padding
$(Q_0, \mathbf{Q}_s, \mathbf{h}) \leftarrow$ rANSDecode$(com[0\,..\,rlen), rlen; n, ln, mn)$; **if** $= \perp$ **then return** $\perp$
for each hint coefficient h **do if** $h \notin [0, H)$ **then return** $\perp$
end for ▷ range-check, never reduce mod H
$z_0[k] \leftarrow 2^{b_0} Q_0[k] + R_k^{(0)}$; $\mathbf{z}_{s,i}[k] \leftarrow 2^{b_s} \mathbf{Q}_s[i,k] + R_{i,k}^{(s)}$ ▷ $R^{(\cdot)}$: low bits from R; no mod
$com' \leftarrow$ rANSEncode$(Q_0, \mathbf{Q}_s, \mathbf{h}; R_{\mathrm{rans}})$
if $com' \ne com[0\,..\,rlen)$ **then return** $\perp$
end if ▷ byte-for-byte re-encode (injectivity)
return $(seed_c, \mathbf{z}_1, \mathbf{h})$

Canonical decoding and strong unforgeability. The decoder sigDecode (Algorithm 33) rejects before verification unless every one of the following conditions holds.

1. The packet parses exactly as $seed_c \parallel rlen \parallel com \parallel R$: the verbatim $\frac{\lambda}{4}$-byte challenge seed $seed_c$, the two-byte $rlen$, the fixed-size com reserve holding one rANS stream of length

rlen followed by zero padding, and the fixed-length uncompressed tail R (the raw low bits of all split coefficients); all reserve bytes and all rANS symbols are consumed.

2. Every retained quotient lies in its block-specific modelled support, every stored low part lies in $[0, 2^b)$ for that block's split width b, and reconstruction uses $z = 2^b Q + R$ without modular reduction.

3. Each decoded hint coefficient *must already be* in $[0, 2(q-1)/\alpha_h)$; reducing an out-of-range integer modulo H is forbidden.

4. The challenge-seed field has exactly $\frac{\lambda}{4}$ bytes and is read verbatim as $seed_c$; its consistency is enforced by the verifier's recomputation check $seed'_c = seed_c$.

5. After the prescribed number of symbols, no compressed byte remains, the state is exactly L, and re-encoding the recovered $(\mathbf{z}_1, \mathbf{h}, seed_c)$ reproduces the input byte-for-byte.

These checks define the accepted-packet set $\mathcal{A}$. Deterministic rANS transitions, the fixed symbol order, Euclidean response splitting, and the fixed-length verbatim seed give

$$\mathsf{sigDecode}(\mathsf{sigEncode}(seed_c, \mathbf{z}_1, \mathbf{h})) = (seed_c, \mathbf{z}_1, \mathbf{h}).$$

The terminal-state, full-consumption, padding, range, and byte-for-byte re-encoding checks also make sigDecode injective on $\mathcal{A}$. Consequently two distinct accepted byte strings cannot represent the same triple.

In short, strong unforgeability (SUF-CMA) rests on two complementary facts. First, the hierarchical hashing of Section 2.3 fixes the challenge seed as $seed_c = \mathsf{H}_{\mathsf{ch}}(\mathsf{EncodeCom}(\mathbf{w}_h, \mathbf{w}_0) \parallel \mu)$, binding it to the message, the public key, and the commitment. Although distinct seeds may sample to the same challenge, the verifier recomputes $seed'_c$ from the reconstructed commitment and accepts only when $seed'_c = seed_c$; this pins the transmitted seed to a single hash value, so it cannot be replaced by an aliasing seed without breaking H_{ch}. Hence a forgery that is not a verbatim copy of a queried signature must change at least one of $(seed_c, \mathbf{z}_1, \mathbf{h})$ and is rejected by verification. Second, the canonical, injective encoding above—rANS for $\mathbf{z}_1, \mathbf{h}$ together with a fixed-length verbatim $seed_c$—guarantees that the accepted triple has a *unique* valid byte representation, so an adversary cannot produce a fresh, distinct signature string for an already queried message. Together these rule out both new-message and re-encoding forgeries, which is exactly what SUF-CMA requires.

2.6 Correctness

Before stating correctness, we note that correctness of our scheme is not immediate from the algebraic consistency of signing and verification alone. Since verification reconstructs the omitted response part from the transmitted hint and the least significant bit, the signer explicitly computes the verifier-side reconstructed value $\mathbf{z}'_2$ and checks the same verification bound B_v before outputting a signature. Our proof follows the same line of argument as the compressed verification analysis in [22, Lemma 5], translated into our notation.

We first record the consistency property that makes verification recompute the same challenge as signing. Its crux is that the signer uses the signed (bimodal) challenge c' while the verifier uses the binary challenge c, and the two agree modulo $2q$ thanks to a parity argument.

Lemma 2 (Challenge reconstruction consistency)**.** *Let $c \in \{0,1\}^n$ be the challenge and let c' denote the signed (bimodal) challenge used inside* Sign*, so that $c'_j \in \{0, \pm 1\}$ with $|c'_j| = c_j$ for every coefficient. Then $c' \equiv c \pmod 2$, and consequently*

$$q\,(c' - c) \equiv 0 \pmod{2q}.$$

Hence the modified commitment recomputed by the verifier from the binary challenge,

$$\widetilde{\mathbf{w}}_V = \left(\mathsf{NTT}^{-1}(\hat{\mathbf{A}}_1 \cdot \mathsf{NTT}(\mathbf{z}_1)) - q\, c\, \mathbf{j}\right) \bmod 2q,$$

equals the value $\widetilde{\mathbf{w}}$ formed during signing with c'. Together with Lemma 1, which fixes the low bit $\mathbf{w}_0 = \mathsf{LSB}(\mathbf{w}) = \mathsf{LSB}(\mathbf{y}'_0)\,\mathbf{j}$, the verifier reconstructs the same high-order part $\mathbf{w}_h$ and the same low bit $\mathbf{w}_0 = \mathbf{w}'_0$, and therefore recomputes the same challenge seed $seed'_c = seed_c$.

Proof. Flipping a sign preserves parity and leaves zero coefficients unchanged, so $c'_j \equiv c_j \pmod 2$ for every j; thus $c' - c$ has even coefficients and $q\,(c' - c) \equiv 0 \pmod{2q}$. The signing and verification commitments are computed by the same NTT-domain product and differ only by the term $q\,(c' - c)\,\mathbf{j}$, which vanishes modulo $2q$; hence $\widetilde{\mathbf{w}}_V = \widetilde{\mathbf{w}}$. Feeding equal inputs through the identical $\mathsf{UseHint}$ formulas yields equal high-order parts $\mathbf{w}_h$, while Lemma 1 gives the equal low bit $\mathbf{w}'_0 = \mathsf{LSB}(\mathbf{y}'_0)\,\mathbf{j} = \mathbf{w}_0$. Therefore $seed'_c = \mathsf{H}_{\mathsf{ch}}(\mathsf{EncodeCom}(\mathbf{w}_h, \mathbf{w}_0) \| \mu) = seed_c$. □

Proof. Let $\sigma = (seed_c, \mathbf{z}_1, \mathbf{h})$ be a signature output by Sign on message M, and let $(\mathbf{z}_1, \mathbf{z}_2) \leftarrow \mathbf{z}$ be the decomposition of the response vector computed during signing.

During signing, the signer computes the commitment $\mathbf{w}$, derives the challenge seed

$$seed_c = \mathsf{H}_{\mathsf{ch}}(\mathsf{EncodeCom}(\mathbf{w}_h, \mathbf{w}_0) \| \mu), \qquad \mu = \mathsf{H}_{\mathsf{msg}}(tr \| M),$$

and samples the challenge $c = \mathsf{SampleC}(seed_c)$. The hint $\mathbf{h}$ is generated from $\mathbf{z}_2$ and $\mathbf{w}$ so that the verifier can recover the same high-order commitment information. By Lemma 1 and Lemma 2, the verifier reconstructs the same high-order and low-order commitment information, and therefore recomputes the same challenge seed: $seed'_c = seed_c$. Consequently $seed'_c = seed_c$, so the seed check in verification succeeds (and the challenge $c = \mathsf{SampleC}(seed_c)$ recomputed by the verifier coincides with the one used in signing).

It remains to prove that the reconstructed response satisfies the verification bound. During signing, after computing $\mathbf{h}$, the signer computes $\mathbf{z}'_2$ from the modified commitment, the recovered high-order commitment, and the low-order commitment, and outputs a signature only if

$$\|(\mathbf{z}_1, \mathbf{z}'_2)\|_2 \le B_v.$$

Because verification applies the same deterministic formulas through $\mathsf{UseHint}$ to the inputs $(\mathbf{h}, \mathbf{z}_1, \mathbf{w}'_0, \hat{\mathbf{A}}_1, c)$ recovered from the signature and public key, the verifier obtains the same reconstructed component $\mathbf{z}'_2$ (the signer having formed the identical value from $\widetilde{\mathbf{w}} = \mathbf{w} - 2\mathbf{z}_2$ and $\mathbf{w}'_0$). Hence the verification bound check succeeds, and the verification algorithm returns accept. □

2.7 Parameter Sets

Table 10 lists the proposed parameter sets targeting the required security categories.

Empirical KeyGen rejection rates (norm window). With the fixed lower bound $B'_k = 290$, KeyGen accepts iff $B'_k \le \|\mathsf{StretchS}(1, \mathbf{s}_1, \mathbf{s}'_2)\|_2 \le B_k$. Table 11 reports measured acceptance and rejection rates from Monte Carlo sampling of candidate secrets (3000/2000/1000 trials for SUF-128/256/512, `np.random.seed(0)`). Because each B_k lies essentially at the median of the stretched-norm distribution ($\approx$ 294.9/296.2/292.8), the upper bound $\le B_k$ alone rejects about half of the candidates; the lower bound $B'_k = 290$ removes an additional left-tail fraction, yielding total rejection rates of **53%–66%** (acceptance **34%–47%**).

Table 10: Parametrizations with SUF (KeyGen rejection rates: Table 11)

Security	128	256	512	Description
λ	128	256	512	Target security level (design λ; also fixes seed/hash/digest lengths)
n	256	512	1024	Ring dimension
q	15361	61441	59393	Modulus
l	3	3	3	Number of polynomial components in $\mathbf{s}_1$
m	3	2	2	Number of polynomial components in $\mathbf{s}_2$
σ_1	0.85	0.9	0.9	Gaussian parameter for $\mathbf{s}_1$
σ_2	0.85	1	0.9	Gaussian parameter for $\mathbf{s}_2$
r	825	825	825	Parameter of y
τ	42	58	115	Hamming weight of the challenge c
α_h	1024	1024	2048	Compression parameter for h
α_b	2	2	4	Compression parameter for b
B_s^{enc}	9	10	10	Per-coefficient encoding bound for $\mathbf{s}_1$ $(= L_{\sigma_1})$
B_e^{enc}	10	12	12	Per-coefficient encoding bound for $\mathbf{s}_2'$ $(= L_{\sigma_2} + \lceil \alpha_b/2 \rceil)$
d_s	5	5	5	$\mathbf{s}_1$ coefficient bit-width $(= \lceil \log_2(2B_s^{\mathsf{enc}} + 1) \rceil)$
d_e	5	5	5	$\mathbf{s}_2'$ coefficient bit-width $(= \lceil \log_2(2B_e^{\mathsf{enc}} + 1) \rceil)$
α_1	90	135	144	Gaussian-width divisor for the first block of y
α_s	10	5	3	Gaussian-width divisor for the s-block of y
α_e	5	5	3	Gaussian-width divisor for the e-block of y
B_k	295.06	296.19	292.76	Secret-key rejection bound
B_k'	290	290	290	Lower bound of KeyGen norm window $(B_k' \le \lVert \mathsf{StretchS}(1, \mathbf{s}_1, \mathbf{s}_2') \rVert_2 \le B_k)$
N	29	29	29	Alternating-series truncation length in R (Section 4.1.2)
B_{bdry}	15	15	15	Interval boundaries per R call $(= \lceil N/2 \rceil$; Algorithm 22)
$\eta_{\log}$	$\le 2^{-57}$	$\le 2^{-57}$	$\le 2^{-57}$	Additive log error budget (Theorem 2)
κ_a	80	80	80	$\mathsf{SamplerU}$ exponent bits $(\lceil \kappa_a/8 \rceil = 10$ bytes; Algorithm 26)
κ_b	57	57	57	$\mathsf{SamplerU}$ mantissa bits $(\lceil \kappa_b/8 \rceil = 8$ bytes; $\mathsf{ApproxLog}$ strictly increasing for $\kappa_b \le 61)$
(g, d, Q)	$(2, 13, 62)$	$(2, 13, 62)$	$(2, 13, 62)$	$\mathsf{ApproxLog}$ segments 2^g, degree, Q-scale (Table 1)
B_v	7356.68	10385.53	25194.90	Verification bound
β	2^{-30}	2^{-30}	2^{-30}	Signing-loop rejection rate
pk size	1264	1952	3648	Public key size
sk size	2288	3680	7104	Secret key size
Sign size	1183	2417	5001	Signature size, computed as in Table 8
Combined	2447	4369	8649	Public key plus signature size
MLWE	138.0 (123.4)	279.7 (252.0)	625.0 (553.4)	Estimated MLWE costs in classical (quantum) Core-SVP model
MSIS-sUF	165.3 (145.5)	266.3 (234.4)	522.4 (459.8)	Estimated MSIS-sUF costs in classical (quantum) Core-SVP model

Table 11: Empirical KeyGen acceptance/rejection for $B'_k = 290$ and the per-mode B_k of Table 10.

Mode	B_k	Accept	Reject	Reject ($< B'_k$)	Reject ($> B_k$)
SUF-128	295.06	33.6%	66.4%	17.6%	48.8%
SUF-256	296.19	47.1%	52.9%	2.8%	50.2%
SUF-512	292.76	37.6%	62.4%	11.8%	50.6%

3 Design Rationale

In accordance with the NGCC submission guidelines, this section outlines the fundamental design rationale of SHUTTLE. We present a top-down exposition of our design choices: starting from the foundational hardness assumptions, proceeding to our architectural evolution and core technical innovations, and concluding with our concrete parameter selection strategy and failure analysis.

3.1 Underlying Hard Problems

The security of SHUTTLE is fundamentally based on two well-studied, conservative lattice problems over the polynomial ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. Crucially, our scheme relies on the exact same mathematical assumptions as the NIST standard ML-DSA (CRYSTALS-Dilithium)[19], ensuring equivalent theoretical security foundations:

- **Module Learning with Errors (MLWE):** The pseudo-randomness of the public key and the zero-knowledge property of our signing phase rely on the hardness of the MLWE problem. Unlike [22] which relies on the non-standard NTWE assumption to achieve compactness, we specifically stick to standard MLWE. This ensures that our public keys are computationally indistinguishable from uniform, aligning our security foundation directly with that of ML-DSA and avoiding any potential algebraic vulnerabilities associated with NTRU-like or NTWE structures.
- **Module Short Integer Solution (MSIS):** The unforgeability of the signature relies on the hardness of the MSIS problem. An attacker attempting to forge a signature without knowing the secret key must find a short vector satisfying the verification relation, which directly reduces to solving the standard MSIS problem. This is mathematically identical to the forgery-to-MSIS reduction used in ML-DSA, with the only difference being the concrete norm bounds and module dimensions.

3.2 Architectural Evolution and Core Innovations

To establish a self-contained and rigorous foundation for our design choices, we first review the mathematical mechanics of Gärtner's iterative rejection sampling (IRS) framework [22], highlighting its core distribution-preserving equations, and then detail how SHUTTLE reformulates this paradigm through four core design innovations to eliminate its inherent engineering vulnerabilities.

Mathematical Recapitulation of Gärtner's IRS. Let the target distribution be a discrete Gaussian ρ_g over a lattice Λ. In a Fiat-Shamir signature, the signer wishes to mask a secret-dependent shift vector $v \in \Lambda$ (which, in the iterative setting, is a short column of the secret matrix $\mathbf{s}$). Given an intermediate state $\mathbf{y}$, the transition step R_v must choose between two shifted candidates $\mathbf{y} - v$ and $\mathbf{y} + v$, corresponding to the sign selection of a challenge coefficient.

To ensure that the output state $\mathbf{z}$ is distributed exactly according to ρ_g (conditioned on success), the transition kernel must satisfy the *local balance condition* of the probability mass. Specifically, Gärtner defines two transition probability functions, $f_v(\mathbf{y})$ and $g_v(\mathbf{y})$, representing the probabilities of transitioning to $\mathbf{y} - v$ and $\mathbf{y} + v$, respectively. The remaining probability $1 - f_v(\mathbf{y}) - g_v(\mathbf{y})$ corresponds to outputting the abort symbol $\perp$.

For the output distribution to preserve ρ_g exactly under successful transitions, the probability of entering any state $\mathbf{z}$ from its preimages $\mathbf{z} + v$ and $\mathbf{z} - v$ must balance perfectly with the target density $\rho_g(\mathbf{z})$ scaled by a global repetition factor $M \geq 1$:

$$\rho_g(\mathbf{z} + v) f_v(\mathbf{z} + v) + \rho_g(\mathbf{z} - v) g_v(\mathbf{z} - v) = \frac{\rho_g(\mathbf{z})}{M}. \tag{4}$$

By analyzing the one-dimensional lattice line $\mathbf{z} + \mathbb{Z}v$, Gärtner solved this recurrence relation. Under the boundary condition that the probabilities must vanish as $\|\mathbf{y}\| \to \infty$, the exact solutions are expressed as alternating infinite sums of Gaussian ratios:

$$f_v(\mathbf{y}) = \frac{S_v(\mathbf{y})}{M} \quad \text{and} \quad g_v(\mathbf{y}) = \frac{S_v(-\mathbf{y})}{M}, \tag{5}$$

where the fundamental alternating sum $S_v(\mathbf{y})$ is defined as:

$$S_v(\mathbf{y}) = \sum_{i=0}^{\infty} (-1)^i \frac{\rho_g(\mathbf{y} + iv)}{\rho_g(\mathbf{y})}. \tag{6}$$

A crucial algebraic property of these ideal functions is their difference, which reveals how the probability mass is shifted across the lattice line to correct the Gaussian density:

$$f_v(\mathbf{y}) - g_v(\mathbf{y}) = \frac{1}{M} \sum_{i \in \mathbb{Z}} (-1)^i \frac{\rho_g(\mathbf{y} + iv)}{\rho_g(\mathbf{y})}. \tag{7}$$

This difference $f_v(\mathbf{y}) - g_v(\mathbf{y})$ acts as a local drift corrector. When the transition is successful, this algebraic relation perfectly cancels out the secret-dependent bias of the shift v.

The Vulnerability of Exactness. While mathematically elegant, enforcing the exact invariant in Equation (4) requires that $f_v(\mathbf{y}) + g_v(\mathbf{y}) \leq 1$ holds globally. To prevent this sum from exceeding 1 at the boundary of the transition region, Gärtner must set the scaling factor M to its worst-case upper bound over the "middle strip" $K_v = \{\mathbf{y} : |\langle \mathbf{y}, v \rangle| \leq \|v\|_2^2\}$:

$$M \geq \max_{\mathbf{y} \in K_v} \left(S_v(\mathbf{y}) + S_v(-\mathbf{y}) \right) = \max_{\mathbf{y} \in K_v} \left(1 - \sum_{i \in \mathbb{Z}} (-1)^i \frac{\rho_g(\mathbf{y} + iv)}{\rho_g(\mathbf{y})} \right).$$

Consequently, whenever the state $\mathbf{y}$ falls into a region where the local density cannot be balanced within the probability budget $1/M$, the algorithm is forced to abort ($\perp$) with probability:

$$\Pr[\mathsf{R}_v(\mathbf{y}) = \perp] = 1 - f_v(\mathbf{y}) - g_v(\mathbf{y}).$$

This abort-driven control flow introduces severe vulnerabilities:

1. **Side-Channel Leakage:** The abort probability is directly correlated with the secret-dependent term $f_v(\mathbf{y}) - g_v(\mathbf{y})$ in Equation (7), which depends heavily on the inner product $\langle \mathbf{y}, v \rangle$. An adversary observing the timing, cache, or power profiles of these aborts can easily exploit this correlation to recover the secret-dependent inner products, leading to complete key recovery.

2. **Long-Tail Latency:** In embedded or real-time systems (e.g., automotive CAN bus), cryptographic operations must execute within strict, deterministic *fixed time-windows*. The variable number of restarts caused by $\perp$ outcomes during the iterations creates a long-tail latency distribution that is highly unacceptable.

SHUTTLE resolves these theoretical and engineering bottlenecks through four core design innovations.

Innovation 1: Rejection-Free Approximate Iterative Sampling. To completely eliminate the side-channel leakage and timing variability of traditional IRS, SHUTTLE introduces the paradigm of *Approximate Iterative Rejection Sampling*. We completely remove the abort branch $\perp$ from our transition kernel, defining a new transition $\mathsf{T}_v(\mathbf{y})$ that always returns one of the two shifted candidates ($\mathbf{y} - v$ or $\mathbf{y} + v$) with probabilities $p_v(\mathbf{y})$ and $1 - p_v(\mathbf{y})$, respectively.

To minimize the pointwise drift from the target Gaussian, we define $p_v(\mathbf{y})$ by adapting Gärtner's positive-definite regions:

$$p_v(\mathbf{y}) = \begin{cases} S_v(\mathbf{y}), & \text{if } \langle \mathbf{y}, v \rangle \geq 0, \\ 1 - S_v(-\mathbf{y}), & \text{if } \langle \mathbf{y}, v \rangle < 0. \end{cases} \tag{8}$$

Because we do not scale these probabilities by $1/M$ and do not include an abort branch, the sum of the transition probabilities is identically 1 (i.e., $p_v(\mathbf{y}) + (1 - p_v(\mathbf{y})) = 1$).

This approximate transition introduces a small, bounded pointwise relative error ε_v at each step. Under a Gaussian input ρ_g, the one-step output distribution $\rho'_{g,v}$ deviates from the target ρ_g only within the middle strip K_v:

$$\varepsilon_v = \max_{\mathbf{y} \in K_v} \left| \frac{\rho'_{g,v}(\mathbf{y})}{\rho_g(\mathbf{y})} - 1 \right| = \max_{\mathbf{y} \in K_v} \left| \frac{\sum_{i \in \mathbb{Z}} (-1)^i \rho_g(\mathbf{y} + iv)}{\rho_g(\mathbf{y})} \right|.$$

Using Poisson summation, we bound this local drift as a function of the Gaussian flatness $\beta = r/\|v\|_2$ (Lemma 3). We then leverage the composition properties of relative errors (Lemma 4) to show that the accumulated relative error after all τ iterations remains extremely small (e.g., $\delta_\tau \leq 2^{-45}$). Finally, we apply Rényi divergence (Lemma 5) to prove that the final signature distribution is statistically indistinguishable from the ideal Gaussian. By shifting the burden of exactness from the online algorithm (which required aborts) to the offline security proof (which bounds the divergence), SHUTTLE achieves a perfectly regular, constant-time, and side-channel-resistant signing flow.

Innovation 2: Logarithmic-Domain Transition Kernel. Even after removing aborts, evaluating the transition probabilities naively requires computing infinite alternating sums of Gaussian ratios—essentially evaluating multiple transcendental exp functions over secret-dependent inner products $\langle \mathbf{y}, v \rangle$ inside the loop. To bypass this computational bottleneck, we algebraically transform the randomized decision process into the logarithmic domain.

By mapping the interval-membership tests into the logarithmic domain, the target Gaussian boundaries simplify algebraically into elementary squared-norm differences. Consequently, after a single evaluation of $\log u$ on public randomness, the entire secret-dependent transition loop reduces to simple integer interval comparisons. This mathematical simplification inherently protects the implementation from the side-channel vulnerabilities typically associated with high-precision or floating-point exponential approximations.

Innovation 3: Overcoming MLWE Size Penalty via Asymmetric Scaling. While Gärtner's extreme compactness relies on the non-standard NTWE assumption, instantiating the scheme over standard MLWE naively incurs a severe size penalty. Specifically, the MLWE

secret vector inherently takes the form $\mathbf{s} = (1, \mathbf{s}_1, \mathbf{s}_2)$. The fixed component 1 contributes zero entropy but still occupies a full ring element in the signature response, forcing a larger masking vector and inflating the signature size.

To bridge this gap, we introduce an asymmetric stretch-and-compress mechanism. Because the first secret component is exactly 1, it is significantly shorter than the rest of the secret. Therefore, we can artificially stretch this single constant component by a factor α_1 during the sampling phase without meaningfully increasing the overall Euclidean norm of the secret. This asymmetry allows us to aggressively divide and compress the corresponding first component of the signature response by the same factor α_1 before outputting it, effectively eliminating the redundant size overhead of the MLWE structure.

Innovation 4: Unified Wide Gaussian Sampler with Block-wise Scaling. To optimize the implementation footprint for resource-constrained and hardware platforms, we introduce a block-wise scaling mechanism. We partition the masking vector $\mathbf{x} = (x_0, \mathbf{x}_s, \mathbf{x}_e) \in R^{1+l+m}$ into three distinct blocks (constant, secret, and error blocks) and scale them by different parameters α_1, α_s, and α_e during the stretching map, and compress them via matching division during compression:

$$\mathsf{StretchS}(x_0, \mathbf{x}_s, \mathbf{x}_e) = (\alpha_1 x_0, \alpha_s \mathbf{x}_s, \alpha_e \mathbf{x}_e),$$

$$\mathsf{CompressY}(y_0, \mathbf{y}_s, \mathbf{y}_e) = \left(\left\lfloor \frac{y_0}{\alpha_1} + \frac{1}{2} \right\rfloor, \left\lfloor \frac{\mathbf{y}_s}{\alpha_s} + \frac{1}{2} \right\rfloor, \left\lfloor \frac{\mathbf{y}_e}{\alpha_e} + \frac{1}{2} \right\rfloor \right).$$

By fine-tuning these block-wise scaling parameters for each security level, we are able to use a *single, unified standard deviation* $r = 825$ for the wide Gaussian sampler across all three security categories (128-, 256-, and 512-bit security levels). This unification allows all parameter sets to share the exact same precomputed reverse cumulative distribution table (RCDT), drastically reducing storage overhead and simplifying hardware design.

3.3 Parameter Selection Strategy

Based on the architectural choices above, our concrete parameters are carefully chosen to balance extreme compactness, computational efficiency, and robust security. Any valid parameter set must simultaneously satisfy the following three core mathematical requirements:

Requirement 1: Hardness of SIS and LWE. For a parameter set to be admissible, the hardness of the underlying SIS and LWE instances shall both provide at least λ bits of classical security. These security levels are estimated using the standard lattice estimator [4][2]. For the SIS instance, we evaluate the advantage $\mathsf{Adv}^{\mathrm{SIS}}_{q,m,l+1,2B_v}$ based on the verification bound B_v. For the LWE instance, we evaluate the advantage $\mathsf{Adv}^{\mathrm{LWE}}_{q,m,l,\chi_1,\chi_2}$ based on the secret key distributions. The parameter set must satisfy:

$$\log_2 \left(\frac{1}{\mathsf{Adv}^{\mathrm{SIS}}_{q,m,l+1,2B_v}} \right) \geq \lambda \qquad \text{and} \qquad \log_2 \left(\frac{1}{\mathsf{Adv}^{\mathrm{LWE}}_{q,m,l,\chi_1,\chi_2}} \right) \geq \lambda.$$

The overall computational security level is determined by the smaller of the two estimates.

Requirement 2: Challenge Space Size. Let $\mathcal{C}$ denote the challenge space. In SHUTTLE, each challenge is represented by a polynomial with coefficients in $\{0, 1\}$ and a fixed Hamming weight exactly τ. Therefore, the number of possible challenges is $|\mathcal{C}| = \binom{n}{\tau}$.

To prevent guessing and collision attacks in both classical and quantum settings, the parameter set must simultaneously satisfy the target classical security level λ and the post-quantum

[2]https://github.com/malb/lattice-estimator.

security level λ_q (where $\lambda_q \in \{80, 128, 256\}$ is determined by specific security category requirements). Due to the quadratic speedup of Grover's search algorithm in the Quantum Random Oracle Model (QROM), the quantum search complexity is bounded by $O(\sqrt{|\mathcal{C}|})$. Therefore, to guarantee full λ-bit classical security and λ_q-bit post-quantum security, the entropy of the challenge space must satisfy:

$$\log_2 \binom{n}{\tau} \geq \max\{\lambda, 2\lambda_q\}.$$

This dual-bound constraint ensures that the probability of an adversary successfully finding a challenge collision or forging a signature remains strictly negligible under both classical and quantum computing paradigms.

Requirement 3: Relative Error of Iterative Rejection Sampling. Because the masking vector is produced by an *approximate* iterative rejection sampling (IRS) procedure rather than an exact Gaussian sampler, the accepted output distribution deviates from the ideal one. An admissible parameter set must keep this deviation small enough that it does not erode the claimed security. Concretely, the per-step approximation is required to respect the additive logarithm-approximation budget $\eta_{\log} \leq 2^{-57}$, and its accumulation over the τ transition steps must stay within the target Rényi divergence used in the reduction. This budget, together with the intrinsic and truncation contributions, is established in the error analysis of Section 4.1 (concrete Rényi estimate in Section 4.1.4, log-approximation bound in Theorem 2), where the realized error is shown to lie far below the overall target 2^{-82} for all three parameter sets.

3.4 Rejection Sampling on the Key

In our scheme, the secret key is sampled from the discrete Gaussian distributions $D_{\mathbb{Z},\sigma_1}$ and $D_{\mathbb{Z},\sigma_2}$ and then filtered through a rejection step, as is customary in schemes such as BLISS and HAETAE. After absorbing the public-key rounding error into $\mathbf{s}_2'$, KeyGen retains a candidate key only if the *stretched* secret norm lies in a two-sided window (Table 10, Table 11):

$$B_k' \leq \|\mathsf{StretchS}(1, \mathbf{s}_1, \mathbf{s}_2')\|_2 \leq B_k.$$

Unlike [22], we impose both a lower and an upper threshold on this stretched norm. The lower threshold $B_k' = 290$ helps the iterative rejection-sampling procedure reach the target error level more quickly. The upper threshold B_k excludes rare extreme cases in which $\|\mathsf{StretchS}(1, \mathbf{s}_1, \mathbf{s}_2')\|_2$ is unusually large and would otherwise skew the worst-case error bound; because each B_k sits essentially at the median of the stretched-norm distribution, the upper bound alone rejects about half of the candidates.

This two-sided truncation makes the accepted key distribution more concentrated and therefore entails a modest security loss in the LWE analysis. For our concrete parameter sets, Monte Carlo sampling (Table 11) yields acceptance rates of **34%–47%** per candidate draw—substantially below a naive "≈73%" estimate that would apply to an unstretched norm window. The resulting LWE penalty is correspondingly somewhat larger than a nominal 0.5-bit heuristic, but remains fully absorbed by the MLWE margin of Table 10 (estimated MLWE levels 138/280/625 bits versus target $\lambda = 128/256/512$).

3.5 Signing Failure and Restart Probability Analysis

Unlike traditional Fiat-Shamir with aborts schemes (e.g., Dilithium) which experience high rejection rates during the masking phase, SHUTTLE employs a *rejection-free* iterative sampling technique. This means that the internal iterations for sign determination never abort, completely eliminating timing side-channels associated with secret-dependent rejections.

However, to ensure strict correctness and security bounds, the final signature must still pass a norm check (i.e., $\|(\mathbf{z}_1, \mathbf{z}_2')\|_2 \leq B_v$). If the sampled commitment and the resulting response exceed this bound, the signing algorithm restarts. Because our parameters are tightly optimized to squeeze the signature size, this final norm check introduces a non-negligible restart probability (often referred to as the signing failure rate). We mathematically bound this probability to ensure that the expected number of repetitions per signature remains small (e.g., ≈ 2 to 4 attempts). This deliberate trade-off allows us to achieve significantly smaller public keys and signatures while maintaining a short-tail execution time distribution, ensuring worst-case optimal performance in practical deployments.

4 Security Statement and Analysis

4.1 Distributional Deviation of Approximate Iterative Rejection Sampling

We control the Rényi divergence introduced by the iterative sampling procedure through pointwise relative-error bounds. Since pointwise relative errors compose multiplicatively, we analyze and bound separately the error contributed by each component.

Our roadmap is as follows:

1. We first analyze the *intrinsic error* ε_{int} of the ideal mathematical transition, assuming all calculations are performed with infinite precision.

2. We then bound the *implementation errors* arising from two practical sources: the *truncation error* ε_{tr} from evaluating a finite number of terms in $S_v(y)$, and the *approximation error* $\varepsilon_{\text{approx}}$ from evaluating transcendental functions (such as the logarithm and exponential) in finite precision.

3. Finally, we compose these error components over all τ iterative steps to establish the final Rényi divergence bound of the sampler.

4.1.1 Intrinsic Error of the Ideal Transition

We first analyze the ideal relative error introduced by a single iteration of the sampling procedure under the assumption that all computations are performed exactly. We follow the notation of Gärtner [22] and define the infinite alternating series

$$S_v(y) = \sum_{i=0}^{\infty} (-1)^i \frac{\rho_g(y + iv)}{\rho_g(y)},$$

where

$$\rho_g(y) = \exp\left(-\frac{\|y\|_2^2}{2r^2}\right).$$

Here, v denotes the secret shift vector used in a single iteration. In the full iterative sampler, v is instantiated as a cyclic shift of the secret key vector. We define the ideal transition probability $p_v(y)$ as

$$p_v(y) = \begin{cases} S_v(y), & \text{if } \langle y, v \rangle \geq 0, \\ 1 - S_v(-y), & \text{if } \langle y, v \rangle < 0. \end{cases}$$

This definition yields a valid probability. Indeed, when $\langle y, v \rangle \geq 0$, the terms in the alternating sum defining $S_v(y)$ have strictly decreasing absolute values, which guarantees $0 \leq S_v(y) \leq 1$. The case $\langle y, v \rangle < 0$ follows symmetrically by applying the same argument to $-y$.

Let the input distribution to this transition step be the ideal discrete Gaussian ρ_g. Let $\rho_v' = \mathsf{T}_v(\rho_g)$ denote the output distribution after one ideal transition step. By the definition

of our two-output transition, any point y in the support can be reached only from the two preimages $y+v$ and $y-v$. Hence, the output probability density at y is given by

$$\rho'_v(y) = \rho_g(y+v)p_v(y+v) + \rho_g(y-v)\big(1-p_v(y-v)\big).$$

To analyze this output distribution, we write $V = \|v\|_2^2$ and $t = \langle y, v\rangle$. The signs of the inner products at the preimages are $\langle y+v, v\rangle = t+V$ and $\langle y-v, v\rangle = t-V$. Consequently, the active branch of p_v evaluated at these preimages depends solely on whether y lies inside or outside the middle strip

$$K_v = \{y : -V \le \langle y, v\rangle < V\}.$$

We analyze the resulting distribution by distinguishing these two cases.

Case 1: Outside the strip ($t \ge V$ or $t < -V$). First, suppose $t \ge V$. This implies both preimages lie on the non-negative side, i.e., $\langle y+v, v\rangle \ge 0$ and $\langle y-v, v\rangle \ge 0$. Thus, we have $p_v(y+v) = S_v(y+v)$ and $p_v(y-v) = S_v(y-v)$. Expanding these terms via the definition of S_v yields

$$\rho_g(y+v)p_v(y+v) = \sum_{i=0}^{\infty}(-1)^i\rho_g(y+(i+1)v),$$

and

$$\begin{aligned}\rho_g(y-v)\big(1-p_v(y-v)\big) &= \rho_g(y-v)\big(1-S_v(y-v)\big)\\ &= \rho_g(y-v) - \sum_{i=0}^{\infty}(-1)^i\rho_g(y-v+iv)\\ &= \rho_g(y) - \sum_{i=0}^{\infty}(-1)^i\rho_g(y+(i+1)v).\end{aligned}$$

Summing these two contributions leads to an exact telescoping cancellation:

$$\rho'_v(y) = \sum_{i=0}^{\infty}(-1)^i\rho_g(y+(i+1)v) + \rho_g(y) - \sum_{i=0}^{\infty}(-1)^i\rho_g(y+(i+1)v) = \rho_g(y).$$

By symmetry, applying the same argument to the negative ray $t < -V$ yields $\rho'_v(y) = \rho_g(y)$. Therefore, the ideal transition preserves the target discrete Gaussian distribution exactly outside the strip K_v.

Case 2: Inside the strip ($-V \le t < V$). In this region, the preimages lie on opposite sides of the boundary, namely $\langle y+v, v\rangle \ge 0$ and $\langle y-v, v\rangle < 0$. This triggers different branches of the probability function: $p_v(y+v) = S_v(y+v)$ and $p_v(y-v) = 1 - S_v(v-y)$. The output distribution is

$$\rho'_v(y) = \rho_g(y+v)S_v(y+v) + \rho_g(y-v)S_v(v-y).$$

Expanding the two terms using the symmetry $\rho_g(x) = \rho_g(-x)$ gives

$$\rho_g(y+v)S_v(y+v) = \sum_{j=1}^{\infty}(-1)^{j-1}\rho_g(y+jv),$$

and

$$\rho_g(y-v)S_v(v-y) = \sum_{j=1}^{\infty}(-1)^{j-1}\rho_g(y-jv).$$

Combining these two sums, we write the non-zero terms compactly as

$$\rho'_v(y) = \sum_{j\in\mathbb{Z}\setminus\{0\}}(-1)^{j-1}\rho_g(y+jv).$$

Using the identity $\sum_{i\in\mathbb{Z}}(-1)^i\rho_g(y+iv) = \rho_g(y) - \sum_{j\in\mathbb{Z}\setminus\{0\}}(-1)^{j-1}\rho_g(y+jv)$, we obtain

$$\rho'_v(y) = \rho_g(y) - \sum_{i\in\mathbb{Z}}(-1)^i\rho_g(y+iv).$$

By combining the two cases, the output distribution of the ideal transition is fully characterized as

$$\rho'_v(y) = \begin{cases} \rho_g(y) - \sum_{i\in\mathbb{Z}}(-1)^i\rho_g(y+iv), & \text{if } -\|v\|_2^2 \le \langle y, v\rangle < \|v\|_2^2, \\ \rho_g(y), & \text{otherwise.} \end{cases}$$

This explicit form confirms that the output distribution deviates from the target discrete Gaussian only within the middle strip K_v. To control this deviation, it suffices to bound the intrinsic pointwise relative error over K_v:

$$\varepsilon_{\text{int}}(v) = \max_{y\in K_v}\left|\frac{\rho'_v(y)}{\rho_g(y)} - 1\right| = \max_{y\in K_v}\left|\frac{\sum_{i\in\mathbb{Z}}(-1)^i\rho_g(y+iv)}{\rho_g(y)}\right|.$$

Using a Poisson-summation argument, Gärtner [22] established a tight upper bound on this ratio. Since the normalization factor of the discrete Gaussian cancels out in the relative ratio, the bound applies directly to our setting.

Lemma 3 (Symmetric intrinsic error bound, adapted from Lemma 3 of [22])**.** *Let $v \neq 0$ be the shift vector and let $\beta = r/\|v\|_2$. Let*

$$K_v = \left\{y : \ |\langle y, v\rangle| \le \|v\|_2^2\right\}.$$

For the ideal transition, let ρ'_v denote the output distribution obtained from input distribution ρ_g. Then, for all $y \in K_v$,

$$\left|\frac{\rho'_v(y)}{\rho_g(y)} - 1\right| \le \varepsilon_{\text{int}}(v),$$

where

$$\varepsilon_{\text{int}}(v) = \frac{2\beta\sqrt{2\pi}\,\exp\left(-\frac{\pi^2\beta^2}{2}\right)}{\exp\left(-\frac{1}{2\beta^2}\right)\left(1-\exp\left(-2\pi^2\beta^2\right)\right)}.$$

Proof. For $y \in K_v$, the ideal transition satisfies

$$\frac{\rho'_v(y)}{\rho_g(y)} = 1 - \frac{\sum_{k\in\mathbb{Z}}(-1)^k\rho_g(y+kv)}{\rho_g(y)}.$$

The normalizing factor of the discrete Gaussian distribution cancels in the ratio, and hence

$$\frac{\rho'_v(y)}{\rho_g(y)} = 1 - \frac{\sum_{k\in\mathbb{Z}}(-1)^k\rho_r(y+kv)}{\rho_r(y)}.$$

Define

$$A_v(y) = \frac{\sum_{k\in\mathbb{Z}}(-1)^k\rho_r(y+kv)}{\rho_r(y)}.$$

It remains to upper bound $|A_v(y)|$.

Let

$$t = \frac{\langle y, v\rangle}{\|v\|_2^2}, \qquad \beta = \frac{r}{\|v\|_2}.$$

Since $y \in K_v$, we have $|t| \leq 1$. Moreover,

$$\frac{\rho_r(y + kv)}{\rho_r(y)} = \exp\left(-\frac{k^2 + 2kt}{2\beta^2}\right).$$

Equivalently,

$$\frac{\rho_r(y + kv)}{\rho_r(y)} = \frac{\rho_\beta(k + t)}{\rho_\beta(t)}.$$

Therefore,

$$A_v(y) = \frac{1}{\rho_\beta(t)} \sum_{k\in\mathbb{Z}} (-1)^k \rho_\beta(k + t).$$

Applying the Poisson summation formula to the shifted alternating Gaussian sum gives

$$\sum_{k\in\mathbb{Z}} (-1)^k \rho_\beta(k + t) = \sqrt{2\pi}\beta \sum_{m\in\mathbb{Z}} \exp\left(-2\pi^2\beta^2 \left(m + \frac{1}{2}\right)^2\right) e^{2\pi i(m+1/2)t}.$$

Hence, by the triangle inequality,

$$|A_v(y)| \leq \frac{\sqrt{2\pi}\beta}{\rho_\beta(t)} \sum_{m\in\mathbb{Z}} \exp\left(-2\pi^2\beta^2 \left(m + \frac{1}{2}\right)^2\right).$$

Since $|t| \leq 1$, we have

$$\rho_\beta(t) = \exp\left(-\frac{t^2}{2\beta^2}\right) \geq \exp\left(-\frac{1}{2\beta^2}\right) = \rho_\beta(1).$$

Furthermore,

$$\sum_{m\in\mathbb{Z}} \exp\left(-2\pi^2\beta^2 \left(m + \frac{1}{2}\right)^2\right) \leq 2\exp\left(-\frac{\pi^2\beta^2}{2}\right) \sum_{j=0}^{\infty} \exp\left(-2\pi^2\beta^2 j\right).$$

Thus,

$$\sum_{m\in\mathbb{Z}} \exp\left(-2\pi^2\beta^2 \left(m + \frac{1}{2}\right)^2\right) \leq \frac{2\exp\left(-\frac{\pi^2\beta^2}{2}\right)}{1 - \exp\left(-2\pi^2\beta^2\right)}.$$

Combining the above inequalities, we obtain

$$|A_v(y)| \leq \frac{2\beta\sqrt{2\pi}\,\exp\left(-\frac{\pi^2\beta^2}{2}\right)}{\exp\left(-\frac{1}{2\beta^2}\right)\left(1 - \exp\left(-2\pi^2\beta^2\right)\right)}.$$

Finally,

$$\left|\frac{\rho_v'(y)}{\rho_g(y)} - 1\right| = |A_v(y)|,$$

which proves the claim. □

4.1.2 Implementation Errors

We next analyze the relative errors introduced by the concrete implementation. These errors arise mainly from two sources. First, the transition probability defined by our sampler involves the infinite series $S_v(y)$, which must be truncated in practice. We therefore need to bound the relative error caused by discarding the tail of this series. Second, evaluating the transition probability requires the computation of a transcendental function, which can only be implemented approximately. We must likewise bound the relative error between the distribution produced by the concrete implementation and its ideal counterpart.

Truncation Error. For the truncation error, we use Lemma 2 of [22] to bound the tail of the alternating series defining S_v. Recall that

$$S_v(y) = \sum_{i=0}^{\infty} (-1)^i \frac{\rho_r(y + iv)}{\rho_r(y)}.$$

In the implementation, we replace S_v by the truncated sum

$$S_{v,N}(y) = \sum_{i=0}^{N-1} (-1)^i \frac{\rho_r(y + iv)}{\rho_r(y)}.$$

Equivalently, this corresponds to Lemma 2 of [22] with truncation parameter $t = N - 1$.

By Lemma 2, for any y satisfying $\langle y, v\rangle \geq -\|v\|_2^2$, we have

$$|S_v(y) - S_{v,N}(y)| \leq \frac{\rho_r((N-1)v)}{\rho_r(v) - \rho_r(2v)}.$$

Writing $V = \|v\|_2^2$, this bound is equal to

$$\varepsilon_S(v, N) := \frac{\exp\left(-\frac{(N-1)^2 V}{2r^2}\right)}{\exp\left(-\frac{V}{2r^2}\right) - \exp\left(-\frac{4V}{2r^2}\right)}.$$

Equivalently,

$$\varepsilon_S(v, N) = \frac{\exp\left(-\frac{((N-1)^2-1)V}{2r^2}\right)}{1 - \exp\left(-\frac{3V}{2r^2}\right)}.$$

We define the implemented transition probability $p_{v,N}$ by replacing S_v with $S_{v,N}$ in the definition of p_v. Since both branches of p_v use either $S_v(y)$ or $S_v(-y)$ in a region where Lemma 2 applies, we obtain the uniform probability-error bound

$$|p_v(y) - p_{v,N}(y)| \leq \varepsilon_S(v, N)$$

for every input y relevant to the transition.

Let T_v denote the ideal transition and $T_{v,N}$ the transition using $p_{v,N}$. For Gaussian input ρ_r, their one-step output distributions satisfy

$$T_v(\rho_r)(y) = \rho_r(y + v)p_v(y + v) + \rho_r(y - v)(1 - p_v(y - v)),$$

and

$$T_{v,N}(\rho_r)(y) = \rho_r(y + v)p_{v,N}(y + v) + \rho_r(y - v)(1 - p_{v,N}(y - v)).$$

Therefore,

$$\begin{aligned} |T_{v,N}(\rho_r)(y) - T_v(\rho_r)(y)| &\leq \rho_r(y + v)|p_{v,N}(y + v) - p_v(y + v)| \\ &\quad + \rho_r(y - v)|p_{v,N}(y - v) - p_v(y - v)| \\ &\leq \varepsilon_S(v, N)\big(\rho_r(y + v) + \rho_r(y - v)\big). \end{aligned}$$

Consequently, the relative one-step truncation error is bounded by

$$\varepsilon_{\mathrm{tr}}(v, N) \leq \varepsilon_S(v, N) \cdot \max_y \frac{\rho_r(y + v) + \rho_r(y - v)}{\rho_r(y)}.$$

In the actual analysis, this maximization is only needed over the verification-relevant support, or over the set of points that can be reached during signing. If the analysis is restricted to a bounded region $\mathcal{B}$, then the bound becomes

$$\varepsilon_{\mathrm{tr}}(v, N; \mathcal{B}) \leq \varepsilon_S(v, N) \cdot \max_{y \in \mathcal{B}} \frac{\rho_r(y + v) + \rho_r(y - v)}{\rho_r(y)}.$$

This is the truncation-error term that is included in the implementation error budget.

The support-dependent maximization above can be avoided by exploiting the alternating structure of the transition probabilities. After applying the sign normalization used by the transition algorithm, assume that

$$\langle y, v\rangle \geq 0$$

and define

$$a_i := \frac{\rho_r(y + iv)}{\rho_r(y)}.$$

Then

$$a_0 = 1$$

and

$$\begin{aligned}\frac{a_{i+1}}{a_i} &= \exp\left(-\frac{2\langle y, v\rangle + (2i+1)\|v\|_2^2}{2r^2}\right) \\ &\leq \exp\left(-\frac{\|v\|_2^2}{2r^2}\right).\end{aligned}$$

Define

$$\rho_{v,r} := \exp\left(-\frac{\|v\|_2^2}{2r^2}\right)$$

and

$$C_{v,r} := \frac{1+\rho_{v,r}}{1-\rho_{v,r}} = \frac{1+\exp\left(-\|v\|_2^2/(2r^2)\right)}{1-\exp\left(-\|v\|_2^2/(2r^2)\right)}.$$

The probabilities of the two ideal outputs can be written as

$$p_+ = \sum_{i=0}^{\infty}(-1)^i a_i$$

and

$$p_- = 1 - p_+ = \sum_{i=1}^{\infty}(-1)^{i+1} a_i.$$

Pairing adjacent terms and using $a_{i+1} \leq \rho_{v,r} a_i$ gives

$$\sum_{i=0}^{\infty} a_i \leq C_{v,r} p_+$$

and

$$\sum_{i=1}^{\infty} a_i \leq C_{v,r} p_-.$$

In particular,

$$p_+ \geq \frac{1}{C_{v,r}} \qquad \text{and} \qquad p_- \geq \frac{a_1}{C_{v,r}}.$$

We now convert the absolute tail bound of Lemma 2 of [22] into a relative error bound. For convenience, define

$$B_t(v, r) := \frac{\rho_r(tv)}{\rho_r(v) - \rho_r(2v)} = \frac{\exp\left(-\frac{(t^2-1)\|v\|_2^2}{2r^2}\right)}{1 - \exp\left(-\frac{3\|v\|_2^2}{2r^2}\right)}.$$

Suppose that N terms are retained, namely the terms indexed by $i = 0, \ldots, N-1$. For the positive output, Lemma 2 directly gives

$$|p_+ - p_{+,N}| \leq B_{N-1}(v, r).$$

Since $p_+ \geq 1/C_{v,r}$, it follows that

$$\left|\frac{p_{+,N}}{p_+} - 1\right| \leq C_{v,r} B_{N-1}(v,r).$$

For the negative output, factor out the first term and reindex the series:

$$p_- = a_1 \sum_{j=0}^{\infty} (-1)^j \frac{a_{j+1}}{a_1},$$
$$p_{-,N} = a_1 \sum_{j=0}^{N-2} (-1)^j \frac{a_{j+1}}{a_1}.$$

The normalized series is of the same form as S_v, with base point $y+v$. Applying Lemma 2 with truncation parameter $t = N-2$ gives

$$|p_- - p_{-,N}| \leq a_1 B_{N-2}(v,r).$$

Since $p_- \geq a_1/C_{v,r}$, we obtain

$$\left|\frac{p_{-,N}}{p_-} - 1\right| \leq C_{v,r} B_{N-2}(v,r).$$

Because $B_t(v,r)$ decreases with t, both outputs satisfy the uniform relative-error bound

$$\max_{b\in\{+,-\}} \left|\frac{p_{b,N}}{p_b} - 1\right| \leq C_{v,r} B_{N-2}(v,r).$$

Equivalently,

$$\boxed{\max_{b\in\{+,-\}} \left|\frac{p_{b,N}}{p_b} - 1\right| \leq C_{v,r} \frac{\exp\left(-\frac{((N-2)^2-1)\|v\|_2^2}{2r^2}\right)}{1-\exp\left(-\frac{3\|v\|_2^2}{2r^2}\right)}.}$$

For all proposed parameter sets, we have

$$r = 825 \qquad \text{and} \qquad 290 < \|v\|_2 \leq 296.19.$$

The worst-case bound is obtained at the smallest admissible norm. We therefore conservatively evaluate the bound at $\|v\|_2 = 290$. Let

$$q := \frac{290^2}{2 \cdot 825^2} \approx 0.0617815.$$

Then

$$C_{v,r} \leq \frac{1+e^{-q}}{1-e^{-q}} \approx 32.3825 < 2^{5.02}.$$

The sampler performs at most $\tau \leq 115$ transition steps. To ensure that the accumulated truncation error is at most 2^{-45}, it suffices to require

$$C_{v,r} B_{N-2}(v,r) \leq \frac{2^{-45}}{115}.$$

Retaining $N = 28$ terms is already sufficient. Indeed, the relevant Lemma 2 parameter for the negative output is $t = N-2 = 26$, for which

$$C_{v,r} B_{26}(v,r) \approx 2^{-52.58}.$$

Consequently,

$$115\, C_{v,r} B_{26}(v,r) \approx 2^{-45.73} < 2^{-45}.$$

However, this choice leaves less than one bit of margin with respect to the allocated truncation-error budget. We therefore retain one additional term and choose

$$\boxed{N = 29}.$$

The retained indices are thus $i = 0, \ldots, 28$. For the negative output, the corresponding Lemma 2 parameter is $t = N - 2 = 27$, and

$$C_{v,r} B_{27}(v,r) \approx 2^{-57.31}.$$

Hence, after at most 115 transition steps, the accumulated truncation error is bounded by

$$115\, C_{v,r} B_{27}(v,r) \approx 2^{-50.46} < 2^{-45}.$$

Thus, retaining 29 terms provides more than five bits of additional margin beyond the required truncation-error target.

Logarithm Approximation Error. Our final goal is to bound the relative error between the implemented single-step transition probabilities and their ideal counterparts. The following theorem shows that it suffices to control the pointwise absolute error of the logarithm component $\mathsf{log}_{\mathrm{rel}}$ in order to obtain a uniform relative-error bound on the single-step transition probabilities.

For fixed $\mathbf{y}$ and a nonzero vector $\mathbf{v}$, it suffices to consider the case $\langle \mathbf{y}, \mathbf{v} \rangle \geq 0$. The other case is handled in Algorithm R in the same way by replacing $\mathbf{y}$ with $-\mathbf{y}$.

For convenience, define

$$A_k(\mathbf{y}, \mathbf{v}) := \frac{\|\mathbf{y}\|_2^2 - \|\mathbf{y} + k\mathbf{v}\|_2^2}{2r^2},$$

and

$$a_k(\mathbf{y}, \mathbf{v}) := \exp\big(A_k(\mathbf{y}, \mathbf{v})\big).$$

When no ambiguity arises, we simply write A_k and a_k.

Assume that the truncation parameter N is odd. The ideal truncated transition probability is

$$S_{\mathrm{trunc}}(\mathbf{y}, \mathbf{v}) := \sum_{k=0}^{N} (-1)^k a_k.$$

Accordingly, the ideal probabilities of the two outputs are defined as

$$p_+ := S_{\mathrm{trunc}}(\mathbf{y}, \mathbf{v}) = \sum_{k=0}^{N} (-1)^k a_k,$$

and

$$p_- := 1 - p_+ = \sum_{k=1}^{N} (-1)^{k+1} a_k.$$

Let $\widetilde{p}_+$ and $\widetilde{p}_-$ denote the corresponding output probabilities induced by the approximate logarithm component.

We further define the parameter-dependent constant

$$C_{\mathbf{v},r} := \frac{1 + \exp\left(-\|\mathbf{v}\|_2^2 / 2r^2\right)}{1 - \exp\left(-\|\mathbf{v}\|_2^2 / 2r^2\right)}.$$

The logarithm component is realized in *base two* (Section 2.5.8): $\mathsf{SamplerU}$ returns $\ell = \mathsf{ApproxLog}(a, m) \approx \log_2 U$, and the transition test forms the natural-logarithm comparison

value by the single multiplication $\mathsf{log}_{\mathrm{rel}}(u) = (\ln 2)\,\ell \approx \ln U$. The theorem below is stated in natural-logarithm terms, with which it is applied here; the corresponding additive error is $\delta_{\log} = (\ln 2)\sup_{a,m}|\Delta_{a,b}|$, where $\Delta_{a,b} = \log_2 u - \ell$ is the base-two error bounded in Section 2.5.8. With the realized $|\Delta_{a,b}| \le 2^{-59.37}$ this gives $\delta_{\log} \le (\ln 2)\,2^{-59.37} \le 2^{-59.9}$, comfortably within the budget $\eta_{\log} \le 2^{-57}$.

Theorem 2 (From absolute logarithm error to relative transition error)**.** *Let* $\mathsf{log}_{\mathrm{rel}}$ *be the logarithm component used by the implementation, and define*

$$\Delta_u := \log u - \mathsf{log}_{\mathrm{rel}}(u).$$

Assume that $\mathsf{log}_{\mathrm{rel}}$ *is strictly increasing over its valid input interval, satisfies*

$$\mathsf{log}_{\mathrm{rel}}(1) = 0,$$

and that every comparison threshold A_k *lies in the image of* $\mathsf{log}_{\mathrm{rel}}$*.*

If

$$|\Delta_u| \le \delta_{\log}$$

for every valid input $u \in (0,1]$*, then, for both outputs* $b \in \{+,-\}$*,*

$$\left|\frac{\widetilde{p}_b}{p_b} - 1\right| \le C_{\mathbf{v},r}\big(e^{\delta_{\log}} - 1\big).$$

Proof. In the computation of the transition probability, the implementation compares $\mathsf{log}_{\mathrm{rel}}(u)$ with

$$A_k(\mathbf{y},\mathbf{v}) = \frac{\|\mathbf{y}\|_2^2 - \|\mathbf{y}+k\mathbf{v}\|_2^2}{2r^2}.$$

Since $\mathsf{log}_{\mathrm{rel}}$ is strictly increasing, the k-th comparison has a unique implemented threshold u'_k satisfying

$$\mathsf{log}_{\mathrm{rel}}(u'_k) = A_k(\mathbf{y},\mathbf{v}).$$

By the definition of Δ_u,

$$\log u'_k = \mathsf{log}_{\mathrm{rel}}(u'_k) + \Delta_{u'_k} = A_k(\mathbf{y},\mathbf{v}) + \Delta_{u'_k}.$$

It follows that

$$u'_k = \exp\big(A_k(\mathbf{y},\mathbf{v})\big)\exp(\Delta_{u'_k}) = a_k \exp(\Delta_{u'_k}).$$

Let

$$\widetilde{a}_k := u'_k.$$

Since $|\Delta_{u'_k}| \le \delta_{\log}$, we have

$$e^{-\delta_{\log}} a_k \le \widetilde{a}_k \le e^{\delta_{\log}} a_k,$$

and hence

$$|\widetilde{a}_k - a_k| \le \big(e^{\delta_{\log}} - 1\big)a_k.$$

In particular, since

$$A_0 = 0 \qquad \text{and} \qquad \mathsf{log}_{\mathrm{rel}}(1) = 0,$$

we have

$$a_0 = \widetilde{a}_0 = 1.$$

Moreover,

$$A_{k+1} - A_k = -\frac{2\langle \mathbf{y},\mathbf{v}\rangle + (2k+1)\|\mathbf{v}\|_2^2}{2r^2} < 0.$$

Thus, $(A_k)_{k\geq 0}$ is strictly decreasing. Together with the strict monotonicity of $\mathsf{log}_{\mathrm{rel}}$, this implies that the implemented thresholds $\widetilde{a}_k$ preserve the same ordering as the ideal thresholds. Therefore,

$$\widetilde{p}_+ = \sum_{k=0}^{N} (-1)^k \widetilde{a}_k.$$

Since N is odd and $\widetilde{a}_0 = 1$, we also have

$$\widetilde{p}_- = 1 - \widetilde{p}_+ = \sum_{k=1}^{N} (-1)^{k+1} \widetilde{a}_k.$$

We first consider the output $+$. By the triangle inequality,

$$\begin{aligned} |\widetilde{p}_+ - p_+| &= \left| \sum_{k=0}^{N} (-1)^k (\widetilde{a}_k - a_k) \right| \\ &\leq \sum_{k=0}^{N} |\widetilde{a}_k - a_k| \\ &\leq \left(e^{\delta_{\log}} - 1\right) \sum_{k=0}^{N} a_k. \end{aligned}$$

Define

$$T_+ := \sum_{k=0}^{N} a_k.$$

For every $k \geq 0$,

$$\frac{a_{k+1}}{a_k} = \exp\left(-\frac{2\langle \mathbf{y}, \mathbf{v} \rangle + (2k+1)\|\mathbf{v}\|_2^2}{2r^2} \right) \leq \exp\left(-\frac{\|\mathbf{v}\|_2^2}{2r^2} \right).$$

Let

$$\rho_{\mathbf{v},r} := \exp\left(-\frac{\|\mathbf{v}\|_2^2}{2r^2} \right).$$

Since $\mathbf{v} \neq \mathbf{0}$, we have

$$0 < \rho_{\mathbf{v},r} < 1.$$

Pairing adjacent terms in p_+ gives

$$p_+ = (a_0 - a_1) + (a_2 - a_3) + \cdots + (a_{N-1} - a_N).$$

For every pair,

$$a_{2j+1} \leq \rho_{\mathbf{v},r} a_{2j},$$

and therefore

$$a_{2j} + a_{2j+1} \leq \frac{1 + \rho_{\mathbf{v},r}}{1 - \rho_{\mathbf{v},r}} (a_{2j} - a_{2j+1}).$$

Summing over all pairs yields

$$T_+ \leq C_{\mathbf{v},r} p_+,$$

where

$$C_{\mathbf{v},r} = \frac{1 + \rho_{\mathbf{v},r}}{1 - \rho_{\mathbf{v},r}}.$$

Consequently,

$$|\widetilde{p}_+ - p_+| \leq C_{\mathbf{v},r} \left(e^{\delta_{\log}} - 1\right) p_+,$$

and hence

$$\left|\frac{\widetilde{p}_+}{p_+} - 1\right| \le C_{\mathbf{v},r}\big(e^{\delta_{\log}} - 1\big).$$

We next consider the output $-$. Similarly,

$$|\widetilde{p}_- - p_-| = \left|\sum_{k=1}^{N}(-1)^{k+1}(\widetilde{a}_k - a_k)\right| \le \big(e^{\delta_{\log}} - 1\big)\sum_{k=1}^{N} a_k.$$

Define

$$T_- := \sum_{k=1}^{N} a_k.$$

At the same time,

$$p_- = (a_1 - a_2) + (a_3 - a_4) + \cdots + a_N.$$

For every complete pair,

$$a_{2j+2} \le \rho_{\mathbf{v},r} a_{2j+1},$$

and therefore

$$a_{2j+1} + a_{2j+2} \le C_{\mathbf{v},r}(a_{2j+1} - a_{2j+2}).$$

If the final term a_N is unpaired, then

$$a_N \le C_{\mathbf{v},r} a_N$$

because $C_{\mathbf{v},r} \ge 1$. Summing over all complete pairs and the final unpaired term gives

$$T_- \le C_{\mathbf{v},r} p_-.$$

Therefore,

$$|\widetilde{p}_- - p_-| \le C_{\mathbf{v},r}\big(e^{\delta_{\log}} - 1\big)p_-,$$

and hence

$$\left|\frac{\widetilde{p}_-}{p_-} - 1\right| \le C_{\mathbf{v},r}\big(e^{\delta_{\log}} - 1\big).$$

Thus, for both outputs $b \in \{+, -\}$,

$$\left|\frac{\widetilde{p}_b}{p_b} - 1\right| \le C_{\mathbf{v},r}\big(e^{\delta_{\log}} - 1\big).$$

□

For all proposed parameter sets, we use $r = 825$. Moreover, since $\mathbf{v}$ is a cyclic shift of $\mathbf{s}$, it satisfies

$$\|\mathbf{v}\|_2 = \|\mathbf{s}\|_2,$$

and the proposed parameters ensure that

$$290 < \|\mathbf{v}\|_2 \le 296.19.$$

It follows that

$$C_{\mathbf{v},r} < 2^{5.02}.$$

The iterative sampler performs at most $\tau \le 115$ transition steps. Given an overall relative-error budget of 2^{-45}, it is therefore sufficient to require the pointwise absolute error of the logarithm approximation, *together with the* SamplerU *mantissa quantization* $\varepsilon_{\mathrm{q}} \le 2^{-58}$ (Section 2.5.8), to satisfy

$$\delta_{\log} + \varepsilon_{\mathrm{q}} \le 2^{-56.9}.$$

Indeed, using

$$e^{\delta_{\log}} - 1 \approx \delta_{\log}$$

in this small-error regime (the quantization term enters additively and is amplified by the same factor), the accumulated error is bounded by

$$\tau C_{\mathbf{v},r}\big(\delta_{\log} + \varepsilon_{\mathrm{q}}\big) < 115 \cdot 2^{5.02} \cdot 2^{-56.9} < 2^{-45}.$$

The realized values are $\delta_{\log} + \varepsilon_{\mathrm{q}} = 2^{-59.9} + 2^{-58} = 2^{-57.67}$, giving $2^{-45.8}$ in this bound.

4.1.3 Multi-Step Composition and Rényi Divergence Bound

We now combine the intrinsic error of the ideal transition, the truncation error of the alternating series, and the logarithm approximation error into a single-step pointwise relative-error bound, and then compose this bound over the sequence of all iterations.

Before doing so, we formally define the transition operators. For a shift vector v, let T_v denote the ideal transition operator defined by the exact functions f_v and g_v, where the infinite series S_v is evaluated exactly and exact real arithmetic is used in the interval test. For any input distribution ρ, the output distribution after one ideal transition step is denoted by $\rho' = \mathsf{T}_v(\rho)$. Similarly, let $\widetilde{\mathsf{T}}_v$ denote the concrete transition operator implemented in practice, where the infinite series is truncated after N terms and the logarithm in the interval test is evaluated with an additive error of at most $\eta_{\log}$. The output distribution of this implemented transition step is denoted by $\widetilde{\rho}' = \widetilde{\mathsf{T}}_v(\rho)$.

For a fixed state y and shift vector v, the truncation of the alternating series and the finite-precision evaluation of the logarithm induce an absolute deviation in the implemented transition probability. More precisely, if $\widetilde{p}_v(y)$ denotes the concrete transition probability and $p_v(y)$ denotes the ideal one, then

$$|\widetilde{p}_v(y) - p_v(y)| \le \varepsilon_S(v, N) + \varepsilon_{\log},$$

where

$$\varepsilon_{\log} \le B \cdot \frac{\eta_{\log}}{r^2}.$$

Here, the logarithm approximation error is interpreted as a conditional probability bound: for fixed y and v, the finite-precision interval test differs from the exact interval test only when the ideal value $u = 2r^2 \ln U$ lies within distance $\eta_{\log}$ of one of the B tested interval boundaries.

Let $\rho'_v = \mathsf{T}_v(\rho_g)$ and $\widetilde{\rho}'_v = \widetilde{\mathsf{T}}_v(\rho_g)$ be the output distributions of the ideal and implemented transitions, respectively, when the input distribution is the target Gaussian ρ_g. By expanding the transition step, we have

$$\rho'_v(y) = \rho_g(y+v)p_v(y+v) + \rho_g(y-v)(1 - p_v(y-v)),$$

and

$$\widetilde{\rho}'_v(y) = \rho_g(y+v)\widetilde{p}_v(y+v) + \rho_g(y-v)(1 - \widetilde{p}_v(y-v)).$$

Subtracting these two equations and applying the triangle inequality yields

$$\begin{aligned}|\widetilde{\rho}'_v(y) - \rho'_v(y)| &= \big|\rho_g(y+v)\big(\widetilde{p}_v(y+v) - p_v(y+v)\big) + \rho_g(y-v)\big((1 - \widetilde{p}_v(y-v)) - (1 - p_v(y-v))\big)\big| \\ &\le \big(\rho_g(y+v) + \rho_g(y-v)\big)\big(\varepsilon_S(v,N) + \varepsilon_{\log}\big).\end{aligned}$$

Consequently, on a good set $\mathcal{S}$ supporting the sampler outputs, we define the maximum expansion factor as

$$C_v(\mathcal{S}) = \max_{y \in \mathcal{S}} \frac{\rho_g(y+v) + \rho_g(y-v)}{\rho_g(y)}.$$

Dividing the absolute discrepancy by $\rho_g(y)$ and applying the triangle inequality, we can bound the overall pointwise relative error of the implemented transition relative to the target distribution ρ_g:

$$\left|\frac{\widetilde{\rho}'_v(y)}{\rho_g(y)} - 1\right| \leq \left|\frac{\rho'_v(y)}{\rho_g(y)} - 1\right| + \frac{\rho_g(y+v) + \rho_g(y-v)}{\rho_g(y)}\big(\varepsilon_S(v,N) + \varepsilon_{\log}\big).$$

Combining this with the symmetric intrinsic relative error bound of the ideal transition (Lemma 3) gives the following single-step relative-error bound on $\mathcal{S}$:

$$\varepsilon_v = \varepsilon_{\text{int}}(v) + C_v(\mathcal{S})\left(\varepsilon_S(v,N) + B \cdot \frac{\eta_{\log}}{r^2}\right).$$

That is, for all $y \in \mathcal{S}$, the implemented single-step transition satisfies

$$1 - \varepsilon_v \leq \frac{\widetilde{\mathsf{T}}_v(\rho_g)(y)}{\rho_g(y)} \leq 1 + \varepsilon_v.$$

We next compose this bound over the sequence of transitions. The following lemma formalizes how these pointwise relative errors accumulate, relying only on the positivity and linearity of the transition kernel.

Lemma 4 (Composition of pointwise relative errors)**.** *Let ρ be a distribution satisfying*

$$(1-\delta)\rho_g(x) \leq \rho(x) \leq (1+\delta)\rho_g(x)$$

for all $x \in \mathcal{S}$. Let $\widetilde{\mathsf{T}}_v$ be a Markov transition kernel such that

$$(1-\varepsilon_v)\rho_g(y) \leq \widetilde{\mathsf{T}}_v(\rho_g)(y) \leq (1+\varepsilon_v)\rho_g(y)$$

for all $y \in \mathcal{S}$. Then the output distribution satisfies

$$(1-\delta)(1-\varepsilon_v) \leq \frac{\widetilde{\mathsf{T}}_v(\rho)(y)}{\rho_g(y)} \leq (1+\delta)(1+\varepsilon_v).$$

In particular, the accumulated relative error satisfies

$$\left|\frac{\widetilde{\mathsf{T}}_v(\rho)(y)}{\rho_g(y)} - 1\right| \leq (1+\delta)(1+\varepsilon_v) - 1.$$

Proof. Let $\widetilde{K}_v(x,y)$ be the non-negative transition kernel associated with the operator $\widetilde{\mathsf{T}}_v$, so that $\widetilde{\mathsf{T}}_v(\rho)(y) = \sum_x \rho(x)\widetilde{K}_v(x,y)$. Since $\widetilde{K}_v(x,y) \geq 0$ for all x, y, we can multiply the pointwise inequality $(1-\delta)\rho_g(x) \leq \rho(x) \leq (1+\delta)\rho_g(x)$ by $\widetilde{K}_v(x,y)$ and sum over x to obtain

$$(1-\delta)\sum_x \rho_g(x)\widetilde{K}_v(x,y) \leq \sum_x \rho(x)\widetilde{K}_v(x,y) \leq (1+\delta)\sum_x \rho_g(x)\widetilde{K}_v(x,y),$$

which is equivalent to

$$(1-\delta)\widetilde{\mathsf{T}}_v(\rho_g)(y) \leq \widetilde{\mathsf{T}}_v(\rho)(y) \leq (1+\delta)\widetilde{\mathsf{T}}_v(\rho_g)(y).$$

Using the single-step relative-error bound $(1-\varepsilon_v)\rho_g(y) \leq \widetilde{\mathsf{T}}_v(\rho_g)(y) \leq (1+\varepsilon_v)\rho_g(y)$, we get

$$(1-\delta)(1-\varepsilon_v)\rho_g(y) \leq \widetilde{\mathsf{T}}_v(\rho)(y) \leq (1+\delta)(1+\varepsilon_v)\rho_g(y).$$

Dividing by $\rho_g(y)$, we obtain the desired pointwise bounds. For the symmetric relative error, we observe that

$$1 - (1-\delta)(1-\varepsilon_v) = \delta + \varepsilon_v - \delta\varepsilon_v,$$

and

$$(1+\delta)(1+\varepsilon_v) - 1 = \delta + \varepsilon_v + \delta\varepsilon_v.$$

Since $\delta + \varepsilon_v + \delta\varepsilon_v \geq \delta + \varepsilon_v - \delta\varepsilon_v$ for $\delta, \varepsilon_v \geq 0$, the symmetric relative error is bounded by $(1+\delta)(1+\varepsilon_v) - 1$. □

Applying Lemma 4 inductively to the sequence of transition steps with shift vectors $v_1, \ldots, v_\tau$, the final output distribution ρ_τ satisfies

$$1 - \delta_\tau \leq \frac{\rho_\tau(y)}{\rho_g(y)} \leq 1 + \delta_\tau,$$

where

$$\delta_\tau = \prod_{i=1}^{\tau}(1 + \varepsilon_{v_i}) - 1.$$

If all steps are bounded by a uniform single-step error $\varepsilon_{v_i} \leq \varepsilon_{\max}$, then the accumulated relative error simplifies to

$$\delta_\tau \leq (1 + \varepsilon_{\max})^\tau - 1.$$

Finally, we apply Lemma 3 of [40] to convert this accumulated pointwise relative-error bound into a Rényi divergence bound.

Lemma 5 (Relative-error bound for Rényi divergence [40, Lemma 3])**.** *Let P and Q be two probability distributions with the same support. Assume that there exists some $0 < \delta < 1$ such that, for every x in their common support,*

$$1 - \delta \leq \frac{P(x)}{Q(x)} \leq 1 + \delta.$$

Then, for every $\alpha > 1$, their Rényi divergence satisfies

$$R_\alpha(P\|Q) \leq \left(1 + \frac{\alpha(\alpha-1)\delta^2}{2(1-\delta)^{\alpha+1}}\right)^{\frac{1}{\alpha-1}}.$$

By setting $P = \rho_\tau$, $Q = \rho_g$, and $\delta = \delta_\tau$, we obtain the final Rényi divergence bound for our approximate iterative rejection sampler:

$$R_\alpha(\rho_\tau\|\rho_g) \leq \left(1 + \frac{\alpha(\alpha-1)\delta_\tau^2}{2(1-\delta_\tau)^{\alpha+1}}\right)^{\frac{1}{\alpha-1}}.$$

For a sufficiently small accumulated error δ_τ, this bound behaves asymptotically as

$$R_\alpha(\rho_\tau\|\rho_g) = 1 + \frac{\alpha}{2}\delta_\tau^2 + O(\delta_\tau^3).$$

4.1.4 Concrete Rényi-Divergence Estimate

We now instantiate the preceding bounds with the concrete parameters from Table 10. The purpose of this subsection is to verify that the accumulated distributional loss of the iterative sampler remains below the target Rényi-loss budget for the allowed number of signing queries.

Target Rényi-loss budget. Following the practical use of Prest's relative-error lemma, we measure the sampling loss through the Rényi divergence $R_{2\lambda}(D_{\text{out}}\|D_{\text{tar}})$. If the final output distribution satisfies a pointwise relative-error bound with parameter δ_τ, then Lemma 5 gives

$$R_{2\lambda}(D_{\text{out}}\|D_{\text{tar}}) \leq \left(1 + \frac{(2\lambda)(2\lambda-1)\delta_\tau^2}{2(1-\delta_\tau)^{2\lambda+1}}\right)^{\frac{1}{2\lambda-1}}.$$

For small δ_τ, this behaves as

$$R_{2\lambda}(D_{\text{out}}\|D_{\text{tar}}) = 1 + \lambda\delta_\tau^2 + O(\lambda\delta_\tau^3).$$

Since our signing algorithm allows at most $q_s < 2^{80}$ signing queries, we target the concrete bound

$$R_{2\lambda}(D_{\text{out}} \| D_{\text{tar}}) \leq 1 + 2^{-82}.$$

This follows the same rule of thumb as the practical Prest-style analysis: the Rényi-loss slack is chosen slightly below the inverse of the maximum number of signing queries. Ignoring the negligible factor $(1-\delta_\tau)^{-(2\lambda+1)}$, it suffices to require $\lambda\delta_\tau^2 \leq 2^{-82}$. For $\lambda \leq 256$, this condition is implied by $2^8\delta_\tau^2 \leq 2^{-82}$,or equivalently $\delta_\tau \leq 2^{-45}$. Thus, in the concrete analysis below, it is sufficient to show that the final accumulated relative error satisfies $\delta_\tau \leq 2^{-45}$.

Concrete intrinsic error. For each parameter set, the intrinsic error is maximized when the shift vector has the largest allowed norm. Since each iterative transition uses a shift vector satisfying $\|v\|_2 \leq B_k$, we use the conservative lower bound

$$\beta_{\min} = \frac{r}{B_k}.$$

By Lemma 3, the one-step intrinsic relative error is bounded by

$$\varepsilon_{\text{int}} \leq \frac{2\beta_{\min}\sqrt{2\pi}\exp\left(-\frac{\pi^2\beta_{\min}^2}{2}\right)}{\exp\left(-\frac{1}{2\beta_{\min}^2}\right)\left(1-\exp\left(-2\pi^2\beta_{\min}^2\right)\right)}.$$

Equivalently, in logarithmic form,

$$\log_2 \varepsilon_{\text{int}} = \log_2(2\beta_{\min}\sqrt{2\pi}) - \frac{\pi^2\beta_{\min}^2}{2\ln 2} + \frac{1}{2\beta_{\min}^2\ln 2} - \log_2\left(1-\exp(-2\pi^2\beta_{\min}^2)\right).$$

For the proposed parameter sets, $r = 825$ and the values of B_k are given in Table 10. Substituting these values gives

Security	r	B_k	$\beta_{\min} = r/B_k$
128	825	295.06	2.796
256	825	296.19	2.785
512	825	292.76	2.818

and hence

Security	$\beta_{\min}$	$\log_2 \varepsilon_{\text{int}}$
128	2.796	−51.80
256	2.785	−51.36
512	2.818	−52.72

Thus, the intrinsic one-step relative error is approximately 2^{-51} for all parameter sets.

Accumulated relative error. Let τ denote the maximum number of iterative transition steps. Considering only the intrinsic error, the accumulated relative error is

$$\delta_\tau^{\text{int}} = (1+\varepsilon_{\text{int}})^\tau - 1.$$

Since ε_{int} is very small, we use the standard approximation

$$\delta_\tau^{\text{int}} = \tau\varepsilon_{\text{int}} + O(\tau^2\varepsilon_{\text{int}}^2).$$

In base-2 logarithmic form, this gives

$$\log_2 \delta_\tau^{\text{int}} \approx \log_2 \tau + \log_2 \varepsilon_{\text{int}}.$$

Substituting the concrete values of τ from Table 10, we obtain

Security	τ	$\log_2 \varepsilon_{\text{int}}$	$\log_2 \delta_\tau^{\text{int}}$
128	42	-51.80	-46.40
256	58	-51.36	-45.50
512	115	-52.72	-45.87

where we used

$$\log_2 42 \approx 5.39, \qquad \log_2 58 \approx 5.86, \qquad \log_2 115 \approx 6.85.$$

Therefore, all three parameter sets satisfy the target $\delta_\tau \le 2^{-45}$ at the intrinsic-error level.

Concrete Rényi-divergence values. We next compute the corresponding Rényi-divergence increment. We instantiate the bound with the per-level Rényi order 2λ of the security reduction, i.e. $\alpha = 2\lambda \in \{256, 512, 1024\}$ for SUF-128/256/512. For small δ_τ, Lemma 5 gives

$$R_{2\lambda}(\rho_\tau \| \rho_g) - 1 \approx \frac{2\lambda}{2} \left(\delta_\tau^{\text{int}}\right)^2 = \lambda \left(\delta_\tau^{\text{int}}\right)^2.$$

Taking base-2 logarithms, we get

$$\log_2 \left(R_{2\lambda}(\rho_\tau \| \rho_g) - 1\right) \approx \log_2 \lambda + 2 \log_2 \delta_\tau^{\text{int}}.$$

Substituting the values above gives

Security	$\log_2 \delta_\tau^{\text{int}}$	$\log_2 \left(R_{2\lambda}(\rho_\tau \| \rho_g) - 1\right)$
128	-46.40	-85.80
256	-45.50	-83.00
512	-45.87	-82.74

Consequently, for every proposed parameter set, the intrinsic contribution to the Rényi-divergence increment satisfies

$$R_{2\lambda}(\rho_\tau \| \rho_g) - 1 < 2^{-82}.$$

This meets the target Rényi-loss budget for $q_s < 2^{80}$ signing queries, with about 0.7 bit to spare at the tightest (SUF-512) level.

Concrete truncation error. We now repeat the concrete analysis for the truncation error, in the same form as for the intrinsic error above. Recall from Section 4.1.2 that, after the sign normalization $\langle y, v \rangle \ge 0$, the one-step relative truncation error of the implemented transition (retaining N terms, indexed by $i = 0, \ldots, N-1$) is bounded by

$$\varepsilon_{\text{tr}}(v, N) \le C_{v,r}\, B_{N-2}(v, r),$$

where

$$C_{v,r} = \frac{1 + \exp\!\left(-\|v\|_2^2/(2r^2)\right)}{1 - \exp\!\left(-\|v\|_2^2/(2r^2)\right)}, \qquad B_t(v, r) = \frac{\exp\!\left(-\frac{(t^2-1)\|v\|_2^2}{2r^2}\right)}{1 - \exp\!\left(-\frac{3\|v\|_2^2}{2r^2}\right)}.$$

In contrast to the intrinsic error, the truncation error is maximized at the *smallest* admissible shift norm, since both $C_{v,r}$ and $B_t(v, r)$ decrease as $\|v\|_2$ grows. For all three parameter sets we have $r = 825$ and $290 < \|v\|_2 \le 296.19$, so we conservatively evaluate the bound at the smallest admissible norm $\|v\|_2 = 290$, which yields the uniform per-step estimate

$$C_{v,r} < 2^{5.02}, \qquad \varepsilon_{\text{tr}}(v, N) \le C_{v,r} B_{N-2}(v, r).$$

With the chosen truncation parameter $N = 29$, the relevant parameter for the dominant (negative) output is $t = N - 2 = 27$, and substituting $\|v\|_2 = 290$ gives

$$\varepsilon_{\mathrm{tr}} \le C_{v,r} B_{27}(v, r) \approx 2^{-57.31}.$$

Because this per-step bound is evaluated at the common worst-case norm and truncation parameter, it applies uniformly to all three parameter sets.

Accumulated truncation error. Let τ denote the maximum number of iterative transition steps. As for the intrinsic error, the accumulated truncation error is

$$\delta_\tau^{\mathrm{tr}} = (1 + \varepsilon_{\mathrm{tr}})^\tau - 1 = \tau\varepsilon_{\mathrm{tr}} + O(\tau^2 \varepsilon_{\mathrm{tr}}^2),$$

so that, in base-2 logarithmic form,

$$\log_2 \delta_\tau^{\mathrm{tr}} \approx \log_2 \tau + \log_2 \varepsilon_{\mathrm{tr}}.$$

Substituting the concrete values of τ from Table 10 gives

Security	τ	$\log_2 \varepsilon_{\mathrm{tr}}$	$\log_2 \delta_\tau^{\mathrm{tr}}$
128	42	-57.31	-51.92
256	58	-57.31	-51.45
512	115	-57.31	-50.46

where we used $\log_2 42 \approx 5.39$, $\log_2 58 \approx 5.86$, and $\log_2 115 \approx 6.85$. Thus the accumulated truncation error is below 2^{-50} for every parameter set, more than five bits under the target $\delta_\tau \le 2^{-45}$.

Truncation Rényi-divergence values. We convert the accumulated truncation error into a Rényi-divergence increment exactly as for the intrinsic error, using the same per-level Rényi order $\alpha = 2\lambda$. By Lemma 5, for small $\delta_\tau^{\mathrm{tr}}$,

$$R_{2\lambda}(\rho_\tau^{\mathrm{tr}} \| \rho_g) - 1 \approx \lambda \left(\delta_\tau^{\mathrm{tr}}\right)^2, \qquad \log_2(R_{2\lambda} - 1) \approx \log_2 \lambda + 2 \log_2 \delta_\tau^{\mathrm{tr}}.$$

Substituting the accumulated values above gives

Security	$\log_2 \delta_\tau^{\mathrm{tr}}$	$\log_2 (R_{2\lambda} - 1)$
128	-51.92	-96.84
256	-51.45	-94.90
512	-50.46	-91.92

Hence the truncation-induced Rényi-divergence increment satisfies

$$R_{2\lambda}(\rho_\tau^{\mathrm{tr}} \| \rho_g) - 1 < 2^{-91.9}$$

for every parameter set, more than ten bits below the intrinsic contribution ($\approx 2^{-82.7}$ to $2^{-85.8}$) and far below the overall target 2^{-82}. The truncation error is therefore negligible relative to the intrinsic loss.

Concrete logarithm-approximation error. The per-step logarithm-approximation relative error is controlled by Theorem 2: with the realized base-two bound $|\Delta_{a,b}| \le 2^{-59.37}$ of Section 2.5.8, the induced additive transition error is $\delta_{\log} \le (\ln 2)\, 2^{-59.37} \le 2^{-59.9}$, within the budget $\eta_{\log} \le 2^{-57}$. Adding the SamplerU mantissa quantization $\varepsilon_{\mathrm{q}} \le 2^{-58}$ (Section 2.5.8) and applying the same worst-case amplification $C_{v,r}$ as in the intrinsic analysis gives a per-step relative error $\delta_{\mathrm{tot}}^{\log} \le C_{v,r}(\mathrm{e}^{\delta_{\log}} - 1 + \varepsilon_{\mathrm{q}}) \lesssim 2^{-52.6}$, hence, as in the *Accumulated truncation error* paragraph, $\delta_\tau^{\log} \lesssim 2^{-45.8}$ and, through Lemma 5, $M_5 - 1 \le \lambda(\delta_\tau^{\log})^2 \lesssim 2^{-82.6}$ at the tightest (SUF-512) level—the same order as M_3 and, like it, inside the target 2^{-82}.

Conclusion. For $q_s < 2^{80}$ signing queries, we target

$$R_{2\lambda}(D_{\text{out}} \| D_{\text{tar}}) \leq 1 + 2^{-82}.$$

In the small-error regime this reduces to the sufficient condition $\delta_\tau \leq 2^{-45}$. Substituting the concrete parameters gives $\delta_\tau^{\text{int}} \approx 2^{-45.5}$ to $2^{-46.4}$ and $R_{2\lambda} - 1 \approx 2^{-82.7}$ to $2^{-85.8}$ at the intrinsic-error level. Therefore, the proposed parameters meet the target Rényi-loss budget. The additional truncation error is bounded directly above and contributes a Rényi increment below $2^{-91.9}$ for every parameter set, more than nine bits below the intrinsic contribution; the logarithm-approximation error is analyzed analogously, with the numbers settled above. Together these keep the full implementation loss within the same concrete security budget.

4.2 Theoretical Security

4.2.1 Security Models

We use the standard existential unforgeability notions for digital signatures, in both their ordinary and strong forms. A signature scheme $\mathsf{SIG} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ consists of a randomized key-generation algorithm, a possibly randomized signing algorithm, and a deterministic verification algorithm. For a public key vk and message μ, a pair (μ, σ) is valid if $\mathsf{Verify}(vk, \mu, \sigma) = 1$.

In the *unforgeability under chosen-message attacks* experiment $\mathsf{UF\text{-}CMA}$, the adversary is given the verification key vk and oracle access to $\mathsf{Sign}(sk, \cdot)$. It may adaptively request signatures on messages of its choice and finally outputs a candidate forgery (μ^*, σ^*). Let $\mathcal{M}$ be the set of messages queried to the signing oracle. The adversary wins if

$$\mathsf{Verify}(vk, \mu^*, \sigma^*) = 1 \qquad \text{and} \qquad \mu^* \notin \mathcal{M}.$$

The corresponding advantage is denoted

$$\mathsf{Adv}_{\mathsf{SIG}}^{\mathsf{UF\text{-}CMA}}(\mathcal{A}) := \Pr[\mathcal{A} \text{ wins the } \mathsf{UF\text{-}CMA} \text{ experiment}].$$

This is the usual existential unforgeability notion: after seeing signatures on adaptively chosen messages, the adversary must produce a valid signature on a new message.

The *strong unforgeability under chosen-message attacks* experiment $\mathsf{sUF\text{-}CMA}$ strengthens the winning condition. The challenger now records the full set

$$\mathcal{Q} := \{(\mu, \sigma) : \sigma \leftarrow \mathsf{Sign}(sk, \mu) \text{ was returned to } \mathcal{A}\}.$$

The adversary wins if

$$\mathsf{Verify}(vk, \mu^*, \sigma^*) = 1 \qquad \text{and} \qquad (\mu^*, \sigma^*) \notin \mathcal{Q}.$$

Thus, unlike in $\mathsf{UF\text{-}CMA}$, the forged message μ^* may have been queried before; what is forbidden is outputting exactly a signature that was already returned by the signing oracle. Consequently, a successful $\mathsf{sUF\text{-}CMA}$ adversary may either forge a signature on a fresh message or produce a new, distinct valid signature on a previously signed message. This stronger notion is particularly relevant for randomized or compressed signatures, where the same message may admit multiple syntactic signatures unless uniqueness of responses or an equivalent property is proved.

We also use the weaker *unforgeability under no-message attacks* experiment $\mathsf{UF\text{-}NMA}$ as an intermediate target in the reduction. In this experiment the adversary receives only the verification key and has no access to a signing oracle. It wins if it outputs a valid pair (μ^*, σ^*). Its advantage is denoted $\mathsf{Adv}_{\mathsf{SIG}}^{\mathsf{UF\text{-}NMA}}(\mathcal{A})$. The purpose of the Fiat–Shamir proof below is to reduce adaptive chosen-message security to this no-message security notion: signing queries are gradually replaced by simulated transcripts, and the loss incurred in this replacement is measured by the Rényi divergence between the honest and simulated transcript distributions.

Finally, we record a *runtime-aware* refinement of $\mathsf{UF\text{-}CMA}$ that is needed to analyse the *unbounded* signing algorithm of our scheme, whose iterative rejection sampler carries no a priori cap on the number of iterations. Let $T' = T'(\lambda)$ be a function of the security parameter. The experiment T'-$\mathsf{UF\text{-}CMA}$ is defined exactly as $\mathsf{UF\text{-}CMA}$ above, except that the adversary is *additionally* declared the winner as soon as it issues a signing query on which $\mathsf{Sign}(sk, \cdot)$ runs for more than T' steps before halting. Writing slow for this timeout event, the adversary wins if

$$\big(\mathsf{Verify}(vk, \mu^*, \sigma^*) = 1 \ \wedge \ \mu^* \notin \mathcal{M}\big) \ \vee \ \mathsf{slow},$$

and the corresponding advantage is denoted $\mathsf{Adv}_{\mathsf{SIG}}^{T'\text{-}\mathsf{UF\text{-}CMA}}(\mathcal{A})$. Likewise, the experiment T'-$\mathsf{sUF\text{-}CMA}$ is defined exactly as $\mathsf{sUF\text{-}CMA}$ above, except that the adversary is *additionally* declared the winner on the same timeout event slow—so it wins if $\big(\mathsf{Verify}(vk, \mu^*, \sigma^*) = 1 \wedge (\mu^*, \sigma^*) \notin \mathcal{Q}\big) \vee \mathsf{slow}$—and its advantage is denoted $\mathsf{Adv}_{\mathsf{SIG}}^{T'\text{-}\mathsf{sUF\text{-}CMA}}(\mathcal{A})$. The same modification applied to the single-signature variant $\mathsf{UF\text{-}CMA}_1$ yields T'-$\mathsf{UF\text{-}CMA}_1$.

Because the clause slow only *adds* executions in which the adversary is deemed to win, $\mathsf{UF\text{-}CMA}$ is the *weaker* of the two notions and T'-$\mathsf{UF\text{-}CMA}$ is a strengthening of it: for every scheme SIG and every adversary $\mathcal{A}$,

$$\mathsf{Adv}_{\mathsf{SIG}}^{\mathsf{UF\text{-}CMA}}(\mathcal{A}) \ \le \ \mathsf{Adv}_{\mathsf{SIG}}^{T'\text{-}\mathsf{UF\text{-}CMA}}(\mathcal{A}).$$

The inequality holds unconditionally: running the two experiments on the same key, adversarial coins, and oracle answers, every run in which $\mathcal{A}$ wins the $\mathsf{UF\text{-}CMA}$ game—a valid forgery on a fresh message—is also a winning run of the T'-$\mathsf{UF\text{-}CMA}$ game, whose winning event is the union of that same forgery event with slow; hence the latter probability, and thus the advantage, can only be larger. Consequently T'-$\mathsf{UF\text{-}CMA}$ *security implies* $\mathsf{UF\text{-}CMA}$ *security*: any upper bound established on the T'-$\mathsf{UF\text{-}CMA}$ advantage is at once an upper bound on the $\mathsf{UF\text{-}CMA}$ advantage.

All advantages considered below are parameterized by the number Q_H of random-oracle queries, the number Q_S of signing queries, the running time of the adversary, and the underlying oracle model. We state the bounds both in the classical ROM and in the QROM. In the QROM setting, the adversary is allowed to query the random oracle in quantum superposition, while signing queries remain classical. For the QROM reductions, we distinguish between implementations with and without QRACM. QRACM refers to the ability to access a classical table $x_1, \ldots, x_N$ coherently: given an index register in superposition, the corresponding classical entry can be loaded into a quantum work register without measuring the index. Formally, such access is represented by the unitary

$$U_{\mathrm{QRACM}} : \ |i\rangle|y\rangle \longmapsto |i\rangle|y \oplus x_i\rangle,$$

extended linearly to arbitrary superpositions over the index register.

4.2.2 UF-CMA and sUF-CMA Security

Proof strategy. We establish the UF-CMA and sUF-CMA security of our scheme by adapting the modular framework of Devevey, Fallahpour, Passelègue and Stehlé [15] to our concrete, finite-precision instantiation. The argument proceeds in two stages. In the first stage we work with the *bounded* variant SIG_B of our scheme, i.e. the Fiat–Shamir-with-aborts signature in which the signer performs at most B rejection-sampling iterations before giving up. Theorems 3 and 4 below reduce, respectively, the UF-CMA and the sUF-CMA security of SIG_B to its UF-NMA security: starting from the genuine UF-CMA/sUF-CMA game, the honest signing oracle is replaced, query by query, with the zero-knowledge simulator, and the cost of each replacement is charged to a Rényi divergence between the corresponding transcript distributions. In the second stage, Theorem 5 lifts the result from the bounded scheme SIG_B to our actual (unbounded) scheme $\mathrm{SIG}_\infty = \mathrm{FS}_\infty[\Sigma, H]$: it bounds the gap between the UF-CMA/sUF-CMA advantages

of the two schemes by a term that decays geometrically in B (of order $Q_S\,\beta^B$). Composing the two stages yields a UF-CMA/sUF-CMA-to-UF-NMA reduction for our actual scheme.

The truncation parameter B is chosen so that $\beta^B < 2^{-\lambda}$, where β is the per-iteration restart probability of signing. With $\beta = 2^{-30}$ this amounts to $B > \lambda/30$, giving

$$B_{128} = 5, \qquad B_{256} = 9, \qquad B_{512} = 18$$

for SUF-128/256/512 (target $\lambda = 128/256/512$).

A distinctive feature of our reduction is that it accounts for the way the signer is *actually* implemented: rather than sampling from the ideal discrete-Gaussian / ideal rejection-sampling distribution, the honest signing procedure incurs five concrete sources of numerical error, and the simulated transcript must be related to the honest one through all of them. Accordingly, the Rényi-HVZK simulator of Definition 2 interposes five intermediate distributions, and the security proof traverses five intermediate games $G_{2.1}, \ldots, G_{2.5}$, one per error source, each adjacent pair of games being controlled by its own Rényi divergence of order a_k:

1. the discrete-Gaussian base sampler;
2. the computation of the exponential function (the $\mathsf{ApproxExp}$ routine);
3. the no-rejection error of the ideal (rejection-free) iterative transition;
4. the truncation of the infinite series;
5. the computation of the logarithm.

A final, *exact* step then removes the secret key via the bimodal identity, producing the simulator distribution. Telescoping the six per-step Rényi bounds via Hölder's inequality yields the closed-form prefactor $\prod_{k=1}^{6} \mathrm{M}_k^{BQ_S\,e_k}$ in front of the UF-NMA advantage, as stated in Theorems 3 and 4.

Definition 2 (Rényi Honest-Verifier Zero-Knowledge with Intermediate Distributions)**.** *Let $a_1, \ldots, a_6 > 1$, $M_1, \ldots, M_6 \geq 0$, and $T \geq 0$. A Σ-protocol is $(M_1, \ldots, M_6, T)$-Rényi-HVZK of orders $(a_1, \ldots, a_6)$ if there exist five intermediate-distribution samplers $\mathsf{Sampler}_{\mathrm{int}}^{(1)}, \ldots, \mathsf{Sampler}_{\mathrm{int}}^{(5)}$ (each of which may use the secret key sk) and a simulator Sim with total runtime at most T such that the following holds for every instance x. Consider the chain of seven transcript distributions $P_0, P_1, \ldots, P_6$, where*

- *P_0 is the distribution of an honestly generated transcript (w', c', z') produced by the real interaction between the prover and the verifier of the identification protocol;*
- *P_k, for $1 \leq k \leq 5$, is the output distribution of $\mathsf{Sampler}_{\mathrm{int}}^{(k)}(x, sk)$;*
- *P_6 is the output distribution of the simulator $\mathsf{Sim}(x)$.*

Then every pair of adjacent distributions in the chain is close in Rényi divergence, where the k-th step may use its own order a_k and is controlled by its own bound:

$$R_{a_k}\big(P_{k-1} \,\big\|\, P_k\big) \leq M_k \qquad \text{for } k = 1, 2, 3, 4, 5, 6.$$

All distributions are taken over the full transcript, including aborting ones (those for which $z = \perp$).

For an order tuple $(a_1, \ldots, a_6)$ we abbreviate the per-step Hölder exponent $\rho_k := \frac{a_k - 1}{a_k} \in (0, 1)$ and its cumulative product $e_k := \prod_{i=1}^{k} \rho_i$ (with $e_0 := 1$), so that $1 \geq e_1 \geq \cdots \geq e_6 > 0$. These are exactly the exponents obtained by composing the six probability-preservation steps, and we use them to state the bounds below in their expanded (product) form.

Theorem 3. *(Adapted from [15])Let* $a_1, \ldots, a_6 > 1$, $\mathrm{M}_1, \ldots, \mathrm{M}_6, \alpha, T_{\mathrm{Sim}} \geq 0$, $B \geq 0$, *and let* H *be a hash function modeled as a random oracle. Assume that*

$$\Sigma = \big((P_1, P_2), (V_1, V_2)\big)$$

is an $(\mathrm{M}_1, \ldots, \mathrm{M}_6, T_{\mathrm{Sim}})$*-Rényi-HVZK of orders* $(a_1, \ldots, a_6)$ *public-coin identification protocol, and that the commitment message of the prover has min-entropy* α. *Let* $\mathcal{A}$ *be any adversary against the UF-CMA security of*

$$\mathrm{SIG}_B = \mathsf{FSB}[\Sigma, H]$$

that makes at most Q_H *queries to the random oracle* H *and* Q_S *classical queries to the signing oracle. More precisely, a QRACM operation is defined by*

$$U_{\mathrm{QRACM}} : \ |i\rangle\,|y_i\rangle \ \longmapsto \ |i\rangle\,|y_i \oplus x_i\rangle$$

for all indices i *and all* y_i, *and is extended to general quantum states by linearity. Then there exists a quantum adversary* $\mathcal{B}$ *against the UF-NMA security of* SIG_B *such that:*

- *In the ROM, algorithm* $\mathcal{B}$ *runs in time*

  $$\mathrm{Time}(\mathcal{A}) + O\big((T_{\mathrm{Sim}} \cdot B \cdot Q_S + Q_H)\log(B \cdot Q_S + Q_H)\big),$$

  and $\mathrm{Adv}^{\mathrm{CMA}}_{\mathrm{SIG}_B}(\mathcal{A}) \leq \left(\prod_{k=1}^{6} \mathrm{M}_k^{BQ_S\, e_k}\right) \cdot \mathrm{Adv}^{\text{UF-NMA}}_{\mathrm{SIG}_B}(\mathcal{B})^{e_6} + 2^{-\alpha} \cdot B \cdot Q_S \cdot (B \cdot Q_S + Q_H + 1)$
- *In the QROM, algorithm* $\mathcal{B}$ *runs in time*

  $$\mathrm{Time}(\mathcal{A}) + O\big((T_{\mathrm{Sim}} \cdot B \cdot Q_S + Q_H)\log(B \cdot Q_S)\big)$$

  with QRACM, and

  $$\mathrm{Time}(\mathcal{A}) + O\big((T_{\mathrm{Sim}} \cdot B \cdot Q_S + Q_H) \cdot (B \cdot Q_S)\big)$$

  without QRACM, and
  $\mathrm{Adv}^{\mathrm{CMA}}_{\mathrm{SIG}_B}(\mathcal{A}) \leq \left(\prod_{k=1}^{6} \mathrm{M}_k^{BQ_S\, e_k}\right) \cdot \mathrm{Adv}^{\text{UF-NMA}}_{\mathrm{SIG}_B}(\mathcal{B})^{e_6} + 2^{-\alpha/2} \cdot 3B \cdot Q_S \cdot \sqrt{B \cdot Q_S + Q_H + 1}$

Proof. The proof is similar to that of Theorem 10[15].

Game G_0**.** The first game is the UF-CMA security game.

Game G_1**.** In this game, the challenges of the transcripts are no longer computed by the random oracle, but are instead sampled independently and uniformly each time. Then, the random oracle is reprogrammed according to the new challenges.

To bound the distance between Game G_0 and Game G_1, we construct a wrapper $\mathcal{D}$ around $\mathcal{A}$ that uses $\mathcal{A}$ to solve a reprogramming game. It works as in Figure 16. Note that if $b = 0$ in Figure 16, then $\mathcal{D}$ perfectly simulates G_0, and otherwise it perfectly simulates G_1. Therefore,

$$\left|\Pr\left[1 \Leftarrow G_0^{\mathcal{A}}\right] - \Pr\left[1 \Leftarrow G_1^{\mathcal{A}}\right]\right| \leq \left|\Pr\left[1 \Leftarrow \mathrm{Reprogram}_0^{\mathcal{D}}\right] - \Pr\left[1 \Leftarrow \mathrm{Reprogram}_1^{\mathcal{D}}\right]\right|.$$

During the game, the distinguisher $\mathcal{D}$ makes $B \cdot Q_S$ reprogramming queries and $B \cdot Q_S + Q_H + 1$ random oracle queries. In the ROM, the advantage of $\mathcal{D}$ is bounded by

$$B \cdot Q_S \cdot (B \cdot Q_S + Q_H + 1)\, 2^{-\alpha}.$$

In the QROM, it follows that the advantage of $\mathcal{D}$ is bounded by

$$\frac{3B \cdot Q_S}{2} \cdot \sqrt{(B \cdot Q_S + Q_H + 1)\, 2^{-\alpha}}.$$

Game:

1: $\mathcal{M} := \emptyset$
2: $(vk, sk) \leftarrow \text{KeyGen}(1^\lambda)$
3: $(\mu^*, \sigma^*) \leftarrow \mathcal{A}^{H, \text{Sign}(sk,\cdot)}(vk)$
4: Parse $\sigma^* = (w^*, z^*)$
5: $c^* := H(w^* \,\|\, \mu^*)$
6: **return** $\mu^* \notin \mathcal{M} \wedge V_2(vk, w^*, c^*, z^*)$

Sign(sk, μ)**:**

1: $\mathcal{M} := \mathcal{M} \cup \{\mu\}$
2: $(w, c, z) \leftarrow \text{GetTrans}(\mu)$
3: **if** $z = \perp$ **return** $\perp$
4: **return** $\sigma = (w, z)$

GetTrans(μ)**:**

1: $\kappa := 0$
2: **while** $z = \perp$ **and** $\kappa \leq B$
3: $\quad (w, st) \leftarrow P_1(sk)$
4: $\quad c := H(w \,\|\, \mu)$
5: $\quad z \leftarrow P_2(sk, w, c, st)$
6: $\quad \kappa := \kappa + 1$
7: **return** (w, c, z)

Figure 2: UF-CMA security game (game G_0)

Game :

1: $\mathcal{M} := \emptyset$
2: $(vk, sk) \leftarrow \text{KeyGen}(1^\lambda)$
3: $(\mu^*, \sigma^*) \leftarrow \mathcal{A}^{H, \text{Sign}(sk,\cdot)}(vk)$
4: Parse $\sigma^* = (w^*, z^*)$
5: $c^* := H(w^* \,\|\, \mu^*)$
6: **return** $\mu^* \notin \mathcal{M} \wedge V_2(vk, w^*, c^*, z^*)$

Sign(sk, μ) **:**

1: $\mathcal{M} := \mathcal{M} \cup \{\mu\}$
2: $(w, c, z) \leftarrow \text{GetTrans}(\mu)$
3: **if** $z = \perp$ **return** $\perp$
4: **return** $\sigma = (w, z)$

GetTrans(μ) **:**

1: $\kappa := 0$
2: **while** $z = \perp$ **and** $\kappa \leq B$
3: $\quad (w, st) \leftarrow P_1(sk)$
4: $\quad c \leftarrow U(\mathcal{C})$
5: $\quad z \leftarrow P_2(sk, w, c, st)$
6: $\quad H = H'(w \,\|\, \mu \mapsto c)$
7: $\quad \kappa := \kappa + 1$
8: **return** (w, c, z)

Figure 3: Game G_1. The difference from G_0 is highlighted in blue.

Games $G_{2.1}, \ldots, G_{2.5}$. For $k = 1, \ldots, 5$, let $\mathsf{Sampler}_{\mathrm{int}}^{(k)}$ be the k-th intermediate-distribution sampler of Definition 2 (each of which may still use the secret key sk). In Game $G_{2.k}$ we modify GetTrans so that the transcripts are produced by $\mathsf{Sampler}_{\mathrm{int}}^{(k)}$. These five games realize the chain of intermediate distributions $P_1, \ldots, P_5$ and bridge the honest signing distribution of G_1 (distribution P_0) to the simulator's output (P_6).

Game $G_{2.6}$. Let Sim be the zero-knowledge simulator for Σ. In this game we modify GetTrans such that the transcripts are now produced by Sim and without the secret key (Figure 4). We use the probability-preservation property of the Rényi divergence (in the convention

Game :

$\mathcal{M} := \emptyset$
$(vk, sk) \leftarrow \mathrm{KeyGen}(1^\lambda)$
$(\mu^*, \sigma^*) \leftarrow \mathcal{A}^{H, \mathrm{Sign}(sk,\cdot)}(vk)$
Parse $\sigma^* = (w^*, z^*)$
$c^* := H(w^* \parallel \mu^*)$
return $\mu^* \notin \mathcal{M} \wedge V_2(vk, w^*, c^*, z^*)$

Sign(sk, μ) **:**

$\mathcal{M} := \mathcal{M} \cup \{\mu\}$
$(w, c, z) \leftarrow \mathrm{GetTrans}(\mu)$
if $z = \perp$ **return** $\perp$
return $\sigma = (w, z)$

GetTrans(μ) **:**

$\kappa := 0$
while $z = \perp$ **and** $\kappa \leq B$
  $c \leftarrow U(\mathcal{C})$
  $(w, z) \leftarrow \mathrm{Sim}(vk, c)$
  $H := H^{w \parallel \mu \mapsto c}$
  $\kappa := \kappa + 1$
return (w, c, z)

Figure 4: Game $G_{2.6}$. The difference from $G_{2.5}$ is highlighted in blue.

of Prest): for order $a > 1$ and any event E,

$$P(E) \;\leq\; \big(R_a(P \,\|\, Q) \cdot Q(E)\big)^{(a-1)/a},$$

which follows from Hölder's inequality with conjugate exponents a and $a/(a-1)$; note that the divergence factor lies *inside* the $(a-1)/a$ power. We apply this bound at the k-th step with that step's own Rényi order a_k: since R_{a_k} is multiplicative over the $B \cdot Q_S$ independent transcripts, the per-transcript bound there accumulates to $\mathrm{M}_k^{B \cdot Q_S}$ and the preservation exponent is $\rho_k = (a_k - 1)/a_k$. Writing $G_{2.0} := G_1$, applying the probability-preservation bound to each of the six consecutive intermediate steps gives, for $k = 1, \ldots, 6$,

$$\Pr[1 \Leftarrow G^{\mathcal{A}}_{2.(k-1)}] \leq \big(\mathrm{M}_k^{B \cdot Q_S} \cdot \Pr[1 \Leftarrow G^{\mathcal{A}}_{2.k}]\big)^{\rho_k}.$$

Composing (telescoping) the six steps yields

$$\Pr[1 \Leftarrow G_1^{\mathcal{A}}] \leq \Big(\prod_{k=1}^{6} \mathrm{M}_k^{BQ_S\, e_k}\Big) \cdot \Pr[1 \Leftarrow G^{\mathcal{A}}_{2.6}]^{e_6}.$$

Game G_3. The signing algorithm does not use the signing key anymore and uses the zero-knowledge simulator to answer the sign queries. The technicality lies in how to simulate the random oracle. In the ROM, we use the lazy sampling method. At each query to the random oracle, we return a match if there exists any in the database, otherwise we return an element freshly sampled from the range of H and we add it in the database. In the QROM, we cannot

simulate the random oracle with the lazy sampling method since the access to it is quantum. Therefore, the challenger uses a private random oracle H' to simulate the hash queries of the adversary but also keeps a database of reprogrammed inputs. Whenever it receives a hash query, it first searches the database for a match and, if there is none, it returns the evaluation of the query with the private random oracle H'.

Game G_4. This game only concerns the QROM. We add one more statement to the winning conditions. Let $(\mu^*, (w^*, z^*))$ be the forgery. The game aborts if $H(w^* \,\|\, \mu^*) \neq H'(w^* \,\|\, \mu^*)$ where H' is the random oracle used to simulate the hash queries of the adversary in the previous game. The adversary can distinguish this modification if the value $w^* \,\|\, \mu^*$ has been programmed during the game. This occurs only if the adversary has made a sign query with μ^*. As the winning condition in the UF-CMA game already requires a forgery for a message that has not been queried before, the view of the adversary is identical to that of the previous game.

It remains to reduce the UF-NMA game to the last hybrid game (G_3 in the ROM and G_4 in the QROM). In the QROM, using the UF-NMA game and its random oracle, one can perfectly simulate G_4 for the adversary. If the adversary A finds a forgery (μ^*, σ^*), then the random oracle has not been reprogrammed at this value during the course of G_4 since it has not been queried before. Hence, it would be a valid signature for the UF-NMA game. In the ROM, the reduction from G_3 to UF-NMA works as follows. The reduction uses the random oracle of the UF-NMA game to answer direct random oracle queries. A forged signature (μ^*, σ^*) in G_3 is a forged signature in UF-NMA if $c^* = H'(w^* \| \mu^*)$ where H' is the random oracle of UF-NMA. This holds if the random oracle has not been reprogrammed on input $w^* \| \mu^*$ during the game, which holds since the adversary is not allowed to ask for a signature of μ^*.

Combining the differences between each game, we obtain the final inequality. □

For the reduction proof of strong unforgeability, our analysis differs substantially from that in the reference. In the reference, because the statistical error is symmetric, one can estimate the probability of event 2 in game 1 in order to bound the probability of event 2 in game 2. The setting considered here is different, since Rényi divergence is not symmetric. Therefore, we introduce a new notion, called a verifiable simulator. We prove that the simulator of our algorithm satisfies this property, and then carry out the analysis based on it.

Definition 3 (β-Verifiable Simulator)**.** *Let $\Pi = (P, V)$ be an identification protocol. A probabilistic polynomial-time algorithm* Sim *is called a* β-verifiable simulator *if, on input a statement x, it outputs a transcript (w, c, z) such that*

$$\Pr[z = \perp] \leq \beta,$$

and for all outputs with $z \neq \perp$, the transcript is accepting for the verifier, i.e.,

$$V(x, w, c, z) = 1.$$

Definition 4 (Simulator Commitment Min-Entropy)**.** *For $\alpha \geq 0$, we say that an identification scheme $\Sigma = ((P_1, P_2), (V_1, V_2))$ with instance generator* Gen *has* simulator commitment min-entropy α *if*

$$H_\infty[w \,|\, (w, \mathsf{st}) \leftarrow \mathsf{Sim}_1(x)] \geq \alpha,$$

for all $(x, y) \leftarrow \mathsf{Gen}(1^\lambda)$, where Sim_1 *denotes the first-message simulator of the honest-verifier zero-knowledge simulator.*

Definition 5 ((γ, β)-Correctness, adapted from [15])**.** *Let $\gamma, \beta \geq 0$. A public-coin Σ-protocol $\Sigma = ((P_1, P_2), (V_1, V_2))$ is* (γ, β)-correct *if for every statement $x \in L$ and every valid witness y the following two conditions hold, where all probabilities are taken over $(w, \mathsf{st}) \leftarrow P_1(x, y)$, $c \leftarrow V_1(x, w)$, and $z \leftarrow P_2(x, y, w, c, \mathsf{st})$:*

1. *Conditioned on the prover not aborting, the verifier accepts with probability at least* γ*:*

$$\Pr\big[\, V_2(x,w,c,z) = 1 \;\big|\; z \neq \bot \,\big] \geq \gamma.$$

2. *The prover aborts with probability at most* β*:*

$$\Pr[\, z = \bot \,] \leq \beta.$$

Definition 6 (Computational Unique Response (CUR), adapted from [15])**.** *An identification protocol has* unique responses *if for every commitment–challenge pair* (w,c) *there exists at most one response* z *such that the transcript* (w,c,z) *is accepting. The following definition relaxes this notion against computationally bounded adversaries. Let* $\Sigma = ((P_1,P_2),(V_1,V_2))$ *be an identification scheme with instance generator* Gen*. For any (quantum) adversary* $\mathcal{A}$*, define the advantage*

$$\mathsf{Adv}^{\mathsf{CUR}}_{\Sigma}(\mathcal{A}) = \Pr\left[z \neq z' \;\wedge\; V_2(x,w,c,z) = 1 \;\wedge\; V_2(x,w,c,z') = 1 \;\middle|\; \begin{array}{l} (x,y) \leftarrow \mathsf{Gen}(1^\lambda), \\ (w,c,z,z') \leftarrow \mathcal{A}(x) \end{array} \right].$$

We say that Σ *has computational unique responses if* $\mathsf{Adv}^{\mathsf{CUR}}_{\Sigma}(\mathcal{A})$ *is negligible for every efficient adversary* $\mathcal{A}$*.*

Theorem 4. *(adapted from [15]) Let* $a_1,\ldots,a_6 > 1$*,* $\mathrm{M}_1,\ldots,\mathrm{M}_6,\alpha,\beta,\gamma,T_{\mathrm{Sim}} \geq 0$*,* $B \geq 0$*, and* H *a hash function modeled as a random oracle. Assume that* $\Sigma = ((P_1,P_2),(V_1,V_2))$ *is an* $(\mathrm{M}_1,\ldots,\mathrm{M}_6,T_{\mathrm{Sim}})$*-Rényi-HVZK of orders* $(a_1,\ldots,a_6)$ *and* (γ,β)*-correct public-coin identification protocol, and that its zero-knowledge simulator is also* β*-verifiable, and that both the simulator's commitment and the prover's commitment message have min-entropy at least* α*. Let* A *be any arbitrary adversary against sUF-CMA security of* $\mathrm{SIG}_B = \mathrm{FSB}[\Sigma,H]$ *that issues at most* Q_H *queries to the random oracle* H *and* Q_S *classical queries to the signing oracle. There exist quantum adversaries* B *and* C *respectively against UF-NMA security of* SIG_B *and the computational unique response property of* Σ *as follows:*

- *In the ROM, both* B *and* C *have runtime*

$$\mathrm{Time}(A) + O\big((T_{\mathrm{Sim}} \cdot B \cdot Q_S + Q_H)\log(B \cdot Q_S + Q_H)\big),$$

  and

$$\mathrm{Adv}^{\text{sUF-CMA}}_{\mathrm{SIG}_B}(\mathcal{A}) \leq \Big(\prod_{k=1}^{6} \mathrm{M}_k^{BQ_S\, e_k}\Big) \cdot \big(\mathrm{Adv}^{\text{UF-NMA}}_{\mathrm{SIG}_B}(\mathcal{B}) + \mathsf{Adv}^{\mathsf{CUR}}_{\Sigma}(\mathcal{C}) + Q_S(\beta^B + B \cdot 2^{-\alpha})\big)^{e_6} + 2^{-\alpha} \cdot B \cdot Q_S \cdot \big(B \cdot Q_S + Q_H + 1\big).$$

- *In the QROM, the runtimes of* B *and* C *are*

$$\mathrm{Time}(A) + O\big((T_{\mathrm{Sim}} \cdot B \cdot Q_S + Q_H)\log(B \cdot Q_S)\big)$$

  with QRACM, and

$$\mathrm{Time}(A) + O\big((T_{\mathrm{Sim}} \cdot B \cdot Q_S + Q_H) \cdot (B \cdot Q_S)\big)$$

  without QRACM, and

$$\mathrm{Adv}^{\text{sUF-CMA}}_{\mathrm{SIG}_B}(\mathcal{A}) \leq \Big(\prod_{k=1}^{6} \mathrm{M}_k^{BQ_S\, e_k}\Big) \cdot \big(\mathrm{Adv}^{\text{UF-NMA}}_{\mathrm{SIG}_B}(\mathcal{B}) + \mathsf{Adv}^{\mathsf{CUR}}_{\Sigma}(\mathcal{C}) + Q_S(\beta^B + B \cdot 2^{-\alpha})\big)^{e_6} + 2^{-\alpha/2} \cdot 3B \cdot Q_S \cdot \sqrt{B \cdot Q_S + Q_H + 1}.$$

Proof. The proof is similar to the one of Theorem 11[15].

We base the proof on the hybrid games in the proof of Theorem 3. We modify them as follows. Here, the challenger maintains the list MS of message-signature pairs that were queried by the adversary via the signature oracle. Each game, at its final step, also checks whether the forgery $(\mu^*, (w^*, z^*))$ belongs to this list or not, and returns 0 if it does. With these modifications, everything remains the same up to G_3.

We first discuss the QROM case. The two games G_3 and G_4 behave differently only if we have the following conditions:

$$H(w^* \| \mu^*) \neq H'(w^* \| \mu^*) \wedge (\mu^*, (w^*, z^*)) \notin \mathsf{MS} \wedge V_2(vk, w^*, z^*) = 1.$$

Since $H(w^* \| \mu^*) \neq H'(w^* \| \mu^*)$, the adversary must have queried μ^* before and w^* must have appeared in at least one of the transcripts produced during the rejection sampling of $\mathsf{Sign}(sk, \mu^*)$. Let $\mathcal{G}$ be the list of indices where the adversary asked for such queries. We proceed case by case.

event$_1$: There exists at least one signature query in $\mathcal{G}$ such that the value of w^* has appeared in the final transcript produced during the execution of $\mathsf{Sign}(sk, \mu^*)$ such that its corresponding transcript gets accepted by the verifier. This can be bounded by the computational unique response of Σ. Let $(w_{\mu^*}, z_{\mu^*}) \leftarrow \mathsf{Sign}(sk, \mu^*)$ be the output of this particular signature query and $c_{\mu^*} := H(w_{\mu^*} \| \mu^*)$. It holds that $V_2(vk, w_{\mu^*}, c_{\mu^*}, z_{\mu^*}) = 1$. Moreover, the forgery (w^*, c^*, z^*) passes the verification check. Therefore, the adversary $\mathcal{A}$ can be turned into another adversary $\mathcal{C}$ (that observes $\mathcal{A}$ and outputs the two transcripts) against the computational unique response property of Σ with an advantage at most $\mathsf{Adv}^{\mathsf{CUR}}_{\Sigma}(\mathcal{C})$. Note that $\mathcal{C}$ has the same runtime as $\mathcal{A}$.

event$_2$: For every signature query in $\mathcal{G}$, whenever the value of w^* appears in the final transcript, that transcript is rejected by the verifier; let $\mathcal{G}_{\text{final}}$ be the corresponding subset of queries. We revert the hybrid games back to G_1, where transcripts are generated honestly. By the β-verifiability of the simulator, every non-aborting output ($\mathbf{z} \neq \perp$) is accepting; hence, in $\mathsf{Game\ 3}$, event_2 can occur for a given query only if either (i) all B iterations of that query abort, which by β-verifiability happens with probability at most β^B, or (ii) w^* does not appear in the (rejected) final transcript but coincides with one of the at most B intermediate commitments, each of which has min-entropy α, contributing at most $B \cdot 2^{-\alpha}$. Summing over the Q_S signing queries,

$$\Pr[\text{Event2} \mid G_3^A] \leq Q_S\big(\beta^B + B \cdot 2^{-\alpha}\big). \tag{9}$$

Finally, using the union bound, it holds that:

$$\begin{aligned}\big|\Pr[1 \leftarrow G_3^{\mathcal{A}}] - \Pr[1 \leftarrow G_4^{\mathcal{A}}]\big| &\leq \Pr[\text{Event}_1 \mid G_3^{\mathcal{A}}] + \Pr[\text{Event}_2 \mid G_3^{\mathcal{A}}] \\ &\leq \mathsf{Adv}^{\mathsf{CUR}}_{\Sigma}(\mathcal{C}) + Q_S\big(\beta^B + B \cdot 2^{-\alpha}\big)\ .\end{aligned}$$

The reduction from the $\mathsf{UF\text{-}NMA}$ game to the $\mathsf{sUF\text{-}CMA}$ game is similar to that of Theorem 3.

For the ROM case, we reduce the $\mathsf{UF\text{-}NMA}$ game to G_3 as follows. One can use the random oracle of the $\mathsf{UF\text{-}NMA}$ game to answer the direct hash queries of the adversary in G_3. This preserves the view of the adversary. Therefore, a similar argument and upper bound, as in the QROM, holds here.

□

Theorem 5. *([15] Let $\alpha \geq 0$, $\beta \in (0, 1)$, and let H be a hash function modeled as a random oracle. Assume that $\Sigma = ((P_1, P_2), (V_1, V_2))$ is a (γ, β)-correct identification protocol, and that the commitment message of P_1 has min-entropy α. Let T denote the runtime of one iteration*

of the protocol with the hash function. Let $T' > BT$. For any arbitrary adversary $\mathcal{A}$ against T'-UF-CMA security of

$$\mathrm{SIG}_\infty = \mathrm{FS}_\infty[\Sigma, H],$$

that issues at most Q_H queries to the random oracle H and Q_S classical queries to the signing oracle, and for any fixed integer B, the same adversary $\mathcal{A}$ against UF-CMA security of

$$\mathrm{SIG}_B = \mathrm{FS}_B[\Sigma, H]$$

is such that

$$\left|\mathrm{Adv}^{T'\text{-}UF\text{-}CMA}_{\mathrm{SIG}_\infty}(\mathcal{A}) - \mathrm{Adv}^{\text{UF-CMA}}_{\mathrm{SIG}_B}(\mathcal{A})\right|$$

is bounded as

$$Q_S \cdot \beta^B \;+\; \frac{\beta^B \cdot 2^{-\alpha}}{(1-\beta)^3} \;+\; \begin{cases} 2^{-\alpha} \cdot B \cdot Q_S \cdot (B \cdot Q_S + Q_H + 1) & \text{in the ROM,} \\ 2^{-\alpha/2} \cdot \frac{3BQ_S}{2} \cdot \sqrt{B \cdot Q_S + Q_H + 1} & \text{in the QROM.} \end{cases}$$

This also holds when replacing UF-CMA with UF-CMA$_1$ or sUF-CMA security.

Lemma 6 (Data Processing for Rényi Divergence)**.** *Let $(\Omega, \mathcal{F})$ and $(\Omega', \mathcal{F}')$ be measurable spaces, and let P, Q be two probability measures on $(\Omega, \mathcal{F})$. Let $f : \Omega \to \Omega'$ be a measurable function, and denote by $f(P)$ and $f(Q)$ the pushforward measures induced by f, i.e.,*

$$f(P)(A) := P(f^{-1}(A)), \quad f(Q)(A) := Q(f^{-1}(A)) \quad \text{for all } A \in \mathcal{F}'.$$

Let $a > 1$ and write $\bar{D}_a(P\|Q) := \mathbb{E}_Q\big[(dP/dQ)^a\big]$ for the (unnormalized) order-a divergence, so that the multiplicative Rényi divergence used elsewhere is $R_a(P\|Q) = \bar{D}_a(P\|Q)^{1/(a-1)}$. Then

$$\bar{D}_a(f(P)\|f(Q)) \le \bar{D}_a(P\|Q), \qquad \text{and consequently} \qquad R_a(f(P)\|f(Q)) \le R_a(P\|Q).$$

Proof. Let $L = \frac{dP}{dQ}$ be the Radon-Nikodym derivative of P with respect to Q on the measurable space $(\Omega, \mathcal{F})$, which exists whenever $\bar{D}_a(P\|Q) < \infty$, in which case $P \ll Q$. For the pushforward measures, the condition $P \ll Q$ implies $f(P) \ll f(Q)$, so there exists a Radon-Nikodym derivative $\tilde{L} = \frac{df(P)}{df(Q)}$ on $(\Omega', \mathcal{F}')$.

By the definition of pushforward measures and the change of variables formula, for any $A \in \mathcal{F}'$, we have:

$$\int_A \tilde{L}\, df(Q) = f(P)(A) = P(f^{-1}(A)) = \int_{f^{-1}(A)} L\, dQ.$$

Let $\sigma(f) = \{f^{-1}(A) : A \in \mathcal{F}'\}$ be the σ-algebra generated by f. The identity above implies that $\tilde{L} \circ f$ is a version of the conditional expectation of L given $\sigma(f)$ under measure Q, i.e.,

$$\tilde{L} \circ f = \mathbb{E}_Q[L|\sigma(f)], \quad Q\text{-a.s.}$$

By the change of variables formula again,

$$\bar{D}_a(f(P)\|f(Q)) = \int_{\Omega'} \tilde{L}(y)^a\, df(Q)(y) = \int_\Omega (\tilde{L} \circ f)^a\, dQ = \mathbb{E}_Q[(\tilde{L} \circ f)^a].$$

Substituting the conditional expectation form, we get:

$$\bar{D}_a(f(P)\|f(Q)) = \mathbb{E}_Q[(\mathbb{E}_Q[L|\sigma(f)])^a].$$

Since $a > 1$, the function $\phi(x) = x^a$ is convex for $x \ge 0$. By Jensen's inequality for conditional expectations, we have:

$$(\mathbb{E}_Q[L|\sigma(f)])^a \le \mathbb{E}_Q[L^a|\sigma(f)], \quad Q\text{-a.s.}$$

Taking the expectation with respect to Q on both sides and applying the law of total expectation:

$$\mathbb{E}_Q[(\mathbb{E}_Q[L|\sigma(f)])^a] \le \mathbb{E}_Q[\mathbb{E}_Q[L^a|\sigma(f)]] = \mathbb{E}_Q[L^a] = \bar{D}_a(P\|Q).$$

This proves $\bar{D}_a(f(P)\|f(Q)) \le \bar{D}_a(P\|Q)$; raising both sides to the power $1/(a-1) > 0$ yields $R_a(f(P)\|f(Q)) \le R_a(P\|Q)$. □

4.2.3 Security and Correctness of the Canonical Identification Scheme

The identification scheme corresponding to our signature algorithm and its zero-knowledge simulator are shown in the figure.

CID scheme

$\mathbf{P}(\mathbf{A}, s)$		$\mathbf{V}(\mathbf{A})$
$\mathbf{y} \leftarrow \mathsf{SampleY}$		
$\mathbf{y}' = \mathsf{CompressY}(\mathbf{y})$		
$\mathbf{w} = \mathbf{A}\mathbf{y}' \bmod 2q$		
$\mathbf{w}_0 \leftarrow \mathsf{LSB}(\mathbf{w})$		
$\mathbf{w}_h \leftarrow \lfloor \mathbf{w}/\alpha_h \rceil$	$\xrightarrow{(\mathbf{w}_h, \mathbf{w}_0)}$	$c \leftarrow \mathcal{U}(\mathcal{C})$
	$\xleftarrow{c}$	
$\widetilde{\mathbf{z}} \leftarrow \mathsf{RejectSample}(\mathbf{y}, c, \mathsf{StretchS}(s))$		
$\mathbf{z} = \mathsf{CompressY}(\widetilde{\mathbf{z}})$		
$(\mathbf{z}_1, \mathbf{z}_2) \leftarrow \mathbf{z}$		
$\mathbf{h} \leftarrow \mathsf{MakeHint}(\mathbf{z}_2, \mathbf{w})$		
$\mathbf{z}_2' = (\alpha_h \mathbf{w}_h + \mathbf{w}_0 - \mathbf{w} + 2\mathbf{z}_2)/2 \bmod^{\pm} q$		
if $\|(\mathbf{z}_1, \mathbf{z}_2')\|_2 > B_v$: **Abort**		
$\sigma \leftarrow (\mathbf{z}_1, \mathbf{h})$	$\xrightarrow{\sigma}$	Reject if $\mathrm{Verify}(\sigma, c) = 0$

CID simulator

1 : $\widetilde{\mathbf{z}} \leftarrow \mathcal{D}_{\mathcal{R},r}^{l+m+1}$ (centered discrete Gaussian of width $r = r$)

2 : $c \leftarrow \mathcal{U}(\mathcal{C})$

3 : $\mathbf{z} \leftarrow \mathsf{CompressY}(\widetilde{\mathbf{z}})$

4 : $\mathbf{w} = \mathbf{A}\mathbf{z} - qc\mathbf{j} \bmod 2q$, $\mathbf{w}_0 \leftarrow \mathsf{LSB}(\mathbf{w})$, $\mathbf{w}_h \leftarrow \lfloor \mathbf{w}/\alpha_h \rceil$

5 : $(\mathbf{z}_1, \mathbf{z}_2) \leftarrow \mathbf{z}$, $\mathbf{h} \leftarrow \mathsf{MakeHint}(\mathbf{z}_2, \mathbf{w})$

6 : $\mathbf{z}_2' = (\alpha_h \mathbf{w}_h + \mathbf{w}_0 - \mathbf{w} + 2\mathbf{z}_2)/2 \bmod^{\pm} q$

7 : **if** $\|(\mathbf{z}_1, \mathbf{z}_2')\|_2 > B_v$: **return** $\big((\mathbf{w}_h, \mathbf{w}_0), c, \perp\big)$

8 : **return** $\big((\mathbf{w}_h, \mathbf{w}_0), c, (\mathbf{z}_1, \mathbf{h})\big)$

Figure 5: Canonical identification scheme and simulator corresponding to the compressed version of our signature scheme.

Commitment min-entropy.

Lemma 7 (Min-entropy of rounded discrete-Gaussian LSBs)**.** *Let $A = \alpha_1 \in \mathbb{Z}_{>0}$ and let $\sigma = 825$. Let $X_1, \ldots, X_{256}$ be independent samples from the discrete Gaussian over $\mathbb{Z}$ with standard deviation σ, namely*

$$\Pr[X_i = x] = \frac{\exp(-x^2/(2\sigma^2))}{\sum_{z \in \mathbb{Z}} \exp(-z^2/(2\sigma^2))}.$$

Define

$$Y_i = \left\lfloor \frac{X_i}{A} + \frac{1}{2} \right\rfloor, \qquad B_i = Y_i \bmod 2,$$

and let

$$B = (B_1, \ldots, B_{256}) \in \{0,1\}^{256}.$$

Set

$$\lambda_A = \frac{\pi^2\sigma^2}{2A^2} = \frac{\pi^2 \cdot 825^2}{2A^2},$$

and

$$\varepsilon_A = \frac{2e^{-\lambda_A}}{1 - e^{-3\lambda_A}}.$$

Then

$$H_\infty(B) > 256\left(1 - \log_2(1 + 2\varepsilon_A)\right).$$

Equivalently,

$$H_\infty(B) > 256\left(1 - \log_2\left(1 + \frac{4e^{-\lambda_A}}{1 - e^{-3\lambda_A}}\right)\right).$$

Proof. Let $q = 2A$. For one coordinate, write

$$R = X_i \bmod q.$$

The bit B_i is a deterministic function of R, because

$$\left\lfloor \frac{x + 2A}{A} + \frac{1}{2} \right\rfloor = \left\lfloor \frac{x}{A} + \frac{1}{2} \right\rfloor + 2.$$

Moreover, shifting x by A flips the parity:

$$\left\lfloor \frac{x + A}{A} + \frac{1}{2} \right\rfloor = \left\lfloor \frac{x}{A} + \frac{1}{2} \right\rfloor + 1.$$

Hence among the $q = 2A$ residue classes modulo q, exactly A classes give $B_i = 0$, and exactly A classes give $B_i = 1$.

We first bound the distance between R and the uniform distribution on $\mathbb{Z}/q\mathbb{Z}$. For $r \in \mathbb{Z}/q\mathbb{Z}$,

$$\Pr[R = r] = \frac{\sum_{k\in\mathbb{Z}} \exp(-(r + kq)^2/(2\sigma^2))}{\sum_{z\in\mathbb{Z}} \exp(-z^2/(2\sigma^2))}.$$

By the Poisson summation formula,

$$\sum_{k\in\mathbb{Z}} \exp(-(r + kq)^2/(2\sigma^2)) = \frac{\sqrt{2\pi}\sigma}{q} \sum_{m\in\mathbb{Z}} \exp\left(-\frac{2\pi^2\sigma^2 m^2}{q^2}\right) e^{2\pi i m r/q}.$$

Also,

$$\sum_{z\in\mathbb{Z}} \exp(-z^2/(2\sigma^2)) = \sqrt{2\pi}\sigma \sum_{m\in\mathbb{Z}} \exp(-2\pi^2\sigma^2 m^2).$$

Let

$$\eta_q = 2\sum_{m=1}^{\infty} \exp\left(-\frac{2\pi^2\sigma^2 m^2}{q^2}\right).$$

The preceding Fourier expansion implies

$$\Delta(R, U_q) \le \eta_q,$$

where U_q is the uniform distribution on $\mathbb{Z}/q\mathbb{Z}$.

Since $q = 2A$, we have

$$\frac{2\pi^2\sigma^2}{q^2} = \frac{\pi^2\sigma^2}{2A^2} = \lambda_A.$$

Therefore

$$\eta_q = 2\sum_{m=1}^{\infty} e^{-\lambda_A m^2}.$$

Using $m^2 \geq 1 + 3(m-1)$ for every $m \geq 1$, we get

$$\eta_q \leq 2\sum_{m=1}^{\infty} e^{-\lambda_A(1+3(m-1))} = \frac{2e^{-\lambda_A}}{1-e^{-3\lambda_A}} = \varepsilon_A.$$

Thus

$$\Delta(R, U_q) \leq \varepsilon_A.$$

Let $S_0 \subseteq \mathbb{Z}/q\mathbb{Z}$ be the set of residues giving $B_i = 0$. Since $|S_0| = q/2$, we have $U_q(S_0) = 1/2$. Hence

$$\left|\Pr[B_i = 0] - \frac{1}{2}\right| = |\Pr[R \in S_0] - U_q(S_0)| \leq \Delta(R, U_q) \leq \varepsilon_A.$$

The same bound holds for $B_i = 1$. Therefore

$$\max_{b\in\{0,1\}} \Pr[B_i = b] \leq \frac{1}{2} + \varepsilon_A.$$

The variables $B_1, \ldots, B_{256}$ are independent, since they are deterministic functions of the independent variables $X_1, \ldots, X_{256}$. Hence

$$\max_{b\in\{0,1\}^{256}} \Pr[B = b] \leq \left(\frac{1}{2} + \varepsilon_A\right)^{256}.$$

By the definition of min-entropy,

$$H_\infty(B) = -\log_2 \max_{b\in\{0,1\}^{256}} \Pr[B = b].$$

Therefore

$$H_\infty(B) \geq -256\log_2\left(\frac{1}{2} + \varepsilon_A\right) = 256\left(1 - \log_2(1 + 2\varepsilon_A)\right).$$

This proves the claimed lower bound. □

Lemma 8 (Min-entropy transfer from rooted Renyi divergence)**.** *Let $a > 1$, and let P_1, P_2 be two probability distributions over the same discrete domain Ω. Define the rooted order-a Renyi divergence by*

$$R_a(P_1\|P_2) = \left(\sum_{x\in\Omega} P_1(x)^a P_2(x)^{1-a}\right)^{1/(a-1)}.$$

Suppose that

$$R_a(P_1\|P_2) \leq M$$

and

$$H_\infty(P_2) \geq \alpha.$$

Then

$$H_\infty(P_1) \geq \frac{a-1}{a}\left(\alpha - \log_2 M\right).$$

Equivalently,

$$\max_{x\in\Omega} P_1(x) \leq \left(M \cdot 2^{-\alpha}\right)^{(a-1)/a}.$$

Proof. By the definition of min-entropy, $H_\infty(P_2) \geq \alpha$ means that for every $x \in \Omega$, $P_2(x) \leq 2^{-\alpha}$. Fix any $x \in \Omega$. Since the term indexed by x is one of the nonnegative terms in the Renyi divergence sum, we have

$$P_1(x)^a P_2(x)^{1-a} \leq \sum_{z \in \Omega} P_1(z)^a P_2(z)^{1-a} = R_a(P_1 \| P_2)^{a-1} \leq M^{a-1}.$$

Therefore,

$$P_1(x)^a \leq M^{a-1} P_2(x)^{a-1}.$$

Taking a-th roots gives

$$P_1(x) \leq M^{(a-1)/a} P_2(x)^{(a-1)/a}.$$

Using $P_2(x) \leq 2^{-\alpha}$, we obtain

$$P_1(x) \leq M^{(a-1)/a} 2^{-\alpha(a-1)/a} = \left(M \cdot 2^{-\alpha}\right)^{(a-1)/a}.$$

Since this holds for every $x \in \Omega$,

$$\max_{x \in \Omega} P_1(x) \leq \left(M \cdot 2^{-\alpha}\right)^{(a-1)/a}.$$

Hence

$$H_\infty(P_1) = -\log_2 \max_{x \in \Omega} P_1(x) \geq \frac{a-1}{a} \left(\alpha - \log_2 M\right).$$

This proves the claim. □

Concrete commitment min-entropy. We now combine Lemmas 7 and 8 to bound the min-entropy α of commitment for our concrete parameters. Since the commitment message is $(\mathbf{w}_h, \mathbf{w}_0)$ and min-entropy does not decrease under restriction, $\alpha = H_\infty(\mathbf{w}_h, \mathbf{w}_0) \geq H_\infty(\mathbf{w}_0)$, so it suffices to bound the low bit $\mathbf{w}_0$.

Reduction to the first mask block. In the CID scheme (Figure 5) the honest commitment is $\mathbf{w} = \mathbf{A}\mathbf{y}' \bmod 2q$ with $\mathbf{A} = [\,2(\mathbf{a_{gen}} - \mathbf{b}) + q\mathbf{j} \mid 2\mathbf{A_{gen}} \mid 2\mathbf{I}\,]$ and $\mathbf{y}' = \mathsf{CompressY}(\mathbf{y})$. Every block of $\mathbf{A}$ except the first is divisible by 2, and q is odd, so reducing modulo 2 leaves $\mathbf{w}_0 = \mathsf{LSB}(\mathbf{w}) = \mathsf{LSB}(\mathbf{y}'_0)\,\mathbf{j}$ (Lemma 1). Hence $\mathbf{w}_0$ is a deterministic function of the first compressed block $\mathbf{y}'_0 = \lfloor \mathbf{y}_0/\alpha_1 \rceil \in \mathcal{R}$, and $H_\infty(\mathbf{w}_0) = H_\infty\big(\mathsf{LSB}(\mathbf{y}'_0)\big)$ is the min-entropy of an n-bit vector. Note that the full mask $\mathbf{y} \leftarrow \mathsf{SampleY} \in \mathcal{R}^{l+m+1}$ has $(m+l+1)\,n$ Gaussian coordinates; only the n coordinates of its first block $\mathbf{y}_0$ enter $\mathbf{w}_0$.

Ideal min-entropy (Lemma 7). Were the wide sampler ideal, $\mathbf{y}_0$ would consist of n independent discrete Gaussians of width $r = 825$. Applying Lemma 7 with $A = \alpha_1$ and $\sigma = r = 825$ gives

$$\alpha_0 := H_\infty\big(\mathsf{LSB}(\mathbf{y}_0)^{\text{ideal}}\big) > n\big(1 - \log_2(1+2\varepsilon)\big), \quad \varepsilon = \frac{2e^{-\lambda_A}}{1 - e^{-3\lambda_A}}, \quad \lambda_A = \frac{\pi^2 \cdot 825^2}{2\alpha_1^2}.$$

For the three sets $\alpha_1 = 90, 135, 144$ this yields $\lambda_A \approx 414.7$, 184.3, 162.0, hence $\varepsilon < 2^{-232}$ and a per-bit defect $\log_2(1+2\varepsilon) < 2^{-231}$: the ideal block is statistically indistinguishable from uniform and $\alpha_0 > n - 2^{-220}$.

Passing to the implemented sampler (Lemma 8). The implemented $\mathsf{SampleY}$ deviates from this ideal only by (i) replacing exp with $\mathsf{ApproxExp}$ and (ii) replacing the half-Gaussian with the table-driven $\mathsf{BaseSampler}$. Over the full $(m+l+1)\,n$-coordinate mask these are exactly the deviations controlled by the wide-sampler theorem (Theorem 1): condition (C1) bounds the accept-branch distortion of the $\mathsf{ApproxExp}$ step (order $2\lambda - 1$) and condition (C2) bounds $R_{2\lambda+1}(\mathsf{BaseSampler} \,\|\, D_{\mathbb{Z}^+,\sigma_s})$ of the $\mathsf{BaseSampler}$ step (order $2\lambda + 1$). We bound each step

at its own order by the relative-error lemma. The table-driven BaseSampler step has single-draw divergence $R_{2\lambda+1}(\mathsf{BaseSampler} \,\|\, D_{\mathbb{Z}^+,\sigma_s}) = 1 + 2^{-95.40}$ (condition (C2)); raising the per-component divergence $(1+2^{-95.40})^n$ over the $l+m+1$ ring-element components of $\mathbf{y}$,

$$\mathrm{M}_1 := R_{2\lambda+1}\big(\mathbf{y}_{\mathrm{bs}}^{\mathrm{real}} \,\|\, \mathbf{y}^{\mathrm{ideal}}\big) \le (1+2^{-95.40})^{(l+m+1)\,n} \le 1 + 2^{-82}.$$

The ApproxExp step has per-coordinate emitted distortion $\delta_{\exp} = 2\eta/(1-\eta) \le 2^{-53.49}$, so Lemma 5 together with multiplicativity over the $(m+l+1)\,n$ mask coordinates gives

$$\mathrm{M}_2 := R_{2\lambda-1}\big(\mathbf{y}_{\exp}^{\mathrm{real}} \,\|\, \mathbf{y}^{\mathrm{ideal}}\big) \le \Big(1 + \tfrac{(2\lambda-1)(2\lambda-2)\delta_{\exp}^2}{2(1-\delta_{\exp})^{2\lambda}}\Big)^{(m+l+1)n/(2\lambda-2)}, \qquad \mathrm{M}_2 \le 1 + 2^{-84}.$$

Because $\mathsf{LSB}(\mathbf{y}_0')$ is a deterministic function of $\mathbf{y}$, the data-processing inequality (Lemma 6) carries each *single-step* mask divergence to the low-bit marginal. Write $L^P := \mathsf{LSB}(\mathbf{y}_0')^P$ for the low bit induced by a distribution P, and let P_2 be the fully ideal sampler, P_1 its BaseSampler-ideal variant (the implemented ApproxExp still in place), and P_0 the implemented sampler; then $R_{2\lambda-1}(L^{P_1} \,\|\, L^{P_2}) \le \mathrm{M}_2$ and $R_{2\lambda+1}(L^{P_0} \,\|\, L^{P_1}) \le \mathrm{M}_1$. We therefore apply Lemma 8 *twice* along the chain $P_2 \to P_1 \to P_0$, starting from $H_\infty(L^{P_2}) = \alpha_0$:

$$H_\infty(L^{P_1}) \ge \frac{2\lambda-2}{2\lambda-1}\big(\alpha_0 - \log_2 \mathrm{M}_2\big), \qquad H_\infty(L^{P_0}) \ge \frac{2\lambda}{2\lambda+1}\big(H_\infty(L^{P_1}) - \log_2 \mathrm{M}_1\big).$$

Composing the two applications,

$$\alpha \ge H_\infty(\mathbf{w}_0) = H_\infty(L^{P_0}) \ge \frac{2\lambda}{2\lambda+1}\left(\frac{2\lambda-2}{2\lambda-1}\big(\alpha_0 - \log_2 \mathrm{M}_2\big) - \log_2 \mathrm{M}_1\right).$$

Concrete values. For all three parameter sets $n = 2\lambda$ ($256 = 2{\cdot}128$, $512 = 2{\cdot}256$, $1024 = 2{\cdot}512$), so $\alpha_0 > 2\lambda - 2^{-220}$. Since $\mathrm{M}_1 \le 1 + 2^{-82}$ and $\mathrm{M}_2 \le 1 + 2^{-84}$ make $\log_2 \mathrm{M}_1$ and $\log_2 \mathrm{M}_2$ both below 2^{-81}, the two transfer factors $\frac{2\lambda}{2\lambda+1}$ and $\frac{2\lambda-2}{2\lambda-1}$ give

$$\alpha \ge \frac{2\lambda}{2\lambda+1} \cdot \frac{2\lambda-2}{2\lambda-1} \cdot 2\lambda - 2^{-79} \;>\; 2\lambda - 2 \;=\; n - 2,$$

i.e. at least 254, 510, 1022 bits for SUF-128/256/512, respectively. In every case the commitment min-entropy is $\approx 2\lambda$ bits and thus exceeds the security level λ by roughly λ bits, which is the value α used in Theorems 3–5.

Divergence, verifiability, and min-entropy of the HVZK simulator. Because the Rényi divergence admits no direct triangle inequality, we instantiate Definition 2 through a chain of *seven* transcript distributions $P_0, \ldots, P_6$ and bound the six consecutive steps separately, each at its own Rényi order a_k. Writing λ for the per-mode security level, the wide Gaussian sampler is charged at the order used in its concrete analysis ($a = 2\lambda - 1$ for ApproxExp, $a = 2\lambda + 1$ for the table-driven BaseSampler), while every iterative-rejection-sampler step is charged at the order $a = 2\lambda$ used in the error analysis (Section 4.1.4). The distributions differ in a single component at a time:

- P_0 — the honest signing distribution: the mask is produced by the real wide Gaussian sampler SampleY (finite-precision ApproxExp in the acceptance test, table-driven BaseSampler, truncated alternating series S_v, finite-precision logarithm in the interval test) and shaped by the iterative rejection sampler RejectSample with the secret key;
- P_1 — identical to P_0 except that the table-driven BaseSampler (Algorithm 27) is replaced by a perfect half-Gaussian (the acceptance test still uses the implemented ApproxExp);
- P_2 — identical to P_1 except that the exponential in the acceptance test is computed *exactly*, i.e. the implemented ApproxExp (Algorithm 29) is replaced by the ideal exp, so that the wide sampler emits an exact discrete Gaussian;

- P_3 — identical to P_2 except that the (rejection-free) ideal iterative transition is reinstated in place of a direct draw, so P_2 and P_3 differ only by the *intrinsic* error of the ideal transition (the "no-rejection" vs. "rejection" step);
- P_4 — identical to P_3 except that the alternating series S_v is summed *exactly* (the truncation after N terms is undone);
- P_5 — identical to P_4 except that the logarithm in the interval test is evaluated *exactly*; thus P_5 is the exact target distribution of the iterative sampler, still using the secret key;
- P_6 — the output of the simulator Sim, which samples $\widetilde{\mathbf{z}} \leftarrow \mathcal{D}_{\mathcal{R},r}$, sets $\mathbf{z} = \mathsf{CompressY}(\widetilde{\mathbf{z}})$ and recovers $\mathbf{w} = \mathbf{Az} - qc\mathbf{j} \bmod 2q$ (using $\mathbf{A} \circ [1, \mathbf{s}_1, \mathbf{s}_2]^T = q\mathbf{j} \bmod 2q$ and the bimodal identity $qc'\mathbf{j} \equiv qc\mathbf{j} \bmod 2q$, so that no secret is required), then computes $\mathbf{w}_0, \mathbf{w}_h, (\mathbf{z}_1, \mathbf{z}_2), \mathbf{h}, \mathbf{z}_2'$ as in the figure and aborts iff $\|(\mathbf{z}_1, \mathbf{z}_2')\|_2 > B_v$.

Each adjacent pair applies the *same* deterministic post-processing map ($\mathsf{CompressY}$, the commitment $\mathbf{w} = \mathbf{Az} - qc\mathbf{j}$, the hint, and the abort test) to two masks that differ only in the single component named above; by the Rényi data-processing inequality the transcript divergence is bounded by the corresponding mask divergence, which is exactly the quantity controlled in the error analysis.

Step M_1 (P_0 vs. P_1; perfect $\mathsf{BaseSampler}$; order $a_1 = 2\lambda + 1$). The masks differ only by replacing the table-driven half-Gaussian of Algorithm 27 with the ideal $D_{\mathbb{Z}^+,\sigma_s}$. The *single-draw* divergence is, by Theorem 1, condition (C2), $R_{2\lambda+1}\big(\mathsf{BaseSampler} \,\|\, D_{\mathbb{Z}^+,\sigma_s}\big) = 1 + 2^{-95.40} \le 1 + 2^{-94.91}$ at the binding SUF-512 level. The mask $\mathbf{y} \in \mathcal{R}^{l+m+1}$ has $l + m + 1$ ring-element components, each obtained from n independent such draws, so the per-component divergence is $(1 + 2^{-95.40})^n$ and, raising it over the $l + m + 1$ components by multiplicativity,

$$M_1 := R_{a_1}(P_0 \,\|\, P_1) \le \big((1 + 2^{-95.40})^n\big)^{l+m+1} = (1 + 2^{-95.40})^{(l+m+1)\,n} \le 1 + 2^{-82}.$$

Step M_2 (P_1 vs. P_2; exact exp*; order $a_2 = 2\lambda - 1$).* The masks differ only by replacing $\mathsf{ApproxExp}$ with the exact exp in the rejection test. By the emitted-sample analysis of Theorem 1, each emitted coordinate carries relative distortion $\delta_{\exp} = 2\eta/(1-\eta) \le 2^{-53.49}$ (accept-branch error $\eta \le 2^{-54.49}$), i.e. $1 - \delta_{\exp} \le \mathsf{ApproxExp}(x)/\exp(x) \le 1 + \delta_{\exp}$ pointwise. The relative-error bound for Rényi divergence (Lemma 5) at order $\alpha = 2\lambda - 1$ therefore gives, per coordinate,

$$R_{2\lambda-1} \le \left(1 + \tfrac{(2\lambda-1)(2\lambda-2)\delta_{\exp}^2}{2(1-\delta_{\exp})^{2\lambda}}\right)^{\frac{1}{2\lambda-2}} = 1 + \tfrac{2\lambda-1}{2}\delta_{\exp}^2 + O(\delta_{\exp}^3),$$

and, multiplying over the $(m + l + 1)\,n$ coordinates of one mask,

$$\begin{aligned} M_2 := R_{a_2}(P_1 \| P_2) &\le \left(1 + \frac{(2\lambda-1)(2\lambda-2)2^{-106.98}}{2(1-2^{-53.49})^{2\lambda}}\right)^{\frac{(m+l+1)n}{2\lambda-2}} \\ &\le 1 + 2^{-84}, \end{aligned}$$

$$(\lambda, n, l, m) \in \{(128, 256, 3, 3), (256, 512, 3, 2), (512, 1024, 3, 2)\}.$$

Condition (C1) is exactly the requirement that this loss stay within one bit when aggregated over all $Q_{\exp}$ emitted samples of the key lifetime.

Step M_3 (P_2 vs. P_3; ideal rejection transition; order $a_3 = 2\lambda$). The masks differ by the intrinsic deviation of the ideal rejection-free iterative transition from the target Gaussian ρ_g. By the concrete intrinsic-error analysis (Section 4.1.4), accumulated over the at most τ transition steps and converted through Lemma 5, $R_{a_3}(P_2 \,\|\, P_3) =: M_3 = R_{2\lambda}\big(\rho_\tau^{\mathrm{int}} \,\|\, \rho_g\big) < 1 + 2^{-82}$, with about half a bit of slack at the tightest (SUF-512) level.

Step M_4 (P_3 vs. P_4; exact series; order $a_4 = 2\lambda$). The masks differ by the truncation of the alternating series S_v after $N = 29$ terms. By the concrete truncation analysis (Section 4.1.4), $R_{a_4}(P_3 \,\|\, P_4) =: M_4 = R_{2\lambda}\big(\rho_\tau^{\mathrm{tr}} \,\|\, \rho_g\big) < 1 + 2^{-91.9}$.
Step M_5 (P_4 vs. P_5; exact logarithm; order $a_5 = 2\lambda$). The masks differ by the finite-precision evaluation of the logarithm in the interval test, whose single-step relative contribution is $C_{v,r}\,(\mathrm{e}^{\delta_{\log}} - 1 + \varepsilon_{\mathrm{q}})$, where $\varepsilon_{\mathrm{q}} \le 2^{-58}$ is the $\mathsf{SamplerU}$ mantissa quantization (Section 2.5.8); it is composed over the τ steps, giving $M_5 - 1 \lesssim 2^{-82.6}$ at the tightest level—the same order as M_3 and inside the reduction's Rényi-loss budget (Section 4.1.4).
Step M_6 (P_5 vs. P_6; secret removal; any order a_6). The rejection-free iterative sampler is designed so that its exact output $\widetilde{\mathbf{z}} = \mathbf{y} + \mathsf{StretchS}(s)\, c'$ is distributed as the centered $\mathcal{D}_{\mathcal{R},r}$, independently of the secret. Hence P_5 and P_6 apply the *same* deterministic map $\widetilde{\mathbf{z}} \mapsto (\mathbf{w}, c, \mathbf{z})$ to the *same* distribution $\mathcal{D}_{\mathcal{R},r}$; the only difference is that P_5 forms the commitment with the secret while P_6 forms $\mathbf{w} = \mathbf{A}\mathbf{z} - qc\mathbf{j}$ without it, and by $\mathbf{A} \circ [1, \mathbf{s}_1, \mathbf{s}_2]^T = q\mathbf{j}$ and the bimodal identity $qc'\mathbf{j} \equiv qc\mathbf{j} \bmod 2q$ the two coincide. Therefore $P_5 = P_6$ and $R_{a_6}(P_5 \,\|\, P_6) =: M_6 = 1$ for every order a_6.

Hence the scheme is $(M_1, \ldots, M_6, T_{\mathrm{Sim}})$-Rényi-HVZK of orders $(a_1, \ldots, a_6) = (2\lambda + 1,\ 2\lambda - 1,\ 2\lambda,\ 2\lambda,\ 2\lambda,\ 2\lambda)$. *Verifiability:* the simulator applies the same $\mathbf{z}_2'$ reconstruction and bound B_v that Verify applies, so every non-aborting output ($\mathbf{z} \neq \perp$) is accepting; the simulator is therefore β-verifiable with $\beta = \Pr[\|(\mathbf{z}_1, \mathbf{z}_2')\|_2 > B_v]$. The commitment min-entropy is obtained by the same argument as above.
Correctness ($\gamma = 1$). The compressed CID protocol is (γ, β)-correct (Definition 5) with $\gamma = 1$. Indeed, the only source of abort is the final norm test $\|(\mathbf{z}_1, \mathbf{z}_2')\|_2 > B_v$, which contributes the abort probability $\beta = \Pr[\|(\mathbf{z}_1, \mathbf{z}_2')\|_2 > B_v]$. Conditioned on the prover *not* aborting, acceptance is deterministic: the inner iterative sampler is rejection-free, and for an honestly generated non-aborting transcript the response relation $\mathbf{z} = \mathsf{CompressY}(\mathbf{y}) + s\, c'$ together with $\mathbf{h} = \mathsf{MakeHint}(\mathbf{z}_2, \mathbf{w})$ lets $\mathsf{UseHint}$ reconstruct exactly the same high-order commitment $\mathbf{w}_h$ and least significant bit $\mathbf{w}_0$ used to form the challenge, so the recomputed Fiat–Shamir hash matches c and the very same norm $\|(\mathbf{z}_1, \mathbf{z}_2')\|_2 \le B_v$ that the signer already verified is re-checked by Verify. Every non-aborting signature therefore passes verification with probability exactly 1, i.e. $\gamma = 1$, and the only correctness slack is the restart probability β analysed in Section 3.

Computational Uniqueness of Response (CUR). The adversary $\mathcal{A}$ would instead produce $w_0, c, z_{a,1}, h_a, z_{b,1}, h_b$, with $(z_{a,1}, h_a) \neq (z_{b,1}, h_b)$. Assuming that this guarantees that the reconstructed z_a' and z_b' differ, As the reconstructed vector z' is given by

$$z' = \left[z_1^T,\ z_2'^T\right]^T,$$

the equality $z_a' = z_b'$ implies that $z_{a,1} = z_{b,1}$. Additionally, given $z_{a,1} = z_{b,1}$, in order for $z_{a,2} = z_{b,2}$ to hold, it is required that

$$w_{0,a} \equiv w_{0,b} \pmod{2(q-1)},$$

which is only possible if

$$h_a \equiv h_b \pmod{2(q-1)}.$$

To ensure computational uniqueness of the response, the hint is encoded as its canonical representative modulo $2(q-1)$: the encoding $\mathsf{PolyToBytes}(\cdot, d_h)$ packs each hint coefficient in the canonical range $[0, 2(q-1)/\alpha_h)$, and $\mathsf{sigDecode}$ returns $\perp$ on any out-of-range coefficient, so that a signature carrying a non-canonical hint is rejected already at decoding. Consequently the map $(z_1, h) \mapsto z'$ is injective, and two distinct accepting transcripts for the compressed CID scheme imply two distinct reconstructed vectors z_a' and z_b', both of which have norm at most B_v. This directly yields a short nonzero kernel vector of the verification matrix and hence, after

replacing the public key by a uniform one via MLWE, a solution to MSIS with bound $2B_v$, as we now make precise.

Reduction from CUR to MLWE and MSIS. A successful CUR adversary $\mathcal{C}$ outputs, for a single commitment–challenge pair $(\mathbf{w}, c)$, two accepting transcripts whose reconstructed responses $\mathbf{z}'_a \neq \mathbf{z}'_b$ satisfy $\|\mathbf{z}'_a\|_2, \|\mathbf{z}'_b\|_2 \leq B_v$ and the *same* verification relation

$$\mathbf{A}\mathbf{z}'_a \equiv \alpha_h \mathbf{w}_h + \mathbf{w}'_0 + qc\mathbf{j} \equiv \mathbf{A}\mathbf{z}'_b \pmod{2q}, \qquad \mathbf{A} = [\, q\mathbf{j} - 2\mathbf{b} \mid 2\mathbf{A}_0 \mid 2\mathbf{I}\,].$$

Subtracting the two relations, $\boldsymbol{\Delta} := \mathbf{z}'_a - \mathbf{z}'_b$ is *nonzero*, has $\|\boldsymbol{\Delta}\|_2 \leq 2B_v$, and lies in the kernel of $\mathbf{A}$ modulo $2q$:

$$\mathbf{A}\,\boldsymbol{\Delta} \equiv \mathbf{0} \pmod{2q}.$$

When $(\mathbf{A}_0, \mathbf{b})$ is uniform, such a short nonzero $\boldsymbol{\Delta}$ is by definition a solution of $\mathsf{MSIS}_{q,m,\ell+1,2B_v}$ for $[\, q\mathbf{j} - 2\mathbf{b} \mid 2\mathbf{A}_0\,]$ with the identity block $2\mathbf{I}$ (the last m coordinates of $\boldsymbol{\Delta}$ being fixed modulo q by the first $\ell + 1$). The only gap to the real scheme is that there $\mathbf{b} = \mathbf{A}_0\mathbf{s}_1 + \mathbf{s}_2$ is an MLWE sample; we close it with one game hop, exactly as in the UF-NMA lemma.

Let Game_0 be the real CUR experiment—honest key conditioned on the KeyGen acceptance event acc, $\Pr[\mathsf{acc}] = \delta$—and let Game_1 be the same experiment with $(\mathbf{A}_0, \mathbf{b})$ uniform. An MLWE distinguisher $\mathcal{B}$ receives $(\mathbf{A}_0, \mathbf{t})$, installs $\mathbf{b} := \mathbf{t}$ *directly* as the verification key (no key rejection), runs $\mathcal{C}$, and outputs 1 iff $\mathcal{C}$ returns a valid CUR collision. If $\mathbf{t} = \mathbf{A}_0\mathbf{s}_1 + \mathbf{s}_2$ then $\mathbf{b}$ has the unconditioned key law, so $\Pr[\mathcal{B} = 1 \mid \mathsf{MLWE}] \geq \delta \cdot \mathsf{Adv}^{\mathsf{CUR}}_{\Sigma}(\mathcal{C})$; if $\mathbf{t}$ is uniform then $\Pr[\mathcal{B} = 1 \mid \mathsf{uniform}] = G'_1$. As the MLWE advantage is defined with an absolute value,

$$\mathsf{Adv}^{\mathsf{MLWE}}_{q,m,\ell,\mathcal{D}_{\mathcal{R},\sigma}}(\mathcal{B}) \geq \delta \cdot \mathsf{Adv}^{\mathsf{CUR}}_{\Sigma}(\mathcal{C}) - G'_1.$$

In Game_1 the matrix $\mathbf{A}$ is built from uniform $(\mathbf{A}_0, \mathbf{b})$, so the collision $\boldsymbol{\Delta}$ is precisely an $\mathsf{MSIS}_{q,m,\ell+1,2B_v}$ solution and $G'_1 \leq \mathsf{Adv}^{\mathsf{MSIS}}_{q,m,\ell+1,2B_v}(\mathcal{B}')$ for the induced solver $\mathcal{B}'$. Combining the two inequalities,

$$\mathsf{Adv}^{\mathsf{CUR}}_{\Sigma}(\mathcal{C}) \leq \frac{1}{\delta}\Big(\mathsf{Adv}^{\mathsf{MLWE}}_{q,m,\ell,\mathcal{D}_{\mathcal{R},\sigma}}(\mathcal{B}) + \mathsf{Adv}^{\mathsf{MSIS}}_{q,m,\ell+1,2B_v}(\mathcal{B}')\Big).$$

Hence computational uniqueness of response reduces to MLWE (public-key indistinguishability) and MSIS with the doubled norm bound $2B_v$—the same twice-looser MSIS instance already used to assess strong unforgeability in Section 4.3.2.

4.2.4 UF-NMA Security

Definition 7 ($\mathsf{MLWE}_{q,m,\ell,\mathcal{D}}$ Problem)**.** *For integers m, ℓ, and a probability distribution $\mathcal{D}$, we say that the advantage of algorithm $\mathcal{A}$ in solving the decisional $\mathsf{MLWE}_{q,m,\ell,\mathcal{D}}$ problem is*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{MLWE}}_{q,m,k,\mathcal{D}} := \Big|&\Pr[b = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{m\times\ell}; \mathbf{t} \leftarrow \mathcal{R}_q^m; b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{t})] \\ &- \Pr[b = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{m\times\ell}; \mathbf{s}_1 \leftarrow \mathcal{D}^\ell; \mathbf{s}_2 \leftarrow \mathcal{D}^m; b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2)]\Big|.\end{aligned}$$

This **BimodalSelfTargetMSIS** problem is parametrized by a size bound $B_v > 0$ and integers $q, m, \ell > 0$. Furthermore, it depends on a hash function H_0 from $\mathcal{R}_{2q}^m \times \mathcal{M}$ to $\mathcal{C}$ where $\mathcal{M} \subseteq \{0,1\}^*$ is a message space and $\mathcal{C} \subseteq \mathcal{R}_2$ is a challenge space.

Definition 8 ($\mathsf{BimodalSelfTargetMSIS}_{\mathsf{H},q,m,\ell,B_v}$ problem)**.** *Given $\mathbf{A}_0 \in \mathcal{R}_q^{m\times\ell}$ and $\mathbf{b} \in \mathcal{R}_q^m$ with elements sampled uniformly at random in $\mathcal{R}_q$, an adversary $\mathcal{A}$ against the $\mathsf{BimodalSelfTargetMSIS}_{\mathsf{H},q,m,\ell,B_v}$ problem is to produce $(\mathbf{z}, c, \mu) \in (\mathcal{R}^{m+\ell+1}, \mathcal{C}, \mathcal{M})$ such that $\mathbf{z} \neq \mathbf{0}$, $\|\mathbf{z}\|_2 < B_v$ and*

$$\mathsf{H}(\mathbf{A}\mathbf{z} - qc\mathbf{j} \pmod{2q}, \mu) = c$$

where $\mathbf{A} = [q\mathbf{j} - 2\mathbf{b}, 2\mathbf{A}_0, 2\mathbf{I}] \pmod{2q}$.

The UF-NMA security of the uncompressed version of our scheme is almost directly given by the assumed hardnesses of the decisional MLWE problem and the BimodalSelfTargetMSIS problem. For the compressed version of our scheme, the UF-NMA security similarly relies on the assumed hardness of these problems, but not quite as obviously. For both versions of our scheme, we argue for their UF-NMA security by the following lemma.

Lemma 9. *Let $\mathcal{A}$ be an adversary against the UF-NMA security of our signature scheme. Then, there exist two adversaries $\mathcal{B}$ and $\mathcal{B}'$, each using $\mathcal{A}$ once and performing negligible additional work, such that*

$$\mathsf{Adv}^{UF\text{-}NMA}(\mathcal{A}) \leq \frac{1}{\delta}\left(\mathsf{Adv}^{\mathsf{MLWE}}_{q,m,\ell,\mathcal{D}_{\mathcal{R},\sigma}}(\mathcal{B}) + \mathsf{Adv}^{\mathsf{BimodalSelfTargetMSIS}}_{\mathsf{H},q,m,\ell,B_v}(\mathcal{B}')\right),$$

where δ is the probability of an accepted secret key occurring.

Proof. We define two games Game_0 and Game_1. Game_0 represents the UF-NMA security experiment for our scheme, so the advantage of $\mathcal{A}$ in Game_0 corresponds directly to $\mathsf{Adv}^{\text{UF-NMA}}(\mathcal{A})$ against the real scheme. For Game_1, $\mathbf{A}_0$ and $\mathbf{b}$ in the public key are uniformly random instead of given by an MLWE instance; the advantage of $\mathcal{A}$ in Game_1 is denoted G_1.

The key observation is that the honestly generated verification key is distributed as an MLWE sample *conditioned* on passing the norm-based key-acceptance test of KeyGen (the two-sided bound on $\|\mathsf{StretchS}(1, \mathbf{s}_1, \mathbf{s}_2')\|_2$), an event we denote acc with $\Pr[\mathsf{acc}] = \delta$.

To bound the gap between the two games, the MLWE distinguisher $\mathcal{B}$ is given an instance $(\mathbf{A}_0, \mathbf{b})$, sets it *directly* as the verification key for $\mathcal{A}$ (performing *no* key rejection), and outputs 1 if and only if $\mathcal{A}$ produces a valid forgery. If $(\mathbf{A}_0, \mathbf{b})$ is an MLWE instance, then $\mathbf{b}$ follows the *unconditioned* key distribution; writing $\mathsf{Adv}^{\text{UF-NMA}}(\mathcal{A})$ for the forgery probability against the real scheme (i.e. conditioned on acc), we have

$$\Pr[\mathcal{B} = 1 \mid \mathsf{MLWE}] = \Pr[\mathsf{forge}] \geq \Pr[\mathsf{forge} \wedge \mathsf{acc}] = \delta \cdot \mathsf{Adv}^{\text{UF-NMA}}(\mathcal{A}).$$

If $(\mathbf{A}_0, \mathbf{b})$ is uniform, then $\Pr[\mathcal{B} = 1 \mid \mathsf{uniform}] = G_1$. Since the MLWE distinguishing advantage is defined with an absolute value, the inequality $|x| \geq x$ gives

$$\mathsf{Adv}^{\mathsf{MLWE}}_{q,m,\ell,\mathcal{D}_{\mathcal{R},\sigma}}(\mathcal{B}) = \left|\Pr[\mathcal{B} = 1 \mid \mathsf{MLWE}] - \Pr[\mathcal{B} = 1 \mid \mathsf{uniform}]\right| \geq \delta \cdot \mathsf{Adv}^{\text{UF-NMA}}(\mathcal{A}) - G_1,$$

which rearranges to

$$\delta \cdot \mathsf{Adv}^{\text{UF-NMA}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathsf{MLWE}}_{q,m,\ell,\mathcal{D}_{\mathcal{R},\sigma}}(\mathcal{B}) + G_1.$$

The algorithm $\mathcal{B}'$ instead relies on whatever advantage $\mathcal{A}$ has in Game_1. To achieve this, the input to the target BimodalSelfTargetMSIS problem is used to construct the verification key that is provided to $\mathcal{A}$. This verification key is thus constructed from uniformly random $\mathbf{A}_0$ and $\mathbf{b}$. For the uncompressed version of our signature scheme, a forgery produced by $\mathcal{A}$ directly corresponds to a solution for the BimodalSelfTargetMSIS problem of $\mathcal{B}'$, which proves the statement for the uncompressed version.

For the compressed version of our signature scheme, the random oracle that $\mathcal{B}'$ provides when simulating $\mathcal{A}$ is given by

$$\mathsf{H}_{\mathsf{ch}}(\mathsf{EncodeCom}(\mathbf{w}_h, \mathbf{w}_0')\|\mu) = \mathsf{H}(\alpha_h \mathbf{w}_h + \mathbf{w}_0', \mu)$$

where H is the random oracle for the BimodalSelfTargetMSIS instance. A forgery from the signature scheme consists of a signature $(\mathbf{z}_1, \mathbf{h}, c)$ and a hashed message μ such that

$$\mathsf{H}_{\mathsf{ch}}(\mathsf{EncodeCom}(\mathbf{w}_h, \mathbf{w}_0')\|\mu) = \mathsf{H}(\alpha_h \mathbf{w}_h + \mathbf{w}_0', \mu) = c$$ [3]

[3] In the detailed description of the signing algorithm, the output of H_{ch} is a seed $seed_c$, which will be used to derive c. This does not affect the security analysis; therefore, we directly assume that the output is c.

for $\mathbf{w}_h$ and $\mathbf{w}'_0$ defined as in Verify$(\cdot)$ algorithm. A valid forgery additionally corresponds to a vector $\mathbf{z}' = (\mathbf{z}_1, \mathbf{z}'_2)$ such that $\|\mathbf{z}'\|_2 \leq B_v$, where $\mathbf{z}'_2 = (\alpha_h \mathbf{w}_h + \mathbf{w}'_0 - \mathbf{A}_1\mathbf{z}_1 + qc\mathbf{j})/2$. Given such a forgery, we claim that $(\mathbf{z}', c, \mu)$ solves the target BimodalSelfTargetMSIS instance. This is the case as

$$\mathbf{A}\mathbf{z}' = \mathbf{A}_1\mathbf{z}_1 + 2\mathbf{z}'_2 = \alpha_h\mathbf{w}_h + \mathbf{w}'_0 + qc\mathbf{j}$$

and thus

$$\mathsf{H}(\mathbf{A}\mathbf{z}' - qc\mathbf{j}, \mu) = \mathsf{H}(\alpha_h\mathbf{w}_h + \mathbf{w}'_0, \mu) = c$$

as claimed. Furthermore, valid signatures satisfy that $\|\mathbf{z}'\|_2 \leq B_v$ and therefore the norm bound for the BimodalSelfTargetMSIS instance is satisfied. That is $G_1 \leq \mathsf{Adv}^{\mathsf{BimodalSelfTargetMSIS}}_{\mathsf{H},q,m,\ell,B_v}(\mathcal{B}')$. □

4.2.5 Reduction from BimodalSelfTargetMSIS to MSIS

The lemma above bounds the UF-NMA advantage by the hardness of MLWE and BimodalSelfTargetMSIS. Modelling H_0 as a random oracle, we now complete the chain by reducing BimodalSelfTargetMSIS itself to plain (homogeneous) MSIS via rewinding, so that the unforgeability of SHUTTLE rests entirely on the standard MLWE and MSIS assumptions; the concrete cost of the resulting MSIS instance is estimated in Section 4.3.2. Let $\mathcal{A}$ make at most Q queries to H_0 and solve $\mathsf{BimodalSelfTargetMSIS}_{\mathsf{H},q,m,\ell,B_v}$ with probability ϵ. A winning output $(\mathbf{z}, c, \mu)$ obeys $c = \mathsf{H}_0(\mathbf{w}, \mu)$ with $\mathbf{w} = \mathbf{A}\mathbf{z} - qc\mathbf{j} \bmod 2q$; except with probability $1/|\mathcal{C}|$ the pair $(\mathbf{w}, \mu)$ was queried, say as the $i^\star$-th distinct query. The solver $\mathcal{B}$ runs $\mathcal{A}$ a second time on identical coins and identical oracle answers through the $i^\star$-th query, but reprograms that answer to a fresh independent $c' \in \mathcal{C}$. By the general forking lemma [39, 9], with probability at least $\epsilon\big(\epsilon/Q - 1/|\mathcal{C}|\big)$ the second run also wins and forks at the same query, returning $(\mathbf{z}', c', \mu)$ with the *same* $(\mathbf{w}, \mu)$ but $c' \neq c$.

Both forgeries hash the same commitment, i.e. $\mathbf{A}\mathbf{z} - qc\mathbf{j} \equiv \mathbf{A}\mathbf{z}' - qc'\mathbf{j} \pmod{2q}$. Splitting $\mathbf{z} = (z_0, \mathbf{z}_1, \mathbf{z}_2)$ along the columns of $\mathbf{A} = [\, q\mathbf{j} - 2\mathbf{b} \mid 2\mathbf{A}_0 \mid 2\mathbf{I}\,]$ and writing $\boldsymbol{\Delta} = \mathbf{z} - \mathbf{z}'$, $\Delta c = c - c'$, the difference reads

$$q(\Delta z_0 - \Delta c)\,\mathbf{j} \;+\; 2\big(\mathbf{A}_0\Delta\mathbf{z}_1 + \Delta\mathbf{z}_2 - \mathbf{b}\,\Delta z_0\big) \;\equiv\; \mathbf{0} \pmod{2q}.$$

Reducing modulo 2 and using that q is odd and $\mathbf{j} = [1, 0, \ldots, 0]^\top$ forces $\Delta z_0 \equiv \Delta c \pmod 2$, so the bimodal term $q(\Delta z_0 - \Delta c)\mathbf{j}$ is a multiple of $2q$ and drops out, leaving the homogeneous relation

$$\mathbf{A}_0\Delta\mathbf{z}_1 + \Delta\mathbf{z}_2 - \mathbf{b}\,\Delta z_0 \;\equiv\; \mathbf{0} \pmod{q}.$$

Since $c \neq c'$ in $\mathcal{R}_2$ we have $\Delta c \not\equiv 0 \pmod 2$, hence $\Delta z_0 \neq 0$ and $\boldsymbol{\Delta} \neq \mathbf{0}$; moreover $\|\boldsymbol{\Delta}\|_2 \leq \|\mathbf{z}\|_2 + \|\mathbf{z}'\|_2 < 2B_v$. Thus $(\Delta\mathbf{z}_1, \Delta\mathbf{z}_2, \Delta z_0)$ is a short nonzero solution of $\mathsf{MSIS}_{q,m,\ell+1,2B_v}$ for the uniform matrix $[\, \mathbf{A}_0 \mid \mathbf{I}_m \mid -\mathbf{b}\,]$. Collecting the forking probability through $\mathsf{Adv}^{\mathsf{MSIS}}_{q,m,\ell+1,2B_v}(\mathcal{B}) \geq \epsilon(\epsilon/Q - 1/|\mathcal{C}|)$ yields

$$\mathsf{Adv}^{\mathsf{BimodalSelfTargetMSIS}}_{\mathsf{H},q,m,\ell,B_v}(\mathcal{A}) \;\leq\; \frac{Q}{|\mathcal{C}|} \;+\; \sqrt{Q \cdot \mathsf{Adv}^{\mathsf{MSIS}}_{q,m,\ell+1,2B_v}(\mathcal{B})}\,.$$

4.3 Concrete Security Estimates

We rely on the Core-SVP model [5] to derive concrete security estimates in Table 10. This model adopts a conservative approach by equating the runtime of the BKZ algorithm (with block size β) to that of a single SVP oracle query in the same dimension β. When β is sufficiently large, the most efficient SVP solver is sieving, whose currently best known complexity is $2^{0.292\beta}$ [8] in the classical setting and $2^{0.257\beta}$ [13] in the quantum setting.

4.3.1 Cost of MLWE

Our security evaluation for MLWE accounts for the main attack families against module-lattice schemes: primal attack [5, 3], BDD (hybrid) attacks [26, 2, 24] and dual attacks [12, 35]. Other approaches such as BKW attack [1], algebraic attack [6] and combinatorial attacks [36, 31] are excluded from our analysis because they are less competitive than primal and dual attacks in the parameter regime of Shuttle. To evaluate the hardness of MLWE, one usually first transformed MLWE instances into standard LWE instances and then assess the concrete attack costs. The study in [24] exploits the special structure of $\mathbb{Z}_q[X]/(X^n + 1)$ to speed up BDD hybrid attacks [26, 2] and dual attack in [12]. We observe that this ring structure can be even more fully exploited in the dual attack framework of [12] compared to the analysis in [24], and that the asymmetry between σ_1 and σ_2 in our parameter sets provides further advantage for this dual attack [12]. Additionally, we also consider it prudent to include the dual attack from [35] in our cost estimates, as noted in [23], notwithstanding the controversial assumptions underlying [35]. We incorporate all the above-mentioned methods into our security evaluation of MLWE.

Table 12 lists the individual MLWE attack costs for each parameter set under the Core-SVP model, obtained using the lattice estimator [4] together with our adapted evaluation code from [12]. The minimum cost in each row is bolded.

Table 12: Attack costs against Shuttle's underlying MLWE instance in the Core-SVP model.

Cost model	**Primal**	**BDD**	**Dual [35]**	**Dual [12]**
		Shuttle-128		
Classical	144.2	145.2	138.5	**138.0**
Quantum	127.0	127.9	**123.4**	133.1
		Shuttle-256		
Classical	300.8	301.8	333.7	**279.7**
Quantum	264.7	265.7	297.9	**252.0**
		Shuttle-512		
Classical	661.7	662.5	∞	**625.0**
Quantum	582.4	583.2	∞	**553.4**

4.3.2 Cost of MSIS

For the MSIS problem, one often transforms the MSIS instance into SIS instances to solve [19]. Solving the SIS problem under the ℓ_2-norm is reduced to finding a short vector in a q-ary lattice. The lattice estimator carries out this evaluation automatically: it determines the minimum BKZ block size required to find a vector of the specified length, and then estimates the attack cost in the Core-SVP moel. Since the norm bound is smaller than q, the attack of [18] does not apply to the parameters of Shuttle. All our MSIS security estimates are obtained through the lattice estimator.

4.3.3 Cost of Self-Target-MSIS

The unforgeability of Shuttle ultimately rests on the $\mathsf{BimodalSelfTargetMSIS}$ problem defined above—a bimodal variant, following HAETAE [14], of the $\mathsf{SelfTargetMSIS}$ problem introduced for Dilithium [19]. We estimate its hardness exactly as in HAETAE. Because H_0 is modeled as a random oracle, the challenge $c = \mathsf{H}_0(\cdot)$ is uniform and independent of its argument, and the

oracle is oblivious to the algebraic structure of its input. Consequently the only known strategy for an adversary is to (i) choose a target commitment $\mathbf{w} \in \mathcal{R}_{2q}^m$, (ii) obtain $c = \mathsf{H}_0(\mathbf{w}, \mu)$ for some message μ, and (iii) compute a short vector $\mathbf{z}$ with $\|\mathbf{z}\|_2 < B_v$ satisfying

$$\mathbf{A}\mathbf{z} - qc\mathbf{j} \equiv \mathbf{w} \pmod{2q}.$$

Such a $\mathbf{z}$, together with the unit coefficient on the $\mathbf{w}$-column, is precisely a solution of $\mathsf{MSIS}_{q,m,\ell+2,\sqrt{B_v^2+1}}$ for the augmented matrix $[\,\mathbf{w} \mid \mathbf{A}\,]$. We denote this MSIS instance as MSIS-UF. We therefore estimate the cost of BimodalSelfTargetMSIS as that of MSIS with norm bound $\sqrt{B_v^2+1} \approx B_v$, evaluated with the method described in Subsection 4.3.2. Following the approach in [14], we additionally evaluate the cost of MSIS with a bound that is twice (i.e. $2B_v$) as loose, which we denote as MSIS-sUF, so as to assess the strong unforgeability of Shuttle scheme. Since $2B_v$ is still smaller than q, the attack of [18] does not apply to these parameters either. The corresponding results are reported in Table 13.

Table 13: Attack costs for MSIS-UF and MSIS-sUF in the Core-SVP model.

Scheme	Scenario	Classical	Quantum
Shuttle-128	MSIS-UF	202.9	178.6
	MSIS-sUF	165.3	145.5
Shuttle-256	MSIS-UF	322.1	283.5
	MSIS-sUF	266.3	234.4
Shuttle-512	MSIS-UF	617.6	543.6
	MSIS-sUF	522.4	459.8

4.4 Side-Channel Resistance and Implementation Security

4.4.1 Timing attacks.

Scope and trust model. The implementation-security claim made for Shuttle is *constant-time execution*: across the reference and the AVX2/AVX512 backends, neither the program control flow nor the sequence of memory-access addresses depends on any secret value. Concretely, this is the guarantee that resists *timing* and *cache-timing* attacks, in which the adversary may repeatedly trigger signing and observe wall-clock latency and microarchitectural cache state (e.g. Flush+Reload or Prime+Probe on a shared core), but obtains no power or electromagnetic trace. The data treated as secret are: the signing key (s, e) and the seeds from which it is unpacked; all per-signature randomness (rnd, ρ', $seed_y$); the discrete-Gaussian mask y and every intermediate Gaussian/uniform sample, sign bit, and acceptance value produced by the samplers of Algorithm 28, Algorithm 27, Algorithm 29, Algorithm 25, Algorithm 26, and Algorithm 23; the Iterated-Rejection-Sampler (IRS) internal state; and the response z and its parts (z_1, z_2') *before* they are committed to the public signature. Once a value is written into the public key or an emitted signature it is *declassified* and may thereafter drive data-dependent code. The single quantity an adversary is permitted to observe in time is the Fiat–Shamir-with-aborts *signing-attempt count*; we show this is a function only of public/declassified data and is independent of the key.

Constant-time by construction: the per-kernel argument. Every secret-dependent computation in the signing path is realised with data-oblivious arithmetic; we walk through them in execution order.

*Discrete-Gaussian sampling (*SampleY/SampleDGauss*, Algorithm 28).* The wide coefficient is formed by the straight-line convolution $z = kx + y$ (a fixed multiply-add, no branch).

Acceptance is a branchless unsigned compare $\mathsf{ct_lt}(u, \hat{p})$ between the 64-bit Bernoulli draw u and the Q64 threshold $\hat{p}$, implemented with the Hacker's-Delight borrow identity rather than a compare-and-jump. The $z = 0$ half-mass correction is folded with an independent candidate bit f as $\mathit{keep} = \mathit{accept} \wedge \big(1 \oplus (z_0 \wedge f)\big)$, where $z_0 = [\![z = 0]\!]$ is computed by a branchless zero test on the full magnitude z, and the output sign is applied separately by the masked select $\mathsf{ct_sel}(\mathit{sign}, -z, z)$. Each mini-batch consumes a fixed amount of random material regardless of how many candidates are accepted. The revised backends must follow the same initial/refill schedule to obtain byte-identical outputs. The rejection-timing observable considered here is the accept count of the current mini-batch. This count is a function solely of fresh sampler coins and public thresholds; it is not a function of the secret key or of any secret-dependent memory access. Under the random-oracle/XOF idealization for the seed expander, observing predicates of these fresh coins does not enable seed recovery or prediction of domain-separated future coins. Moreover, the count is statistically independent of the emitted Gaussian value: conditioning on the event that the first acceptance occurs at any fixed attempt r leaves the accepted proposal distributed exactly as the proposal conditioned on acceptance. Hence the timing signal associated with the accept count carries no information about the secret key and does not bias or disclose the sampled mask y.

For the revised independent-fold sampler, this independence is a distributional statement in the independent-uniform-coins/XOF model. The magnitude and fold candidates are i.i.d., and their acceptance flags are independent of the output sign bit. Let $\alpha > 0$ be the per-candidate acceptance probability, T the first accepted attempt, M its magnitude, and Q the law of a single candidate magnitude conditioned on acceptance. For any set A and $r \geq 1$,

$$\Pr[T = r, M \in A] = (1 - \alpha)^{r-1} \alpha Q(A).$$

This factorisation proves that the retry count is independent of the accepted magnitude. The independent output sign preserves this independence for the signed value, even though it is held fixed across retries. With ideal arithmetic the emitted distribution is the target Gaussian; with finite precision it is the corresponding accepted-candidate distribution. This argument concerns the retry count only: constant-time candidate arithmetic and output placement still require implementation verification.

*Base sampler (*96-bit RCDT scan*, Algorithm 27).* The cumulative-distribution table is never indexed by the sample. Instead the kernel performs a *full-table* linear scan over all public rows with no value-dependent early exit, so its timing depends only on the public table length. Each row is compared against the secret 96-bit draw $v = (v_0, v_1, v_2)$ by a single-compare borrow-fold chain $b_0 = \mathsf{ct_lt}(v_0, Z_0)$, $b_1 = \mathsf{ct_lt}(v_1, Z_1 + b_0)$, $b_2 = \mathsf{ct_lt}(v_2, Z_2 + b_1)$, with the borrow folded into the public threshold limb (never into v, which would underflow). A table invariant guaranteeing that no middle/high limb equals `0xFFFFFFFF` makes the single-compare form bit-exact to the textbook less-or-equal-with-borrow comparison; the invariant constrains only the public table and is asserted offline. The output index is accumulated by the unconditional add of a $\{0, 1\}$ value. The AVX2 fork maps the unsigned order onto the signed comparator `vpcmpgtd` by a monotone bit-63 flip and a flip-commute identity, and the AVX512 fork uses the native `vpcmplt_epu32` mask; both read the identical limb layout and are byte-exact to the scalar oracle. This is the precise countermeasure to the Flush, Gauss, and Reload cache attack on the BLISS CDT sampler [11], whose binary-search touched a cache line determined by the secret sample.

*Bernoulli evaluation (*ApproxExp*, Algorithm 29).* Shuttle is integer-only. ApproxExp uses *no table, no branch, no division, and no floating point*: it is a fully unrolled degree-8 Horner polynomial in a Q64 `__int128` accumulator followed by seven squarings (a fixed 15 multiplies) to realise the 128th power, with a certified worst-case relative error of $2^{-54.49}$ and the degree/squaring count pinned by static assertions. Because there is no data-dependent address and no compare-jump, the acceptance threshold is computed by the identical instruction stream

for every (x, y). This directly removes the variable-time Bernoulli/1/cosh rejection operator that was exploited in BLISS [21, 10].

*IRS transition test (*ApproxLog/SamplerU/RejectSample*, Algorithm 25, Algorithm 26, Algorithm 23).* The IRS is *rejection-free*: RejectSample applies *exactly* τ R-transitions per call and never aborts, so there is no secret-dependent loop count and SamplerU consumes a fixed 18τ pseudorandom bytes drawn in a single bulk squeeze. ApproxLog is a full-table segmented $\log_2$ evaluation: its row fetch ORs every one of the 4×14 table entries under an arithmetic equality mask ($z = a \oplus b$, $nz = (z \mid (\sim z{+}1)) \gg 63$, $mask = nz - 1$), so the load address is data-independent, followed by a branch-free Q62 Horner with rounded high-half multiplies. SamplerU extracts an 80-bit exponent by a branchless count-leading-zeros over OR-with-sentinel limbs selected by arithmetic nonzero masks; the underlying `lzcnt`/`bsr` are not variable-latency and require no integer→float conversion. Inside RejectSample, sign normalisation is a two's-complement mask flip $(t - 1) \gg 63$; the 15-pair interval test runs *all* fixed boundary pairs and ORs the result; and the polynomial update $z_i \mathrel{-}= f \cdot v_i$ with $f \in \{-1, +1\}$ is unconditional. The one permitted data-dependent branch is the ascending-j test $c_j \stackrel{?}{=} 1$ on the challenge support, which the verifier recomputes from $seed_c$ and is therefore public. No division or modulo appears; the rate constant is applied by a fixed-point reciprocal multiply. The *only* timing observable created by the whole signing path is the outer-loop count gated by the *public* norm test $\|(z_1, z_2')\|_2 \le B_v$, whose operands are public/declassified.

Response and NTT arithmetic. The secret residues of s, y, z and the commitment pass through a straight-line uint16 Montgomery NTT with data-oblivious butterflies. The freeze operation is a division-free Barrett reduction with a compile-time reciprocal followed by a single masked conditional subtraction $r - (ge \wedge q)$, where $ge = 0 - [r \ge q]$ is a sign mask; this is bit-identical to $a \bmod q$ and uses no runtime divide and no branch. SIMD kernels use only fixed permutations, so backend-native slot order is an internal representation, never a declassification.

rANS encoder and signature packing (Section 2.5.10). The rANS codec and `pack_sig` operate *only on public, post-signing data*: by the time the symbol stream is serialised, the secret mask y has been consumed and the emitted z is about to become the signature. Consequently the data-dependent renormalisation `while` loops and cursor bounds are not a leak. The sole timing adjacency to secret behaviour is the out-of-support restart counter, which equals the already-public Fiat–Shamir-with-aborts attempt count and reveals nothing about the key. One hard rule is nonetheless enforced because rANS renormalisation is a division acting on the symbol stream (the KyberSlash class): it must be a precomputed-reciprocal multiply, and the object-code scanner forbids *every* division mnemonic in the rANS translation unit with no allowlist exception. Hint decoding range-checks each value into $[0, H_h)$ and *rejects* out-of-range hints (it never reduces $\bmod H_h$, which would forge a hint), accumulating the per-coefficient reject bits and branching once at the end.

Audit methodology. The constant-time property is not merely asserted; it is verified by a multi-tool audit matrix, with one machine-code gate designated authoritative and the remaining tools serving as cross-checks. The methods, their target leakage class, the build/parameter coverage, and their result are summarized in Table 14.

The *object-code forbidden-mnemonic scan* is the primary defence and the reason the audit descends below source level: source-level constant-time code is insufficient because `gcc -Os` re-emits a genuine hardware `idiv` for a divide-by-constant that `-O3` would lower to a reciprocal multiply. The scanner therefore compiles every secret-handling translation unit—the samplers, the IRS, the rANS and packing code, the NTT and reduction layer, and the active XOF backend—across the entire matrix, disassembles them, and flags any instruction in the variable-latency divide, secret gather/scatter, `sqrt`, or integer↔float convert/compare families. The float family is included precisely because Shuttle is integer-only, so any `cvtsi2sd`/`comisd` would betray a hidden floating-point path. Only two classes of exception are allowlisted, each citing the bound-

Table 14: Constant-time audit methods for SHUTTLE. The object-code forbidden-mnemonic scan is the single authoritative instruction-level gate; the remaining methods are subordinate cross-checks. The coverage column abbreviates the build matrix $\{\mathsf{ref}, \mathsf{avx2}, \mathsf{avx512}\} \times \{128, 256, 512\} \times \{\mathsf{gcc}, \mathsf{clang}\} \times \{\text{-O3}, \text{-Os}\} \times \{\mathsf{ngcc}, \mathsf{sha3}\}$.

Method	**Tool / harness**	**Leakage class**	**Coverage**	**Result**
Object-code mnemonic scan *(authoritative)*	KYBERSLASH-style disassembly of compiled secret-handling objects	variable-latency divide, secret gather/scatter, `sqrt`, int↔float convert/compare	full matrix (backend × mode × {gcc,clang} × {-O3,-Os} × {ngcc,sha3})	Zero flagged instructions in every secret object; rANS unit division-free with *no* allowlist; only public-length divisions and public-lane SHAKE gathers allowlisted.
Statistical timing	dudect Welch t-test, per-kernel + full-sign	first-order timing dependence on secret input	backend × mode; N=512 / $2{\times}10^5$	Per-kernel cells PASS (worst $\lvert t\rvert \approx 2.5$) once inputs are pre-generated; full-sign t is the public B_v rejection channel, not a leak.
Dynamic taint tracking	patched Valgrind/ctgrind (timecop) on tainted secrets	secret operand reaching a variable-latency instruction	ref,avx2, all modes (avx512: EVEX/VBMI2 SIGILL)	No tainted operand reaches a variable-latency instruction; `cmov` idioms suppressed, divide class kept live.
Cross-backend KAT + diff. fuzz	known-answer vectors + randomized fuzzing	SIMD-vs-scalar divergence on the CT paths	backend × mode × {ngcc,sha3}	ref ≡ avx2 ≡ avx512 byte-identical; SIMD kernels run the same straight-line logic as the scalar oracle.
Boundary + fault + parser	boundary doc, fault harness, malformed-sig corpus	misclassified declassification, parser nondeterminism	backend × mode	Every exception cites a boundary entry; every malformed signature is a deterministic reject; no fault forges.
Memory safety	ASan/LSan over keygen→sign→verify	heap errors / leaks in the Gaussian path	backend × mode × {ngcc,sha3}	All cells PASS, no leak.

ary document: divisions on *public-length* block counts inside named DRBG/XOF/Gaussian-length functions, and the public-lane pointer gathers of the four- and eight-way SHAKE permutations. The rANS unit, the single most dangerous division site, carries no exception at all. The scan reports zero flagged instructions in every secret object across the matrix; its verdict is authoritative and the remaining tools are subordinate. The *dudect* harness applies Welch's t-test online over fixed-versus-random secret inputs, with a cropped-percentile family to suppress tail outliers; a load-bearing lesson is that earlier SIMD-only $|t| = 10-26$ readings were *harness artifacts* from class-correlated work executed before the timing window (a buffer write triggering store-to-load-forwarding and 4K-aliasing penalties on wide loads, and constant-folded fixed-class literals racing a modulo on the random class), not kernel leaks; pre-generating all inputs outside the timed loop removed them, after which every cell passes. We stress that dudect is a smoke test on a shared host and the object-code scan, not the t-statistic, is the authoritative gate. The *timecop* patched-Valgrind run taints the secret region of the unpacked key and the IRS-feeding randomness and dynamically confirms that no tainted operand reaches a variable-latency instruction; it covers the reference and AVX2 backends, with AVX512 falling back to the static scan because Valgrind faults on EVEX/VBMI2 encodings. The *cross-backend KAT and differential fuzz* establish that the reference, AVX2, and AVX512 implementations are byte-identical on every input and every parameter set, which both validates functional correctness and pins that the vectorised constant-time kernels execute the same oblivious logic as the scalar reference. Finally, the *boundary, fault, and negative-parser* matrix verifies that every declassification is documented, that every malformed signature (wrong length, bad challenge seed, rANS decode failure, out-of-range hint) is a deterministic public reject, and that faults never produce a forged acceptance, while the sanitizer gate certifies memory safety of the bulk-preset Gaussian allocation path.

Contrast with BLISS. Auditors are right to scrutinise Gaussian-plus-rejection signatures, because the canonical scheme of this family, BLISS [17], was broken by an entire campaign of side-channel attacks. Shuttle is designed so that none of those leak vectors has an analog. (i) Flush, Gauss, and Reload [11] recovered the Gaussian mask through the *secret-indexed cache line* of the CDT binary search; Shuttle replaces value-dependent table lookup with a full-table branchless scan whose access pattern is identical for every draw. (ii) The relative-norm and integer-LWE attacks [21, 10] exploited that the BLISS *acceptance probability is a function of the secret quadratic form* $\|Sc\|^2$, making the retry count leak a linear system in $SS^\top$; Shuttle's acceptance test is the public inequality $\|(z_1, z_2')\|_2 \le B_v$ on already-declassified quantities, the IRS inner sampler is rejection-free with a fixed 18τ-byte budget, so the per-iteration behaviour carries no key information and the only public signal is the abort count. (iii) The branch-tracing and EM variants [21] read the *non-constant-time control flow* of the Gaussian sampler and the secret-dependent Bernoulli evaluation; Shuttle's sampler is straight-line and its ApproxExp is an integer, branch-free, division-free polynomial. (iv) The one-bit timing attack [45] showed that BLISS's single bimodal *sign-flip bit*, the only mask of the public response's mean, gave a total break when leaked; Shuttle carries no such load-bearing bimodal sign—all sign handling is a branchless two's-complement select and is not the linchpin of any zero-mean argument. (v) Loop-abort faults [20] truncated the mask; Shuttle's fixed-iteration sampler and final public norm check make a truncated mask fail verification. Crucially, the constant-time redesign of BLISS, GALACTICS [7], is itself a cautionary tale: although provably constant-time, it still fell to a machine-learning *power* analysis of its secret-dependent Gaussian arithmetic [34]—evidence that constant time is necessary but not sufficient against the power/EM adversary that we explicitly place out of scope here, and that we delegate to masking at deployment. Shuttle follows the GALACTICS lesson by construction from the start: its sampler data flow is oblivious by design, and because the per-iteration accept rate is a published key-independent constant, the decoupling of the accept/reject decision from any

secret quadratic form is structural rather than a patched-in fix.

Summary. The reference and AVX2/AVX512 implementations of SHUTTLE are constant-time by construction and verified by the multi-tool audit matrix above, with the object-code forbidden-mnemonic scan as the authoritative instruction-level gate. The only timing-observable quantity in the entire signing path is the public Fiat–Shamir-with-aborts signing-attempt count—a function of public and declassified data that is independent of the secret key.

4.4.2 Power/EM and other implementation analysis.

Unlike timing attacks on high-performance CPUs, SHUTTLE does not provide intrinsic resistance against power or EM side-channel analysis. Without appropriate countermeasures (e.g., masking), many intermediate states may leak sensitive information through power or EM emissions during SHUTTLE's execution. For application scenarios where resistance to such side-channel attacks is required, developers should deploy appropriate implementation-level countermeasures, as is common practice for most post-quantum cryptographic schemes. Note that the discussion above concerns only data-dependent leakage. To mitigate instruction-dependent leakage, developers should also follow the guidance in Section 4.4.1 and eliminate any secret-dependent branches.

Fault attacks, whether physically induced or caused by memory fault mechanisms (e.g., Rowhammer [29]), also pose a serious threat to security-certified cryptographic devices. Protecting an implementation against fault attacks generally requires hardware-assisted or low-level implementation techniques (e.g., tamper detection, active shielding, or secure erase), which cannot be addressed solely at the algorithmic level.

4.5 Other Security Properties

Non-Resignability via the salted BUFF transform. Non-resignability informally requires that, given a signature on a message M under a public key pk but *without* knowing M, one cannot produce a valid signature on the same M under a different public key pk'. We obtain this property by composing SHUTTLE with the BUFF transform, relying on the detailed analysis of Don, Fehr, Huang, and Struck [16]. That work revisits non-resignability carefully: in the plain model it shows, by a simple attack, that the (unsalted) BUFF transform does *not* achieve non-resignability whenever the message retains high entropy given the signature, and that the natural plain-/random-oracle-model definition is in fact unsatisfiable (so earlier positive claims rested on faulty arguments). On the positive side, the same work proves that the *salted* BUFF transform satisfies a slightly weakened, statistical variant of non-resignability in the random oracle model, against both classical and quantum adversaries.

Accordingly, SHUTTLE can adopt the salted BUFF transform: each signature additionally carries a random salt together with the binding hash $\mathsf{H}(pk \parallel M)$ (with H modeled as a random oracle). With this transform, SHUTTLE achieves the non-resignability notion established in [16].

5 Performance Evaluation and Implementation Analysis

This section reports the measured software performance of SHUTTLE on both high performance CPU (i.e. x86) and embedded MCU (i.e. ARM Cortex-M4), compares it with representative lattice signatures.

5.1 Performance on x86 architecture.

All measurements in this section were taken on the same Intel Core i7-11700K platform under Linux 6.11, with Turbo Boost and Hyper-Threading disabled. Every operation is benchmarked over 10,000 runs, and the reported cycle counts (both the median $p50$ and the 99th percentile $p99$) are computed from these 10,000 samples. For SHUTTLE we report the SHA3/SHAKE mode only; the implementation also supports an SM3-based NGCC mode, but using SHA3/SHAKE keeps the hash and extendable-output functions aligned with the comparison schemes.

5.1.1 Benchmarking Results

Table 15 gives exact classical security estimates, public-key size, signature size, their sum, and the median cycle counts for KeyGen, Sign, and Verify together with the 99th-percentile signing time. The rows are partitioned by security level. Within each partition, portable reference implementations are listed first, followed by vectorized (SSE2/AVX2) implementations; SHUTTLE additionally includes an AVX-512 implementation. Following current naming, we refer to Dilithium as ML-DSA.

5.1.2 Implementation Analysis

SHUTTLE provides a portable ISO C reference implementation and optimized AVX2 and AVX-512 implementations. The vectorized backends preserve the same byte format and KAT boundary as the reference implementation. The main speedups come from vectorized NTT arithmetic, multi-stream SHAKE expansion, batched Gaussian sampling, and vectorized arithmetic in the iterative sampling decision.

The size measurements in Table 15 use the rANS signature format specified in Section 2.5.10. In particular, the realized signature sizes are 1183, 2417, and 5001 bytes for the 128-, 256-, and 512-bit parameter sets, and the combined public-key-plus-signature sizes are 2447, 4369, and 8649 bytes.

5.2 Performance on ARM Cortex-M4

In this section, we evaluate SHUTTLE on an ARM Cortex-M4 microcontroller (STM32L4R5ZI). This platform is chosen because its 640 KB SRAM is sufficient to accommodate even the reference implementation of SHUTTLE-512 without memory constraints. Cycle counts are measured using the ARM Data Watchpoint and Trace (DWT) unit integrated into the Cortex-M4 core. Each reported result is the median of $N_{\text{test}} = 100$ independent measurements. Stack usage is estimated following the methodology adopted by *pqm4* [27]: the available stack is first filled with a canary pattern, after which the maximum overwritten region is measured during execution.

5.2.1 Benchmark Results

Table 16 summarizes the cycle counts and stack usage of SHUTTLE on the STM32L4R5ZI (ARM Cortex-M4). Following the benchmarking methodology of *pqm4* [27], the MCU is clocked at 20 MHz so that instruction fetches incur zero wait states.

Compared with ML-DSA, the performance gap of SHUTTLE decreases as the security level increases. While SHUTTLE-128 requires approximately 7.8× more cycles than ML-DSA-44 for key generation, the gap is reduced to 4.1× for the reference implementation of SHUTTLE-256 versus ML-DSA-87, and further to only 2.7× after assembly optimization. Notably, the optimized implementation of SHUTTLE-256 achieves a verification time lower than that of ML-DSA-87.

Table 15: Size and performance comparison. Sec. is in classical bits; sizes are in bytes; timings are CPU cycles in thousands over 10,000 runs, rounded to the nearest integer.

Algorithm	**Sec.**	**Impl.**	$\|pk\|$	$\|\sigma\|$	$\|pk\|+\|\sigma\|$	**KeyGen**	**Sign**	**Sign-p99**	**Verify**
Shuttle-128	138	ref	1264	1183	2447	2015	2083	2162	425
ML-DSA-44	123	ref	1312	2420	3732	287	1008	4392	315
HAETAE2	119	ref	992	1474	2466	1021	3065	18857	212
Shuttle-128	138	avx2	1264	1183	2447	566	614	638	131
ML-DSA-44	123	avx2	1312	2420	3732	80	186	758	86
Falcon-512	120	sse2	897	666	1563	10445	824	852	47
HAETAE2	119	avx2	992	1474	2466	413	1163	7074	92
Shuttle-128	138	avx512	1264	1183	2447	367	444	468	112
ML-DSA-65	183	ref	1952	3309	5261	526	1701	7047	504
HAETAE3	180	ref	1472	2349	3821	1683	4662	24205	373
ML-DSA-65	183	avx2	1952	3309	5261	137	310	1232	139
HAETAE3	180	avx2	1472	2349	3821	623	1537	7529	147
Shuttle-256	266	ref	1952	2417	4369	2993	3636	3704	671
ML-DSA-87	252	ref	2592	4627	7219	769	2059	7720	806
HAETAE5	256	ref	2080	2948	5028	1451	5775	33899	461
Shuttle-256	266	avx2	1952	2417	4369	787	1058	1077	197
ML-DSA-87	252	avx2	2592	4627	7219	215	388	1285	217
Falcon-1024	273	sse2	1793	1280	3073	38050	1663	1702	89
HAETAE5	256	avx2	2080	2948	5028	666	1804	10856	173
Shuttle-256	266	avx512	1952	2417	4369	500	765	785	167
Shuttle-512	522	ref	3648	5001	8649	5880	8367	8450	1322
Shuttle-512	522	avx2	3648	5001	8649	1544	2478	2508	376
Shuttle-512	522	avx512	3648	5001	8649	962	1841	1880	312

Notes.

Security. The Sec. column reports exact classical security estimates: for Shuttle, the minimum of the MLWE and MSIS-sUF estimates in Table 10; for ML-DSA, Falcon, and HAETAE, the values from the comparison table in the Lithium paper.

Sizes. The ML-DSA, Falcon, and HAETAE logs include public-key and signature sizes, so the comparison uses $|pk|+|\sigma|$.

Benchmark sources. ML-DSA/Dilithium (`https://github.com/pq-crystals/dilithium.git`, commit `bba9534`); Falcon (`https://github.com/pornin/c-fn-dsa.git`, commit `3aa5048`); HAETAE (`https://github.com/kpqc-cryptocraft/KpqClean_ver2.git`, commit `0896ebe`, tag `v2.0.0`).

Falcon. The Falcon reference implementation (c-fn-dsa) relies on SSE2/floating-point intrinsics rather than portable scalar C, so we label it `sse2` and group it with the optimized implementations; we do not include the separately optimized AVX2 Falcon.

Hashing. All Shuttle rows use SHA3/SHAKE for fair comparison with the other schemes; the same code base also supports an SM3-based mode.

Signing-time tail. Tail behavior differs markedly across Fiat–Shamir-with-aborts schemes: our measurements give ML-DSA signing $p99/p50$ ratios of about 3.31–4.36 and HAETAE ratios of about 4.90–6.15, while Shuttle stays near one (1.01–1.05 across the reported SHA3/SHAKE rows), confirming that the fixed-iteration signing design removes the long signing-time tail.

Table 16: Size and performance comparison. Sec. is in classical bits; sizes are in bytes; timings are median MCU cycles in thousands; stack is peak usage in KB as KeyGen / Sign / Verify (ARM NUCLEO-L4R5ZI @ 20 MHz).

Algorithm	**Sec.**	**Impl.**	$\lvert pk\rvert$	$\lvert\sigma\rvert$	$\lvert pk\rvert + \lvert\sigma\rvert$	**KeyGen**	**Sign**	**Verify**	**Stack [KB]**
Shuttle-128	138	ref	1264	1183	2447	14432	14112	5070	39/105/ 54
ML-DSA-44	123	clean	1312	2420	3732	1871	5992	2063	37/ 51/ 35
Shuttle-128	138	m4f	1264	1183	2447	10636	11648	2194	38/105/ 54
ML-DSA-44	123	m4f	1312	2420	3732	1423	2965	1422	37/ 44/ 9
ML-DSA-65	183	clean	1952	3309	5261	3206	10673	3377	59/ 78/ 56
ML-DSA-65	183	m4f	1952	3309	5261	2516	4525	2416	59/ 67/ 10
Shuttle-256	266	ref	1952	2417	4369	21797	24576	8219	61/163/ 79
ML-DSA-87	252	clean	2592	4627	7219	5342	10789	5610	95/120/ 91
Shuttle-256	266	m4f	1952	2417	4369	11588	19161	3351	61/163/ 79
ML-DSA-87	252	m4f	2592	4627	7219	4275	7228	4193	95/105/ 12
Shuttle-512	522	ref	3648	5001	8649	43257	58445	17215	121/325/153
Shuttle-512	522	m4f	3648	5001	8649	29204	52418	6596	121/325/153

Notes.

Benchmark sources. ML-DSA-clean (`https://github.com/pqclean/pqclean`, commit `c3e6861f`); ML-DSA-m4f (`https://github.com/mupq/pqm4`, commit `cc2c1b9`).

Stack. Peak stack usage per operation (KeyGen / Sign / Verify), in kilobytes (1 KB = 1024 B), rounded to integers.

Hashing. All Shuttle rows use SHA3/SHAKE for fair comparison with the other schemes.

5.3 Transmission and Storage Overhead

Minimizing communication bandwidth and persistent key-storage overhead is a critical design goal for post-quantum signatures, especially in bandwidth-limited network protocols, embedded devices, and secure elements. Table 17 reports the concrete sizes of the public key (pk), the serialized secret key (sk), and the signature (σ) for the three parameter sets of SHUTTLE.

Table 17: Communication and Storage Sizes of SHUTTLE

Parameter Set	$\|pk\|$ **(Bytes)**	$\|sk\|$ **(Bytes)**	$\|\sigma\|$ **(Bytes)**	$\|pk\| + \|\sigma\|$ **(Bytes)**
SHUTTLE-128	1264	2288	1183	2447
SHUTTLE-256	1952	3680	2417	4369
SHUTTLE-512	3648	7104	5001	8649

The public key and signature sizes determine the communication overhead transmitted over the network, whereas the secret-key size determines the amount of persistent secure storage required by a signer. In SHUTTLE, the secret key is stored in its fully expanded serialized form:

$$sk = (seed_A, b, K, tr, s_1, s_2'),$$

which contains the public matrix seed, the rounded public-key component, auxiliary hashing/randomization material, and the secret vectors needed for signing. This representation avoids repeated key expansion during the signing phase, thereby minimizing signing latency on performance-critical platforms.

By virtue of our asymmetric structural compression and efficient rANS entropy coding, SHUTTLE achieves exceptional compactness. At the 128-bit security level, the standalone signature of SHUTTLE-128 requires only 1183 bytes. This is highly advantageous for real-world network protocols, as it comfortably fits within the standard 1500-byte Ethernet Maximum Transmission Unit (MTU) limit. Consequently, transmitting a standalone signature does not trigger IP-layer packet fragmentation, significantly reducing the packet loss rate and handshake latency in UDP-based protocols (e.g., DTLS or DNSSEC) and automotive CAN FD platforms. Even when the public key and signature are transmitted together, the combined overhead of 2447 bytes remains highly competitive, requiring at most two standard Ethernet frames.

6 Features Statement

In accordance with the New Generation Commercial Cryptography (NGCC) submission guidelines, this section highlights the distinctive theoretical and engineering advantages of SHUTTLE. By shifting the design paradigm of lattice-based signatures, SHUTTLE achieves an unprecedented combination of security, compactness, and deterministic real-time performance.

6.1 Innovation

The core innovation of SHUTTLE lies in its "Two-Birds-with-One-Stone" design philosophy, which harmonizes post-quantum compactness with constant-time execution.

- **Rejection-Free Iterative Sampling:** Unlike traditional Fiat-Shamir with aborts schemes (e.g., ML-DSA and HAETAE) that rely on secret-dependent restarts, SHUTTLE introduces a novel approximate iterative transition mechanism. By bounding the accumulated statistical drift via Rényi divergence, we completely eliminate intermediate abort branches. This simultaneously eradicates the risk of timing/cache side-channel leakage and the notorious "long-tail" signing latency.

- **Asymmetric Structural Compression:** To overcome the size penalty inherent in standard MLWE, we introduce a tailored stretch-and-compress mechanism. By scaling the constant component of the secret by α_1, we compress the response vector $\mathbf{z}$ aggressively. This achieves signature sizes that rival floating-point-based schemes (like Falcon) while retaining the simplicity and robustness of integer-only arithmetic.
- **Logarithmic-Domain Decision Logic:** We map the randomized decision logic into the logarithmic domain. By leveraging the monotonicity of base-two logarithms, we replace heavy transcendental evaluations (infinite Gaussian sums and exponentials) with simple, nested integer interval comparisons, reducing computational overhead to just a single logarithmic evaluation on public randomness.

6.2 Compatibility and Real-World Applicability

SHUTTLE is engineered for seamless integration into existing security infrastructures and bandwidth-constrained network protocols.

- **Standardized Cryptographic API Compatibility:** In strict compliance with NGCC guidelines, SHUTTLE treats the extendable-output function (XOF) as a black-box primitive. It is fully compatible with standard hash functions (such as SHA3/SHAKE) and can be seamlessly transitioned to the SM3-based commercial cryptography suite without altering the underlying signature logic.
- **IPsec and TLS 1.3 Friendly Bandwidth Profile:** At security Level 1, the combined public key and signature size of SHUTTLE-128 is only 2447 bytes (a *34.4% reduction* compared to ML-DSA-44's 3732 bytes). This compact size comfortably fits within the standard Ethernet Maximum Transmission Unit (MTU) of 1500 bytes for standalone signatures ($\sigma = 1183$ bytes), preventing IP fragmentation and packet loss in high-throughput network protocols like TLS 1.3 [42] and IPsec/IKEv2 [28].
- **Deterministic Real-Time Scheduling:** By eliminating the signing rejection loop, SHUTTLE guarantees a nearly flat signing latency profile (with a $p99/p50$ ratio of 1.01–1.05). This deterministic execution is essential for time-triggered networks and real-time systems, such as automotive CAN bus authentication and industrial control loops, where sudden latency spikes cannot be tolerated.

6.3 Flexibility and Scalability

The mathematical and structural architecture of SHUTTLE offers excellent adaptability across diverse hardware and software ecosystems.

- **Unified Hardware Footprint:** By fine-tuning the block-wise scaling parameters $(\alpha_1, \alpha_s, \alpha_e)$ for each security level, all three security categories (128-, 256-, and 512-bit) utilize a single, unified wide Gaussian standard deviation $r = 825$. This allows all parameter sets to share the exact same precomputed Reverse Cumulative Distribution Table (RCDT), drastically reducing the storage footprint and simplifying coprocessor design for resource-constrained IoT and smart card devices.
- **Cross-Platform Vectorization Scalability:** The integer-only design of SHUTTLE avoids any floating-point or emulation overhead. The implementation scales naturally from low-end embedded microcontrollers (e.g., ARM Cortex-M4) to high-performance servers, where it can leverage AVX2 and AVX-512 vector extensions to achieve up to 2.6× speedups over state-of-the-art bimodal schemes.

References

[1] Martin R. Albrecht, Carlos Cid, Jean-Charles Faugère, Robert Fitzpatrick, and Ludovic Perret. On the complexity of the BKW algorithm on LWE. *Des. Codes Cryptogr.*, 74(2):325–354, 2015.

[2] Martin R. Albrecht, Benjamin R. Curtis, and Thomas Wunderer. Exploring trade-offs in batch bounded distance decoding. In Kenneth G. Paterson and Douglas Stebila, editors, *Selected Areas in Cryptography - SAC 2019 - 26th International Conference, Waterloo, ON, Canada, August 12-16, 2019, Revised Selected Papers*, volume 11959 of *Lecture Notes in Computer Science*, pages 467–491. Springer, 2019.

[3] Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving usvp and applications to LWE. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, Lecture Notes in Computer Science, pages 297–322. Springer, 2017.

[4] Martin R Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.

[5] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange - A new hope. In Thorsten Holz and Stefan Savage, editors, *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016*, pages 327–343. USENIX Association, 2016.

[6] Sanjeev Arora and Rong Ge. New algorithms for learning in presence of errors. In Luca Aceto, Monika Henzinger, and Jirí Sgall, editors, *Automata, Languages and Programming - 38th International Colloquium, ICALP 2011, Zurich, Switzerland, July 4-8, 2011, Proceedings, Part I*, volume 6755 of *Lecture Notes in Computer Science*, pages 403–415. Springer, 2011.

[7] Gilles Barthe, Sonia Belaïd, Thomas Espitau, Pierre-Alain Fouque, Mélissa Rossi, and Mehdi Tibouchi. GALACTICS: Gaussian sampling for lattice-based constant-time implementation of cryptographic signatures, revisited. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019*, pages 2147–2164. ACM, 2019.

[8] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In Robert Krauthgamer, editor, *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2016, Arlington, VA, USA, January 10-12, 2016*, pages 10–24. SIAM, 2016.

[9] Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In *Proceedings of the 13th ACM Conference on Computer and Communications Security (CCS 2006)*, pages 390–399. ACM, 2006.

[10] Jonathan Bootle, Claire Delaplace, Thomas Espitau, Pierre-Alain Fouque, and Mehdi Tibouchi. LWE without modular reduction and improved side-channel attacks against BLISS. In Thomas Peyrin and Steven D. Galbraith, editors, *Advances in Cryptology — ASIACRYPT 2018*, volume 11272 of *Lecture Notes in Computer Science*, pages 494–524. Springer, 2018.

[11] Leon Groot Bruinderink, Andreas Hülsing, Tanja Lange, and Yuval Yarom. Flush, gauss, and reload — A cache attack on the BLISS lattice-based signature scheme. In Benedikt Gierlichs and Axel Y. Poschmann, editors, *Cryptographic Hardware and Embedded Systems — CHES 2016*, pages 323–345. Springer, 2016.

[12] Kévin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of matzov's dual attack on kyber. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part I*, volume 16000 of *Lecture Notes in Computer Science*, pages 444–476. Springer, 2025.

[13] André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part IV*, volume 13093 of *Lecture Notes in Computer Science*, pages 63–91. Springer, 2021.

[14] Jung Hee Cheon, Hyeongmin Choe, Julien Devevey, Tim Güneysu, Dongyeon Hong, Minhyeok Kang, Taekyung Kim, Jeongbeen Ko, Markus Krausz, Georg Land, Marc Möller, Junbum Shin, Damien Stehlé, and MinJune Yi. HAETAE: Shorter lattice-based Fiat–Shamir signatures. Algorithm Specification and Supporting Documentation, 2024.

[15] Julien Devevey, Pouria Fallahpour, Alain Passelègue, Damien Stehlé, and Keita Xagawa. A detailed analysis of fiat-shamir with aborts. In *Annual International Cryptology Conference*, pages 327–357. Springer, 2023.

[16] Jelle Don, Serge Fehr, Yu-Hsuan Huang, and Patrick Struck. On the (In)Security of the BUFF transform: Non-resignability and Φ-non-malleability in the plain and the random oracle model. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, volume 14920 of *Lecture Notes in Computer Science*, pages 246–275. Springer, 2024.

[17] Léo Ducas, Alain Durmus, Tancrède Lepoint, and Vadim Lyubashevsky. Lattice signatures and bimodal gaussians. In *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 40–56. Springer, 2013.

[18] Léo Ducas, Thomas Espitau, and Eamonn W. Postlethwaite. Finding short integer solutions when the modulus is small. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III*, volume 14083 of *Lecture Notes in Computer Science*, pages 150–176. Springer, 2023.

[19] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(1):238–268, 2018.

[20] Thomas Espitau, Pierre-Alain Fouque, Benoît Gérard, and Mehdi Tibouchi. Loop-abort faults on lattice-based Fiat–Shamir and hash-and-sign signatures. In Roberto Avanzi and Howard M. Heys, editors, *Selected Areas in Cryptography — SAC 2016*, volume 10532 of *Lecture Notes in Computer Science*, pages 140–158. Springer, 2016.

[21] Thomas Espitau, Pierre-Alain Fouque, Benoît Gérard, and Mehdi Tibouchi. Side-channel attacks on BLISS lattice-based signatures: Exploiting branch tracing against strongswan and electromagnetic emanations in microcontrollers. In Bhavani Thuraisingham, David

Evans, Tal Malkin, and Dongyan Xu, editors, *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017*, pages 1857–1874. ACM, 2017.

[22] Joel Gärtner. Compact lattice signatures via iterative rejection sampling. In *Annual International Cryptology Conference*, pages 514–547. Springer, 2025.

[23] Lewis Glabush, Patrick Longa, Michael Naehrig, Chris Peikert, Douglas Stebila, and Fernando Virdia. Frodokem: A cca-secure learning with errors key encapsulation mechanism. *IACR Commun. Cryptol.*, 2(3):25, 2025.

[24] Jianhua Hou, Haodong Jiang, and Tabitha Ogilvie. Careful with the ring! concrete hardness gaps between lwe and mlwe. *To appear at CRYPTO 2026*, 2026.

[25] James Howe, Thomas Prëst, Thomas Ricosset, and Mélissa Rossi. Isochronous Gaussian sampling: From inception to implementation. In *Post-Quantum Cryptography – PQCrypto 2020*, volume 12100 of *Lecture Notes in Computer Science*, pages 53–71. Springer, 2020.

[26] Nick Howgrave-Graham. A hybrid lattice-reduction and meet-in-the-middle attack against NTRU. In Alfred Menezes, editor, *Advances in Cryptology - CRYPTO 2007, 27th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2007, Proceedings*, volume 4622 of *Lecture Notes in Computer Science*, pages 150–169. Springer, 2007.

[27] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking NIST PQC on ARM cortex-m4. *IACR Cryptol. ePrint Arch.*, 2019:844, 2019.

[28] Charlie Kaufman, Paul Hoffman, Yoav Nir, Pasi Eronen, and Tero Kivinen. Internet key exchange protocol version 2 (ikev2). RFC 7296, Internet Engineering Task Force (IETF), October 2014.

[29] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. Flipping bits in memory without accessing them: An experimental study of dram disturbance errors. In *2014 ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*, pages 361–372, 2014.

[30] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.

[31] Eunmin Lee, Joohee Lee, and Yuntao Wang. Improved meet-lwe attack via ternary trees. *IACR Cryptol. ePrint Arch.*, 2024:824, 2024.

[32] Vadim Lyubashevsky. Fiat-shamir with aborts: Applications to lattice and coding-based signatures. In *Advances in Cryptology – ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 598–616. Springer, 2009.

[33] Vadim Lyubashevsky. Lattice signatures without trapdoors. In *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 738–755. Springer, 2012.

[34] Soundes Marzougui, Nils Wisiol, Patrick Gersch, Juliane Krämer, and Jean-Pierre Seifert. Machine-learning side-channel attacks on the GALACTICS constant-time implementation of BLISS. In *ARES 2022: The 17th International Conference on Availability, Reliability and Security*, pages 34:1–34:11. ACM, 2022.

[35] MATZOV. Report on the security of LWE: Improved dual lattice attack. Technical report, MATZOV, April 2022.

[36] Alexander May. How to meet ternary LWE keys. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 701–731. Springer, 2021.

[37] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Technical Report FIPS PUB 202, U.S. Department of Commerce, 2015.

[38] National Institute of Standards and Technology. Module-lattice-based digital signature standard. Technical Report FIPS PUB 204, U.S. Department of Commerce, 2024.

[39] David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *Journal of Cryptology*, 13(3):361–396, 2000.

[40] Thomas Prest. Sharper bounds in lattice-based cryptography using the Rényi divergence. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 347–374. Springer, 2017.

[41] Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon: Fast-fourier lattice-based compact signatures over NTRU. Technical report, NIST PQC Standardisation Round 1 Submission, 2018.

[42] Eric Rescorla. The Transport Layer Security (TLS) protocol version 1.3. RFC 8446, 2018.

[43] Mélissa Rossi. *Extended Security of Lattice-Based Cryptography*. PhD thesis, Paris Sciences et Lettres University, France, 2020.

[44] Florian Skopik, Albert Treytl, Arjan Geven, Bernd Hirschler, Thomas Bleier, Andreas Eckel, Christian El-Salloum, and Armin Wasicek. Towards secure time-triggered systems. In *International Conference on Computer Safety, Reliability, and Security*, pages 365–372. Springer, 2012.

[45] Mehdi Tibouchi and Alexandre Wallet. One bit is all it takes: A devastating timing attack on BLISS's non-constant-time sign flipping. *Journal of Mathematical Cryptology*, 15(1):131–142, 2021.

[46] Armin Rudolf Wasicek. *Security in time-triggered systems*. PhD thesis, Technische Universität Wien, 2011.